データサイエンス実践A 講義ノート集（第２講〜第11講）

新潟大学 創生学部（2026年度）

はじめに

- 第２〜11講の講義ノートを1冊にまとめたもの。自分でデータをグラフにするときに、書き方を調べる参考資料として使う。
- **調べるとき**は、次の「グラフの種類から探す」表 → その講の「まとめ」の表（やりたいこと｜書き方）→ 本文の節、の順に見る。「第４講 H-3」のような文字はリンクで、押すとその節に飛ぶ。PDF のしおりや目次からも飛べる。
- はじめから通して読むときは、第２講から順に読む。各講は、前の講で学んだことを使って進む。
- 何度も出てくる説明は1か所にまとめ、ほかの講からは**講と節の番号**で参照している。節の記号（E-1、Z-2 など）は講義のときのままなので、アルファベットが飛んだり、順番が前後したりしているところがある。
- コードを試すときは、notebook と `dsa2026data.xlsx` を**同じ場所**（ホームフォルダ）に置く（第５講 L-1）。第11講は `dsa2026review.xlsx`（第11講 AU-1）、第10講 AH は画像 `sweets_montblanc.png` も使う。置き場が分からないときは、セルで次の1行を実行すると、notebook と同じ場所にダウンロードされる。
  ```
  !curl -O https://raw.githubusercontent.com/CreativeStudies-NGT/lectures/main/dsa/dsa2026data.xlsx
  ```
- 新しい notebook では、まず次のセルを実行してから、各節のコードを貼る。
  ```
  import numpy as np
  import pandas as pd
  import matplotlib.pyplot as plt
  import japanize_matplotlib
  ```
- 節のコードだけを試すと、その講の前の方で作った変数（df、x など）がなくて `NameError` になることがある。**その講の最初の方にある、データを用意するセル**（「準備」と書いてある講もある）を先に実行し、それでも出るときは、その節より上で変数を作っているセルを探して実行する（第２講「セルの実行順序」）。
- **エラーが出たら**、第11講 AF の表を見る（ほかに 第２講 C-3、第５講 L-4）。日本語が □ になるときは 第２講 C-1。
- 授業中に出た質問のうち、調べるときに役立つものを、巻末の付録 よくある質問にまとめた。

グラフの種類から探す

やりたいこと	基本	細かい指定・応用	まとめの表
グラフの種類			
折れ線グラフ	第３講 E-0、第３講 E-1	第３講 E-2、第３講 E-3、第３講 E-4、第３講 F-1、第３講 F-2、第３講 F-3	第３講
散布図	第４講 H-0、第４講 H-1	第４講 H-2、第４講 H-3、第４講 H-4、第４講 H-5、第４講 H-6、第４講 H-7、第４講 H-8、折れ線との違い（第４講）、第５講 L-5	第４講
平均・標準偏差・補助線	第４講 G-1、第４講 G-3、第４講 G-4	第４講 H-9、第６講 N-6、第７講 S-6、第９講 AA-5、第４講 I-2	第４講
棒グラフ	第６講 N-0、第６講 N-1	第６講 N-2、第６講 N-3、第６講 N-4、第６講 N-5、第６講 N-8、第６講 P-4、第７講 R-1、第７講 R-2、第７講 R-3	第６講、第７講
横棒グラフ・並べ替え	第６講 N-7	第６講 P-1、第11講 AV-1	第６講
積み上げ棒グラフ	第７講 R-4	第11講 AV-2	第７講
円グラフ・ドーナツ	第６講 O-0、第６講 O-1	第６講 O-2、第６講 O-3、第６講 O-4、第６講 O-5、第６講 O-6、第６講 O-7、第６講 O-8、第６講 P-3、第10講 AH-4	第６講
箱ひげ図	第７講 S-0、第７講 S-2	第７講 S-1、第７講 S-3、第７講 S-4、第７講 S-5、第７講 S-7、第７講 T-1	第７講
ヒストグラム	第９講 AA-0、第９講 AA-1	第９講 AA-2、第９講 AA-3、第９講 AA-4、第９講 AA-6、第９講 AC-2、第９講 AC-3	第９講
エラーバー	第９講 AB-3、第９講 AB-4	第９講 AB-5、第９講 AB-6、第９講 AB-7	第９講
複合グラフ（両 y 軸）	第８講 Z-1、第８講 Z-2	第８講 Z-3、第８講 Z-4、指数化（第８講）	第８講
文字・矢印を書く（注釈）	第８講 V-1	第８講 V-2、第８講 V-3、第４講 H-10	第８講

ローソク足（株価）	第７講 R-4 応用		第７講
2本の線の間を塗る	第11講 AV-4	第４講 I-2	第11講
ヒートマップ	第11講 AV-5		第11講
矢印で向き（風・流れ）	第11講 AV-8		第11講
グラフを並べる	第10講 AI-1	第11講 AV-6	第10講
画像・記号のマーカー、背景の画像	第10講 AH-1、第10講 AH-2、第10講 AH-3		第10講
手書き風・動くグラフ	第10講 AJ-1	第10講 AJ-2、第10講 AJ-3、第10講 AJ-4、第10講 AJ-5	第10講
共通の作業			
Excel を読む・列を取り出す	第５講 L-1、第５講 L-2、第５講 L-3	第５講 L-4、整っていない Excel・CSV（第５講）、第７講 Q-1	第５講
グラフを保存する	第５講 K-1、第５講 K-2	第５講 K-3、第５講 K-4	第５講
色・カラーマップ	第４講 H-4、第６講 P-2	色の組み合わせを替える（第６講）	
日本語の表示・import	第２講 C-1		
Jupyter の操作	ショートカットと関数の調べ方（第２講）	第３講 F-4	
エラーが出たとき	第11講 AF	第２講 C-3、第５講 L-4	
伝わるグラフか確かめる	第11講 AT-3	第11講 AT-1、第11講 AT-2	
グラフの型・書く順番の一覧	第11講 AU-3	第11講 AU-4、第11講 AU-5、第11講 AV-9	

ここにないグラフや指定は、第11講 AU-3 と 第11講 AV-9 の一覧も見る。

目次

第２講：Python入門 ― 最初のグラフを描くまで

- Jupyter の使い方
- 0. 導入
- A. 値を扱う
  A-1 print：結果を表示する　A-2 変数と代入　A-3 計算　A-4 文字列
- B. データをまとめる
  B-1 リスト　B-2 要素数と要素の取り出し
- C. 道具を使う
  C-1 import：ライブラリを読み込む　C-2 関数と引数　C-3 エラーの読み方
- D. グラフを描く
- まとめ

第３講：折れ線グラフを整える

- E. 折れ線グラフを整える
  E-0 完成形　E-1 マーカーの種類・サイズ・色　E-2 軸の範囲　E-3 タイトル・軸の名前とフォントサイズ　E-4 2本の線を重ねる
- F. グラフのオプション
  F-1 凡例の場所・枠　F-2 線の種類・太さと短い書き方　F-3 目盛り・透明度・目盛り線　F-4 Markdown セルで文章を書く
- まとめ

第４講：平均・標準偏差と補助線、散布図

- G. 平均・標準偏差と補助線
  G-0 平均と補助線の完成形　G-1 平均　G-2 最大・最小・合計・中央値・四分位数　G-3 標準偏差　G-4 補助線
- H. 散布図を描く
  H-0 散布図の完成形　H-1 散布図の基本　H-2 マーカーの種類と色　H-3 点の大きさ　H-4 点ごとに色を変える　H-5 2つのグループを重ねる　H-6 重なる順番　H-7 透明度　H-8 凡例とその場所　H-9 散布図に補助線を引く　H-10 点に名前を付ける
- I. 発展
  I-1 np.array でまとめて計算する　I-2 平均 ± 標準偏差の範囲を塗る
- まとめ

第５講：グラフの保存・Excel の読み込み

- K. グラフの保存
  K-1 画像ファイルに保存する　K-2 細かさと背景の透明化　K-3 保存の形式と、保存した図を直す　K-4 PowerPoint に貼る　K-5 画面の表示をきれいにする（retina）
- L. Excel のデータ
  L-1 Excel をいつもの場所に置く　L-2 Excel ファイルを読む　L-3 列を取り出す　L-4 よくあるエラー　L-5 Excel のデータで散布図　L-6 ipynb ファイルを人と交換する
- 発展
  K-6 スタイルシート
- まとめ

第６講：棒グラフ・円グラフ

- N. 棒グラフ
  N-0 完成形　N-1 棒グラフの基本　N-2 棒の色　N-3 棒の幅　N-4 外枠線と網掛け　N-5 データを増やす　N-6 平均の補助線　N-7 横棒グラフ　N-8 Excel のデータで棒グラフ
- O. 円グラフ
  O-0 完成形　O-1 円グラフの基本と始まりの位置　O-2 ラベル　O-3 色　O-4 一部を切り出す　O-5 割合を表示する　O-6 灰色だけで描く　O-7 Excel のデータで円グラフ　O-8 境目の線・ラベルの位置・ドーナツグラフ
- P. 発展
  P-1 並べ替えて上位だけ描く　P-2 カラーマップで色をまとめて決める　P-3 二重のドーナツグラフ　P-4 棒が多いときに1本だけ色を変える
- まとめ

第７講：値を数える・辞書・棒グラフの補足・箱ひげ図

- Q. 値を数える・辞書
  Q-1 Excel の列の値を数える　Q-2 辞書の基礎　Q-3 オプションの辞書
- R. 棒グラフの補足
  R-1 横軸の名前を書き換える　R-2 項目の順番を決める　R-3 棒の上に値を書く　R-4 積み上げ棒グラフ
- S. 箱ひげ図
  S-0 完成形　S-1 箱ひげ図の読み方　S-2 箱ひげ図の基本　S-3 項目の名前　S-4 箱の幅と塗りつぶし　S-5 平均を表示する　S-6 補助線を引く　S-7 Excel のデータで箱ひげ図
- T. 発展
  T-1 箱ごとに色を変える　T-2 棒を横に並べる
- まとめ

第８講：複合グラフと注釈

- Z. 複合グラフ・両 y 軸
  Z-0 完成形　Z-1 2つのグラフを重ねる　Z-2 右にもう1本 y 軸を作る（ひな形）　Z-3 単位を変える　Z-4 対数目盛（データはそのまま）
- V. 注釈：グラフに文字を書く
  V-1 plt.text の基本　V-2 文字の大きさ・色・位置　V-3 矢印付きの注釈
- まとめ

第９講：ヒストグラムとエラーバー

- AA. ヒストグラム
  AA-0 完成形（ヒストグラム）　AA-1 ヒストグラムの基本：まず手で数えてみる　AA-2 区切りの数　AA-3 色と枠線　AA-4 区切りの範囲　AA-5 平均の補助線　AA-6 2つのグループを重ねる
- AB. エラーバー
  AB-0 完成形（エラーバー）　AB-1 平均だけでは分からないこと　AB-2 リストを np.array にする　AB-3 平均と標準偏差を求める　AB-4 エラーバーを描く　AB-5 5教科のエラーバー　AB-6 棒グラフにエラーバーを付ける　AB-7 エラーバーと箱ひげ図の使い分け
- AC. 発展
  AC-1 区切りを15点刻みにする　AC-2 ヒストグラムを並べる・横向きにする　AC-3 ヒストグラムの1本だけ色を変える　AC-4 標準誤差と信頼区間　AC-5 関数とメソッド
- まとめ

第10講：遊び心のあるグラフ・グラフを並べる

- AH. 遊び心のあるグラフ
  AH-0 完成形　AH-1 画像をマーカーにする　AH-2 記号をマーカーにする　AH-3 背景に画像を敷く　AH-4 ドーナツグラフ・ゲージ
- AI. 複数のグラフを並べる
  AI-1 横に並べる・縦に並べる
- AJ. 発展
  AJ-1 手書き風のグラフ　AJ-2 手書き風フォントを入れると（紹介）　AJ-3 動くグラフ（アニメーション）　AJ-4 別の道具で動かす：plotly（紹介）　AJ-5 スライダーで動かす（ipywidgets）
- まとめ

第11講：伝わるグラフ・総復習＋α

- AT. 伝わるグラフ・誤解を招くグラフ
  AT-1 縦軸を途中から始めた棒グラフ　AT-2 項目が多すぎる円グラフ　AT-3 グラフのチェックリスト
- AU. 総復習
  AU-1 総復習のデータ　AU-2 総復習のグラフの一覧　AU-3 グラフの型の一覧　AU-4 共通の指定　AU-5 書く順番
- AV. ＋α：新しい形のグラフ
  AV-1 値の順に並べる（横棒グラフ）　AV-2 割合に直して比べる（100% 積み上げ棒）　AV-3 2つの列から新しい量を作る　AV-4 2本の線の間を塗る　AV-5 色の濃さで表す（ヒートマップ）　AV-6 グラフを並べて1枚にまとめる　AV-7 株価の動きを描く（ローソク足）　AV-8 矢印で向きを表す（風・流れ）　AV-9 ＋α のグラフの型の一覧
- AF. よくあるエラーと対処
- まとめ

付録 よくある質問

第 2 講：Python入門 — 最初のグラフを描くまで

Python の基本（値・変数・リスト・import・関数）を学び、折れ線グラフを1枚描くところまで進む。ここで描いたグラフを、第 3 講で見やすく整える。

Jupyter の使い方

起動方法

- ① **Jupyter Notebook を起動** → ② **ブラウザのファイル一覧から ipynb をクリックして開く**の順番で開く
  - Windows：スタートメニュー →「Anaconda (anaconda3)」→「Jupyter Notebook」
  - Mac：Anaconda Navigator を起動 →「Jupyter Notebook」の「Launch」
- **ipynb ファイルをダブルクリックしても起動しない。**「このファイルを開くアプリ」を聞かれても、選ばずに閉じる
- **黒い画面（Mac はターミナル）は閉じない。**ブラウザの画面は、黒い画面を通じて Python 本体（カーネル）を動かしている
- 終わるときは、ファイルを保存してから、ブラウザと黒い画面を閉じる

セルの実行順序

- セルの左の `[1]` `[2]` … は、**実行した順番。**上から並んでいても、その順に実行したとは限らない
- 変数を作るセルを実行しないまま、その変数を使うセルを実行すると **NameError** になる

▶ やってみよう：下の2つのセルのうち、**先に2つ目のセルだけ**を実行してエラーを確かめる。次に1つ目、2つ目の順に実行する。

```
# 1つ目：変数 score を作る
score = 80
```

```
# 2つ目：score を使う（1つ目を実行していないと NameError）
print(score)
```

```
80
```

- 困ったら「カーネル」→「カーネルを再起動してすべてのセルを実行」（英語表示では「Kernel」→「Restart Kernel and Run All Cells」）で、上から順にすべて実行し直す
- notebook を開き直したときは、結果の表示は残っていても**変数は消えている。**もう一度上から実行する

ショートカットと関数の調べ方

- セルの左が青い状態が「コマンドモード」（Esc で入る）、セルの中にカーソルがある状態が「編集モード」（Enter で入る）。下の表の多くはコマンドモードで使う
- Mac は Ctrl を command（⌘）に、Alt を option に読み替える

キー	動作	モード
Shift＋Enter	実行して次のセルへ	どちらでも

キー	動作	モード
Ctrl＋Enter	実行して、そのセルにとどまる	どちらでも
Alt＋Enter	実行して、下に新しいセルを追加	どちらでも
A ／ B	上 ／ 下にセルを追加	コマンド
D D（2回）	セルを削除	コマンド
Z	セルの削除を取り消す	コマンド
X ／ C ／ V	セルの切り取り ／ コピー ／ 貼り付け	コマンド
Shift＋M	選んだセルを結合	コマンド
Y ／ M ／ R	コードセル ／ Markdown セル ／ Raw セルに変更	コマンド
1 〜 6	Markdown の見出し（# 〜 ######）に変更	コマンド
O	出力（実行結果）の表示 ／ 非表示	コマンド
Shift＋L	行番号の表示 ／ 非表示	コマンド
I I（2回）	実行を中断（止まらなくなったとき）	コマンド
0 0（2回）	カーネルを再起動（変数はすべて消える）	コマンド
Ctrl＋S	保存	どちらでも
Ctrl＋/	選んだ行をコメント（#）にする ／ 戻す	編集
Shift＋Tab	関数の説明（引数・オプションの一覧）を表示	編集
Ctrl＋F	検索（ノートブック全体が対象）	どちらでも
Ctrl＋G ／ Ctrl＋Shift＋G	次 ／ 前の一致へ移動	検索中

関数の使い方を調べる（Shift＋Tab）

- セルの中で、関数名の後ろ（例： `plt.legend(` の「(」の後ろ）にカーソルを置いて Shift＋Tab を押すと、その関数の説明が小さな窓に出る。Esc で閉じる。Mac も同じ Shift＋Tab
- 説明は英語。Parameters の欄に、使えるオプション（ `loc` 、 `fontsize` など）と意味が並んでいる
- `plt.legend` のようにライブラリの関数を調べるときは、先に import のセルを実行しておく（実行していないと `plt` が何か分からないので、説明が出ない）
- カーソルが行の先頭（空白だけの位置）にあるときは、Shift＋Tab は字下げを戻す動作になる

検索・置換

- Ctrl＋F で、ノートブックの右上に検索ボックスが出る（メニューの「編集」→「検索...」でも同じ）
- 置換したいときは、検索ボックスの左端のボタン（「置換を表示」）を押すと置換欄が出る。置き換える文字を入れて「置換」（1つずつ）か「すべてを置換」を押す
- ノートブック以外（ブラウザのアドレスバーなど）を選んだ状態で Ctrl＋F を押すと、ブラウザの検索が開く。セルを一度クリックしてから押す

ネット上の古い表（Notebook 6 版）との違い

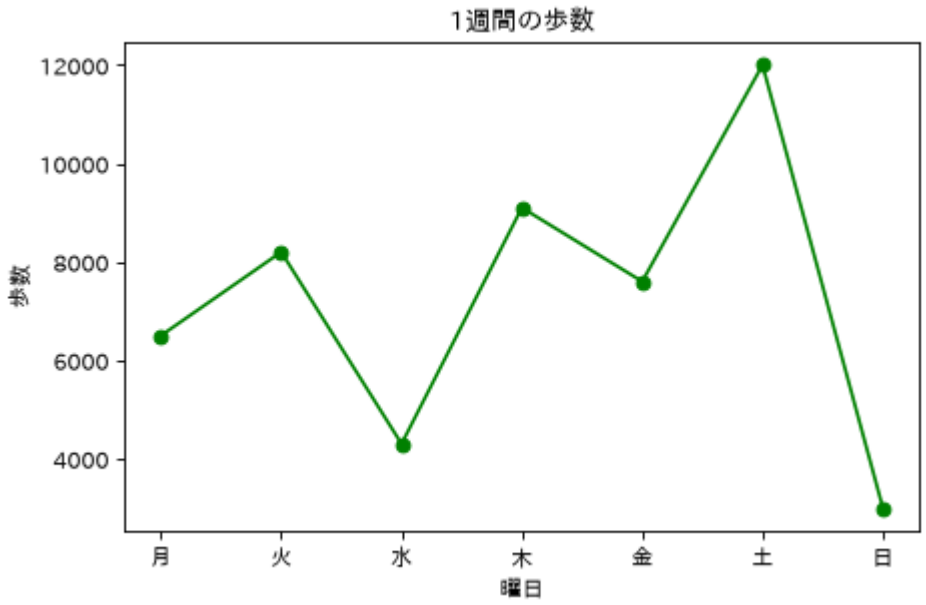
- H（ショートカット一覧）は使えない。一覧は Ctrl＋,（カンマ）で設定エディタを開き、「キーボードショートカット」で見る

- コマンドパレットは Ctrl＋Shift＋C、検索は F ではなく Ctrl＋F（Ctrl＋Shift＋F は左のファイル一覧の表示切替）
- S、L、Shift＋O、Q、Space、Shift＋V は、今の版では割り当てがない（行番号は L ではなく Shift＋L）

---

0. 導入

この先で描くグラフ



データ可視化の流れ

段階	第 2 講で学ぶもの
① データを用意する	変数、リスト
② 道具を読み込む	import
③ 描く	関数（`plt.plot` など）

この notebook の内容は、すべてこの①〜③のどれかに当たる。

A. 値を扱う

A-1 print：結果を表示する

- `print(...)` で、( ) の中身を画面に表示する。1つの print で1行
- 数値はそのまま書く。文字はクォーテーション `'...'` または `"..."` で囲む（どちらでもよい。このテキストではシングル `'` に統一）

```
print(100)
print('Good morning!')
```

```
print("Good morning!")
```

```
100
Good morning!
Good morning!
```

- カンマ `,` で区切ると、複数の値を並べて表示できる（間に半角スペースが入る）

```
print('身長は', 170, 'cm です')
```

```
身長は 170 cm です
```

> **つまずきやすい点**：括弧 `( )` やクォーテーション `'` を**全角**で書いたり、**閉じ忘れ**たりするとエラーになる（→ C-3 で扱う）。

A-2 変数と代入

コメント `#`

- `#` から行の終わりまでは**実行されない**。人間向けのメモ（コメントという）
- ここから先のひな形には `# 変数 steps に 8000 を代入する` のようなコメントが出てくる。これは「この下に何を書くか」の指示なので、**消さずに残してよい**
- `#` は**半角**で書く（全角の `＃` はコメントにならず、エラーになる）

```
# この行はコメントなので実行されない
print('コメントの行は無視される')   # 行の途中から書いてもよい
```

```
コメントの行は無視される
```

変数と代入

- 変数＝値に付ける**名札**
- `=` は「**等しい**」ではなく、「**右の値を左の名前に入れる**」という意味（代入という）

```
# 変数 sleep に 7 を代入する
sleep = 7

print(sleep)
```

```
7
```

- 数学では成り立たない `b = b + 1` も、Python では「今の b に 1 を足して、b に入れ直す」という意味になる

```
b = 10
b = b + 1   # 今の b（10）に 1 を足して、b に入れ直す

print(b)
```

```
11
```

- 変数名のルール
  - 英字（またはアンダースコア `_`）で始める
  - **大文字と小文字は別物**（`temp` と `Temp` は別の変数。→ C-3 の NameError）

A-3 計算

演算子	意味	例	結果
`+`	足し算	`3 + 2`	5
`-`	引き算	`3 - 2`	1
`*`	掛け算	`3 * 2`	6
`/`	割り算	`3 / 2`	1.5
`**`	べき乗	`3 ** 2`	9

- 計算の順番は数学と同じ（`**` → `* /` → `+ -`）。迷ったら `( )` を付ける

```
print(2 + 3 * 4)      # 掛け算が先
print((2 + 3) * 4)    # 括弧の中が先
```

```
14
20
```

- 変数を使って計算できる。例：税込価格、円の面積

```
price = 1200              # 税抜価格（円）
total = price * 1.1       # 税込価格

print(total)
```

```
1320.0
```

```
r = 3                     # 半径
area = 3.14 * r**2        # 円の面積

print(area)
```

```
28.26
```

- 割り算や小数を含む計算の結果は小数になる（`1320.0` のように `.0` が付く）
- 小数の計算では、**桁がたくさん出たり、ごくわずかにずれたりすることがあるが正常**（コンピュータの小数の仕組みによるもの）

```
print(10 / 4)
print(0.1 + 0.2)
```

```
2.5
0.30000000000000004
```

A-4 文字列

- 文字列は `'...'` または `"..."` で囲む（`'気温'` と `"気温"` は同じ。始めと終わりはそろえる）

- 文字列どうしは `+` でつなげる（連結）

```
name = '新潟' + '大学'

print(name)
```

新潟大学

- **同じ `+` でも、数値なら足し算、文字列なら連結**になる

```
print(10 + 20)       # 数値 → 足し算
print('10' + '20')   # 文字列 → 連結
```

```
30
1020
```

- 値には**種類（型）**がある：数値（`10`）と文字列（`'10'`）は見た目が似ていても別物
- ここで作る文字列は、D でグラフのタイトルや軸ラベルになる

B. データをまとめる

B-1 リスト

- 複数の値を `[ ]` で囲み、カンマで区切って1つにまとめたもの
- 数値のリストも、文字列のリストも作れる
- **グラフの x 軸・y 軸のデータは、リストで渡す**

```
temp = [22, 24, 19, 21, 25, 27, 23]              # 月～日の最高気温
days = ['月', '火', '水', '木', '金', '土', '日']   # 曜日

print(temp)
print(days)
```

```
[22, 24, 19, 21, 25, 27, 23]
['月', '火', '水', '木', '金', '土', '日']
```

B-2 要素数と要素の取り出し

- `len(リスト)` で要素の数を調べる

```
print(len(temp))
```

```
7
```

- `リスト[番号]` で要素を取り出す。**番号は 0 から始まる**

曜日	月	火	水	木	金	土	日
番号	0	1	2	3	4	5	6
temp	22	24	19	21	25	27	23

```
print(days[2], temp[2])   # 3番目（水曜）
```

```
水 19
```

- 7つの要素の最後は `[6]`。`[7]` は存在しないので **IndexError** になる（下のセルはエラーになるのが正しい）

```
print(temp[7])
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[17], line 1
----> 1 print(temp[7])

IndexError: list index out of range
```

- （発展）`temp[-1]` で最後の要素を取り出せる

```
print(temp[-1])
```

```
23
```

C. 道具を使う

C-1 import：ライブラリを読み込む

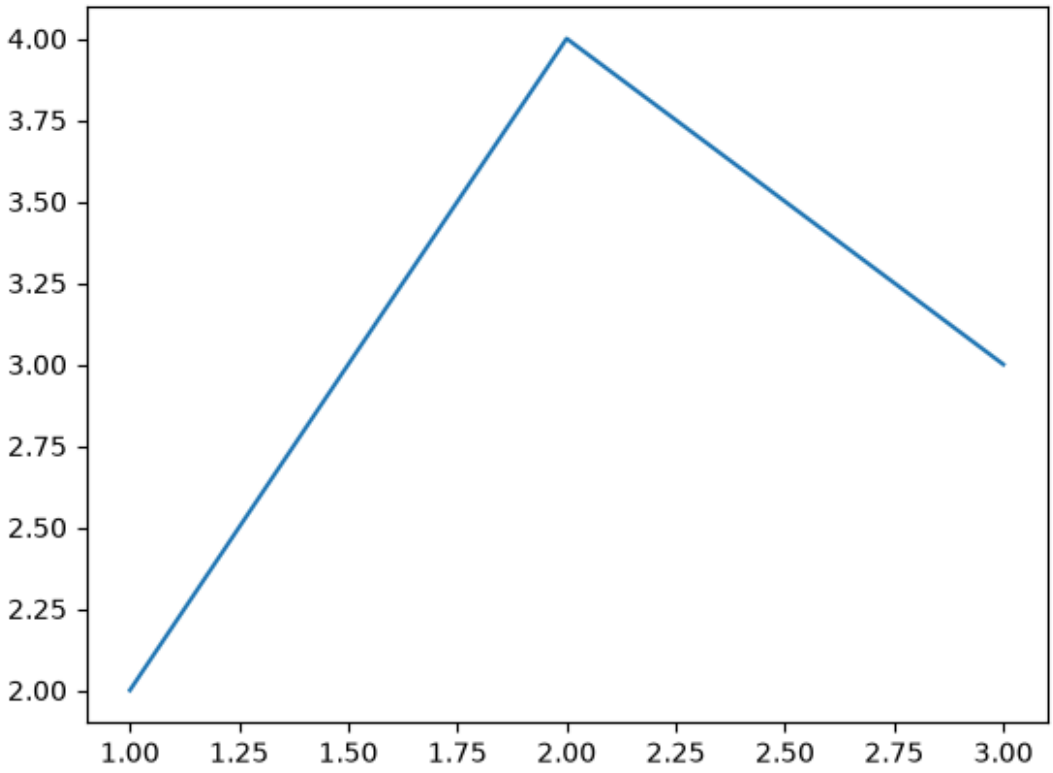
- ライブラリ＝他の人が作った便利な道具の詰め合わせ。グラフを描く道具は **matplotlib**
- `import ○○ as △△`：「○○ を △△ という短い名前で使えるようにする」

matplotlib とは

- **matplotlib**（マットプロットリブ）は、グラフを描くための**ライブラリ**。`import` してから使う
- `import matplotlib.pyplot as plt` は「matplotlib の中の **pyplot**（グラフを描く道具をまとめた部分）を `plt` という短い名前で使えるようにする」という意味
- matplotlib の道具は `plt.道具の名前(…)` の形で使う。例：`plt.plot(x, y)` は「x と y で折れ線を描く」、`plt.show()` は「描いたグラフを表示する」
- 折れ線グラフ・散布図・棒グラフ・ヒストグラム・円グラフなど、この授業のグラフはすべて matplotlib で描く
- Anaconda には最初から入っているので、インストールはいらない（日本語を表示する japanize_matplotlib は別。下で説明する）

```
import matplotlib.pyplot as plt

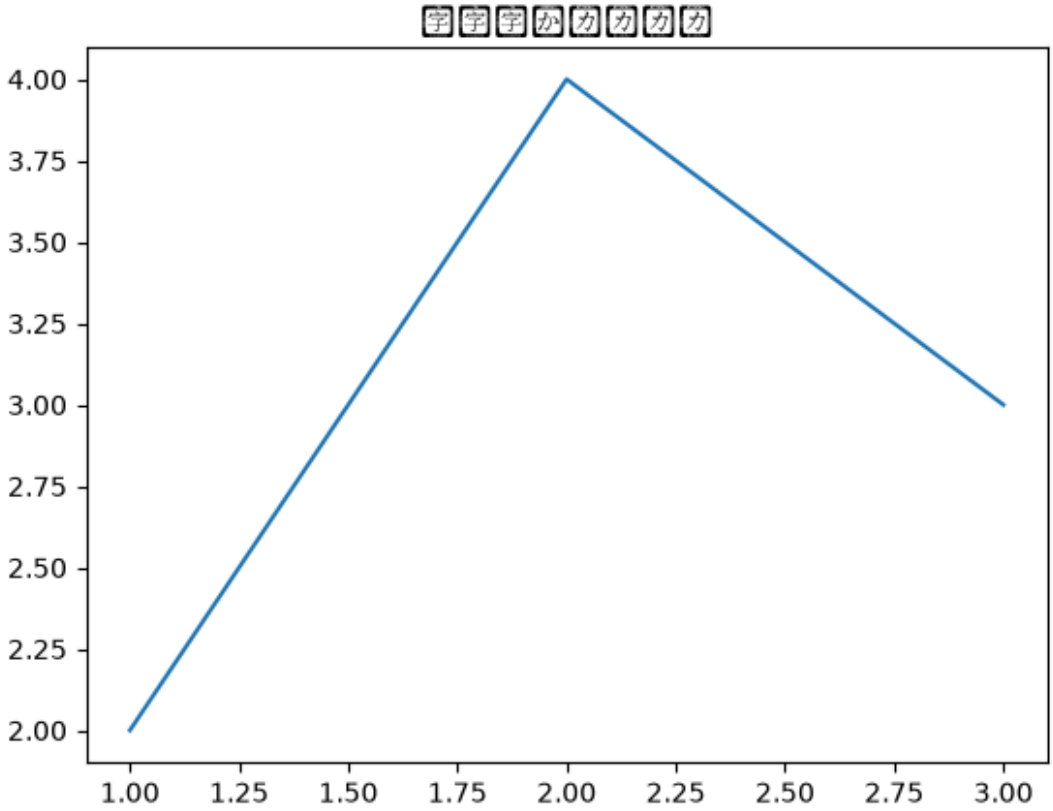
plt.plot([1, 2, 3], [2, 4, 3])
plt.show()
```



- matplotlib は、そのままでは**日本語が文字化けする**（□□□ のようになる）

```
plt.plot([1, 2, 3], [2, 4, 3])
plt.title('日本語のタイトル')
plt.show()
```

```
C:\Users\kumano\anaconda3\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyp
h 26085 (\N{CJK UNIFIED IDEOGRAPH-65E5}) missing from font(s) DejaVu Sans.
  fig.canvas.print_figure(bytes_io, **kw)
C:\Users\kumano\anaconda3\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyp
h 26412 (\N{CJK UNIFIED IDEOGRAPH-672C}) missing from font(s) DejaVu Sans.
  fig.canvas.print_figure(bytes_io, **kw)
C:\Users\kumano\anaconda3\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyp
h 35486 (\N{CJK UNIFIED IDEOGRAPH-8A9E}) missing from font(s) DejaVu Sans.
  fig.canvas.print_figure(bytes_io, **kw)
C:\Users\kumano\anaconda3\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyp
h 12398 (\N{HIRAGANA LETTER NO}) missing from font(s) DejaVu Sans.
  fig.canvas.print_figure(bytes_io, **kw)
C:\Users\kumano\anaconda3\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyp
h 12479 (\N{KATAKANA LETTER TA}) missing from font(s) DejaVu Sans.
  fig.canvas.print_figure(bytes_io, **kw)
C:\Users\kumano\anaconda3\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyp
h 12452 (\N{KATAKANA LETTER I}) missing from font(s) DejaVu Sans.
  fig.canvas.print_figure(bytes_io, **kw)
C:\Users\kumano\anaconda3\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyp
h 12488 (\N{KATAKANA LETTER TO}) missing from font(s) DejaVu Sans.
  fig.canvas.print_figure(bytes_io, **kw)
C:\Users\kumano\anaconda3\Lib\site-packages\IPython\core\pylabtools.py:170: UserWarning: Glyp
h 12523 (\N{KATAKANA LETTER RU}) missing from font(s) DejaVu Sans.
  fig.canvas.print_figure(bytes_io, **kw)
```



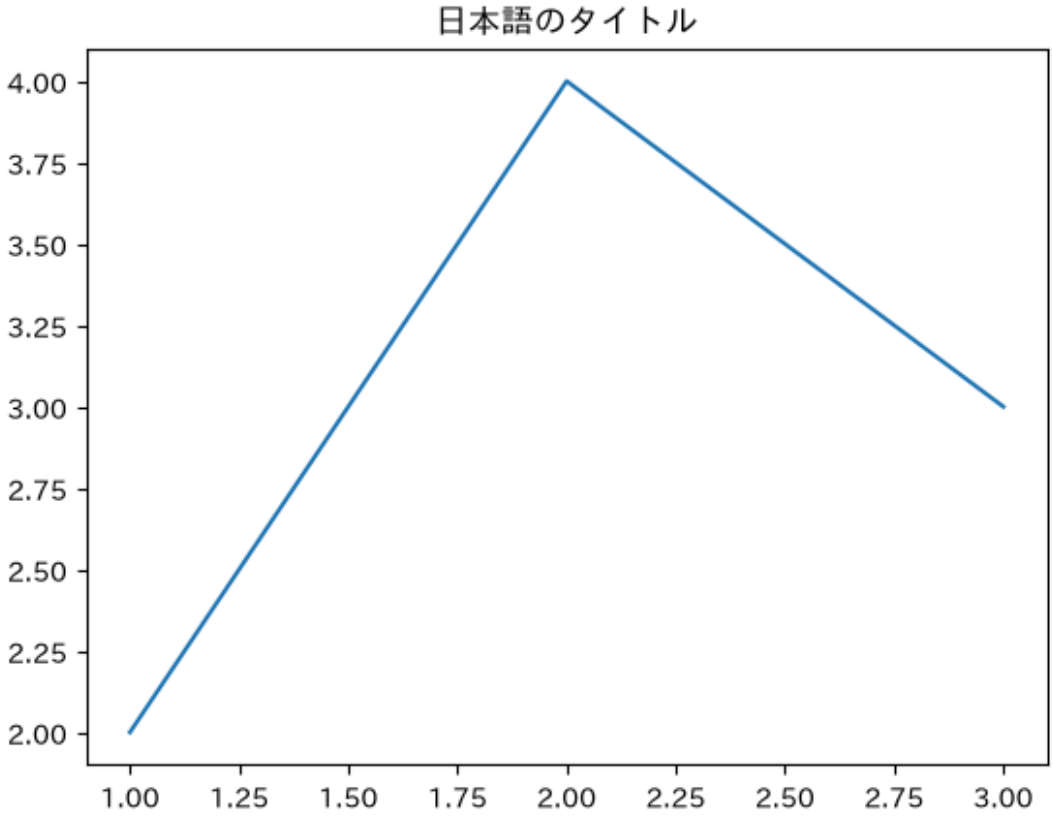
- グラフに**日本語**（タイトル・軸の名前など）を表示するには、`import japanize_matplotlib` を書く
- 初めて使うときは、インストールが必要（1回だけ実行すればよい。次からは import だけでよい）
- notebook のセルに次の1行を書いて実行する。先頭の `!` は「Python ではなく、パソコンへの命令として実行する」という印

```
!pip install japanize-matplotlib
```

  - Anaconda Prompt（Mac はターミナル）で `pip install japanize-matplotlib` と打っても同じ
  - `Successfully installed …` または `Requirement already satisfied`（すでに入っている）と出れば成功
  - `No module named 'distutils'` と出た場合は、`!pip install setuptools` も実行する
  - インストールしたのに import でエラーになるときは、メニューの Kernel → Restart Kernel を選んでから、もう一度実行する

```
import japanize_matplotlib

plt.plot([1, 2, 3], [2, 4, 3])
plt.title('日本語のタイトル')
plt.show()
```



C-2 関数と引数

- `print(...)` や `len(...)` は**関数**。「**名前(引数)**」の形で使う。`( )` の中に渡す値を**引数**という
- `max` 、`min` で最大値・最小値、`sum` で合計
- 平均は「合計 ÷ 個数」なので、`sum(score) / len(score)` で求められる
- セルの最後の行に書いた値は、`print` を使わなくても表示される。何行か表示したいときは `print` を使う

```
score = [65, 80, 72, 90, 58]

print(max(score))
print(min(score))
print(sum(score))
print(sum(score) / len(score))
```

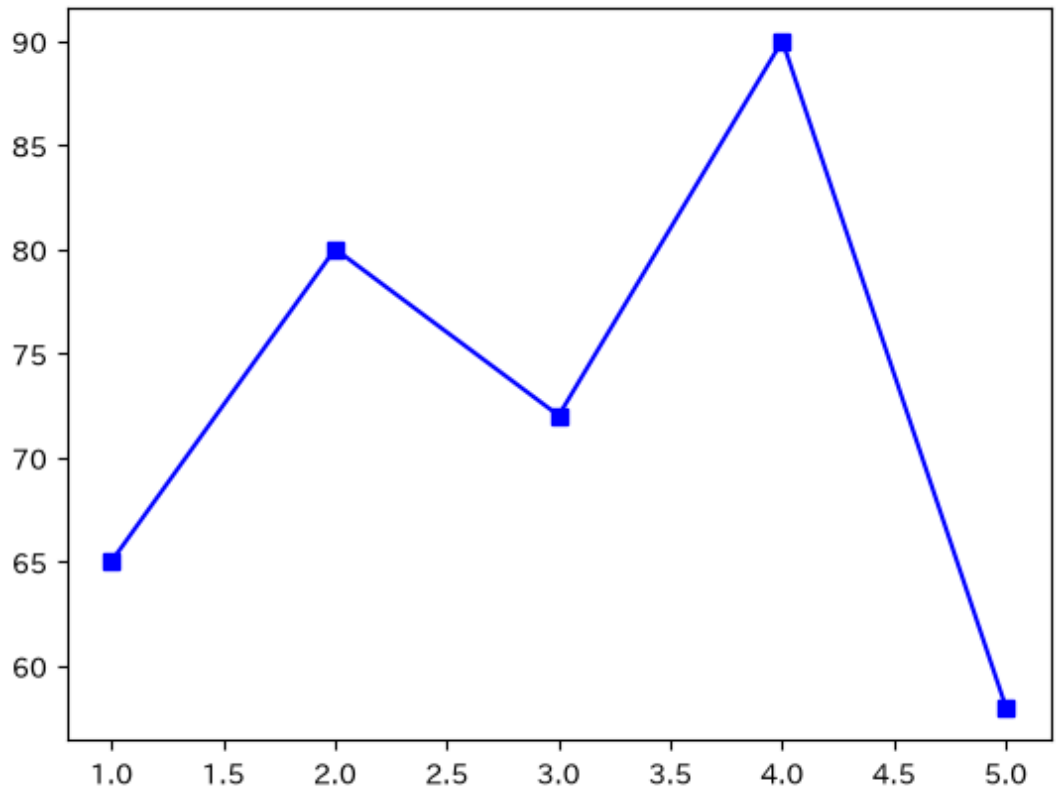
```
90
58
365
73.0
```

- 引数には2種類ある
  - **順番で意味が決まる引数**：`plt.plot(x, y)` （1番目が x、2番目が y）
  - **名前で指定する引数**（キーワード引数）：`color='b'` 、`marker='s'` のように「名前=値」で書く

色 `color`		**マーカー** `marker`	
`'r'`	赤	`'o'`	丸
`'g'`	緑	`'s'`	四角
`'b'`	青	`'^'`	三角

```
x = [1, 2, 3, 4, 5]

plt.plot(x, score, color='b', marker='s')
plt.show()
```



- `plt.` で始まるもの（`plot`、`title`、`show` …）は、「plt という道具箱の中の関数」

C-3 エラーの読み方

- エラーは怖くない。**メッセージの最後の行**を見る
- よく出る3つ

エラー	意味	よくある原因
SyntaxError	文法の誤り	括弧・クォーテーションの閉じ忘れ、全角文字（A-1）
NameError	知らない名前	つづり間違い、大文字・小文字の違い（A-2）
ValueError	値がおかしい	x と y のリストの要素数が違う（B-2）

以下の3つのセルは、**わざとエラーになるように書いてある**。実行して、最後の行を読んでみよう。

> 括弧の閉じ忘れは、Jupyter では `SyntaxError` ではなく `incomplete input`（入力が終わっていない）と表示されることがある。意味は同じ。

```
# SyntaxError：クォーテーションの閉じ忘れ
print('こんにちは)
```

```
  Cell In[24], line 2
    print('こんにちは)
          ^
SyntaxError: unterminated string literal (detected at line 2)
```

```
# NameError：大文字・小文字の違い（定義したのは temp）
print(Temp)
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[25], line 2
      1 # NameError：大文字・小文字の違い（定義したのは temp）
----> 2 print(Temp)

NameError: name 'Temp' is not defined
```

```
# ValueError：x は 5 個、y は 4 個
plt.plot([1, 2, 3, 4, 5], [65, 80, 72, 90])
plt.show()
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[26], line 2
      1 # ValueError：x は 5 個、y は 4 個
----> 2 plt.plot([1, 2, 3, 4, 5], [65, 80, 72, 90])
      3 plt.show()

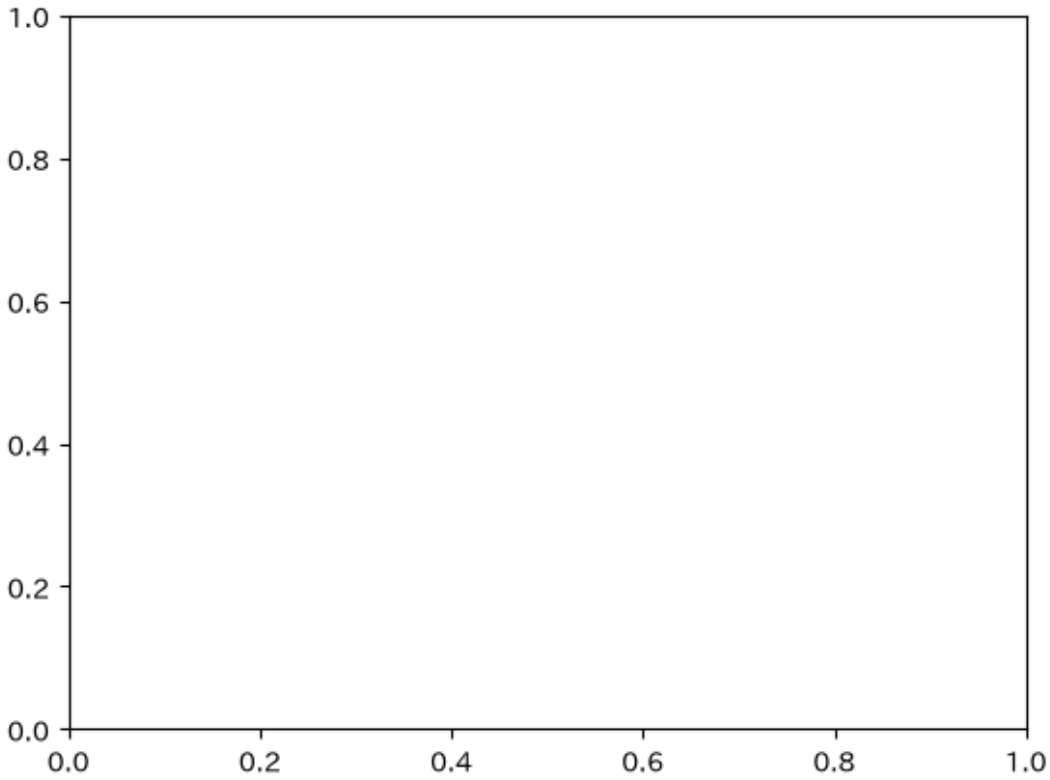
File ~\anaconda3\Lib\site-packages\matplotlib\pyplot.py:4043, in plot(scalex, scaley, data, *
args, **kwargs)
   4035 @_copy_docstring_and_deprecators(Axes.plot)
   4036 def plot(
   4037     *args: float | ArrayLike | str,
   (...)   4041     **kwargs,
   4042 ) -> list[Line2D]:
-> 4043     return gca().plot(
   4044         *args,
   4045         scalex=scalex,
   4046         scaley=scaley,
   4047         **({"data": data} if data is not None else {}),
   4048         **kwargs,
   4049     )

File ~\anaconda3\Lib\site-packages\matplotlib\axes\_axes.py:1792, in Axes.plot(self, scalex,
scaley, data, *args, **kwargs)
   1549 """
   1550 Plot y versus x as lines and/or markers.
   1551
   (...)   1789 (``'green'``) or hex strings (``'#008000'``).
   1790 """
   1791 kwargs = cbook.normalize_kwargs(kwargs, mlines.Line2D)
-> 1792 lines = [*self._get_lines(self, *args, data=data, **kwargs)]
   1793 for line in lines:
   1794     self.add_line(line)

File ~\anaconda3\Lib\site-packages\matplotlib\axes\_base.py:331, in _process_plot_var_args.__
call__(self, axes, data, return_kwargs, *args, **kwargs)
    329     this += args[0],
    330     args = args[1:]
--> 331 yield from self._plot_args(
    332     axes, this, kwargs, ambiguous_fmt_datakey=ambiguous_fmt_datakey,
    333     return_kwargs=return_kwargs
    334 )

File ~\anaconda3\Lib\site-packages\matplotlib\axes\_base.py:509, in _process_plot_var_args._p
lot_args(self, axes, tup, kwargs, return_kwargs, ambiguous_fmt_datakey)
    506     axes.yaxis.update_units(y)
    508 if x.shape[0] != y.shape[0]:
--> 509     raise ValueError(f"x and y must have same first dimension, but "
    510                      f"have shapes {x.shape} and {y.shape}")
    511 if x.ndim > 2 or y.ndim > 2:
    512     raise ValueError(f"x and y can be no greater than 2D, but have "
    513                      f"shapes {x.shape} and {y.shape}")

ValueError: x and y must have same first dimension, but have shapes (5,) and (4,)
```



- 直し方：**1つ直したら、もう一度実行する**（エラーは1つずつしか出ないことが多い）
- ValueError のときは、`len()` で x と y の要素数を確かめるとよい

D. グラフを描く

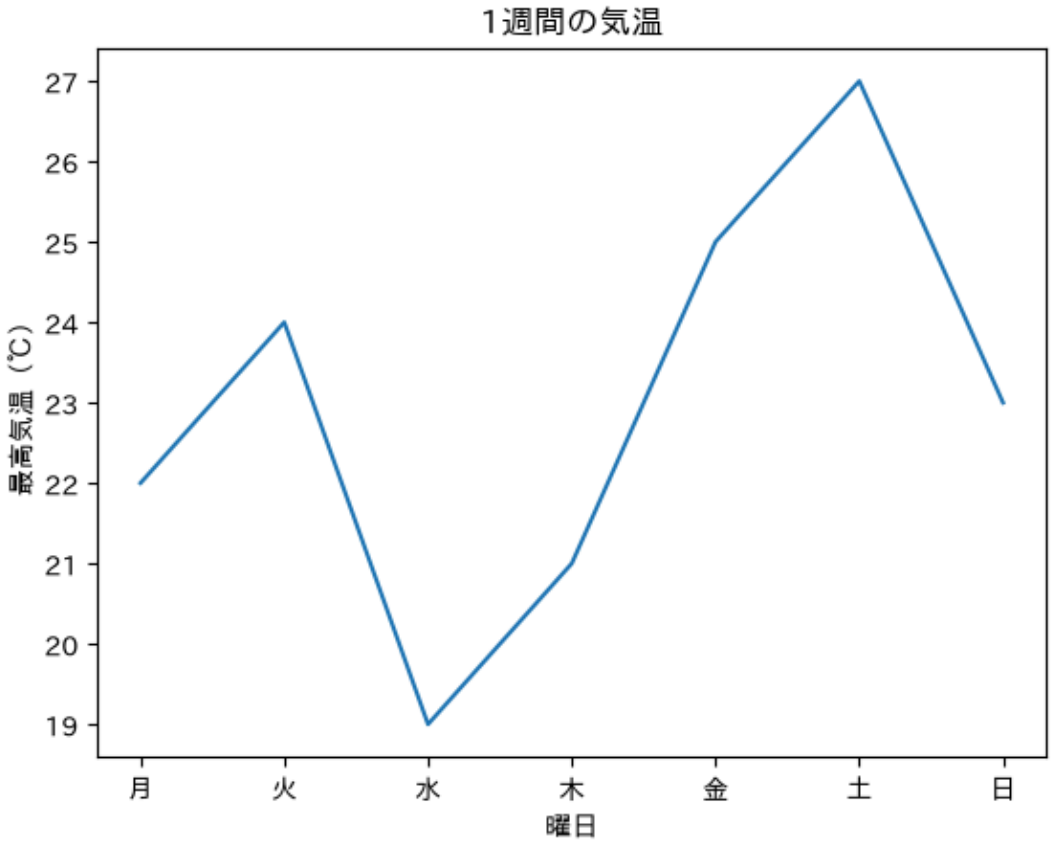
ここまでの内容を組み合わせて、1週間の気温の折れ線グラフを描く。各行が、どこで学んだものかを確認しよう。

グラフの部品	関数
線	`plt.plot(x, y)`
タイトル	`plt.title('...')`
x 軸ラベル	`plt.xlabel('...')`
y 軸ラベル	`plt.ylabel('...')`
表示	`plt.show()`

```
import matplotlib.pyplot as plt   # C-1 import
import japanize_matplotlib        # C-1 日本語表示

days = ['月', '火', '水', '木', '金', '土', '日']   # B-1 リスト
temp = [22, 24, 19, 21, 25, 27, 23]

plt.plot(days, temp)              # C-2 関数（x, y の順）
plt.title('1週間の気温')          # A-4 文字列
plt.xlabel('曜日')
plt.ylabel('最高気温（℃）')
plt.show()                        # 表示
```



まとめ

キーワード	一言で
`print`	画面に表示する
コメント `#`	実行されないメモ
変数・代入 `=`	右の値を、左の名前に入れる
演算子	`+ - * / **`
文字列	`'...'` または `"..."` で囲む。`+` で連結
リスト	`[ ]` で値をまとめる。**番号は 0 から**
`import`	ライブラリ（道具）を読み込む
関数・引数	`名前(引数)` 。キーワード引数は `名前=値`
エラー	**最後の行**を読む

データを用意する（リスト）→ 道具を読み込む（import）→ 描く（関数）

第 3 講：折れ線グラフを整える

第 2 講 D で描いた折れ線グラフに、マーカー・色・軸の範囲・タイトル・凡例などの**部品**を足して見やすくする（E）。F では、凡例の場所・線の種類・目盛りなどのオプションと、文章を書く Markdown セルを学ぶ。ここで学ぶ部品の書き方は、第 4 講以降の散布図・棒グラフでもほぼそのまま使う。

E. 折れ線グラフを整える

第 2 講 D で描いたグラフに、**見やすくするための指定**を足していく。

E-0 完成形

このグラフを目指す。

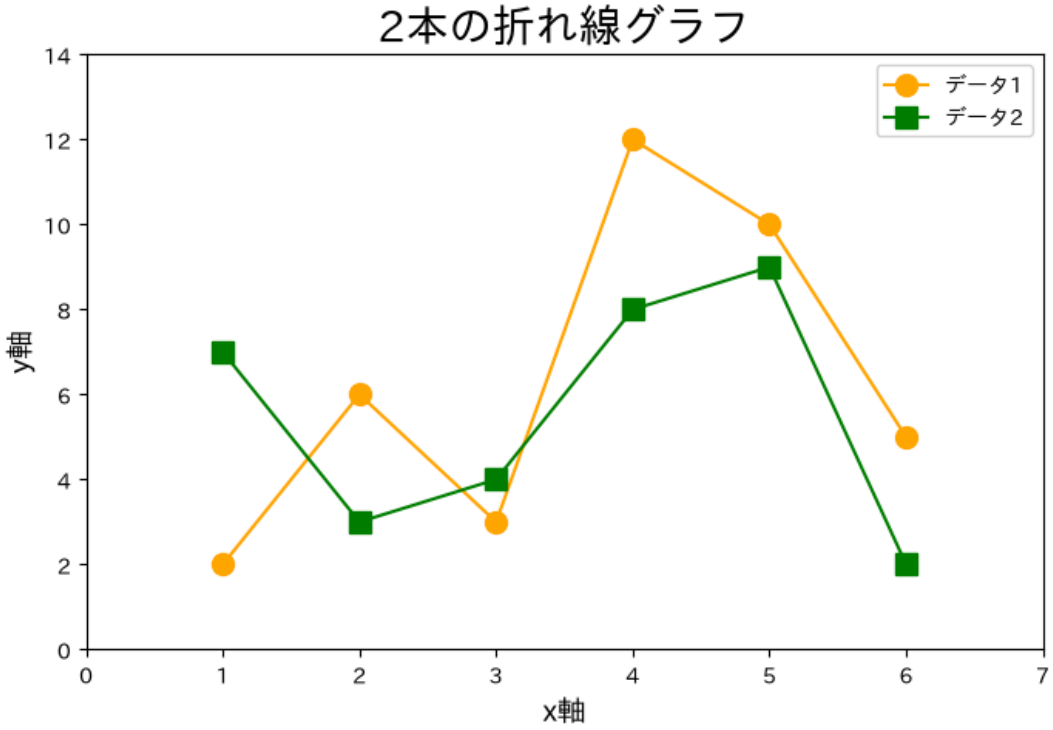
1. **全部コピーして**自分の notebook のセルに貼り付け、実行する（`Shift + Enter`）
2. まずは**数値や文字を変えて**、グラフのどこが変わるかを確かめる
3. そのあと1行ずつ分解して、各部品（[0]〜[7]）の働きを学ぶ（E-1〜E-4）

最初の行 `plt.figure(figsize=(8, 5))`（[0]）は**グラフの大きさを決める基本設定**で、この講のグラフはすべてこの行から書き始める。

```
import matplotlib.pyplot as plt
import japanize_matplotlib

x = [1, 2, 3, 4, 5, 6]
y1 = [2, 6, 3, 12, 10, 5]
y2 = [7, 3, 4, 8, 9, 2]

plt.figure(figsize=(8, 5))  # [0] グラフの大きさ
plt.plot(x, y1, marker='o', markersize=10, color='orange', label='データ1')  # [1] 線 [2] マーカー [3] 色 [7] 名前
plt.plot(x, y2, marker='s', markersize=10, color='g', label='データ2')  # [7] 2本目の線
plt.xlim(0, 7)  # [4] 横軸の範囲
plt.ylim(0, 14)  # [4] 縦軸の範囲
plt.title('2本の折れ線グラフ', fontsize=20)  # [5] タイトル [6] 文字の大きさ
plt.xlabel('x軸', fontsize=14)  # [5] 横軸の名前 [6]
plt.ylabel('y軸', fontsize=14)  # [5] 縦軸の名前 [6]
plt.legend()  # [7] 凡例
plt.show()
```



変えてみよう（1か所変えたら、もう一度実行する）

- 色：`'orange'` → `'r'`、`'skyblue'`、`'pink'`
- マーカー：`'o'` → `'^'`、`'*'`、`'D'`。大きさ `markersize=10` → `20`
- グラフの大きさ：`figsize=(8, 5)` → `(10, 3)`（横長）
- 軸の範囲：`plt.ylim(0, 14)` → `plt.ylim(0, 30)`
- 文字：タイトルを自分の好きな言葉に、`fontsize=20` → `30`
- データ：`y1` の数字を変える（x と同じ6個のまま）
- ある行を消す（または行頭に `#` を付ける）と、その部品が無いグラフになる。どの行が何をしているかを確かめよう

番号	部品	書き方	短い書き方	学ぶところ
[0]	グラフの大きさ（基本設定）	`plt.figure(figsize=(8, 5))`		E-0
[1]	線	`plt.plot(x, y)`		E の最初
[2]	マーカーの種類・サイズ	`marker='o'`、`markersize=10`	`ms=10`	E-1
[3]	色	`color='orange'`	`c='orange'`	E-1
[4]	軸の範囲	`plt.xlim(0, 7)`、`plt.ylim(0, 14)`		E-2
[5]	タイトル・軸の名前	`plt.title`、`plt.xlabel`、`plt.ylabel`		D・E-3
[6]	文字の大きさ	`fontsize=20`		E-3
[7]	2本目の線と凡例（仕上げ）	`plt.plot` を2回、`label='...'`、`plt.legend()`		E-4

- 全部覚えなくてよい。この表は**部品の地図**として、グラフを描くときに見返す

[0] グラフの大きさ（基本設定）：`plt.figure(figsize=(横, 縦))`

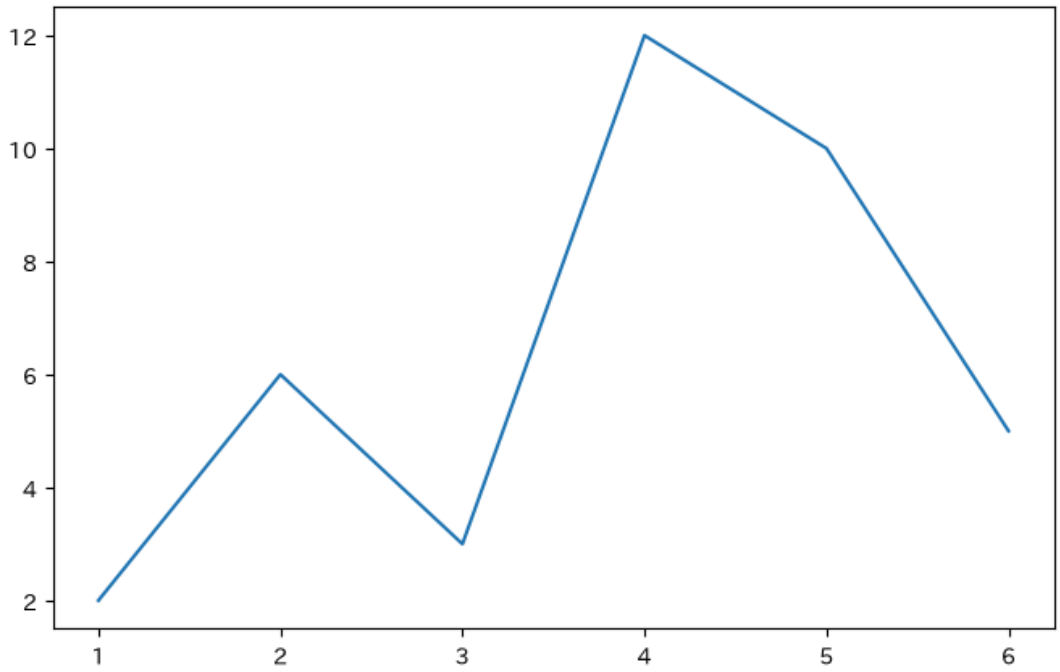
- 単位はインチ（1インチ＝約2.5cm）。何も指定しないと `(6.4, 4.8)`
- **`plt.plot` より前に**書く（「これから描く紙の大きさ」を先に決める）
- 横長にすると、時間の流れ（日ごとの気温など）が見やすくなる。例：`figsize=(10, 3)`

ここから[1]の線だけのグラフに戻って、1つずつ足していく。

```
import matplotlib.pyplot as plt
import japanize_matplotlib

x = [1, 2, 3, 4, 5, 6]
y = [2, 6, 3, 12, 10, 5]

plt.figure(figsize=(8, 5))
plt.plot(x, y)
plt.show()
```

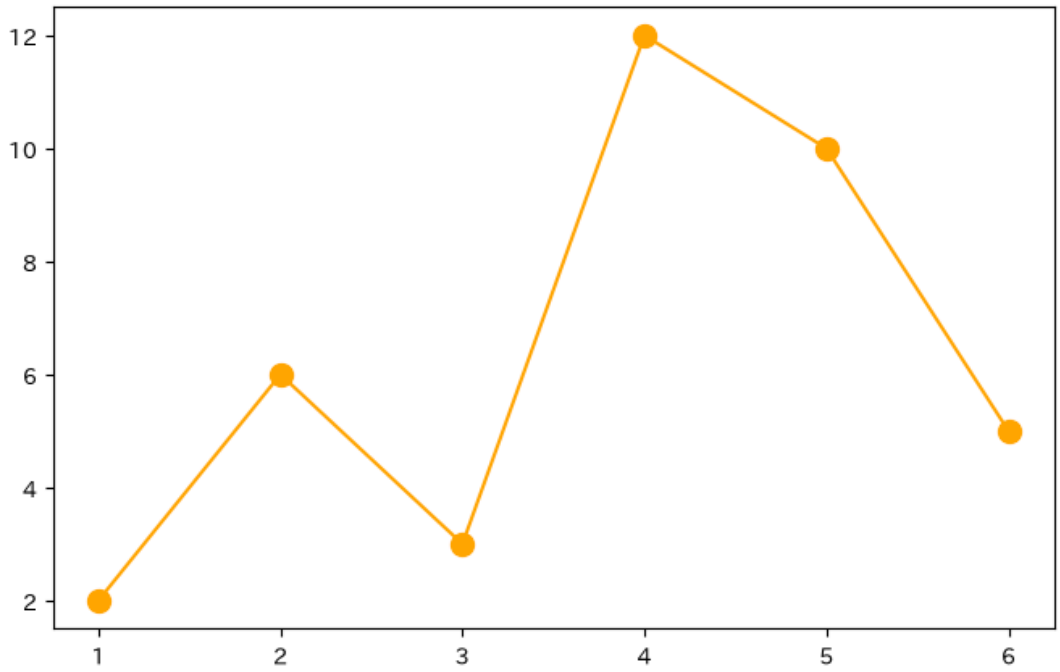


E-1 マーカーの種類・サイズ・色

- `plt.plot` に**キーワード引数**（C-2）を足す

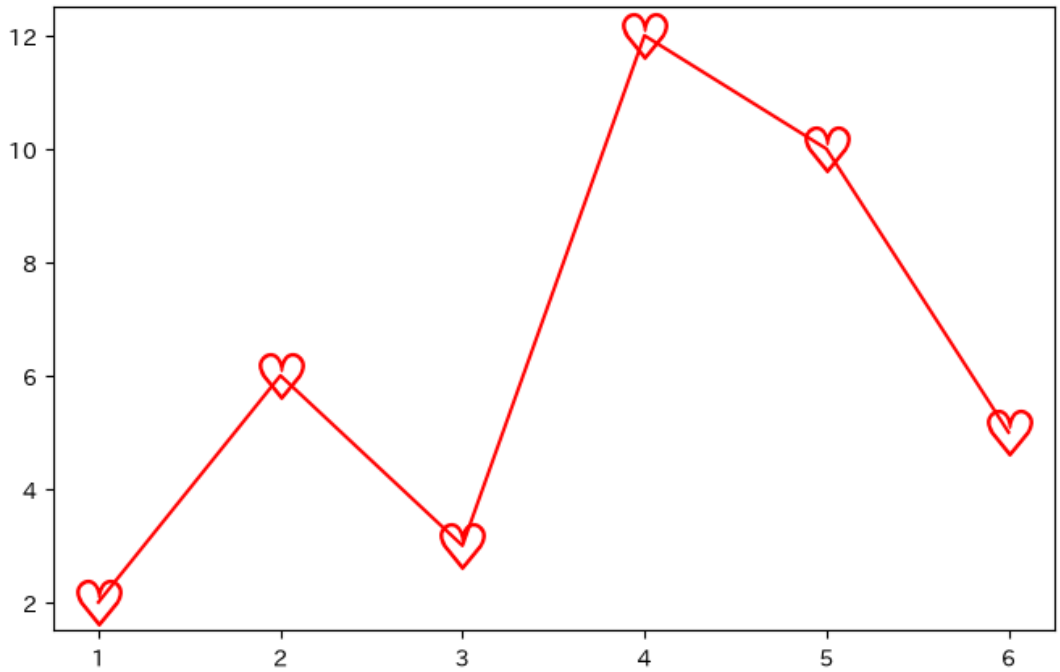
指定	書き方	例
マーカーの種類	`marker=`	`'o'` 丸、`'s'` 四角、`'^'` 三角、`'*'` 星、`'D'` ひし形
マーカーのサイズ	`markersize=`	`10` 、`15`（数値。大きいほど大きい）
色	`color=`	`'r'` 赤、`'g'` 緑、`'b'` 青、`'orange'` 、`'tan'` など色の名前も使える

```
plt.figure(figsize=(8, 5))
plt.plot(x, y, marker='o', markersize=10, color='orange')
plt.show()
```



- 記号をマーカーにすることもできる。`$` で囲んだ部分は数式の書き方（TeX）で解釈される
  - ハート `'$\\heartsuit$'`、スペード `'$\\spadesuit$'`、クラブ `'$\\clubsuit$'`、ダイヤ `'$\\diamondsuit$'`
  - `\` は Windows のキーボードでは `¥` キーで入力する。2つ重ねて書くことに注意

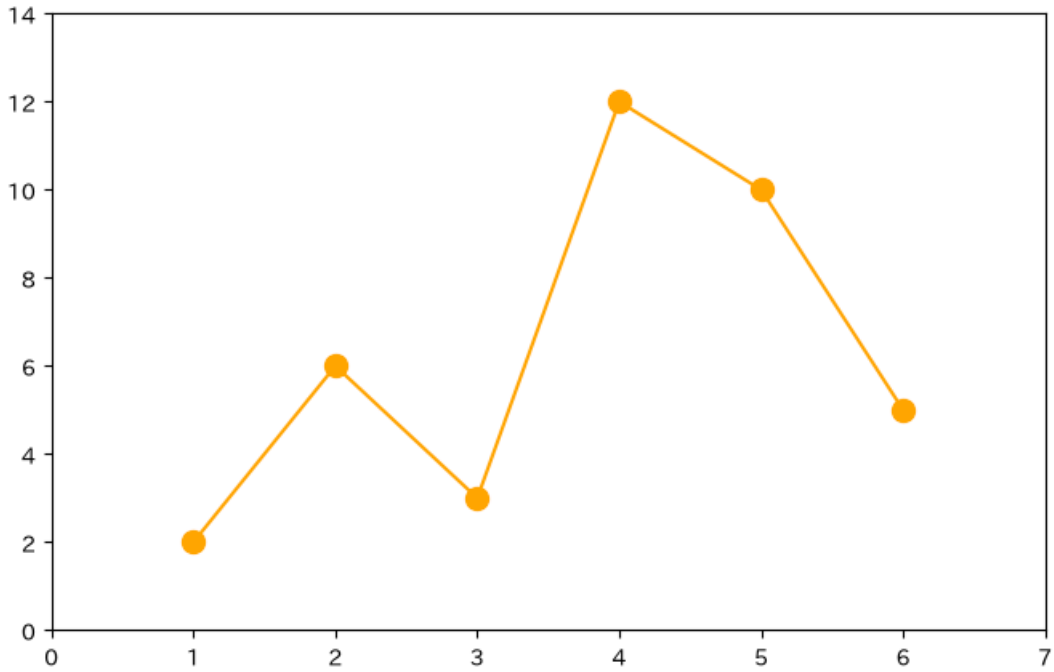
```
plt.figure(figsize=(8, 5))
plt.plot(x, y, marker='$\\heartsuit$', markersize=20, color='r')
plt.show()
```



E-2 軸の範囲

- 何も指定しないと、軸の範囲はデータに合わせて自動で決まる
- `plt.xlim(最小, 最大)` 、`plt.ylim(最小, 最大)` で範囲を決められる
- 例：y 軸を 0 から始めると、値の大小を正しく比べやすい

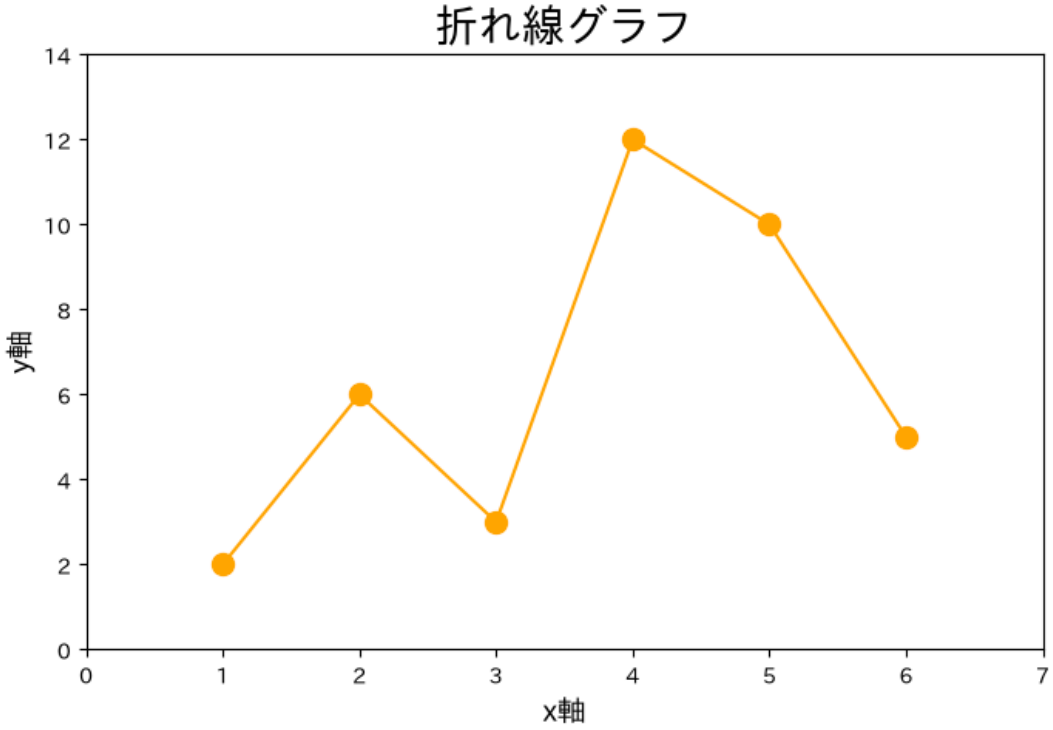
```
plt.figure(figsize=(8, 5))
plt.plot(x, y, marker='o', markersize=10, color='orange')
plt.xlim(0, 7)
plt.ylim(0, 14)
plt.show()
```



E-3 タイトル・軸の名前とフォントサイズ

- D で使った `plt.title` 、`plt.xlabel` 、`plt.ylabel` に、`fontsize=` を足すと文字の大きさを変えられる

```
plt.figure(figsize=(8, 5))
plt.plot(x, y, marker='o', markersize=10, color='orange')
plt.xlim(0, 7)
plt.ylim(0, 14)
plt.title('折れ線グラフ', fontsize=20)
plt.xlabel('x軸', fontsize=14)
plt.ylabel('y軸', fontsize=14)
plt.show()
```



- 上から順に「描く → 範囲 → 文字」と並べたが、`plt.show()` より前なら順番は入れ替えてもよい

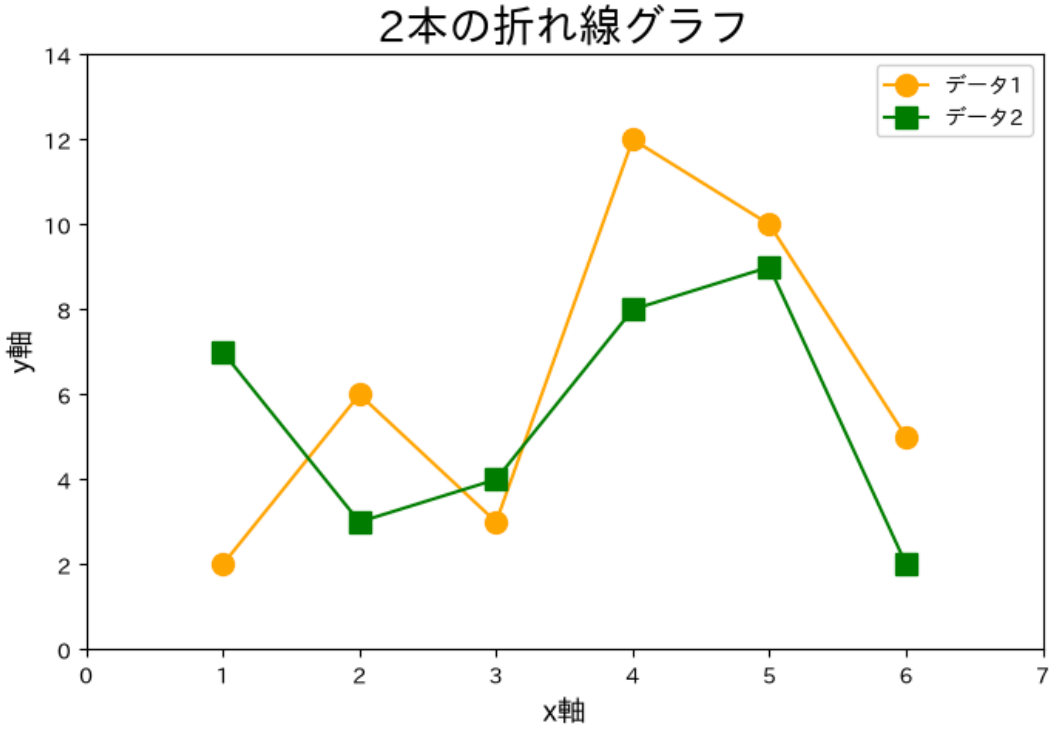
E-4 2本の線を重ねる

1本の線のグラフに、2本目の線と凡例（[7]）を足して仕上げる（E-0 の完成形になる）。

- `plt.plot` を**2回書く**と、同じグラフに2本の線が重なる
- `label=` で線に名前を付け、`plt.legend()` で**凡例**（どの線が何か）を表示する
- 色を指定しないと、1本目は青、2本目はオレンジのように自動で色分けされる

```
y1 = [2, 6, 3, 12, 10, 5]
y2 = [7, 3, 4, 8, 9, 2]

plt.figure(figsize=(8, 5))
plt.plot(x, y1, marker='o', markersize=10, color='orange', label='データ1')    # label を追加
plt.plot(x, y2, marker='s', markersize=10, color='g', label='データ2')         # 2本目
plt.xlim(0, 7)
plt.ylim(0, 14)
plt.title('2本の折れ線グラフ', fontsize=20)
plt.xlabel('x軸', fontsize=14)
plt.ylabel('y軸', fontsize=14)
plt.legend()                                    # 凡例
plt.show()
```



F. グラフのオプション

必要に応じて読み、自分のグラフで試してみよう。

- x, y1, y2 は E-4 で作ったものを使う。notebook を開き直したときは、E の最初のセルと E-4 のセルを先に実行する

F-1 凡例の場所・枠

関数には、ここまで使ったもの以外にも**いろいろなオプション（キーワード引数）**がある。F-1〜F-3 でよく使うものを紹介する。

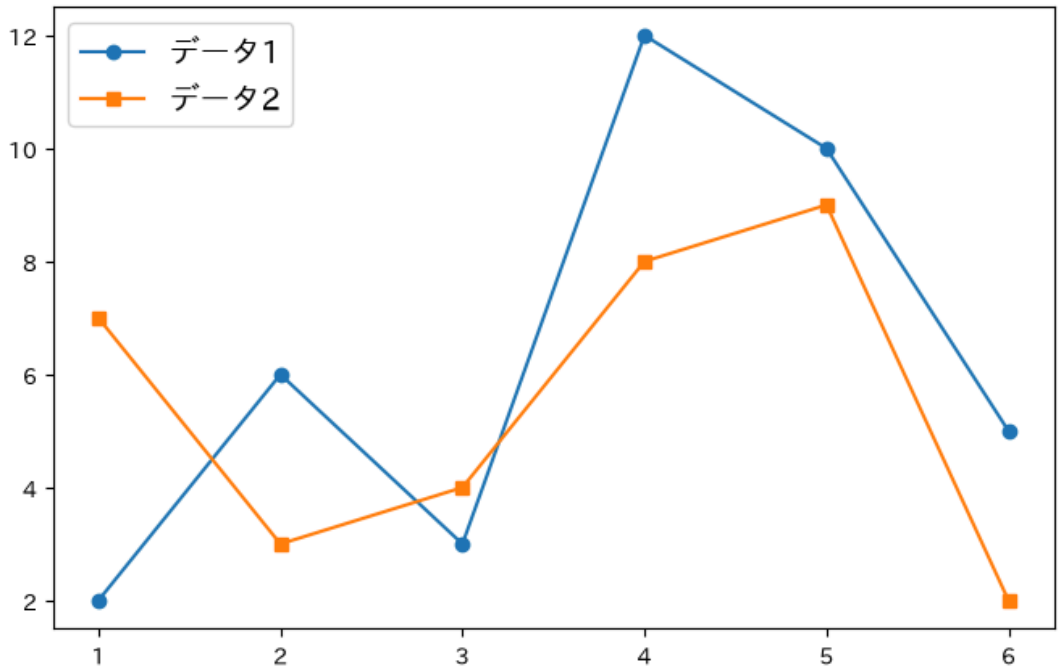
凡例の場所：`plt.legend(loc='...')`

- 何も指定しないと `'best'`（線と重なりにくい場所を自動で選ぶ）。それでも線に重なるときは、場所を指定する

`loc=`	場所
`'upper right'`	右上
`'upper left'`	左上
`'lower right'`	右下
`'lower left'`	左下
`'center'`	中央

- 凡例の文字の大きさも `fontsize=` で変えられる（E-3 と同じ）

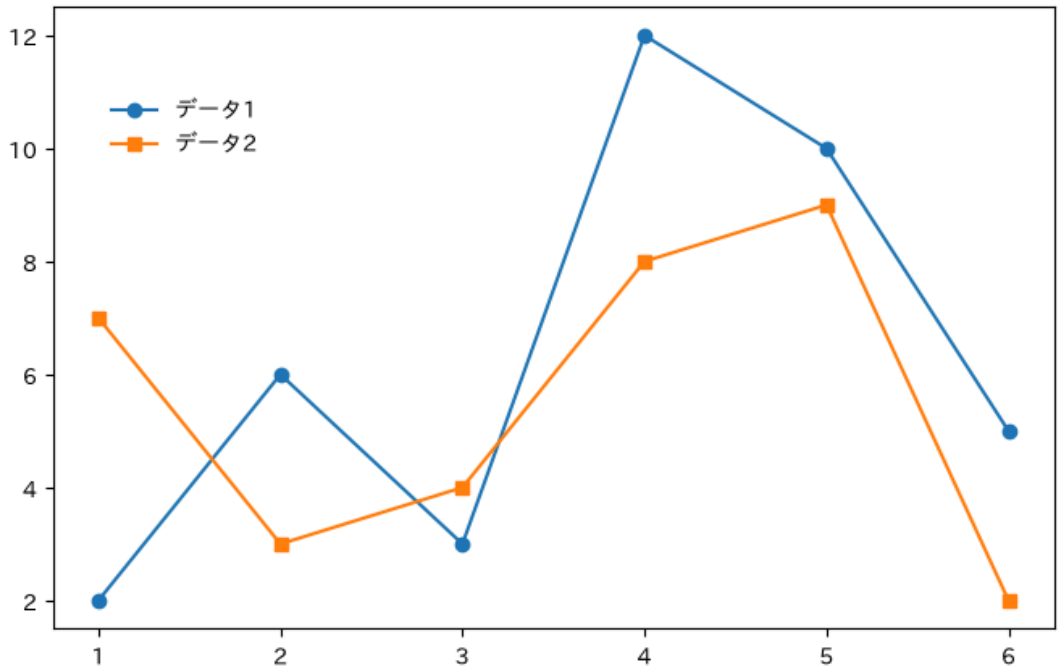
```
plt.figure(figsize=(8, 5))
plt.plot(x, y1, marker='o', label='データ1')
plt.plot(x, y2, marker='s', label='データ2')
plt.legend(loc='upper left', fontsize=14)
plt.show()
```



凡例の場所を数値で指定する：`loc=(横, 縦)`

- グラフの枠の**左下を `(0, 0)`、右上を `(1, 1)`** として、凡例の**左下の角**を置く位置を割合で指定する
- 例：`loc=(0.05, 0.75)` は「横は左端から少し右、縦は上から4分の1」のあたり。線と重なる場合は、数値を少しずつ変えて、空いている場所を探す
- `frameon=False` を足すと、凡例の**枠が消える**（すっきり見える）

```
plt.figure(figsize=(8, 5))
plt.plot(x, y1, marker='o', label='データ1')
plt.plot(x, y2, marker='s', label='データ2')
plt.legend(loc=(0.05, 0.75), frameon=False)
plt.show()
```

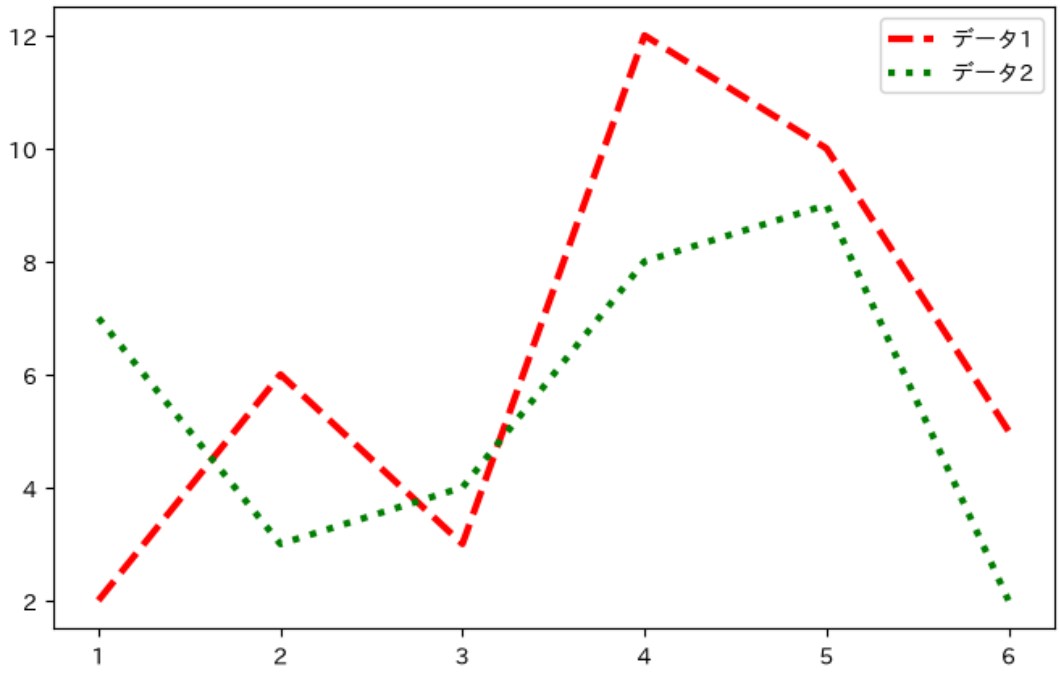


F-2 線の種類・太さと短い書き方

線の種類・太さ：`linestyle=`、`linewidth=`

指定	書き方	例
線の種類	`linestyle=`	`'-'` 実線、`'--'` 破線、`':'` 点線、`'-.'` 一点鎖線
線の太さ	`linewidth=`	`1`、`3`（数値。大きいほど太い）

```
plt.figure(figsize=(8, 5))
plt.plot(x, y1, color='r', linestyle='--', linewidth=3, label='データ1')
plt.plot(x, y2, color='g', linestyle=':', linewidth=3, label='データ2')
plt.legend()
plt.show()
```



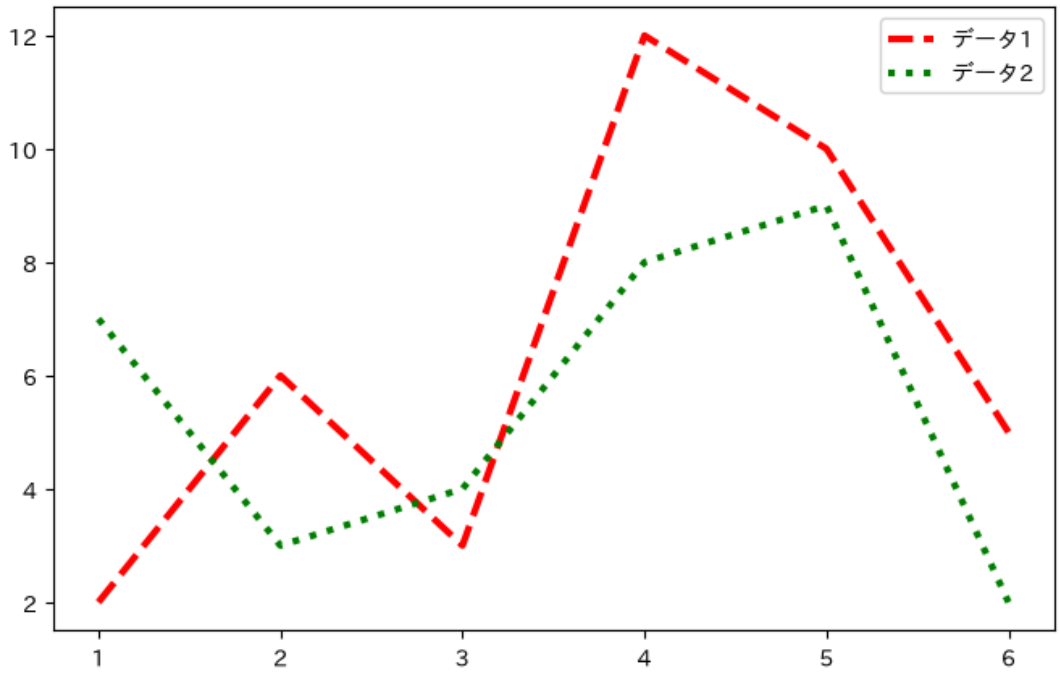
短い書き方：`c=`、`ms=`、`ls=`、`lw=`

- よく使う4つには**短い名前**がある。意味はまったく同じ

長い書き方	短い書き方
`color='r'`	`c='r'`
`markersize=10`	`ms=10`
`linestyle='--'`	`ls='--'`
`linewidth=3`	`lw=3`

- 下のセルは、上のセルと**同じグラフ**になる。インターネットや生成AIのコードには短い書き方もよく出てくるので、読めるようにしておこう

```
plt.figure(figsize=(8, 5))
plt.plot(x, y1, c='r', ls='--', lw=3, label='データ1')
plt.plot(x, y2, c='g', ls=':', lw=3, label='データ2')
plt.legend()
plt.show()
```



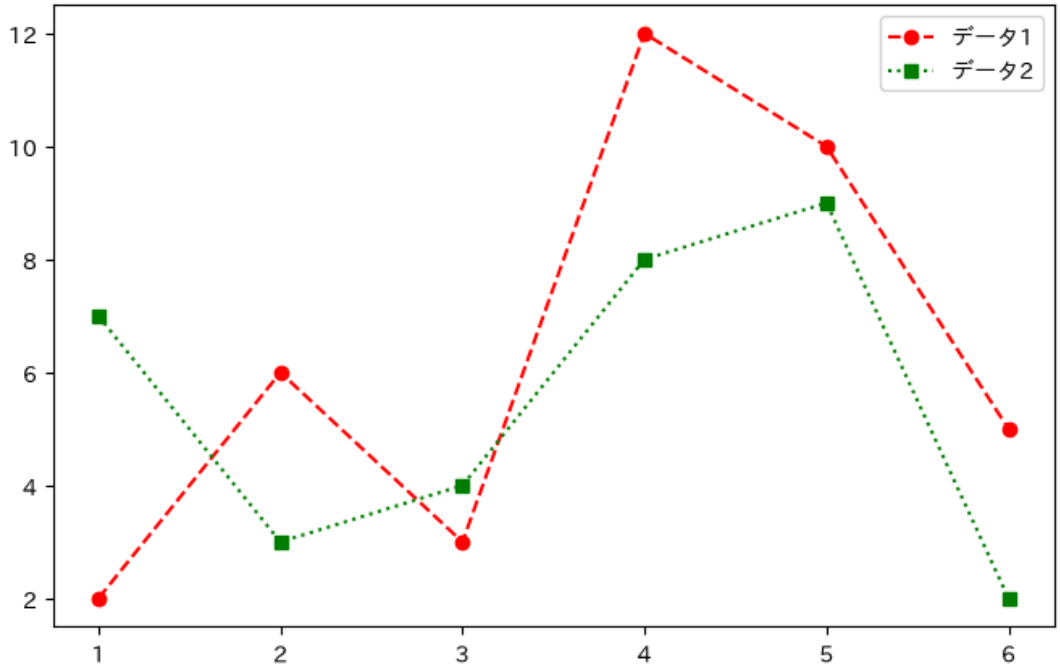
マーカー・線・色を1つの文字列で指定する

- `plt.plot(x, y, 'o--r')` のように、x, y の後ろに**文字列を1つ**書くと、マーカー・線の種類・色をまとめて指定できる
- 中身は「マーカー」「線の種類」「色」の記号を並べたもの（順番は入れ替えてもよい。使わないものは省略できる）

文字列	意味
`'o--r'`	丸マーカー＋破線＋赤
`'s:g'`	四角マーカー＋点線＋緑
`'^-b'`	三角マーカー＋実線＋青

文字列	意味
`'o'`	丸マーカーだけ（線なし。散布図のようになる）

```
plt.figure(figsize=(8, 5))
plt.plot(x, y1, 'o--r', label='データ1')
plt.plot(x, y2, 's:g', label='データ2')
plt.legend()
plt.show()
```



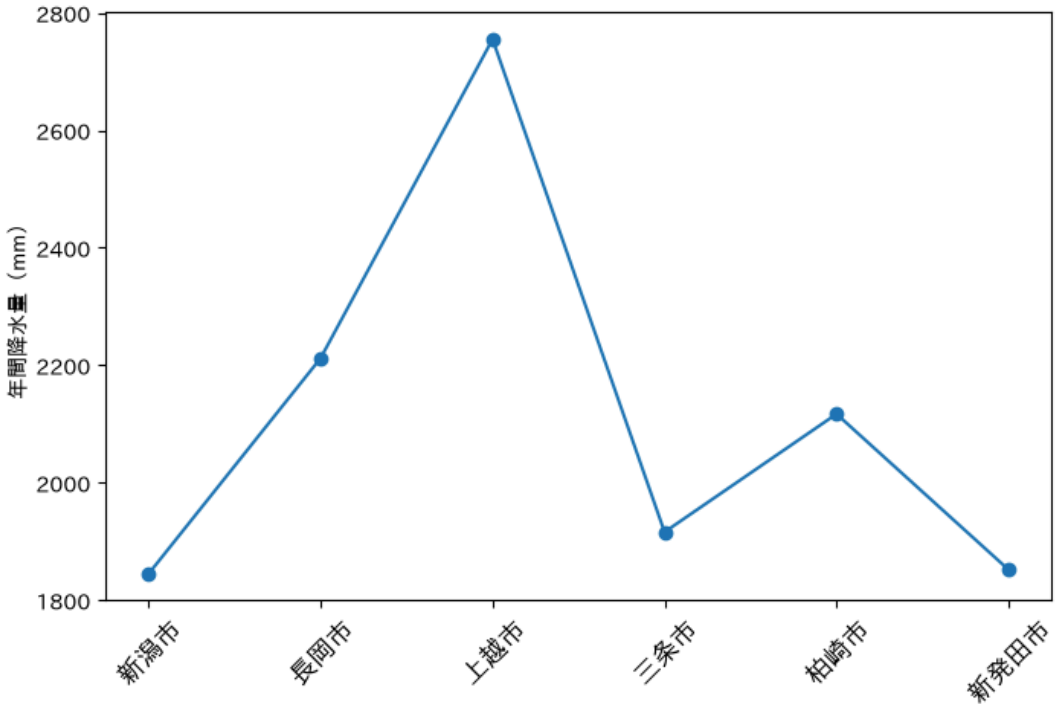
F-3 目盛り・透明度・目盛り線

目盛りの文字を斜めにする：`plt.xticks(rotation=角度)`

- x 軸の文字が長くて重なるときに使う。角度は度（`20`、`45`、`90` など）
- `fontsize=` で目盛りの文字の大きさも変えられる

```
city = ['新潟市', '長岡市', '上越市', '三条市', '柏崎市', '新発田市']
rain = [1845, 2212, 2755, 1916, 2117, 1852]

plt.figure(figsize=(8, 5))
plt.plot(city, rain, marker='o')
plt.xticks(rotation=45, fontsize=12)
plt.ylabel('年間降水量（mm）')
plt.show()
```

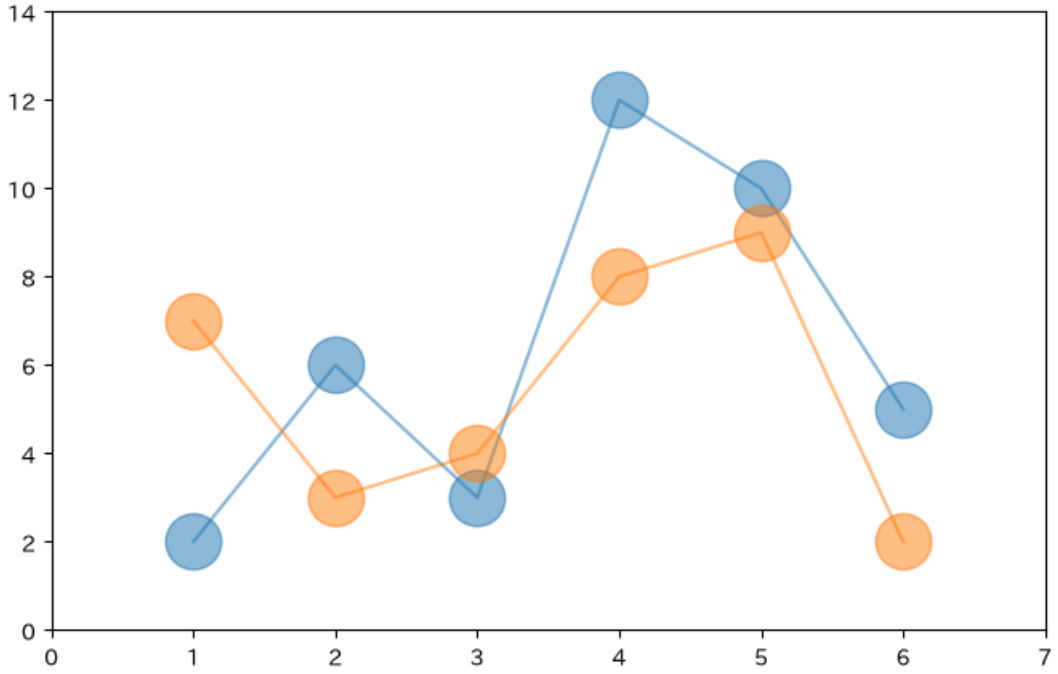


- 上のデータは、目盛りの説明用に作った例

透明度：`alpha=`

- `0`（完全に透明）～ `1`（不透明）の数値。重なった部分が透けて見える

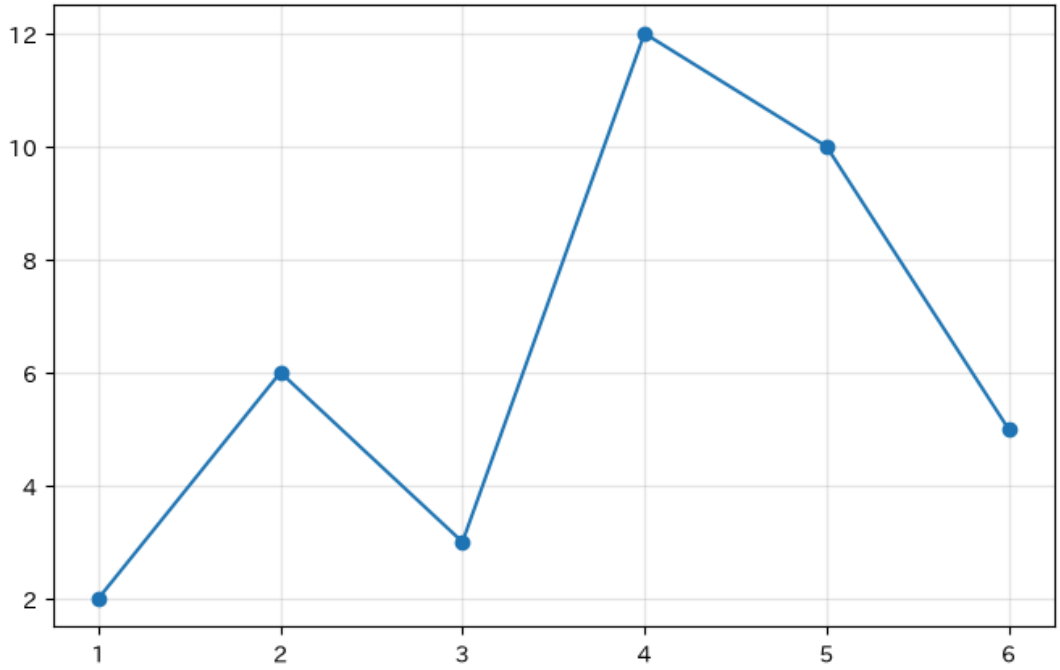
```
plt.figure(figsize=(8, 5))
plt.plot(x, y1, marker='o', markersize=25, alpha=0.5)
plt.plot(x, y2, marker='o', markersize=25, alpha=0.5)
plt.xlim(0, 7)
plt.ylim(0, 14)
plt.show()
```



目盛り線：`plt.grid()`

- グラフに方眼のような線を引く。値を読み取りやすくなる
- `alpha=` で薄くすると、グラフの邪魔になりにくい（透明度と組み合わせる）

```
plt.figure(figsize=(8, 5))
plt.plot(x, y1, marker='o')
plt.grid(alpha=0.3)
plt.show()
```



オプションの調べ方

- 関数名の後ろ（`plt.legend(` の `(` の後など）にカーソルを置いて **`Shift + Tab`** を押すと、使い方の説明が出る（英語）。Mac も同じ `Shift + Tab`
- 「matplotlib legend 位置」のように**検索**するのも早い。生成AIに「matplotlib で凡例を右下に出すには？」と聞いてもよい

F-4 Markdown セルで文章を書く

- セルの種類を **Markdown** にすると、プログラムではなく**文章**（見出し・箇条書き・太字・リンク）を書ける。グラフの説明やレポートのメモに使う
- 変え方：セルを選んで `Esc` → `M`（コードに戻すのは `Y`）。上のメニューの「Code」と書かれたプルダウンを「Markdown」にしてもよい
- `Shift + Enter` で実行すると、整った表示になる。**ダブルクリック**すると書き直せる
- 記号（`#`、`-`、`*` など）は**半角**で書き、後ろに**半角スペース**を1つ入れる

改行に注意

- Markdown では、**Enter で改行しただけでは改行されない**（表示すると前の行とつながる）
- 改行したいときは、**行の最後に半角スペースを2つ**入れてから Enter を押す。見た目では分からないので注意
- **空行を1行**入れると、段落が分かれる（行の間が少し空く）
- `<br>` と書いても改行になる

よく使う書き方

書きたいもの	書き方
見出し（大〜小）	`# 見出し` 、`## 見出し` 、`### 見出し` （`#` は6個まで）
箇条書き	`- 項目` （`+ 項目` 、`* 項目` でもよい）
箇条書きの中の箇条書き	行の先頭に半角スペース4つを入れて `- 項目`
番号付きの箇条書き	`1. 項目` （番号は自動で振られるので、全部 `1.` でもよい）
太字	`**文字**`
斜体	`*文字*` （日本語ではほとんど使わない）
取り消し線	`~~文字~~`
リンク	`[表示する文字](https://…)`
改行	行の最後に**半角スペース2つ**、または ` `
段落を分ける	空行を1行入れる

- `#` の数が多いほど小さい見出しになる。この notebook の見出しも同じ書き方
- コードセルの `#` は**コメント**（実行されない行）、Markdown セルの `#` は**見出し**。同じ記号でも意味が違う

下は Markdown セルの中身の例。自分の notebook の Markdown セルに貼り、`Shift + Enter` で表示してみよう。

```
# レポート
## レポート
### レポート
#### レポート

- その１
- **その２**
    - その２の中の項目
- ~~その３~~

1. 番号付き
1. 番号付き

この行の最後には半角スペースが2つある
だから、ここで改行される。
この行の最後には何もないので、
前の行とつながって表示される。

[Qiita へのリンク](https://qiita.com/)
[創生学部](https://create.niigata-u.ac.jp/)
```

線・数式・そのほか

書きたいもの	書き方
横線（区切り線）	1行に `---` だけを書く（前の行との間に空行を入れる）

書きたいもの	書き方
数式（文の中）	`$` で囲む：`$y = ax + b$`
数式（1行で中央に）	`$$` で囲む：`$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i$$`
引用	行の先頭に `>`
コード（文の中）	`` ` `` で囲む：`` `plt.plot(x, y)` ``
コード（何行か）	前後の行に ```` ``` ```` を書いて挟む（ ```` ```python ```` とすると色が付く）
画像	`![説明](画像のファイル名)`（notebook と同じ場所の画像）
記号をそのまま表示	前に `\` を付ける：`\*` 、`\#`（Mac は option + ¥ で `\`）
表示しないメモ（コメント）	`<!-- メモ -->`（`<!--` と `-->` の間は、表示されない）

- 数式の書き方は **LaTeX**（ラテック）という書き方。よく使うもの：分数 `\frac{a}{b}` 、2乗 `x^2` 、添字 `x_i` 、合計 `\sum` 、平方根 `\sqrt{x}` 、平均の記号 `\bar{x}` 、ギリシャ文字 `\alpha` 、`\sigma`
- 表は、`| 月 | 気温（℃） |` のように縦棒で列を区切った行の次に `|---|---|` の行を書き、その後にデータの行を並べる（下の例）
- `^` や `_` の後ろが2文字以上のときは `{ }` で囲む（ `x^{10}` 、`x_{12}` ）

下は、上の書き方で書いた Markdown セルの中身と、それを表示したもの。

````
回帰直線の式は $y = a + bx$ と書ける。平均は次の式で求める。

$$\bar{x} = \frac{1}{n} \sum_{i=1}^{n} x_i$$

標準偏差は $\sigma = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (x_i - \bar{x})^2}$ である。

---

> 引用：グラフは「見る人」のために描く。

折れ線グラフは `plt.plot(x, y)` で描く。

```python
plt.plot(x, y, marker='o')
plt.show()
```

| 月 | 気温（℃） |
|---|---|
| 1 | 4.3 |
| 8 | 28.0 |

\*これは斜体にならない\*

<!-- このメモは、表示すると見えない。自分用の覚え書きに使える -->
この上の行にはコメントが書いてある（ダブルクリックすると見える）。
````

表示すると：

回帰直線の式は $y = a + bx$ と書ける。平均は次の式で求める。

$$\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

標準偏差は $\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2}$ である。

> 引用：グラフは「見る人」のために描く。

折れ線グラフは `plt.plot(x, y)` で描く。

```
plt.plot(x, y, marker='o')
plt.show()
```

月	気温 (℃)
1	4.3
8	28.0

これは斜体にならない

この上の行にはコメントが書いてある（ダブルクリックすると見える）。

まとめ

やりたいこと	書き方
グラフの大きさ（基本設定）	`plt.figure(figsize=(8, 5))`（`plot` より前）
折れ線グラフ	`plt.plot(x, y)`
マーカーの種類	`marker='o'`
マーカーのサイズ	`markersize=10`（短く `ms=10`）
色	`color='r'`（短く `c='r'`）
軸の範囲	`plt.xlim(最小, 最大)`、`plt.ylim(最小, 最大)`
タイトル・軸の名前	`plt.title('...')`、`plt.xlabel('...')`、`plt.ylabel('...')`
文字の大きさ	`fontsize=14`
凡例	線に `label='...'` を付けて `plt.legend()`
以下は F	
凡例の場所	`plt.legend(loc='upper left')`、数値なら `loc=(0.05, 0.75)`
凡例の枠を消す	`plt.legend(frameon=False)`
線の種類・太さ	`linestyle='--'`、`linewidth=3`
短い書き方	`c='r'`、`ms=10`、`ls='--'`、`lw=3`
まとめて指定	`plt.plot(x, y, 'o--r')`（マーカー・線・色）

やりたいこと	書き方
目盛りの文字を斜めに	`plt.xticks(rotation=45)`
透明度	`alpha=0.5`
目盛り線	`plt.grid()`（薄くするなら `plt.grid(alpha=0.3)`）
Markdown セルにする	`Esc` → `M`（コードに戻すのは `Y`）
見出し・箇条書き・太字・リンク	`# 見出し`、`- 項目`、`**文字**`、`[文字](URL)`
Markdown の改行	行の最後に**半角スペース2つ**（または ` `）
横線・数式	`---`、`$y = ax + b$`

第 4 講：平均・標準偏差と補助線、散布図

numpy で平均・標準偏差を計算し、**補助線**としてグラフに引く（G）。次に、2つの量の関係を見る**散布図**を描く（H）。散布図の書き方は第 3 講の折れ線グラフとほぼ同じで、違うところ（点の大きさ `s=` など）だけ覚えればよい。

G. 平均・標準偏差と補助線

第 3 講の折れ線グラフ `plt.plot` に `linestyle='none'` を付けると、**線が消えて点だけ**のグラフになる（見た目は散布図。散布図の正式な描き方は H で学ぶ）。この「点だけのグラフ」に、データの**平均**と**標準偏差**を**補助線**として描く。

numpy とは

- **numpy**（ナムパイ）は、数の計算をするための**ライブラリ**（道具の詰め合わせ）。グラフの matplotlib と同じく、`import` してから使う（第 2 講 C-1）
- `import numpy as np` は「numpy を `np` という短い名前で使えるようにする」という意味。`import matplotlib.pyplot as plt` の `plt` と同じ考え方
- numpy の道具は `np.道具の名前(…)` の形で使う。例：`np.mean(score)` は「numpy の mean（平均）を score に使う」
- 平均・標準偏差などの統計の計算や、たくさんの数をまとめて計算するときに使う。データ分析ではほぼ必ず使う
- Anaconda には最初から入っているので、インストールはいらない

G-0 平均と補助線の完成形

8人のテストの点数を点だけのグラフにし、**平均**の線と、**平均 ± 標準偏差**の線を引いたもの。

1. まずは**点数を変えて**、線が自動で動くことを確かめる
2. そのあと部品（[0]〜[5]）の働きを1つずつ学ぶ（G-1〜G-4）

```
import numpy as np
import matplotlib.pyplot as plt
import japanize_matplotlib

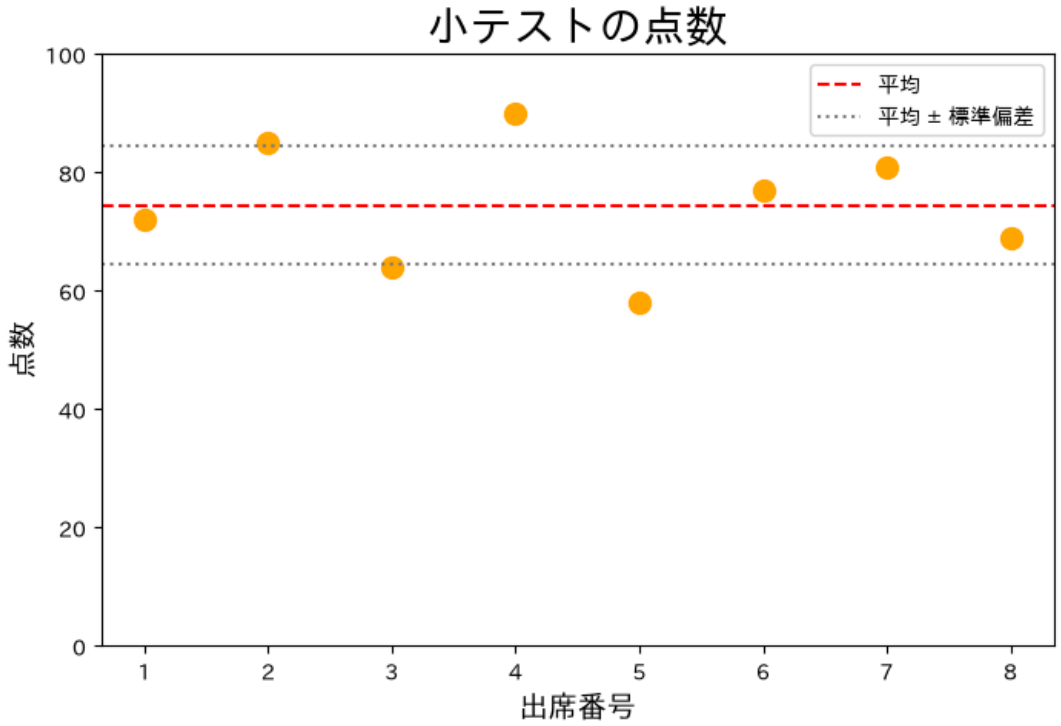
score = [72, 85, 64, 90, 58, 77, 81, 69]
number = [1, 2, 3, 4, 5, 6, 7, 8]

mean = np.mean(score)  # [1] 平均
std = np.std(score)  # [2] 標準偏差
print('平均', mean, '標準偏差', std)

plt.figure(figsize=(8, 5))  # [0] グラフの大きさ
plt.plot(number, score, marker='o', markersize=10, linestyle='none', color='orange')  # 線を消した折れ線（点だけ）
plt.axhline(y=mean, color='r', linestyle='--', label='平均')  # [3] 平均の補助線
plt.axhline(y=mean + std, color='gray', linestyle=':', label='平均 ± 標準偏差')  # [4] 平均 + 標準偏差
plt.axhline(y=mean - std, color='gray', linestyle=':')  # [4] 平均 - 標準偏差
```

```
plt.ylim(0, 100)  # [5] 軸・文字・凡例
plt.title('小テストの点数', fontsize=20)
plt.xlabel('出席番号', fontsize=14)
plt.ylabel('点数', fontsize=14)
plt.legend()
plt.show()
```

```
平均 74.5 標準偏差 10.111874208078342
```



変えてみよう（1か所変えたら、もう一度実行する）

- 点数：`90` → `40` に変えると、平均と標準偏差の線がどう動くか
- 線の種類：`linestyle='--'` → `'-'`、`'-.'`
- `plt.axhline` の行頭に `#` を付けると、どの線が消えるか
- `linestyle='none'` を消すと、点が線でつながる（第 3 講の折れ線グラフに戻る）

番号	部品	書き方	学ぶところ
[0]	グラフの大きさ（基本設定）	`plt.figure(figsize=(8, 5))`	第 3 講
[1]	平均	`np.mean(リスト)`	G-1
[2]	標準偏差	`np.std(リスト)`	G-3
[3]	平均の補助線	`plt.axhline(y=mean)`	G-4
[4]	平均 ± 標準偏差の補助線	`plt.axhline(y=mean + std)`	G-4
[5]	軸・文字・凡例	`plt.ylim`、`plt.title` など	第 3 講

G-1 平均

- **平均**は、全部足して個数で割ったもの。`np.mean(リスト)` で計算できる
- 第 2 講 C-2 の `sum(リスト) / len(リスト)` と同じ結果になる

```
print(np.mean(score))
print(sum(score) / len(score))
```

```
74.5
74.5
```

- 計算した値は、変数（ここでは `mean`）に入れておくと、あとで何度も使える
- 平均の値を**自分で計算して数字で書き込まない**。`np.mean(score)` と書いておけば、データを変えても自動で計算し直される

G-2 最大・最小・合計・中央値・四分位数

- 第 2 講 C-2 では `max`、`min`、`sum` を使った。numpy にも同じ働きの道具がある

知りたいもの	書き方
最大	`np.max(リスト)`
最小	`np.min(リスト)`
合計	`np.sum(リスト)`
中央値（大きさの順に並べて真ん中の値）	`np.median(リスト)`
第一四分位数・第三四分位数	`np.percentile(リスト, 25)`、`np.percentile(リスト, 75)`

```
print('最大', np.max(score))
print('最小', np.min(score))
print('合計', np.sum(score))
print('中央値', np.median(score))
```

```
最大 90
最小 58
合計 596
中央値 74.5
```

- 中央値は、極端な値が1つあっても動きにくい。年収や家の値段など、一部に大きい値があるデータでは平均と中央値がずれる

```
income = [300, 350, 400, 420, 450, 3000]
print('平均', np.mean(income))
print('中央値', np.median(income))
```

```
平均 820.0
中央値 410.0
```

四分位数

- **四分位数**は前期に学んだ。データを小さい順に並べて**4等分する区切り**の値。numpy で求められる
  - 第一四分位数（下から 1/4）：`np.percentile(リスト, 25)`
  - 第二四分位数（真ん中）：**中央値**と同じ。`np.percentile(リスト, 50)` は `np.median(リスト)` と同じ値
  - 第三四分位数（下から 3/4）：`np.percentile(リスト, 75)`
- `percentile` は**百分位数**。`25` は「下から 25% の位置」という意味

```
print('第一四分位数', np.percentile(score, 25))
print('中央値', np.percentile(score, 50))
print('第三四分位数', np.percentile(score, 75))
```

```
第一四分位数 67.75
中央値 74.5
第三四分位数 82.0
```

- 第一四分位数から第三四分位数までに、**真ん中の半分**（50%）のデータが入る。この幅（第三四分位数 − 第一四分位数）を**四分位範囲**という
- 四分位範囲が広いほど、真ん中の半分がばらついている。第 7 講の**箱ひげ図**の「箱」が、この範囲になる
- 四分位数の求め方はいくつかある。numpy は高校の教科書の手計算（前半・後半それぞれの中央値）と少し違う値になることがある（この score では、numpy は 67.75 と 82.0、教科書の方法では 66.5 と 83）

G-3 標準偏差

- **標準偏差**は、データの**ばらつき**（平均からどれくらい離れているか）を表す数。`np.std(リスト)` で計算できる
- 平均が同じでも、ばらつきが違うことがある。下の2つのクラスは平均がどちらも 70 点

```
class_a = [68, 70, 72, 69, 71, 70]
class_b = [40, 100, 55, 85, 60, 80]

print('A組 平均', np.mean(class_a), '標準偏差', np.std(class_a))
print('B組 平均', np.mean(class_b), '標準偏差', np.std(class_b))
```

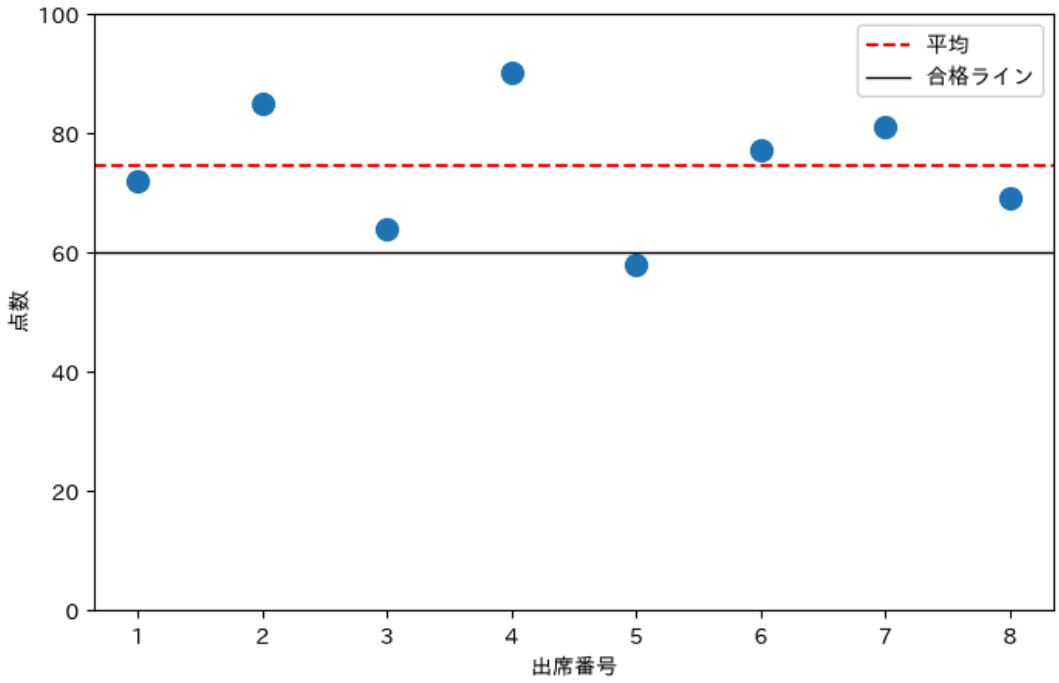
```
A組 平均 70.0 標準偏差 1.2909944487358056
B組 平均 70.0 標準偏差 20.207259421636902
```

- 標準偏差が**小さい**＝みんな平均の近く、**大きい**＝ばらばら
- 多くのデータでは、「平均 ± 標準偏差」の範囲に全体のおよそ3分の2が入る
- 標準偏差は第 9 講のエラーバー（AB）でも使う

G-4 補助線

- `plt.axhline(y=値)` で、その高さに**横の線**を引く（axis horizontal **line**）
- `plt.axvline(x=値)` で、その位置に**縦の線**を引く（axis vertical **line**）
- 線の位置に、G-1〜G-3 で**計算した値**（平均・標準偏差）を使える
- 折れ線グラフにも散布図にも使える（散布図は H-9）

```
plt.figure(figsize=(8, 5))
plt.plot(number, score, marker='o', markersize=10, linestyle='none')
plt.axhline(y=np.mean(score), color='r', linestyle='--', label='平均')
plt.axhline(y=60, color='k', linewidth=1, label='合格ライン')
plt.ylim(0, 100)
plt.xlabel('出席番号')
plt.ylabel('点数')
plt.legend()
plt.show()
```



- 平均の線は `np.mean(score)` で、合格ラインの線は `60` と数を直接書いている。「データで決まる線」と「決まっている線」の違い

線の色・種類・太さ・名前

折れ線グラフ（第 3 講 F-2）の線と同じ書き方。

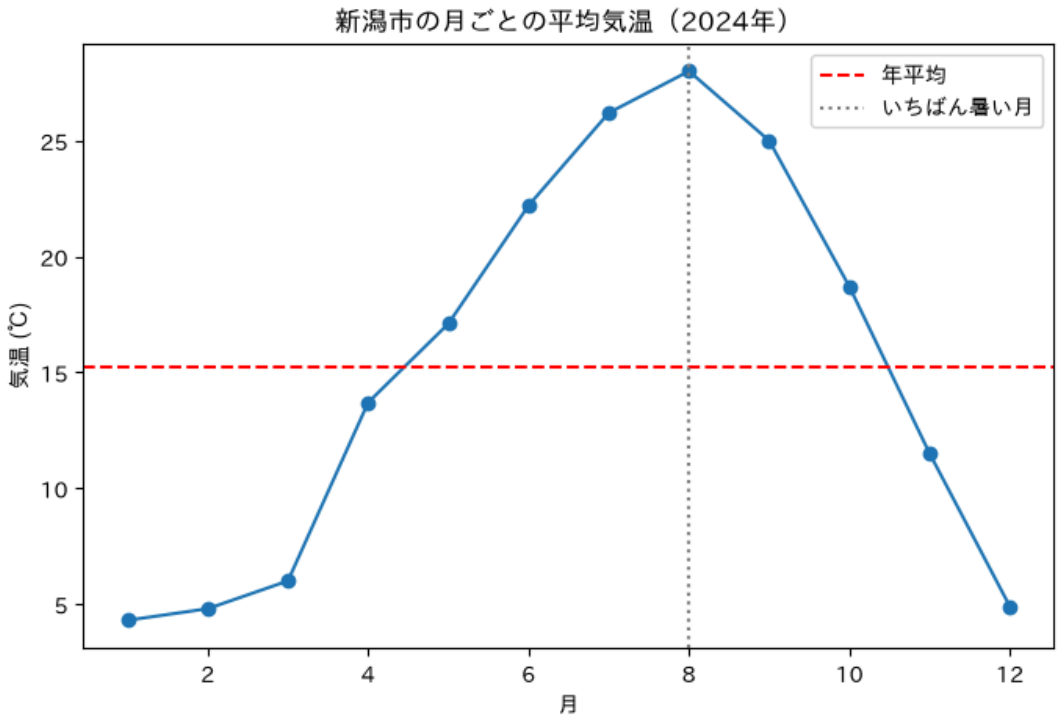
やりたいこと	書き方	短い書き方
色	`color='r'`	`c='r'`
線の種類	`linestyle='--'`（`'-'` 実線、`'--'` 破線、`':'` 点線、`'-.'` 一点鎖線、`'none'` 線なし）	`ls='--'`
線の太さ	`linewidth=2`	`lw=2`
凡例の名前	`label='平均'`（`plt.legend()` と一緒に使う）	

折れ線グラフに引く

- ふつうの折れ線グラフにも同じように引ける。縦の線も一緒に引ける

```
month = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
temp = [4.3, 4.8, 6.0, 13.7, 17.1, 22.2, 26.2, 28.0, 25.0, 18.7, 11.5, 4.9]

plt.figure(figsize=(8, 5))
plt.plot(month, temp, marker='o')
plt.axhline(y=np.mean(temp), color='r', linestyle='--', label='年平均')
plt.axvline(x=8, color='gray', linestyle=':', label='いちばん暑い月')
plt.title('新潟市の月ごとの平均気温（2024年）')
plt.xlabel('月')
plt.ylabel('気温（℃）')
plt.legend()
plt.show()
```



使ってみよう

- 上のセルの `x=8` を、`temp` が最大になる月を自動で探して書けるか考えてみよう（ヒント：I-1 の発展編）
- `score` のグラフに、平均 + 標準偏差・平均 − 標準偏差の2本の線を、色を変えて引いてみよう

H. 散布図を描く

H-0 散布図の完成形

散布図では、このグラフを目指す。

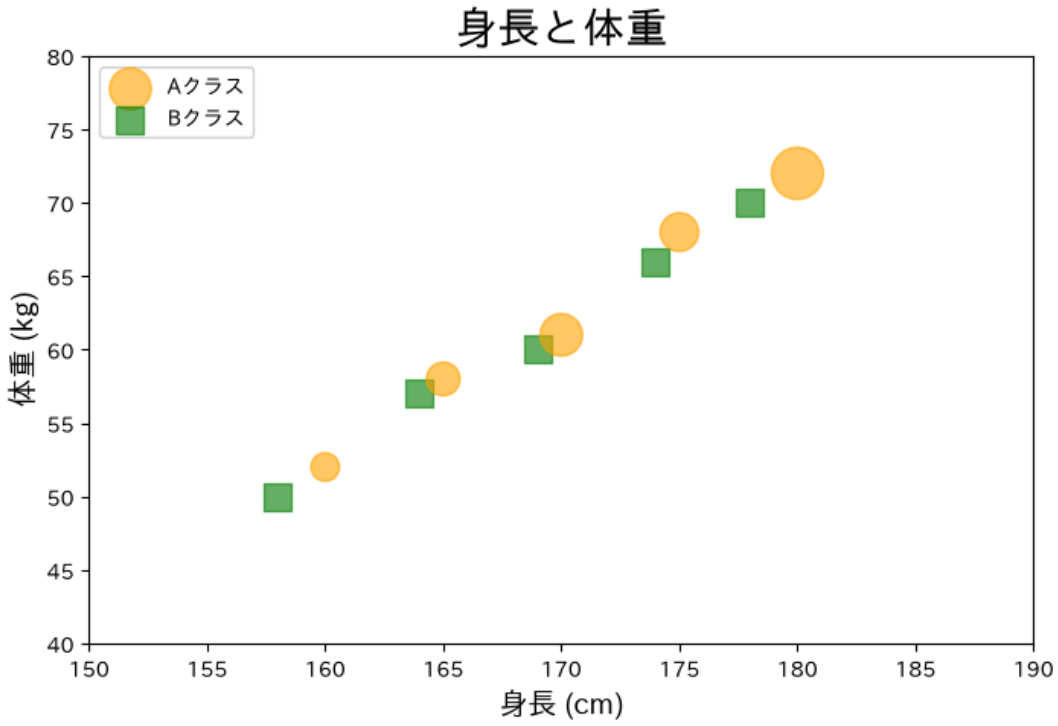
1. まずは**数値や文字を変えて**、グラフのどこが変わるかを確かめる
2. そのあと1行ずつ分解して、各部品（[0]〜[9]）の働きを学ぶ（H-1〜H-8）

```
import matplotlib.pyplot as plt
import japanize_matplotlib

height_a = [160, 165, 170, 175, 180]
weight_a = [52, 58, 61, 68, 72]
size_a = [180, 250, 400, 330, 600]
height_b = [158, 164, 169, 174, 178]
weight_b = [50, 57, 60, 66, 70]

plt.figure(figsize=(8, 5))  # [0] グラフの大きさ
plt.scatter(height_a, weight_a, marker='o', s=size_a, color='orange',
            alpha=0.6, zorder=2, label='Aクラス')  # [1] 点 [2] マーカー・色 [3] 大きさ [7] 透明度 [6] 重なる順番 [8] 名前
plt.scatter(height_b, weight_b, marker='s', s=150, color='g',
            alpha=0.6, zorder=1, label='Bクラス')  # [5] 2つ目のグループ
plt.xlim(150, 190)  # [9] 横軸の範囲
plt.ylim(40, 80)  # [9] 縦軸の範囲
plt.title('身長と体重', fontsize=20)  # [9] タイトル
```

```
plt.xlabel('身長 (cm)', fontsize=14)  # [9] 横軸の名前
plt.ylabel('体重 (kg)', fontsize=14)  # [9] 縦軸の名前
plt.legend(loc='upper left')  # [8] 凡例
plt.show()
```



上の1つ目の `plt.scatter(...)` は1つの命令なので、1行に並べて書くと次のようになる。

```
plt.scatter(height_a, weight_a, marker='o', s=size_a, color='orange', alpha=0.6, zorder=2, label='Aクラス')
```

- これでも同じグラフになるが、長くて読みにくい。そこで、見やすさのために `,` の後ろで**改行**してよい（`(` の中なら、次の行も続きとして読まれる）
- 改行の位置は自由。上では、点の見た目（`marker`・`s`・`color`）までを1行目、残り（`alpha`・`zorder`・`label`）を2行目に書いている

変えてみよう（1か所変えたら、もう一度実行する）

- 色：`'orange'` → `'r'`、`'skyblue'`、`'pink'`
- マーカー：`'o'` → `'^'`、`'*'`、`'D'`。Bクラスの大きさ `s=150` → `s=600`
- 透明度：`alpha=0.6` → `alpha=1`（重なった点が隠れる）、`alpha=0.2`
- 重なる順番：Aクラスの `zorder=2` と Bクラスの `zorder=1` を入れ替える
- 凡例の場所：`'upper left'` → `'lower right'`
- ある行を消す（または行頭に `#` を付ける）と、その部品が無いグラフになる。どの行が何をしているかを確かめよう

番号	部品	書き方	短い書き方	学ぶところ
[0]	グラフの大きさ（基本設定）	`plt.figure(figsize=(8, 5))`		第 3 講
[1]	点を打つ	`plt.scatter(x, y)`		H-1
[2]	マーカーの種類・色	`marker='o'`、`color='orange'`	`c='orange'`	H-2

番号	部品	書き方	短い書き方	学ぶところ
[3]	点の大きさ	`s=150` 、`s=リスト`		H-3
[4]	点ごとの色	`color=リスト`	`c=リスト`	H-4
[5]	2つのグループ	`plt.scatter` を2回		H-5
[6]	重なる順番	`zorder=2`		H-6
[7]	透明度	`alpha=0.6`		H-7
[8]	凡例とその場所	`label='...'` 、`plt.legend(loc='upper left')`		H-8
[9]	軸の範囲・タイトル・軸の名前	`plt.xlim` 、`plt.ylim` 、`plt.title` 、`plt.xlabel` 、`plt.ylabel`		第 3 講

- 全部覚えなくてよい。この表は**部品の地図**として、グラフを描くときに見返す
- [4] 点ごとの色は、完成形には入っていない（H-4 で学ぶ）。H-3 の後半（3つ目の値で大きさを変える）、H-10（点に名前を付ける）も完成形にはない
- [9] は第 3 講の折れ線グラフと同じ書き方

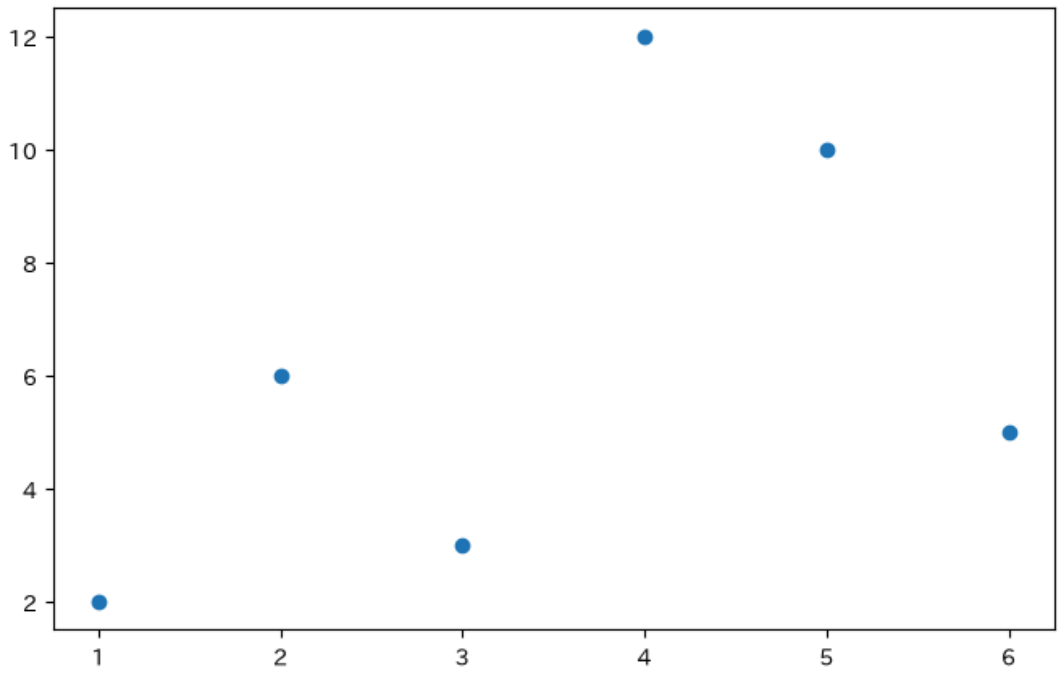
ここから [1] の点だけのグラフに戻って、1つずつ足していく。

H-1 散布図の基本

- `plt.plot(x, y)` を `plt.scatter(x, y)` に変えるだけ
- x と y は**同じ個数**にする（違うと `ValueError: x and y must be the same size` ）

```
x = [1, 2, 3, 4, 5, 6]
y = [2, 6, 3, 12, 10, 5]

plt.figure(figsize=(8, 5))
plt.scatter(x, y)
plt.show()
```



折れ線グラフとの違い

	折れ線グラフ	散布図
関数	`plt.plot(x, y)`	`plt.scatter(x, y)`
線	点を線でつなぐ	線なし（点だけ）
向いているもの	時間の流れなど、**順番に意味があるデータ**	2つの量の**関係**
点（マーカー）	何も書かないと点なし。`marker='o'` で付く	何も書かないと `'o'`
マーカーの種類・色	`marker=` 、`color=`	`marker=` 、`color=` （同じ）
点の大きさ	`markersize=10` （ `ms=10` ）	**`s=100`** （名前が違う！ **面積**なので、`s` は `markersize` のおよそ2乗）
点ごとの大きさ・色	できない（全部同じ）	`s=リスト` 、`color=リスト` （H-3・H-4）
点のふちの色	`markeredgecolor='k'`	**`edgecolor='k'`** （名前が違う）
何も書かないときの重なる順番	`zorder` は 2	`zorder` は 1（線が点の上になる。H-9）
線の種類・太さ	`linestyle=` 、`linewidth=`	使わない（点を線でつながないので）

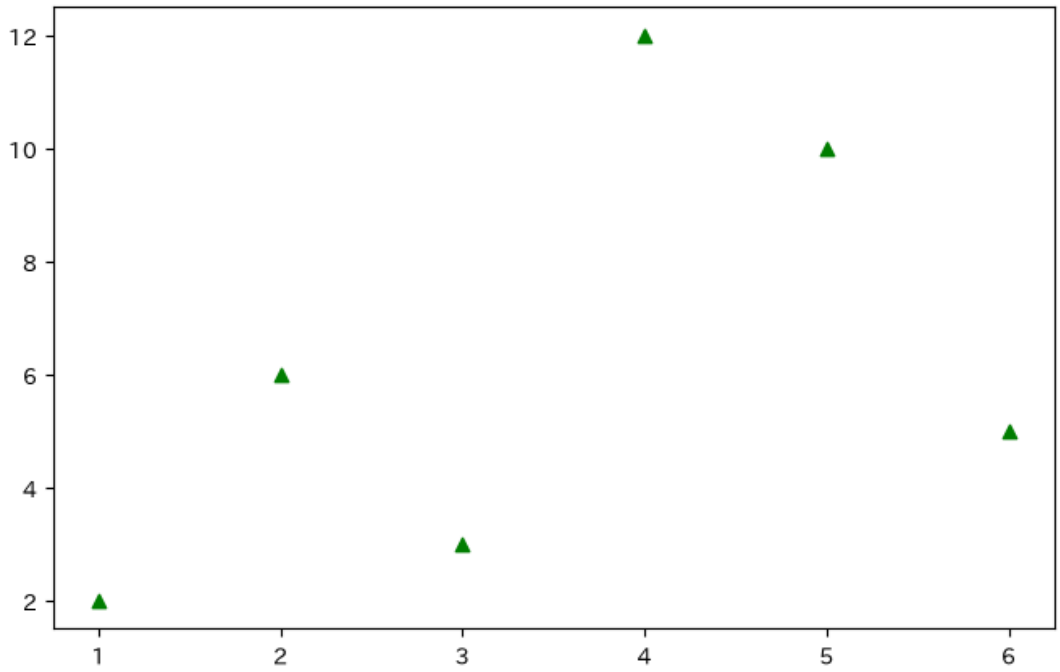
- 名前の違うオプションを書くとエラーになる（散布図に `markersize=` 、折れ線に `s=` や `edgecolor=` ）

H-2 マーカーの種類と色

- 折れ線グラフと同じく、`marker=` と `color=` を足す

書き方	形	書き方	形
`'o'`	● 丸（何も書かないときはこれ）	`'*'`	★ 星
`'s'`	■ 四角	`'D'`	◆ ひし形
`'^'`	▲ 上向き三角	`'x'`	× バツ

```
plt.figure(figsize=(8, 5))
plt.scatter(x, y, marker='^', color='g')
plt.show()
```

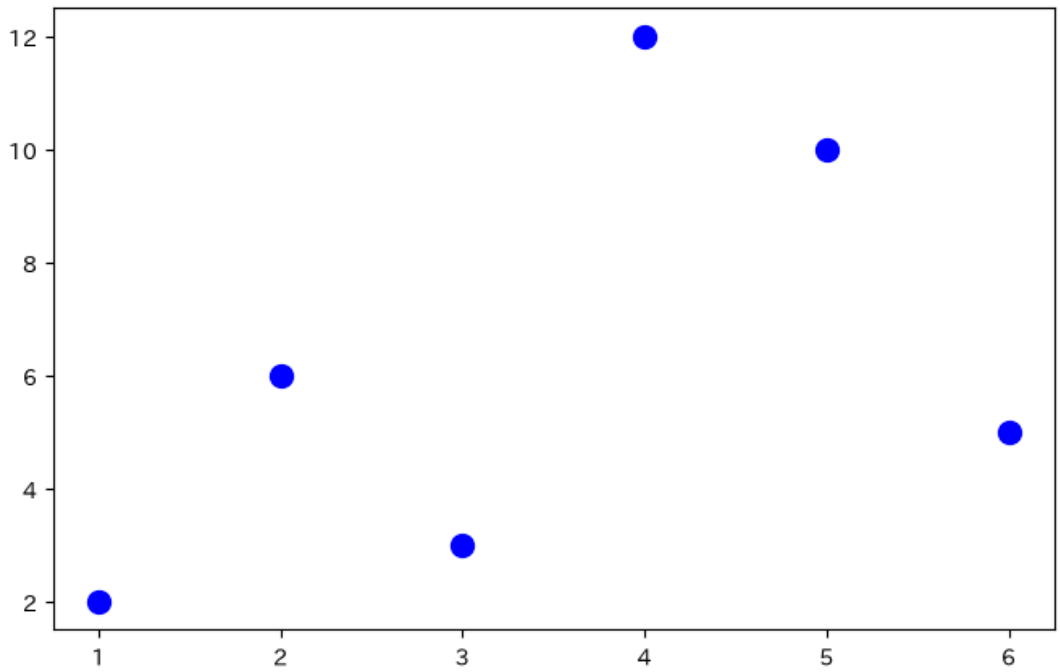


- `color=` の代わりに `c=` と書いてもよい（`color='g'` / `c='g'`）

H-3 点の大きさ

- 散布図の点の大きさは `s=` で指定する（**`markersize=` ではない**）
- `s=` は点の**面積**。何も書かないときは `s=36` くらい。`s=100` で折れ線の `markersize=10` と同じくらい
- 面積なので、`s=100` → `s=400`（4倍）で、見た目の幅は2倍になる

```
plt.figure(figsize=(8, 5))
plt.scatter(x, y, marker='o', color='b', s=100)
plt.show()
```

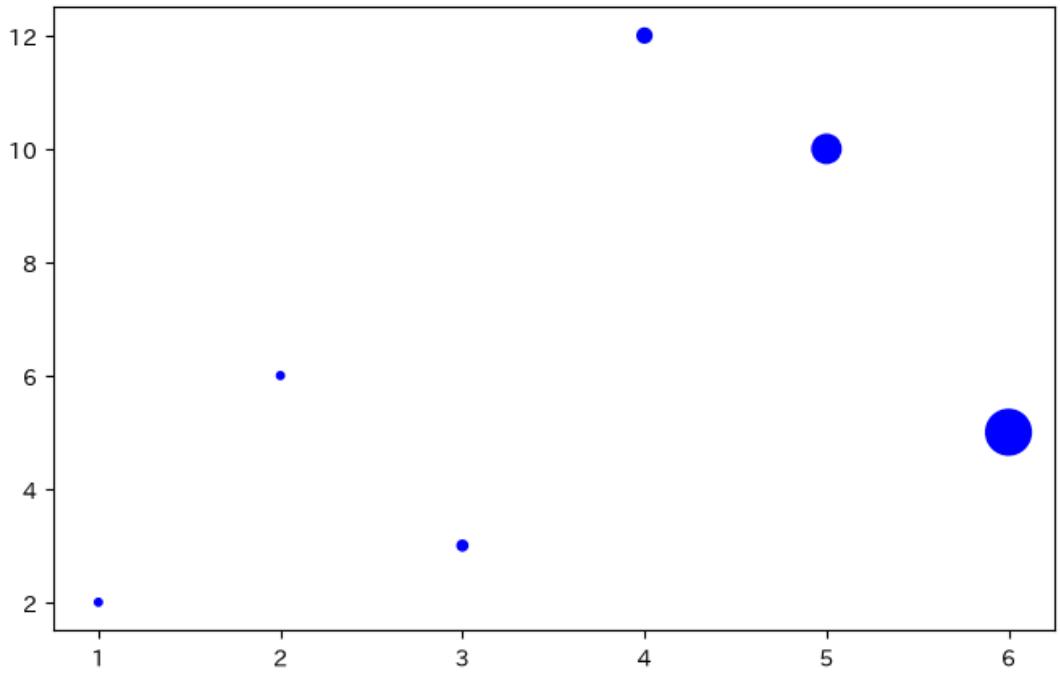


点ごとに大きさを変える：`s=リスト`

- `s=` に**リスト**を渡すと、1つ目の点にはリストの1つ目、2つ目の点には2つ目…の大きさが付く
- リストの数は**自分で決めて**書いてよい。同じ数が並んでもよい
- リストの個数は、**点の個数（x・y の個数）と同じ**にする

```
size_list = [10, 10, 20, 40, 160, 400]

plt.figure(figsize=(8, 5))
plt.scatter(x, y, marker='o', color='b', s=size_list)
plt.show()
```



- 左の2つ（`10`）は、何も書かないとき（`s=36` くらい）より小さい。右の2つ（`160`、`400`）は大きく目立つ
- どの点を目立たせるかを、大きさで決められる。リストの数を書き換えて、どの点が大きくなるか確かめよう
  `AttributeError: PathCollection.set() got an unexpected keyword argument 'markersize'`

3つ目の値で大きさを変える（バブルチャート）

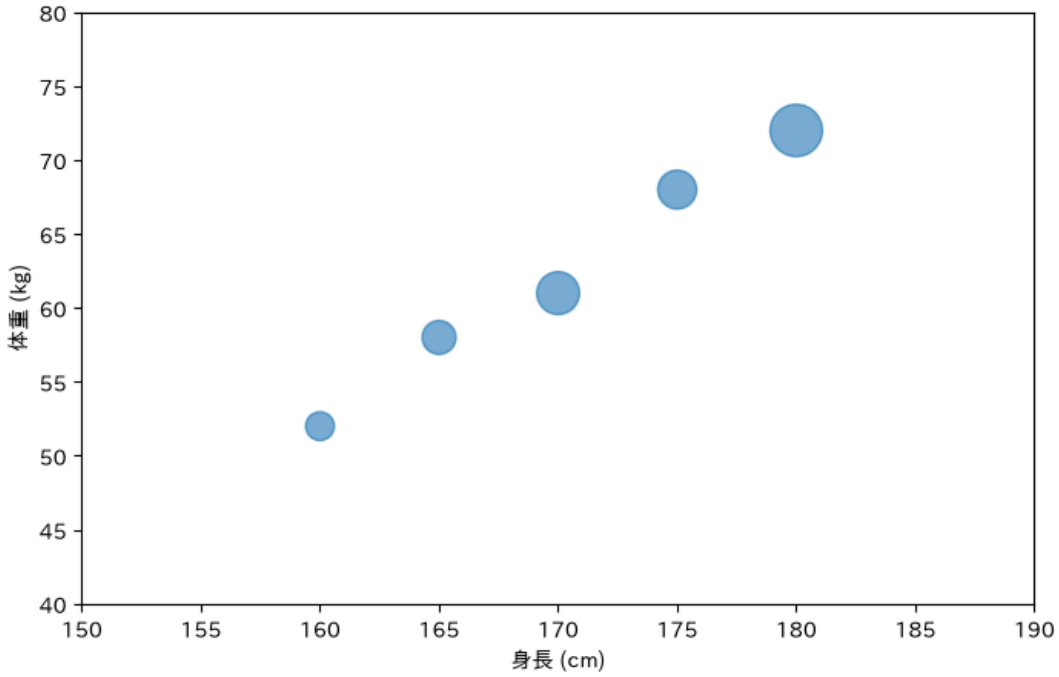
- 上では大きさの数を手で書いた。代わりに、**データの値**（年齢、人口、売上など）のリストを `s=` に渡すと、x・y に続く**3つ目の情報**を点の大きさで見せられる（**バブルチャート**）
- 下は、H-0 の Aクラスの身長（x）と体重（y）に、年齢 `age` を点の大きさとして加えたもの
- 年齢の数（18〜60）をそのまま `s=` に渡すと、点が小さくて見えにくい。そこで `np.array(age) * 10` で**全部10倍**にしている（なぜ `np.array` にするかは発展 I-1）

```
age = [18, 25, 40, 33, 60]
age_size = np.array(age) * 10
print(age_size)

plt.figure(figsize=(8, 5))
plt.scatter(height_a, weight_a, s=age_size, alpha=0.6)
plt.xlim(150, 190)
plt.ylim(40, 80)
```

```
plt.xlabel('身長 (cm)')
plt.ylabel('体重 (kg)')
plt.show()
```

```
[180 250 400 330 600]
```



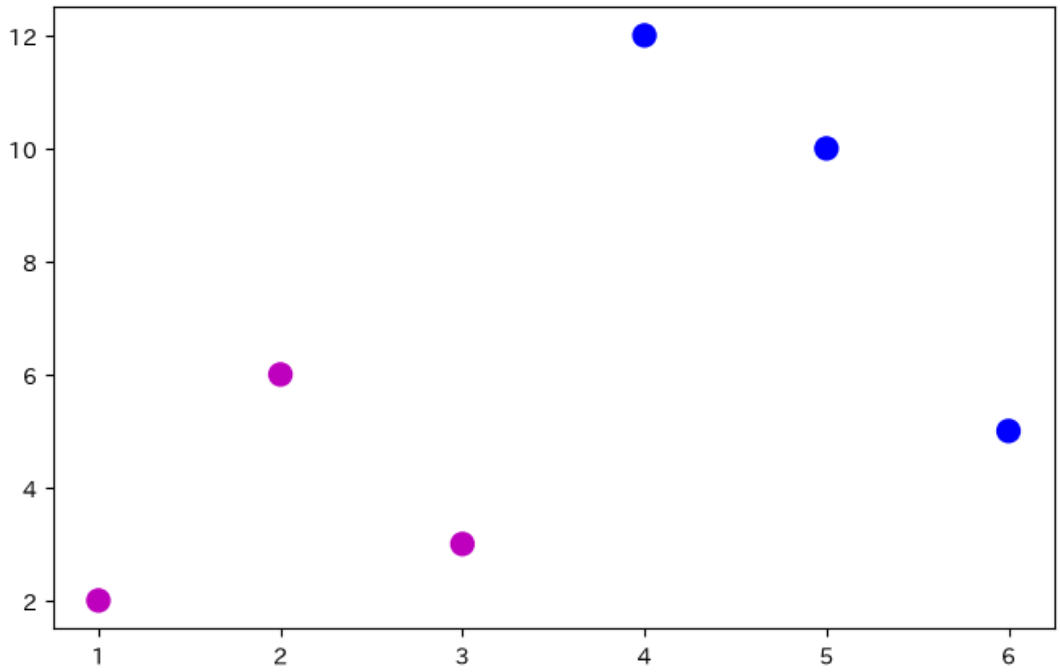
- 点が大きいほど年齢が高い。「x・y・大きさ」で、**3つの量を1枚に**描ける
- 実は H-0 の `size_a = [180, 250, 400, 330, 600]` は、この年齢を10倍したもの
- 大きさは**面積**なので、値が2倍でも見た目の幅は2倍にならない（H-3 の最初）。大きさの違いはおおまかに読む

H-4 点ごとに色を変える

- `color=` にも**リスト**を渡せる。点ごとに色が変わる
- 例：最初の3つ（x が 1, 2, 3）は女性、残りの3つは男性として、色で分ける
- リストは**手で書けばよい**（短く書く方法は第 6 講 P-4）

```
mycolors = ['m', 'm', 'm', 'b', 'b', 'b']

plt.figure(figsize=(8, 5))
plt.scatter(x, y, marker='o', s=100, color=mycolors)
plt.show()
```



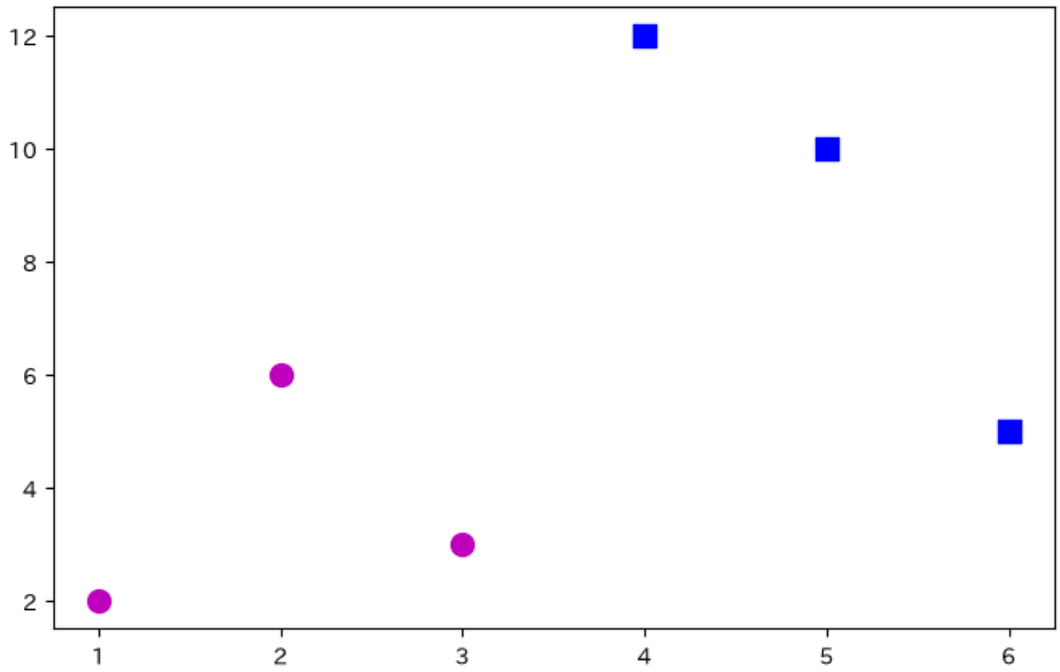
- 色のリストの個数が点の個数と違うとエラーになる： `ValueError: 'c' argument has 2 elements, which is inconsistent with 'x' and 'y' with size 6.`

H-5 2つのグループを重ねる

- データが**グループごとに別のリスト**になっているときは、`plt.scatter` を**2回書く**（第 3 講 E-4 の `plt.plot` を2回書いたのと同じ）
- グループごとに色やマーカーを変えると区別しやすい

```
female_x = [1, 2, 3]
female_y = [2, 6, 3]
male_x = [4, 5, 6]
male_y = [12, 10, 5]

plt.figure(figsize=(8, 5))
plt.scatter(female_x, female_y, marker='o', s=100, color='m')
plt.scatter(male_x, male_y, marker='s', s=100, color='b')
plt.show()
```

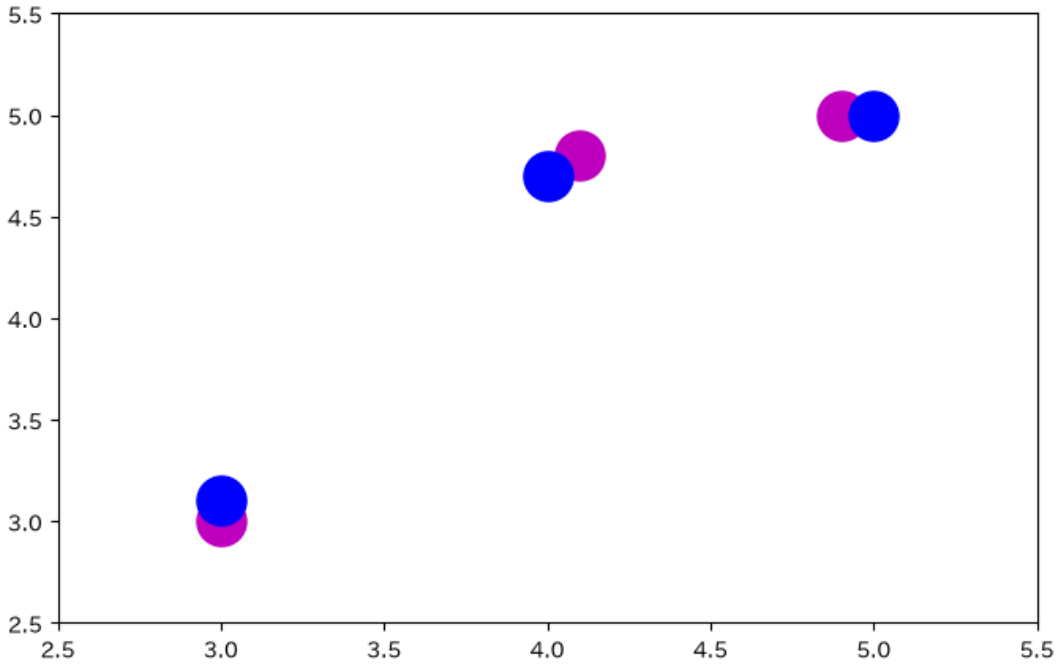


- H-4（色のリスト）と同じ見た目になる。データが1つのリストなら H-4、グループごとのリストなら H-5 の書き方

H-6 重なる順番

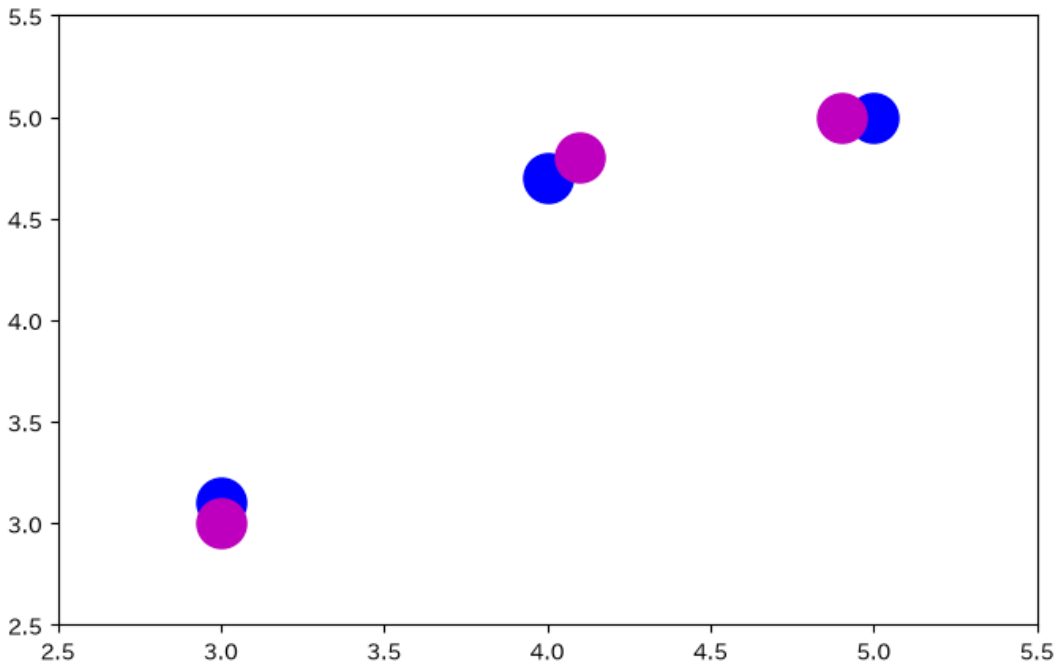
```
female_x = [3, 4.1, 4.9]
female_y = [3, 4.8, 5]
male_x = [3, 4, 5]
male_y = [3.1, 4.7, 5]

plt.figure(figsize=(8, 5))
plt.scatter(female_x, female_y, s=500, color='m')
plt.scatter(male_x, male_y, s=500, color='b')
plt.xlim(2.5, 5.5)
plt.ylim(2.5, 5.5)
plt.show()
```



- 男性（青）を後から描いたので、青が上になっている
- `zorder=` で重なる順番を決められる。**数が大きいほど上（手前）**に描かれる
- 大きな点が枠からはみ出しそうなときは、`plt.xlim`・`plt.ylim` で範囲を少し広げる

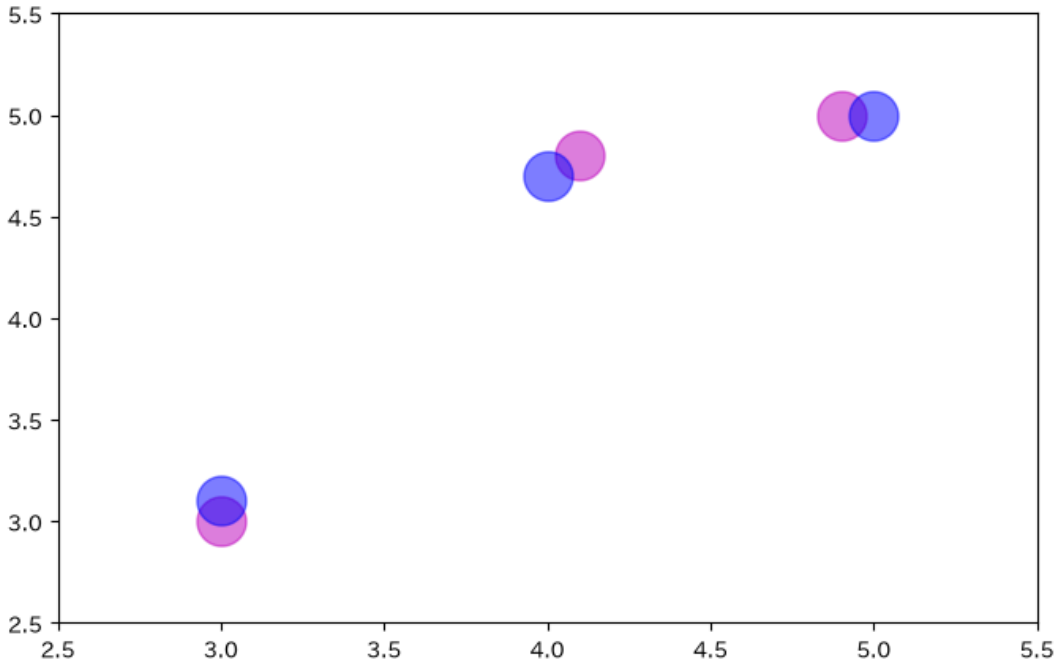
```
plt.figure(figsize=(8, 5))
plt.scatter(female_x, female_y, s=500, color='m', zorder=2)
plt.scatter(male_x, male_y, s=500, color='b', zorder=1)
plt.xlim(2.5, 5.5)
plt.ylim(2.5, 5.5)
plt.show()
```



H-7 透明度

- `alpha=` で点を**透けさせる。0 で透明、1 で不透明**（何も書かないときは 1）
- 0.5 くらいにすると、重なった部分の下の点も透けて見える
- 点の数が多いとき（数百〜数千）は、`alpha=0.3` くらいにすると、点が集まっている場所が濃く見えて分かりやすい

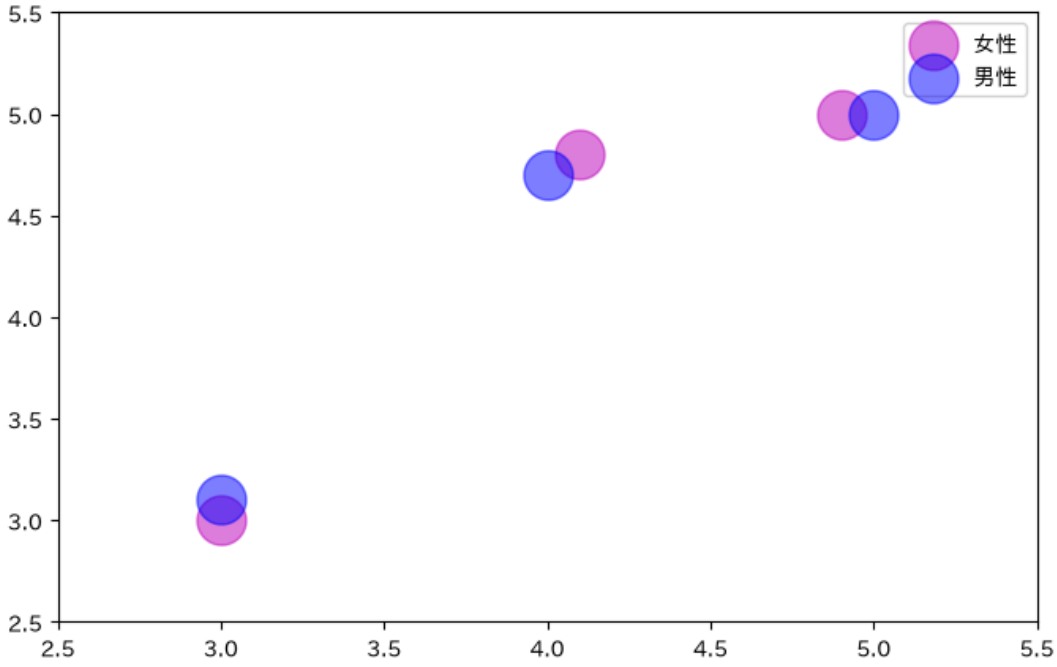
```
plt.figure(figsize=(8, 5))
plt.scatter(female_x, female_y, s=500, color='m', alpha=0.5)
plt.scatter(male_x, male_y, s=500, color='b', alpha=0.5)
plt.xlim(2.5, 5.5)
plt.ylim(2.5, 5.5)
plt.show()
```



H-8 凡例とその場所

- 折れ線グラフと同じく、`label='...'` でグループに名前を付け、`plt.legend()` で**凡例**を表示する

```
plt.figure(figsize=(8, 5))
plt.scatter(female_x, female_y, s=500, color='m', alpha=0.5, label='女性')
plt.scatter(male_x, male_y, s=500, color='b', alpha=0.5, label='男性')
plt.xlim(2.5, 5.5)
plt.ylim(2.5, 5.5)
plt.legend()
plt.show()
```

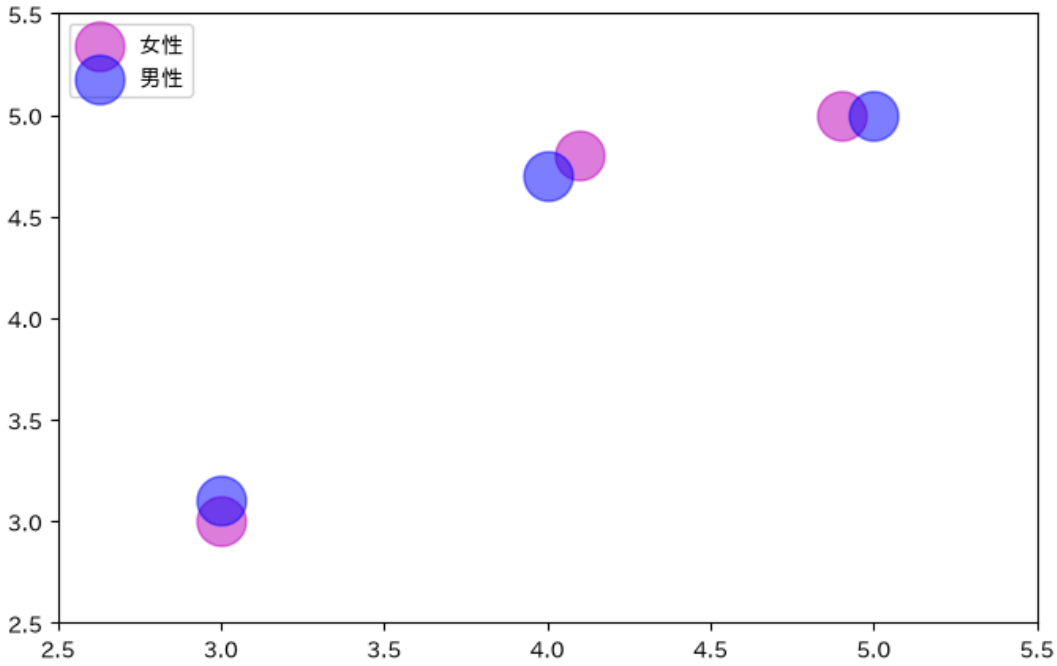


- `label=` を書かずに `plt.legend()` だけ書くと、凡例は出ず `No artists with labels found to put in legend.` という注意が出る

凡例の場所を言葉で指定する：`loc='...'`

loc=	場所	loc=	場所
`'upper left'`	左上	`'upper right'`	右上
`'lower left'`	左下	`'lower right'`	右下
`'center'`	中央	`'best'`	点と重なりにくい場所（何も書かないときはこれ）

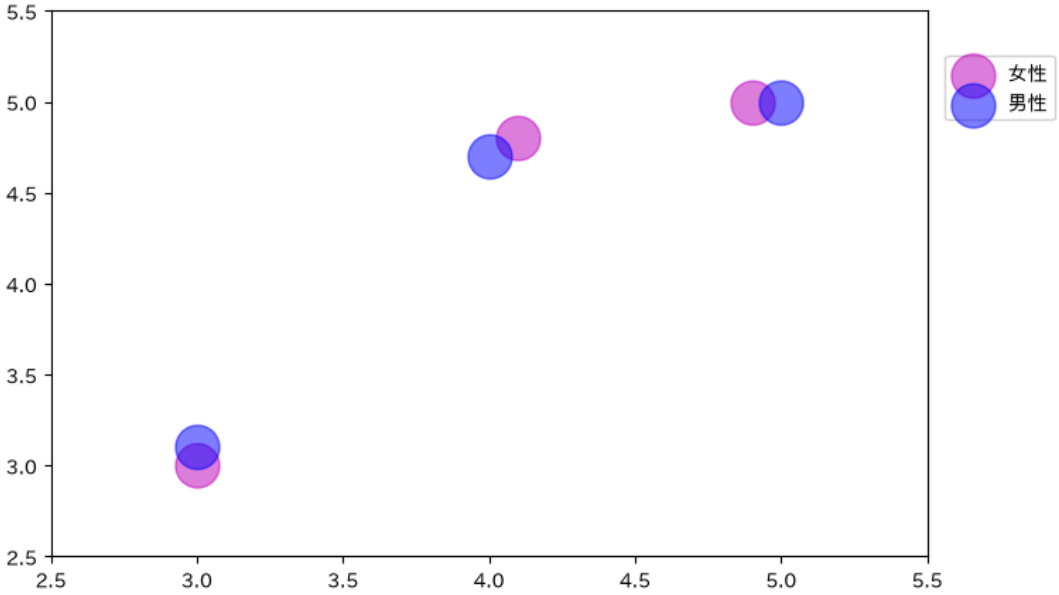
```
plt.figure(figsize=(8, 5))
plt.scatter(female_x, female_y, s=500, color='m', alpha=0.5, label='女性')
plt.scatter(male_x, male_y, s=500, color='b', alpha=0.5, label='男性')
plt.xlim(2.5, 5.5)
plt.ylim(2.5, 5.5)
plt.legend(loc='upper left')
plt.show()
```



凡例の場所を数値で指定する：`loc=(横, 縦)`

- グラフの枠の**左下**を `(0, 0)`、**右上**を `(1, 1)` として、凡例の**左下の角**を置く位置を指定する
- 0〜1 の範囲なら枠の中、それ以外なら**枠の外**に出る。例：`loc=(1.02, 0.8)` で枠の右外

```
plt.figure(figsize=(8, 5))
plt.scatter(female_x, female_y, s=500, color='m', alpha=0.5, label='女性')
plt.scatter(male_x, male_y, s=500, color='b', alpha=0.5, label='男性')
plt.xlim(2.5, 5.5)
plt.ylim(2.5, 5.5)
plt.legend(loc=(1.02, 0.8))
plt.show()
```

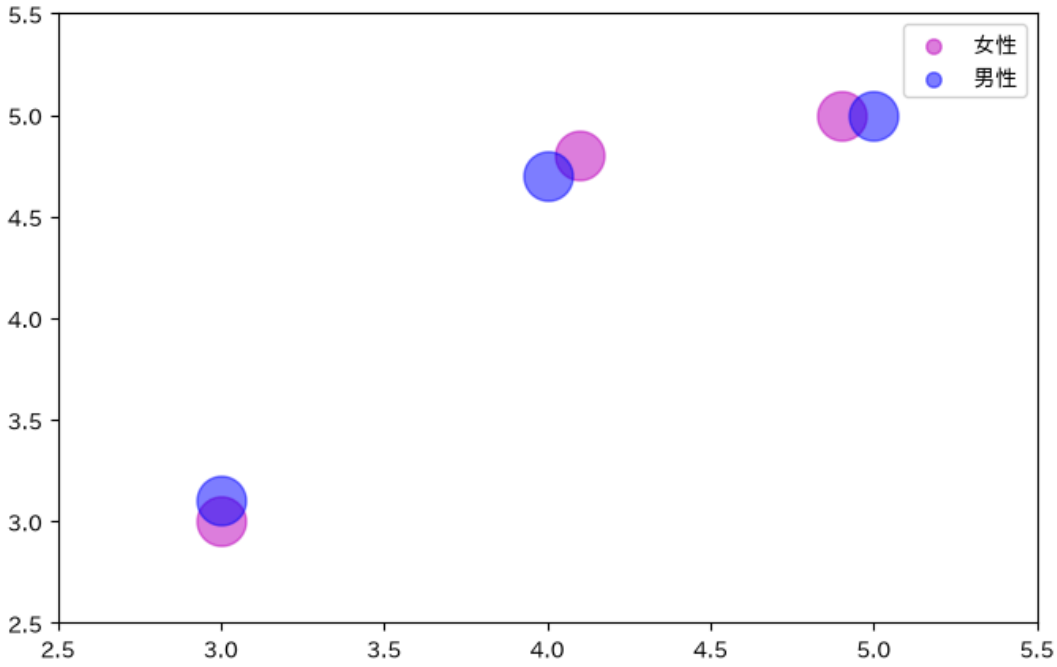


- 枠の外に出すと、保存した画像で凡例が切れることがある。そのときは `plt.show()` の前に `plt.tight_layout()` を書く

凡例の点の大きさだけ変える：`markerscale=`

- 凡例の点は、グラフの点と**同じ大きさ**で描かれる。`s=500` のように点が大きいと、凡例も大きくなる
- `plt.legend(markerscale=0.3)` で、凡例の点だけを**グラフの点の 0.3 倍**の大きさにできる（グラフの点の大きさは変わらない）
- `loc=` と一緒に書くときは、`plt.legend(loc='upper left', markerscale=0.3)` のようにカンマで区切る

```
plt.figure(figsize=(8, 5))
plt.scatter(female_x, female_y, s=500, color='m', alpha=0.5, label='女性')
plt.scatter(male_x, male_y, s=500, color='b', alpha=0.5, label='男性')
plt.xlim(2.5, 5.5)
plt.ylim(2.5, 5.5)
plt.legend(markerscale=0.3)
plt.show()
```



H-9 散布図に補助線を引く

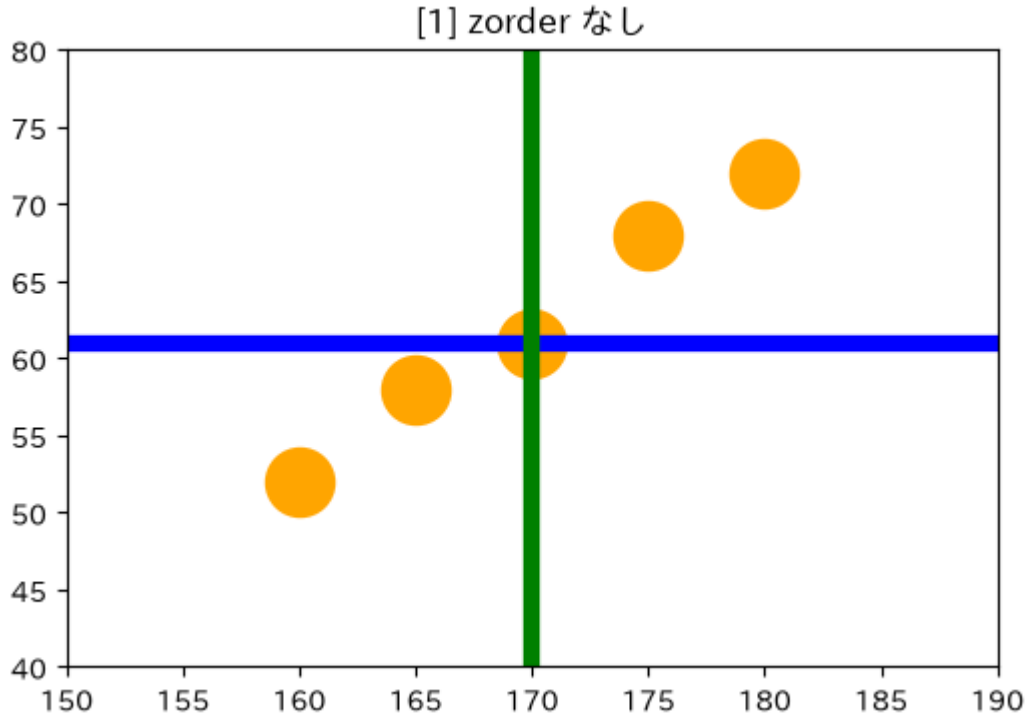
- G-4 の補助線（`plt.axhline`・`plt.axvline`）は、散布図にも同じように引ける
- 散布図では、何も書かないと**線が点の上**に描かれる。重なる順番は H-6 の `zorder=` で決める

点と補助線の重なる順番：`zorder`

- H-6 の `zorder=` は、点と補助線の間でも使える。**数が大きいほど上（手前）**に描かれる
- 何も書かないときは、**点**（**`plt.scatter`**）**は 1、線**（**`plt.axhline`**・**`plt.axvline`**・**`plt.plot`**）は 2**。だから、何もしないと**線が点の上**に描かれる
- 違いが分かりやすいように、点を大きく（`s=600`）、線を太く（`linewidth=6`）して、同じグラフを3通りで比べる

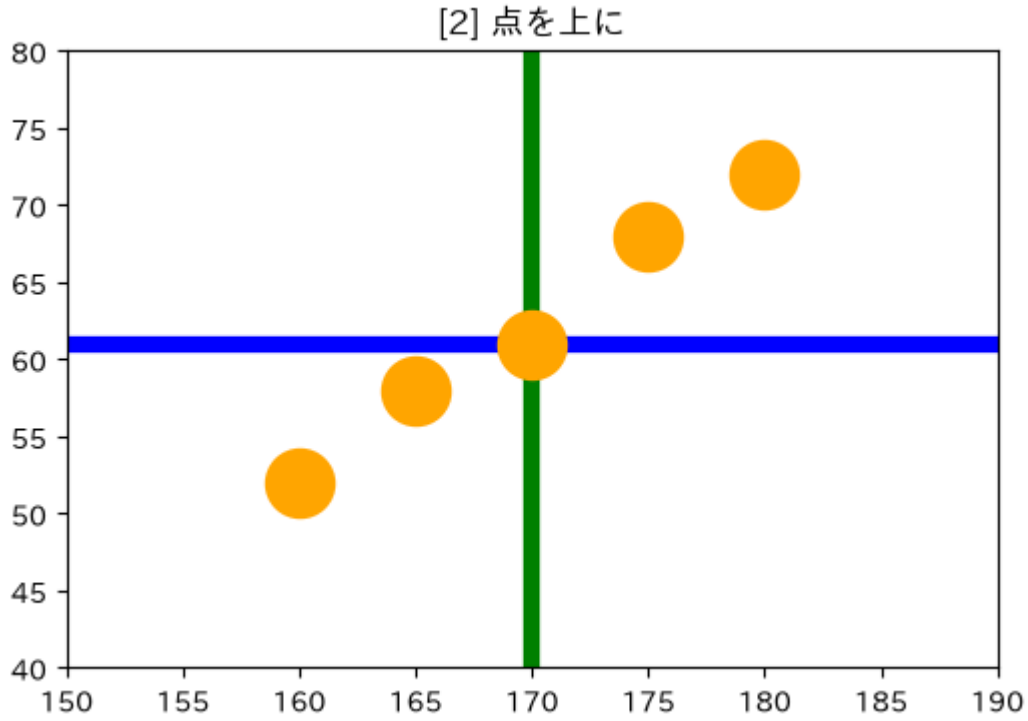
[1] 何も書かない：線（2）が点（1）の上。点が線で切られて見える

```
plt.figure(figsize=(6, 4))
plt.scatter(height_a, weight_a, s=600, color='orange')
plt.axhline(y=61, color='b', linewidth=6)
plt.axvline(x=170, color='g', linewidth=6)
plt.xlim(150, 190)
plt.ylim(40, 80)
plt.title('[1] zorder なし')
plt.show()
```



[2] 点に `zorder=3`：点（3）が線（2）の上。点が線に隠れない

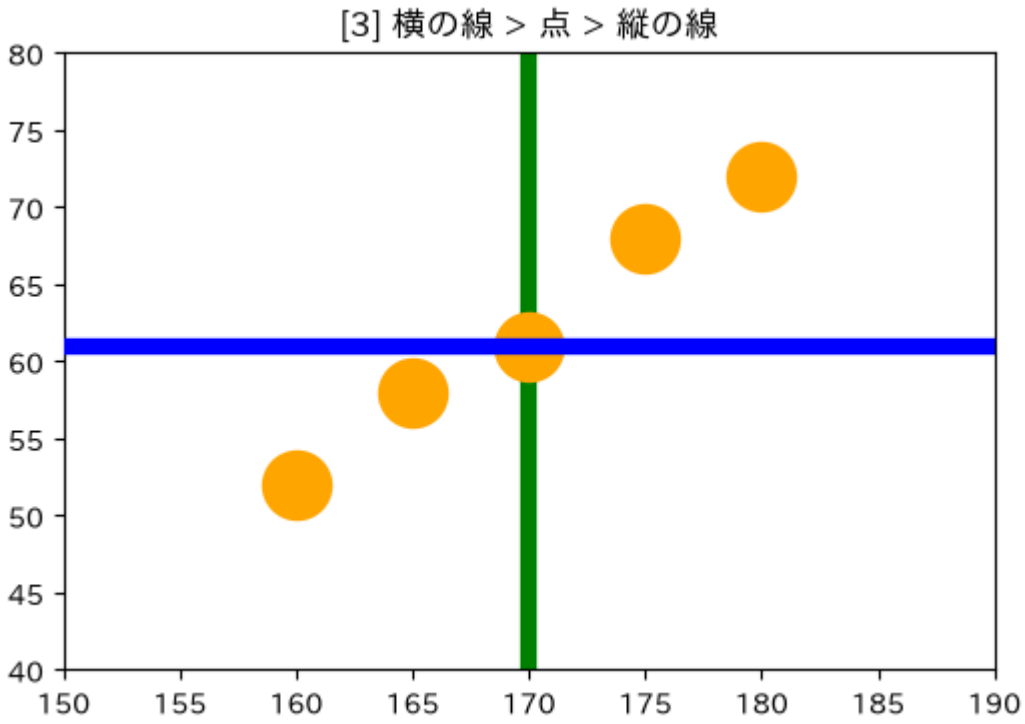
```
plt.figure(figsize=(6, 4))
plt.scatter(height_a, weight_a, s=600, color='orange', zorder=3)
plt.axhline(y=61, color='b', linewidth=6)
plt.axvline(x=170, color='g', linewidth=6)
plt.xlim(150, 190)
plt.ylim(40, 80)
plt.title('[2] 点を上に')
plt.show()
```



[3] 線ごとに変える：横の線に `zorder=4`、縦の線に `zorder=1`。点は `zorder=3`

- 上から順に、横の線（4）→ 点（3）→ 縦の線（1）

```
plt.figure(figsize=(6, 4))
plt.scatter(height_a, weight_a, s=600, color='orange', zorder=3)
plt.axhline(y=61, color='b', linewidth=6, zorder=4)
plt.axvline(x=170, color='g', linewidth=6, zorder=1)
plt.xlim(150, 190)
plt.ylim(40, 80)
plt.title('[3] 横の線 > 点 > 縦の線')
plt.show()
```



	横の線	点	縦の線	一番上に見えるもの
[1] 何も書かない	2	1	2	線（2本は後に書いた縦の線が上）
[2] 点に `zorder=3`	2	3	2	点
[3] 線ごとに変える	4	3	1	横の線

- 同じ数どうしは、H-6 と同じく**後から書いたものが上**になる（[1] で縦の線が横の線の上にあるのはこのため）
- ふだんは、[2] のように**点に `zorder=3`** を付けておけば、点が補助線に隠れない

変えてみよう

- [3] の数を入れ替えて、縦の線が一番上、横の線が一番下になるようにしてみよう
- 線を細く（`linewidth=1`）、点を小さく（`s=100`）すると、ふだんのグラフではどれくらい違って見えるか確かめよう

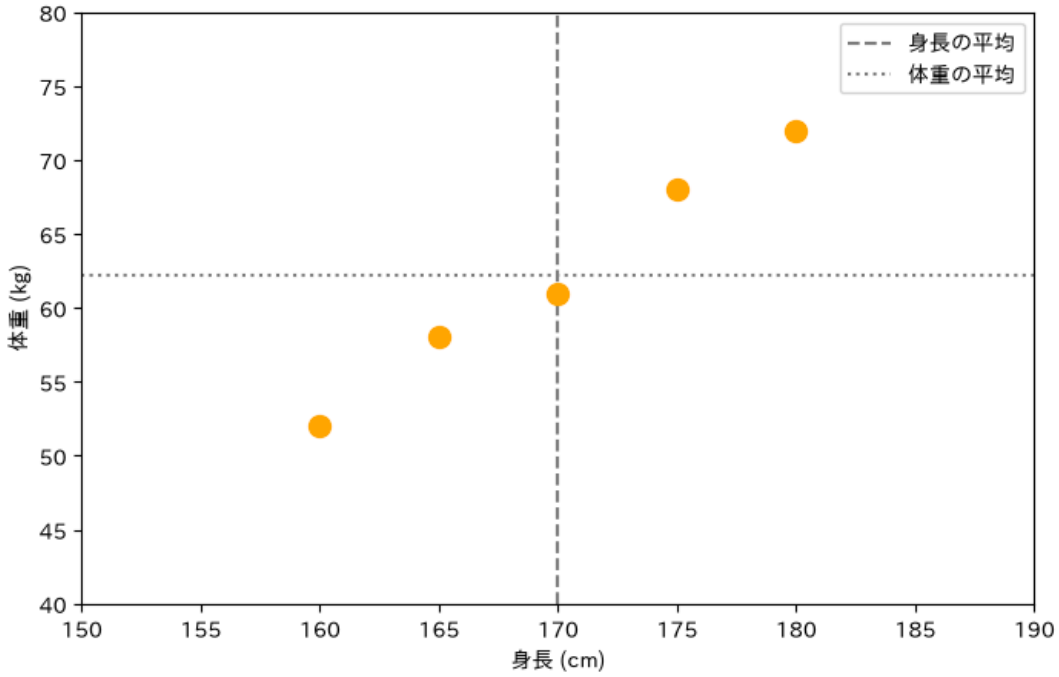
使ってみよう：平均の線を引く

- 身長の平均の縦の線と、体重の平均の横の線を引く。平均は G-1 の `np.mean(リスト)` で計算する

```
mean_h = np.mean(height_a)
mean_w = np.mean(weight_a)
print(mean_h, mean_w)
```

```
plt.figure(figsize=(8, 5))
plt.scatter(height_a, weight_a, s=100, color='orange', zorder=3)
plt.axvline(x=mean_h, color='gray', linestyle='--', label='身長の平均')
plt.axhline(y=mean_w, color='gray', linestyle=':', label='体重の平均')
plt.xlim(150, 190)
plt.ylim(40, 80)
plt.xlabel('身長 (cm)')
plt.ylabel('体重 (kg)')
plt.legend()
plt.show()
```

```
170.0 62.2
```

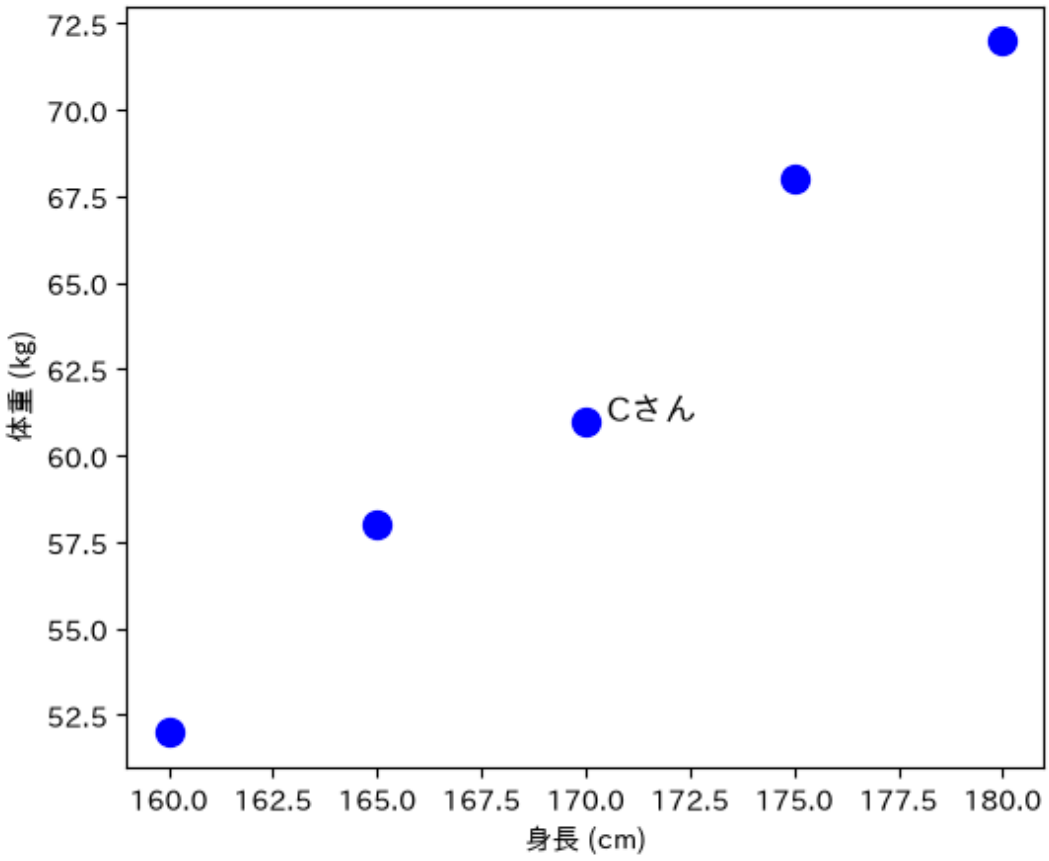


- 平均の線でグラフを4つに区切ると、「身長が平均より高い人は、体重も平均より重い（右上に集まる）」のように、**2つの量の関係**が見やすくなる
- `zorder=3` を付けたので、点が補助線より上に描かれている（上の「点と補助線の重なる順番」）

H-10 点に名前を付ける

- `plt.text(横の位置, 縦の位置, '文字')` で、グラフの**目盛りの位置**に文字を書ける

```
plt.figure(figsize=(6, 5))
plt.scatter(height_a, weight_a, s=100, color='b')
plt.text(170.5, 61, 'Cさん', fontsize=12)  # (170, 61) の点の少し右に書く
plt.xlabel('身長 (cm)')
plt.ylabel('体重 (kg)')
plt.show()
```



- 横の位置を点より少しずらす（170 → 170.5）と、文字が点に重ならない
- 右端の点に付けると、文字が枠の外にはみ出すことがある。そのときは `plt.xlim` で右を空ける
- 文字の色・大きさ・そろえ方は第 8 講 V-2
- 点が5つなら `plt.text` を5回、30個なら30回書くことになる（for 文を使うとまとめて書けるが、この講義ノート集では扱わない）

使ってみよう

- `plt.text` の行をもう1つ足して、(180, 72) の点に「Eさん」と付けてみよう。はみ出したら `plt.xlim(155, 190)` を足す

参考：折れ線と散布図のオプション比較

- 同じ働きでも、名前が違うオプションがある。名前の違うオプションを書くとエラーになる（例：散布図に `markersize=` 、折れ線に `s=` や `edgecolor=` と書くと `AttributeError` ）
- **太字**の行が、特に間違えやすいところ

やりたいこと	折れ線 `plt.plot`	散布図 `plt.scatter`	注意
関数	`plt.plot(x, y)`	`plt.scatter(x, y)`	
点の大きさ	`markersize=10` （ `ms=10` ）	`s=100`	名前も意味も違う。 `markersize` は幅、 `s` は面積で、 `s` はおよそ `markersize` の2乗。何も書かないときは `ms=6` 、 `s=36`

やりたいこと	折れ線 `plt.plot`	散布図 `plt.scatter`	注意
点ごとに大きさを変える	できない（全部同じ）	`s=リスト`	
点（マーカー）	何も書かないと点なし（線だけ）。`marker='o'` で付く	何も書かないと `'o'`	
マーカーの種類	`marker='s'`	`marker='s'`	同じ
色（全部同じ色）	`color='r'`（`c='r'`）	`color='r'`（`c='r'`）	同じ
点ごとに色を変える	できない	`color=リスト`（`c=リスト`）	
数値で色（カラーマップ）	できない	`c=数値のリスト, cmap='viridis'`	
点のふちの色	`markeredgecolor='k'`（`mec`）	`edgecolor='k'`	名前が違う
点の中の色	`markerfacecolor='w'`（`mfc`）	`facecolor='w'`	名前が違う。白抜きの点は、折れ線は `mfc='none'`、散布図は `facecolor='none'`（ふちの色も付ける）
点のふちの太さ	`markeredgewidth=2`（`mew`）	`linewidth=2`	散布図の `linewidth` は点のふちの太さ
点を結ぶ線	引く。`linestyle='none'` で消せる	引かない（引けない）	
線の種類・太さ	`linestyle='--'`（`ls`）、`linewidth=2`（`lw`）	使わない	散布図で書くと、点のふちの線に効く
まとめて指定	`'o--r'` のように書ける	できない	
透明度	`alpha=0.5`	`alpha=0.5`	同じ
凡例の名前	`label='…'`	`label='…'`	同じ。最後に `plt.legend()`

やりたいこと	折れ線 `plt.plot`	散布図 `plt.scatter`	注意
重なる順番	`zorder=`（何も書かないと 2）	`zorder=`（何も書かないと 1）	何もしないと、線（補助線を含む）が点の上に描かれる。点に `zorder=3` を付けると点が上になる（H-9）

同じ見た目（白抜きの点、ふちは黒）を、折れ線と散布図で書くと次のとおり。

```
x = [1, 2, 3, 4, 5, 6]
y = [2, 6, 3, 12, 10, 5]

# 折れ線
plt.plot(x, y, marker='o', markersize=10, markerfacecolor='none',
markeredgecolor='k', markeredgewidth=2)
plt.show()

# 散布図（大きさは 10 の2乗 = 100）
plt.scatter(x, y, marker='o', s=100, facecolor='none', edgecolor='k', linewidth=2)
plt.show()
```

I. 発展

I-1 np.array でまとめて計算する

- リストを `np.array(リスト)` にすると、**全部の要素にまとめて**計算できる（リストのままではできない）
- H-3 のバブルチャートで `np.array(age) * 10` と書いたのはこのため。リストのまま `age * 10` と書くと、10 倍ではなく「リストを10回くり返したリスト」になる
- `np.argmax` で、いちばん大きい要素の**番号**が分かる

```
score_np = np.array(score)
print(score_np + 5)
print(score_np / 100)

temp_np = np.array(temp)
print('いちばん暑い月', month[np.argmax(temp_np)])
```

```
[77 90 69 95 63 82 86 74]
[0.72 0.85 0.64 0.9  0.58 0.77 0.81 0.69]
いちばん暑い月 8
```

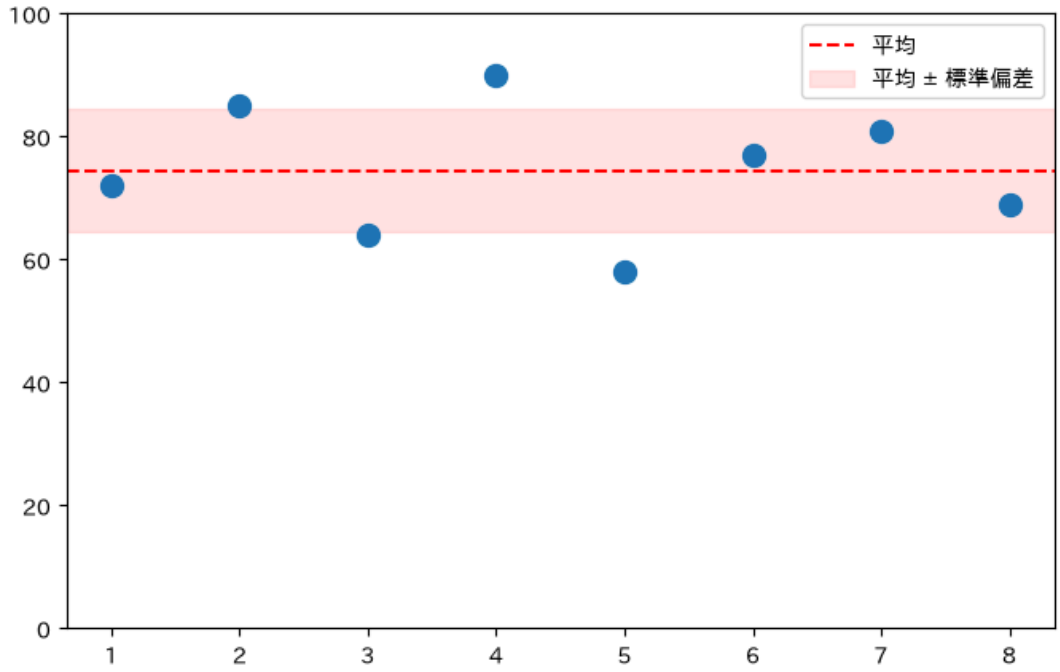
- リストに `+ 5` をするとエラーになる。試してみよう：`score + 5`
- 平均より上の人だけ取り出す：`score_np[score_np > np.mean(score_np)]`

I-2 平均 ± 標準偏差の範囲を塗る

- `plt.axhspan(下, 上)` で、横方向の帯を塗る。`alpha=` で薄くする

```
mean = np.mean(score)
std = np.std(score)
```

```
plt.figure(figsize=(8, 5))
plt.scatter(number, score, s=100, zorder=3)
plt.axhline(y=mean, color='r', linestyle='--', label='平均')
plt.axhspan(mean - std, mean + std, color='r', alpha=0.1, label='平均 ± 標準偏差')
plt.ylim(0, 100)
plt.legend()
plt.show()
```



まとめ

やりたいこと	書き方
線を消した折れ線（点だけ）	`plt.plot(x, y, marker='o', linestyle='none')`
平均	`np.mean(リスト)`（`sum(リスト) / len(リスト)` と同じ）
最大・最小・合計・中央値	`np.max`、`np.min`、`np.sum`、`np.median`
第一四分位数・第三四分位数	`np.percentile(リスト, 25)`、`np.percentile(リスト, 75)`
標準偏差（ばらつき）	`np.std(リスト)`
横の補助線・縦の補助線	`plt.axhline(y=値)`、`plt.axvline(x=値)`（`color=`、`linestyle=`、`linewidth=`、`label=` も使える）
以下は散布図	
グラフの大きさ（基本設定）	`plt.figure(figsize=(8, 5))`（`scatter` より前）
散布図	`plt.scatter(x, y)`
マーカーの種類	`marker='o'`（`'s'` `'^'` `'*'` `'D'` など）
色	`color='r'`（短く `c='r'`）

やりたいこと	書き方
点の大きさ	`s=100`（**面積**。`markersize` ではない）
点ごとの大きさ・色	`s=リスト`、`color=リスト`（点と同じ個数）
2つのグループ	`plt.scatter` を2回書く
重なる順番	`zorder=2`（大きいほど上。何も書かないと点は 1、線は 2）
透明度	`alpha=0.5`（0 透明〜1 不透明）
凡例	`label='...'` を付けて `plt.legend()`
凡例の場所	`plt.legend(loc='upper left')`、数値なら `loc=(0.05, 0.75)`
凡例の点の大きさ	`plt.legend(markerscale=0.3)`（グラフの点の 0.3 倍）
散布図に補助線	点に `zorder=3` を付けると、点が線の上になる
3つ目の値で大きさ（バブルチャート）	`s=np.array(age) * 10`
点に名前を付ける	`plt.text(横の位置, 縦の位置, '文字')`

第 5 講：グラフの保存・Excel の読み込み

作ったグラフを**画像ファイルに保存**してスライドに貼る方法（K）と、**Excel のデータを読んで**グラフにする方法（L）を学ぶ。第 6 講からは、データをこの方法で Excel から読む。

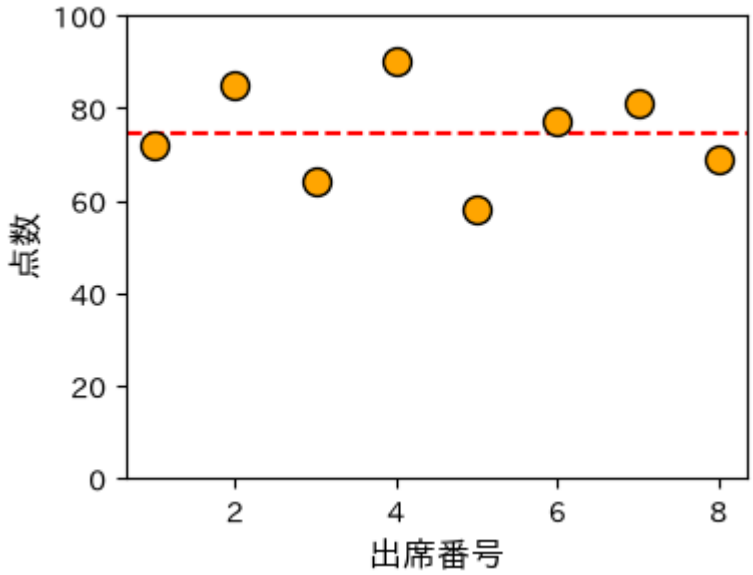
準備：この講で使うライブラリを読み込み、データを用意する。

```
import matplotlib.pyplot as plt
import japanize_matplotlib
import numpy as np

score = [72, 85, 64, 90, 58, 77, 81, 69]
number = [1, 2, 3, 4, 5, 6, 7, 8]

plt.figure(figsize=(4, 3))
plt.scatter(number, score, s=100, c='orange', edgecolor='k', zorder=3)
plt.axhline(y=np.mean(score), color='r', linestyle='--')
plt.ylim(0, 100)
plt.xlabel('出席番号', fontsize=12)
plt.ylabel('点数', fontsize=12)

# x軸目盛を明示的に指定
#plt.xticks([1, 2, 3, 4, 5, 6, 7, 8])  # plt.xticks(range(1,9)) でもよい
plt.show()
```



K. グラフの保存

レポートやスライドに使うには、グラフを**画像ファイル**にする。

K-1 画像ファイルに保存する

- `plt.savefig('ファイル名.png')` で保存する
- **`plt.show()` より前**に書く（後ろに書くと**白紙**が保存される）

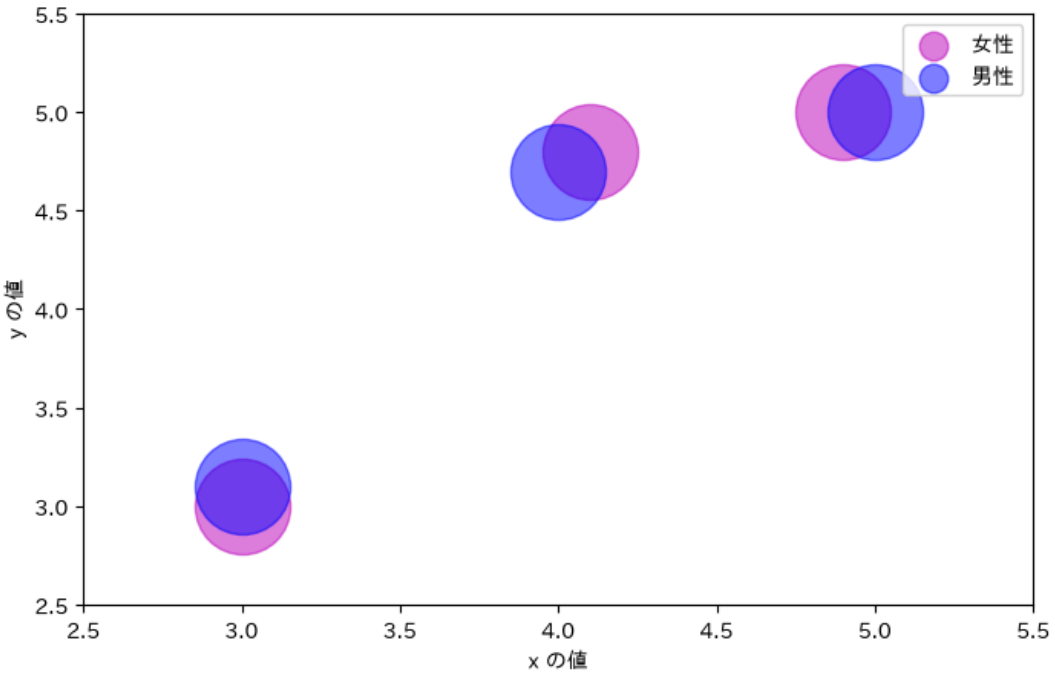
- 保存先は、この notebook と**同じ場所**（いつもの場所）。Jupyter のファイル一覧に出てくる
- 下のセルは、第 4 講 H-5 の2つのグループの散布図に、`plt.savefig` の行を足したもの
- 点を大きく（`s=2000`）すると、凡例の点も同じ大きさになり、グラフの点と重なりやすい。`plt.legend(markerscale=0.3)` で、凡例の点だけをグラフの点の 0.3 倍にしている（`loc=` と一緒に書くときは `plt.legend(loc='upper left', markerscale=0.3)`）

```
import matplotlib.pyplot as plt
import japanize_matplotlib

female_x = [3, 4.1, 4.9]
female_y = [3, 4.8, 5]
male_x = [3, 4, 5]
male_y = [3.1, 4.7, 5]

plt.figure(figsize=(8, 5))
plt.scatter(female_x, female_y, s=2000, color='m', alpha=0.5, label='女性')
plt.scatter(male_x, male_y, s=2000, color='b', alpha=0.5, label='男性')
plt.xlim(2.5, 5.5)
plt.ylim(2.5, 5.5)
plt.xlabel('x の値')
plt.ylabel('y の値')
plt.legend(markerscale=0.3)  # loc を指定しても重なる。 marker scale で凡例のマーカーサイズを指定可

plt.savefig('lec05_legend.png')
plt.show()
```



- ファイル一覧の `lec05_legend.png` をダブルクリックすると、保存した画像を見られる
- この場所が、notebook と同じ場所（**いつもの場所**）。L-1 では、ここに Excel のファイルを置く
- 同じファイル名で保存すると、**上書き**される（前の画像は消える）

K-2 細かさと背景の透明化

- `dpi=300` で細かく保存する（何も書かないと 100）。数が大きいほど細かく、ファイルも大きくなる
- `transparent=True` で背景を透明にする。色の付いたスライドに貼るときに使う

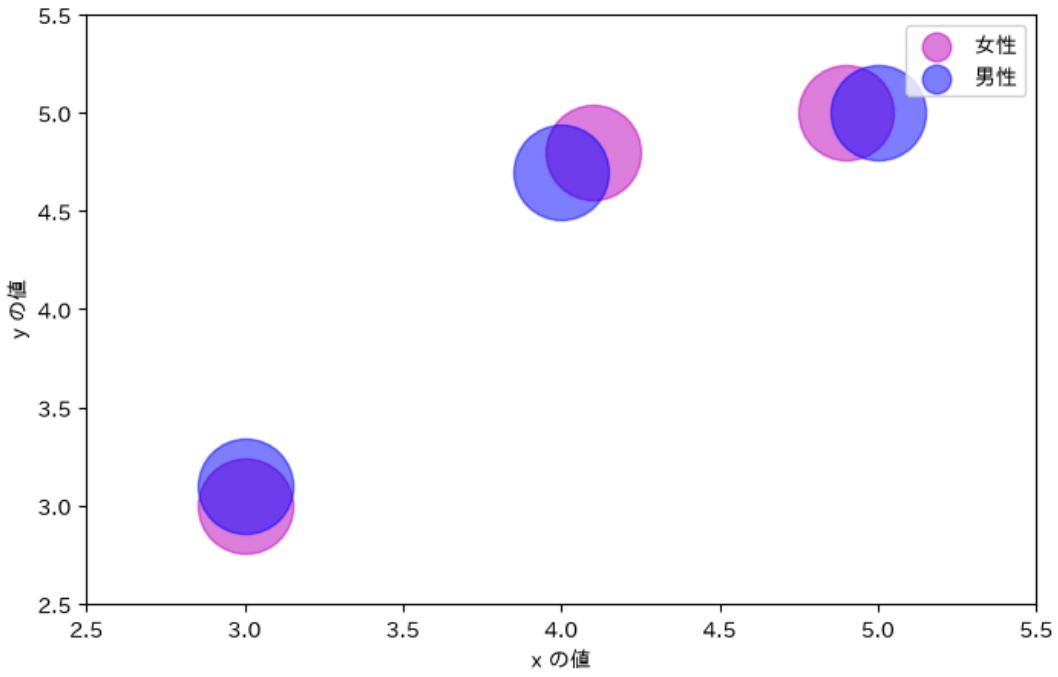
- 余白が大きいときは `bbox_inches='tight'` で余白を詰める
- 下の2つのセルは、K-1 のコードの `plt.savefig` の行を変えたもの

```
import matplotlib.pyplot as plt
import japanize_matplotlib

female_x = [3, 4.1, 4.9]
female_y = [3, 4.8, 5]
male_x = [3, 4, 5]
male_y = [3.1, 4.7, 5]

plt.figure(figsize=(8, 5))
plt.scatter(female_x, female_y, s=2000, color='m', alpha=0.5, label='女性')
plt.scatter(male_x, male_y, s=2000, color='b', alpha=0.5, label='男性')
plt.xlim(2.5, 5.5)
plt.ylim(2.5, 5.5)
plt.xlabel('x の値')
plt.ylabel('y の値')
plt.legend(markerscale=0.3)  # locを指定しても重なる。 marker scaleで凡例のマーカーサイズを指定可

# 解像度指定、見切れ防止を追加
plt.savefig('lec05_legend_dpi300.png', dpi=300, bbox_inches='tight')
plt.show()
```



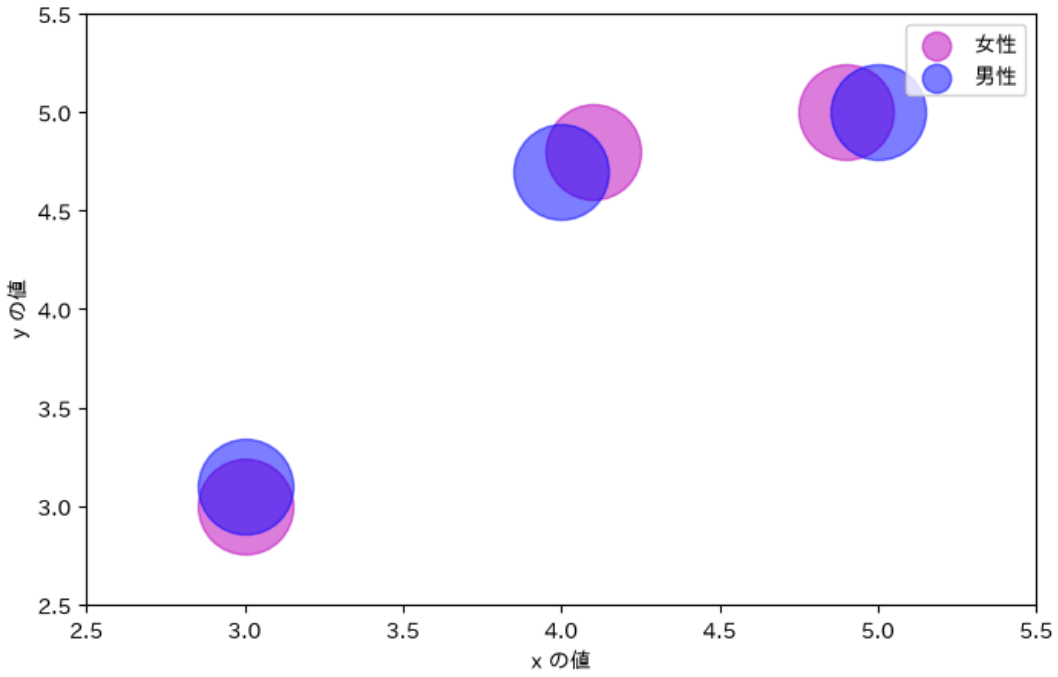

```
import matplotlib.pyplot as plt
import japanize_matplotlib

female_x = [3, 4.1, 4.9]
female_y = [3, 4.8, 5]
male_x = [3, 4, 5]
male_y = [3.1, 4.7, 5]

plt.figure(figsize=(8, 5))
plt.scatter(female_x, female_y, s=2000, color='m', alpha=0.5, label='女性')
plt.scatter(male_x, male_y, s=2000, color='b', alpha=0.5, label='男性')
plt.xlim(2.5, 5.5)
plt.ylim(2.5, 5.5)
plt.xlabel('x の値')
```

```
plt.ylabel('y の値')
plt.legend(markerscale=0.3)  # locを指定しても重なる。 marker scaleで凡例のマーカーサイズを指定可

# 透明化処理追加
plt.savefig('lec05_legend_透明.png', dpi=300, transparent=True, bbox_inches='tight')
plt.show()
```



K-3 保存の形式と、保存した図を直す

ファイル名の最後（**拡張子**）で形式が決まる。

種類	形式（拡張子）	向いている使い方
画像（ラスター）	`png`、`jpg`（`jpeg`）、`tif`（`tiff`）、`gif`、`webp`、`avif`	PowerPoint・Word・Web に貼る。**迷ったら png**
ベクター（拡大しても荒れない）	`svg`、`pdf`、`eps`、`ps`	後から図を直す。ポスター・論文に使う

- **ラスター**は写真と同じ「点（画素）の集まり」、**ベクター**は「ここからここへ線を引く」という情報。ベクターは拡大しても線がなめらか
- ベクターは **svg** を使う。eps は**透明が使えない**（`transparent=True` や `alpha=` が効かない）うえ、新しい PowerPoint には挿入できない
- 下のセルは、K-1 のコードで、ファイル名の最後を `.svg` にしたもの

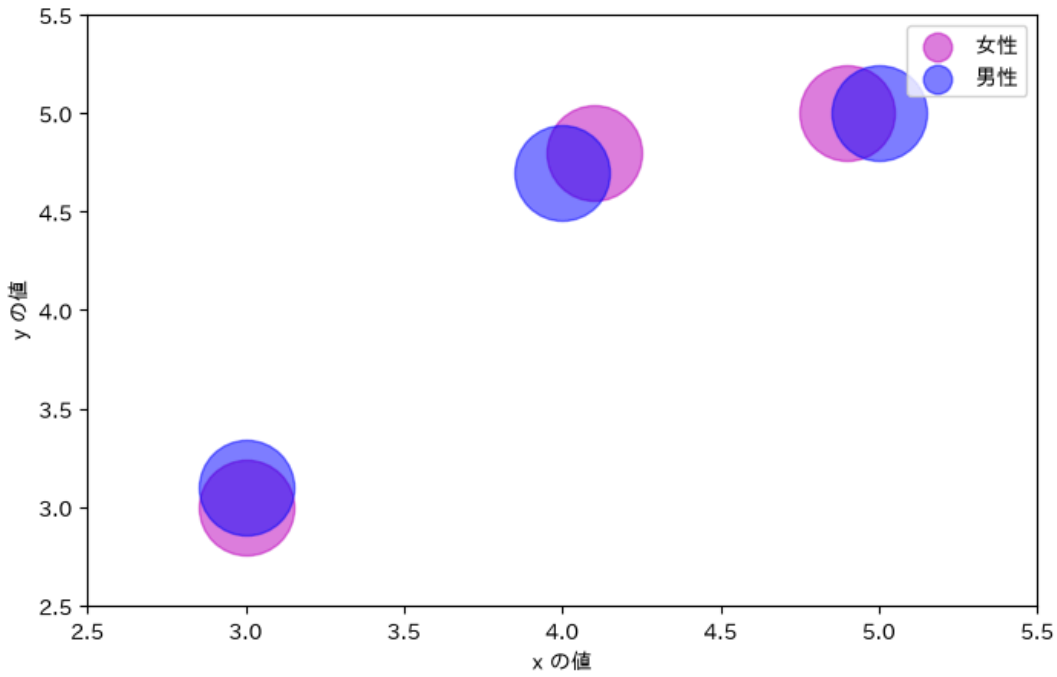
```
import matplotlib.pyplot as plt
import japanize_matplotlib

female_x = [3, 4.1, 4.9]
female_y = [3, 4.8, 5]
male_x = [3, 4, 5]
male_y = [3.1, 4.7, 5]

plt.figure(figsize=(8, 5))
plt.scatter(female_x, female_y, s=2000, color='m', alpha=0.5, label='女性')
plt.scatter(male_x, male_y, s=2000, color='b', alpha=0.5, label='男性')
```

```
plt.xlim(2.5, 5.5)
plt.ylim(2.5, 5.5)
plt.xlabel('x の値')
plt.ylabel('y の値')
plt.legend(markerscale=0.3)  # locを指定しても重なる。 marker scaleで凡例のマーカーサイズを指定可

# svg形式で保存
plt.savefig('lec05_legend.svg')
plt.show()
```



svg を PowerPoint で直す（例：y 軸の名前を縦書きにする）

1. 上のセルで `lec05_legend.svg` を保存する
2. PowerPoint で「挿入」→「画像」から svg を入れる
3. 図を右クリック →「**図形に変換**」
4. 元の y 軸の名前を消し、縦書きのテキストボックスで書き直す。線の色や太さも同じように直せる

- svg の文字は図形（線の集まり）になっているので、文字を打ち直すことはできない。消して書き直す

Illustrator で直す（全学契約で学生も使える）

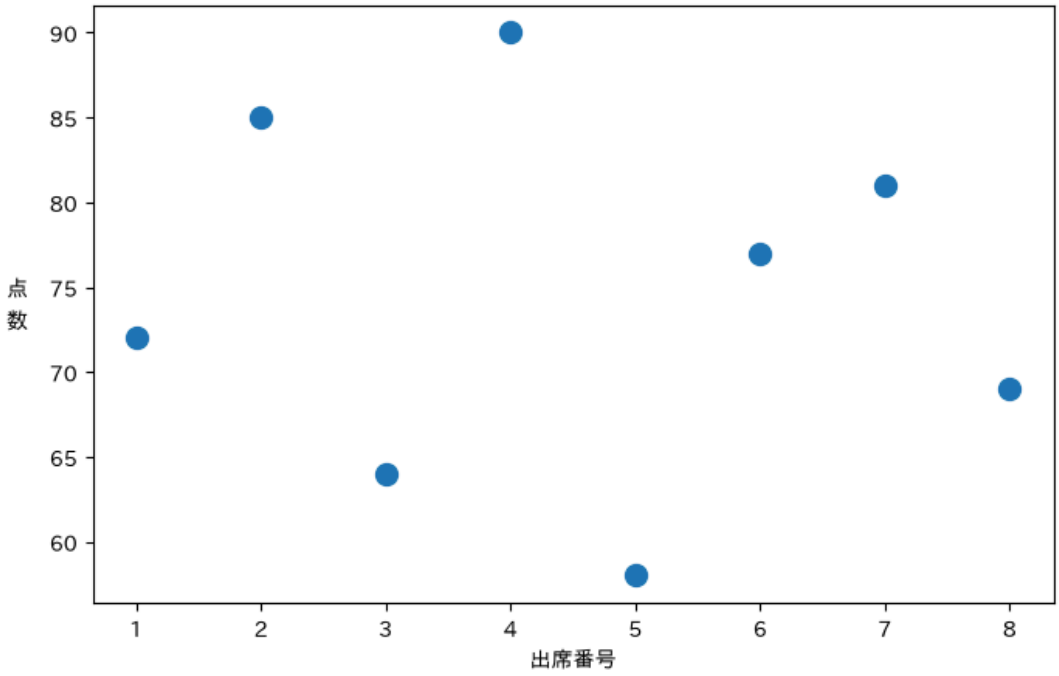
1. Illustrator で svg（または pdf）を「開く」
2. 全体がまとまっているので、「オブジェクト」→「グループ解除」、「オブジェクト」→「クリッピングマスク」→「解除」でばらす
3. 「ダイレクト選択ツール」で、線・点・文字を1つずつ選んで直せる
4. 元の y 軸の名前を消し、「文字（縦）ツール」で縦書きの名前を書く

- PowerPoint より細かく直せる（色をまとめて替える、点の形を変えるなど）。ポスターや論文の図に向く

matplotlib だけで縦書きにする（紹介のみ）

- 1文字ずつ**改行**して、縦に並べる力技
- `rotation=0` で文字を横向きのまま並べ、`labelpad=` で軸から離す。`linespacing=1.5` で行の間を少し空けると読みやすい

```
plt.figure(figsize=(8, 5))
plt.scatter(number, score, s=100)
plt.xlabel('出席番号')
plt.ylabel('点\n数', rotation=0, va='center', labelpad=15, linespacing=1.5)
plt.show()
```



- `'点\n数'` の `\n` が**改行**。縦書き用のかっこ `﹇` `﹈` を使うと、単位も縦に書ける：`'体\n重\n﹇\nkg\n﹈'`

制御文字（エスケープシーケンス）

`\` の後ろに文字を続けると、キーボードで直接打てない文字を表せる。

書き方	意味	例
`\n`	改行	`print('1行目\n2行目')`、`plt.title('身長と体重\n（Aクラス）')`
`\t`	タブ（字下げ）	`print('名前\t点数')`
`\\`	`\` そのもの	`print('C:\\Users')`
`\'`	`'` そのもの（`'…'` の中で使う）	`print('It\'s')`

- `\` の打ち方：Windows は `¥` キー（画面では `¥` に見えることがあるが同じ文字）、Mac は `option` + `¥`
- `print` で試してから、グラフのタイトルや軸の名前で使う（2行のタイトルなど）

```
print('1行目\n2行目')
print('名前\t点数')
print('C:\\Users')
print('It\'s')
```

```
1行目
2行目
名前　　点数
C:\Users
It's
```

K-4 PowerPoint に貼る

1. PowerPoint を開き、「挿入」→「画像」→「このデバイス」（Mac は「画像」→「ファイルからの画像」）
2. いつもの場所（ホームフォルダ）にある `lec05_legend.png` を選ぶ
3. ドラッグして貼ってもよい（Windows はエクスプローラー、Mac は Finder から）

- 画像の大きさは PowerPoint 上で変えられるが、**文字が小さすぎるとき**は notebook に戻って `fontsize=` を大きくして保存し直す
- 画面をスクリーンショットして貼るより、`savefig` で保存したほうがきれい

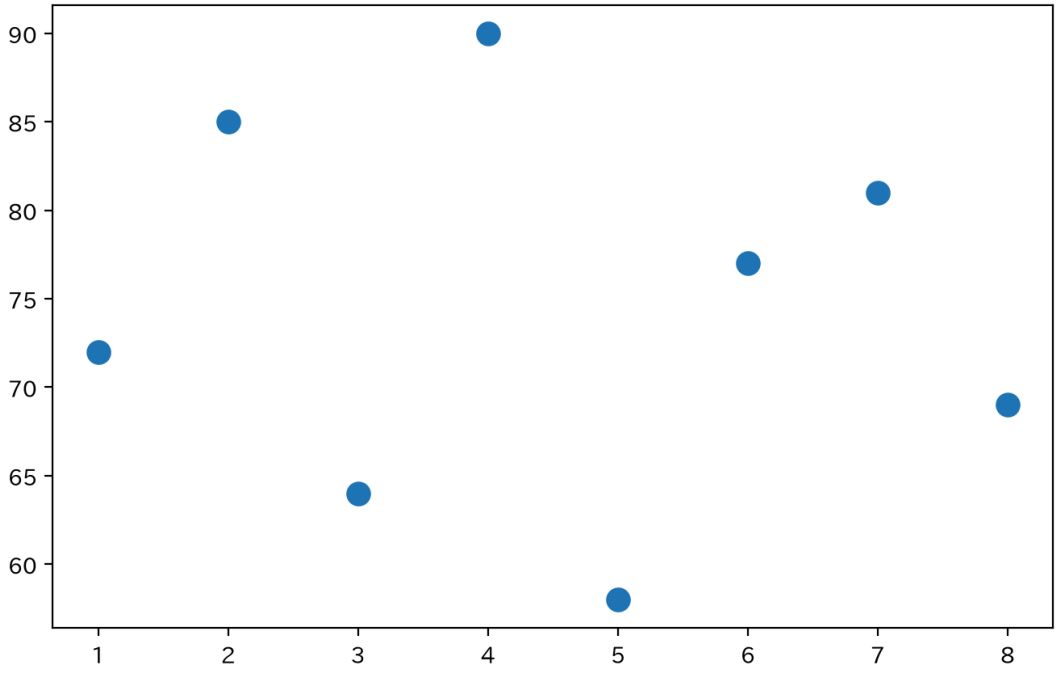
よくある失敗

起きたこと	原因	直し方
保存した画像が真っ白	`savefig` を `show` の**後ろ**に書いた	`savefig` を `show` の前に移す
画像がどこにあるか分からない	保存先は notebook と同じ場所（いつもの場所）	Jupyter のファイル一覧を見る
文字が □ になる	`import japanize_matplotlib` を実行していない	最初の import のセルを実行する
文字が切れている	余白が足りない	`bbox_inches='tight'` を付ける

K-5 画面の表示をきれいにする（retina）

- notebook の中のグラフは、そのままだと少しぼやけて見える（特に Mac や高解像度の画面）
- 次の1行を実行すると、それ以降のグラフが**2倍の細かさ**で表示される。最初の import のセルに入れておくとよい
- これは**画面の表示だけ**の設定。保存する画像の細かさは `savefig` の `dpi=` で決める（別物）

```
%config InlineBackend.figure_format = 'retina'
plt.figure(figsize=(8, 5))
plt.scatter(number, score, s=100)
plt.show()
```



L. Excel のデータ

ここから、グラフにするデータを**Excel ファイルから読む**。Excel のような表は **pandas**（`import pandas as pd`）で扱う。毎回この形：

1. Excel ファイルを **notebook と同じ場所**（いつもの場所＝ホームフォルダ）に置く
2. `pd.read_excel('ファイル名', sheet_name='シート名')` で読む
3. `df['列名']` で1列取り出して、`plt.plot` などに渡す

L-1 Excel をいつもの場所に置く

[1] いつもの場所（ホームフォルダ）とは

- K-1 で保存した `lec05_legend.png` が出てきた場所。Jupyter のファイル一覧に最初に表示される場所。何もしなければ、ipynb はここに保存される
- Windows は `C:\Users\自分の名前`、Mac は家のアイコンのフォルダ
- notebook もここに作る

[2] `dsa2026data.xlsx` をいつもの場所に置く

1. `dsa2026data.xlsx` をダウンロードする（下の「置き場が分からないとき」の1行でもよい）
2. Windows はエクスプローラー（Win＋E）、Mac は Finder で、「ダウンロード」フォルダから、ホームフォルダへドラッグする
   - Windows でホームフォルダが見つからない場合は、エクスプローラー上部のアドレスバーに `%USERPROFILE%` と入力して Enter を押す
   - Mac で Finder の左のサイドバーに家のアイコン（ホームフォルダ）が出ていない場合は、Finder メニューの「設定」→「サイドバー」で、自分の名前のホームアイコンにチェックを入れる

- ファイル名は**変えない**（コードの中のファイル名と1文字でも違うと読めない）

- 中身を Excel で開いて見てもよい。ただし、**開いたままだと読めないことがある**（Windows）ので、見終わったら閉じる

置き場が分からないとき：次の1行をセルに貼って実行すると、`dsa2026data.xlsx` がダウンロードされ、notebook と同じ場所に置かれる（インターネットにつながっている必要がある）

```
!curl -O https://raw.githubusercontent.com/CreativeStudies-NGT/lectures/main/dsa/dsa2026data.xlsx
```

[3] Jupyter のファイル一覧に `dsa2026data.xlsx` が見えれば準備完了

参考：授業用フォルダを作って整理する

ファイルが増えてきたら、授業用のフォルダ（例：`dsa`）を作ってまとめると探しやすい。やりたい人だけでよい。

1. Jupyter のファイル一覧で「New」→「新規フォルダ」（英語表示では「New Folder」）を選び、名前を `dsa` に変える
2. エクスプローラー（Mac は Finder）で、ホームフォルダの ipynb と `dsa2026data.xlsx` を、**両方とも** `dsa` へドラッグする
3. Jupyter で `dsa` をダブルクリックして中に入り、「New」で notebook を作る
4. デスクトップにショートカットを作ると、ファイルを探すときにすぐたどり着ける
    - Windows：ホームフォルダで `dsa` を右クリック →（Windows 11 は「その他のオプションを確認」）→「送る」→「デスクトップ（ショートカットを作成）」
    - Mac：Finder でホームフォルダを開く → `dsa` を **option + command を押しながら**デスクトップへドラッグ

- **notebook と Excel ファイルは、必ず同じフォルダに置く。**片方だけ移すと `FileNotFoundError` になる（L-4）
- フォルダ名やファイル名に日本語が入っていても大丈夫（ユーザー名のフォルダだけは日本語を含まないこと）

L-2 Excel ファイルを読む

- `dsa2026data.xlsx` の中には**シート**（Excel の画面下のタブ）がいくつもある
- `sheet_name=` で読むシートを選ぶ。まずは12か月の降水量と気温が入った `降水量` シートを読む
- 読み込んだ表は、名前 `df`（data frame＝データの表）に入れるのが習慣

```
import pandas as pd

df = pd.read_excel('dsa2026data.xlsx', sheet_name='降水量')
df
```

	月	降水量	気温
0	1	180.0	4.3
1	2	114.5	4.8
2	3	168.0	6.0
3	4	54.0	13.7
4	5	128.0	17.1
5	6	57.0	22.2
6	7	376.5	26.2
7	8	42.0	28.0
8	9	292.0	25.0
9	10	144.5	18.7
10	11	246.5	11.5
11	12	262.5	4.9

- 表が出れば成功。いちばん左の `0, 1, 2, …` は pandas が付けた**行の番号**（Excel のデータではない）
- 1行目（太字）が**列名**。Excel ファイルの1行目がそのまま列名になる
- 行が多い表は `df.head()` で**最初の5行だけ**見る

```
import pandas as pd

df_ramen = pd.read_excel('dsa2026data.xlsx', sheet_name='ramen')
df_ramen.head()
```

	city	Y2021	Y2022	Y2023	Y2024	Y2025
0	札幌市	6041	5722	6533	6972	9376
1	青森市	7875	9487	9660	8982	10630
2	盛岡市	8462	7346	10789	11729	11215
3	仙台市	8067	12480	13074	15534	14013
4	秋田市	8544	10015	8777	12375	14605

- `ramen` シートは、都市ごとのラーメンの外食費（1世帯あたり、円）
- 表を2つ以上使うときは、`df_ramen` のように名前を変えて区別する

L-3 列を取り出す

- `df['列名']` で1列だけ取り出せる。**列名は表に表示されたとおりに**書く（全角・半角、空白も同じに）
- 取り出した列は、**リストと同じように**グラフに渡したり、平均を計算したりできる

```
df['降水量']
```

```
0     180.0
1     114.5
2     168.0
3      54.0
4     128.0
5      57.0
6     376.5
7      42.0
8     292.0
9     144.5
10    246.5
11    262.5
Name: 降水量, dtype: float64
```

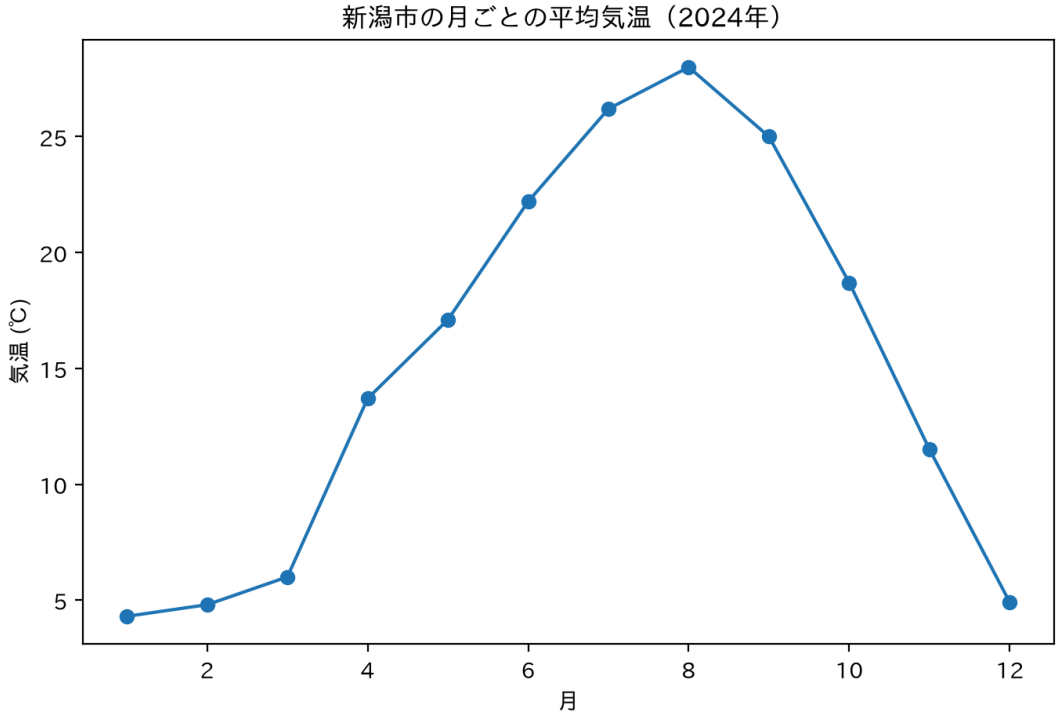
```python
import numpy as np

print('列名の一覧', list(df.columns))
print('降水量の平均', np.mean(df['降水量']))
```

```
列名の一覧 ['月', '降水量', '気温']
降水量の平均 172.125
```

- `list(df.columns)` で列名の一覧が分かる。列名が分からないときに使う
- 取り出した列で、すぐにグラフが描ける（第 3 講の折れ線グラフ）

```python
plt.figure(figsize=(8, 5))
plt.plot(df['月'], df['気温'], marker='o')
plt.title('新潟市の月ごとの平均気温（2024年）')
plt.xlabel('月')
plt.ylabel('気温（℃）')
plt.show()
```



使ってみよう

- 上のセルの `'気温'` を `'降水量'` に、`plt.ylabel` を `'降水量（mm）'` に変えて描いてみよう

- `ramen` シートの `df_ramen['Y2024']` と `df_ramen['Y2025']` で散布図（第 4 講の `plt.scatter` ）を描いてみよう

L-4 よくあるエラー

エラーの最後の行	原因	直し方
`FileNotFoundError: ... 'dsa2026data.xlsx'`	ファイルが notebook と同じ場所にない、またはファイル名の打ち間違い	L-1 の [2] をやり直す。Jupyter のファイル一覧に出ているか確かめる。置き場が分からなければ、L-1 の「置き場が分からないとき」の1行を実行する
`FileNotFoundError` （ファイルは置いてある）	notebook と Excel ファイルが別のフォルダにある（L-1 の参考でフォルダを作ったとき）	両方を同じフォルダに入れる
`ValueError: Worksheet named '…' not found`	シート名の打ち間違い	Excel の画面下のタブの名前と同じにする
`KeyError: '…'`	列名の打ち間違い（全角・半角、空白）	`list(df.columns)` で列名を確かめ、コピーして貼る
`NameError: name 'pd' is not defined`	`import pandas as pd` を実行していない	そのセルの先頭に `import pandas as pd` を書いて実行する
`PermissionError`	Excel でそのファイルを開いたまま（Windows）	Excel を閉じてから、もう一度実行する

CSV ファイルをもらったとき

- Excel ファイル（ `.xlsx` ）は、日本語の列名やデータが**文字化けしない**
- CSV ファイル（ `.csv` 、e-Stat などからダウンロードしたデータに多い）は、そのまま Python で読むと**文字化けやエラー**になることがある
- そこで、**CSV は Excel で開いて「Excel ブック（.xlsx）」で保存し直し、`read_excel` で読む**のがいちばん簡単。こうすれば文字化けの問題は起きない
  - Windows・Mac とも：Excel で開く →「ファイル」→「名前を付けて保存」→ ファイルの形式を「Excel ブック（.xlsx）」にする
  - Excel で開いた時点で文字化けしているときは、Excel の「データ」→「テキストまたは CSV から」で読み込み、「元のファイル」（文字コード）を「65001: Unicode (UTF-8)」にしてから保存する
- 整っていない Excel（表の上に説明の行がある など）や、CSV を Python で直接読む方法は、下の「補足」にまとめた

補足：整っていない Excel を読む

- 第 5 講のデータ（ `dsa2026data.xlsx` ）は「1行目が列名、1列に1項目」に整えてある。自分で集めたデータがそうなっていないときは、 `pd.read_excel` にオプションを付けて読む

Excel の状態	書き方
先頭に表題や説明の行がある	`skiprows=3` （先頭の3行を飛ばす）
列名の行がない	`header=None` （列名は 0, 1, 2, … になる）
シートが何枚もあり、全部読みたい	`sheet_name=None` （シート名をキーにした辞書になる）
シート名ではなく順番で選びたい	`sheet_name=0` （一番左のシート）

```
df = pd.read_excel('ファイル名.xlsx', sheet_name='シート名', skiprows=3)
```

- 迷ったら、Excel で開いて、1行目が列名になるように余分な行を消してから保存し直すのが一番簡単

補足：CSV を Python で直接読む（文字化けしたとき）

- CSV は文字コード（文字を数字に置き換える約束）がファイルによって違い、日本語が読めないことがある。日本語版の Excel で保存した CSV や e-Stat の CSV は Shift_JIS（Windows では cp932）
- `pd.read_csv('ファイル名.csv')` は、何も書かないと UTF-8 として読むので、Shift_JIS のファイルではエラーになる

困ったこと	原因	直し方
`read_csv` で `UnicodeDecodeError: 'utf-8' codec …`	CSV が Shift_JIS	`pd.read_csv('ファイル名.csv', encoding='cp932')`
`encoding='cp932'` でも `UnicodeDecodeError: 'cp932' codec …`	CSV が UTF-8	`encoding=` を外す（または `encoding='utf-8'`）
Python で書き出した CSV を Excel で開くと文字化け	Excel は UTF-8 の CSV を正しく判別できないことがある	書き出すときに `df.to_csv('ファイル名.csv', encoding='utf-8-sig', index=False)`
グラフの日本語が □□□ になる	文字コードではなくフォントの問題	`import japanize_matplotlib` を実行する

- `shift_jis` ではなく `cp932` と書くのは、①や髙など Windows で増えた文字も読めるようにするため

L-5 Excel のデータで散布図

- `exam` シートは、30人の5教科（国語・数学・理科・社会・英語）の点数
- 「Excel を読む → 列を取り出す → 描く」の3段で、第 4 講の散布図がそのまま描ける

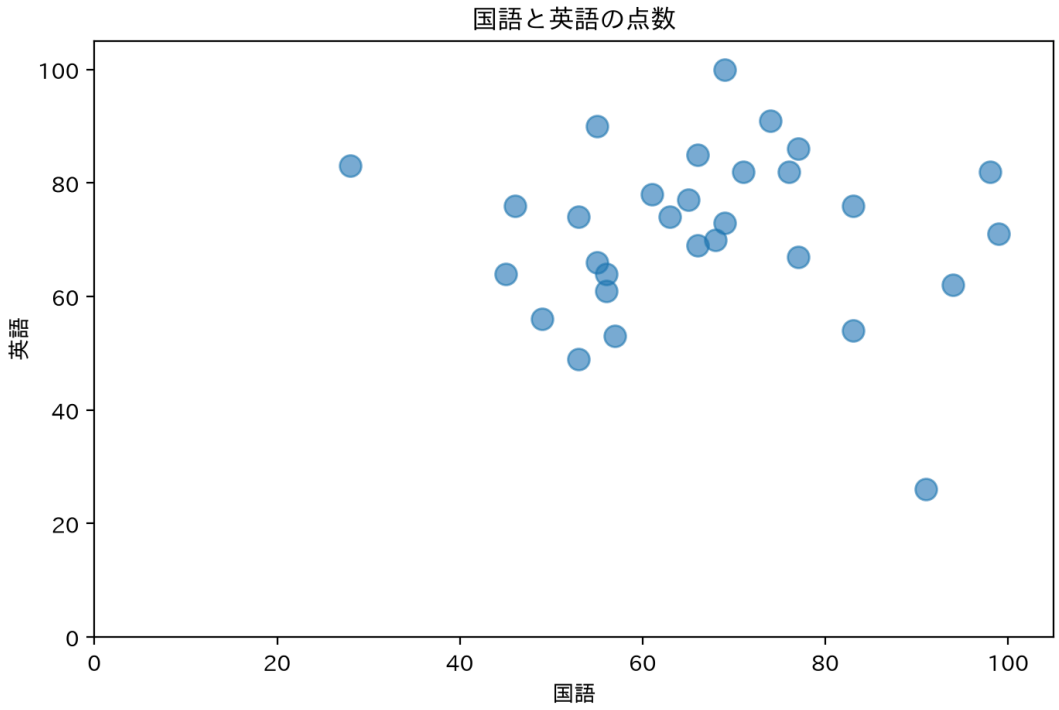
```
import pandas as pd

df_exam = pd.read_excel('dsa2026data.xlsx', sheet_name='exam')
df_exam.head()
```

	name	国語	数学	理科	社会	英語
0	伊藤 涼介	94	76	62	81	62
1	上田 美咲	63	57	59	80	74
2	木村 太郎	55	66	82	78	66
3	佐藤 亜美	61	87	80	65	78
4	鈴木 恵美子	56	58	69	97	61

```
plt.figure(figsize=(8, 5))
plt.scatter(df_exam['国語'], df_exam['英語'], s=100, alpha=0.6)
plt.xlim(0, 105)
plt.ylim(0, 105)
```

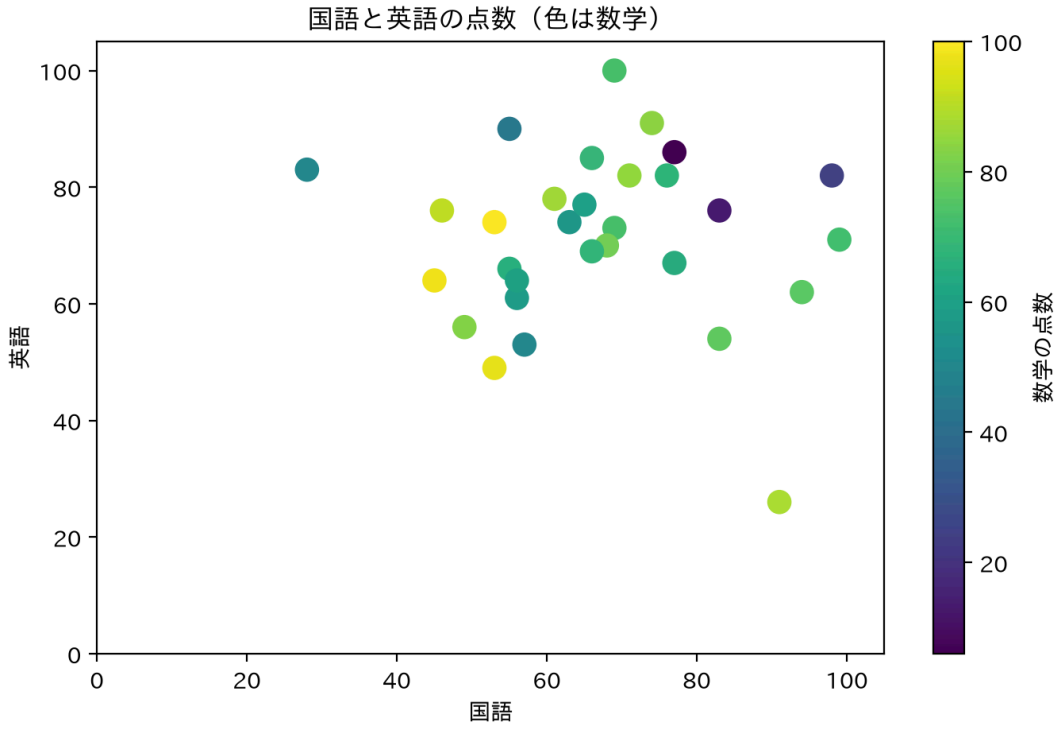
```
plt.title('国語と英語の点数')
plt.xlabel('国語')
plt.ylabel('英語')
plt.show()
```



3つ目の教科を色で見せる（カラーマップ）

- 第 4 講 H-3 では3つ目の値を**点の大きさ**で見せた。**色**で見せることもできる
- `c=` に**数値のリスト**（ここでは3つ目の列 `df_exam['数学']`）、`cmap=` に色の並び（**カラーマップ**）を指定すると、数値に応じて色が自動で付く
- `plt.colorbar()` で、どの色がどの数値かを示す目盛りが付く。`label=` で目盛りに名前を付ける
- `color=` には**色の名前**のリスト、`c=` に**数値**のリストを渡すとカラーマップになる
- 末尾に `_r` を付けると（例：`'viridis_r'`）、色の並びが逆になる

```
plt.figure(figsize=(8, 5))
plt.scatter(df_exam['国語'], df_exam['英語'], s=100, c=df_exam['数学'], cmap='viridis')
plt.colorbar(label='数学の点数')
plt.xlim(0, 105)
plt.ylim(0, 105)
plt.title('国語と英語の点数（色は数学）')
plt.xlabel('国語')
plt.ylabel('英語')
plt.show()
```



使ってみよう

- 横軸・縦軸・色の教科を入れ替えて描いてみよう（例：数学と理科、色は社会）
- `cmap='viridis'` を `'coolwarm'`（青→赤）や `'Reds'`（白→赤）に変えてみよう
- 末尾に `_r` を付けて（例：`'viridis_r'`）、色の並びを逆にしてみよう

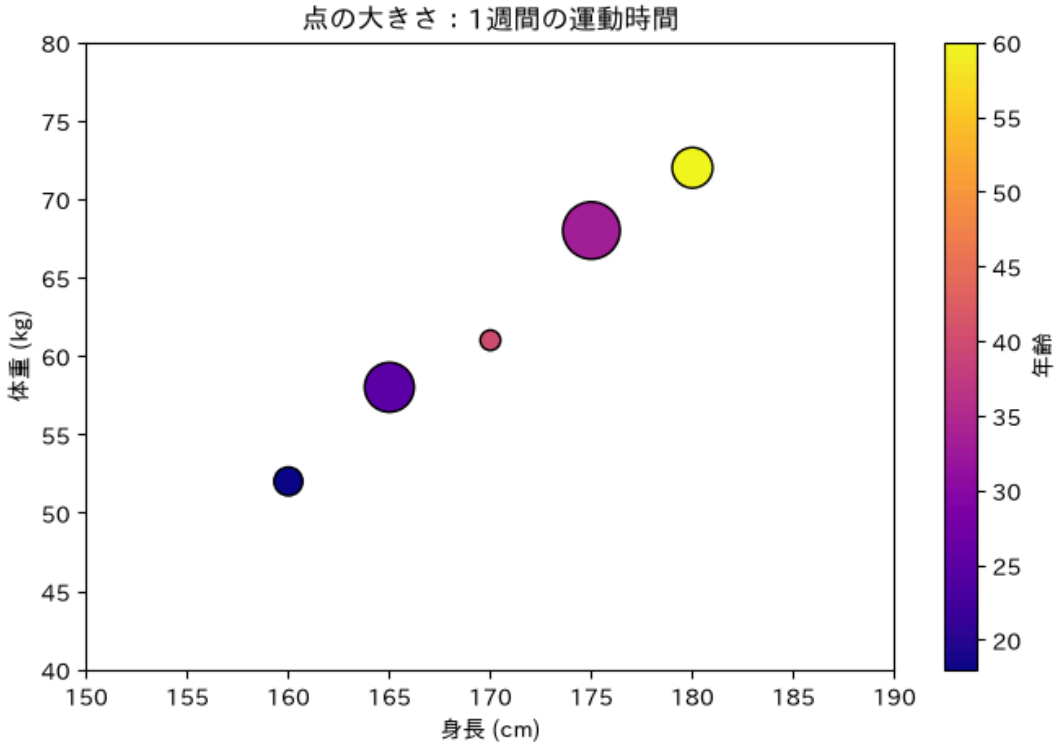
大きさと色の両方を使う

- 点の**大きさ**（第 4 講 H-3 のバブルチャート）と**色**の両方を使うと、x・y に加えて**4つ目の情報**まで見せられる
- 下のセルは、身長と体重の散布図に、年齢を**色**、1週間の運動時間（`exercise`）を**大きさ**で加えたもの。大きさは第 4 講 H-3 と同じく `np.array(…) * 数` で決める

```
import numpy as np
import matplotlib.pyplot as plt
import japanize_matplotlib

height_a = [160, 165, 170, 175, 180]
weight_a = [52, 58, 61, 68, 72]
age = [18, 25, 40, 33, 60]
exercise = [2, 6, 1, 8, 4]  # 1週間の運動時間（時間）
ex_size = np.array(exercise) * 80

plt.figure(figsize=(8, 5))
plt.scatter(height_a, weight_a, s=ex_size, c=age, cmap='plasma', edgecolor='k')
plt.colorbar(label='年齢')
plt.xlim(150, 190)
plt.ylim(40, 80)
plt.title('点の大きさ：1週間の運動時間')
plt.xlabel('身長 (cm)')
plt.ylabel('体重 (kg)')
plt.show()
```



L-6 ipynb ファイルを人と交換する

- Excel ファイルと同じく、notebook（ `.ipynb` ）も**ファイルとして人に渡したり、もらったり**できる。コードの共有などで使う
- もらった ipynb は、自分の notebook と同じように **Jupyter のファイル一覧から開いて**、実行できる
- 中身は、コードと、保存したときの**出力**（表やグラフ）。開いてすぐ出力が見えるが、もう一度実行すると自分のパソコンで計算し直す

やってみよう：講義の notebook を開く

[1] 講義の notebook をダウンロードする

- 配布された講義の notebook（例： `DSA2026_第４講_散布図.ipynb` ）をダウンロードする
- クリックしても文字だらけの画面が出てダウンロードされないときは、リンクを**右クリック**→「名前を付けてリンク先を保存」（Mac の Safari は「リンク先のファイルをダウンロード」）

[2] いつもの場所に入れる

- Windows はエクスプローラー、Mac は Finder で、「ダウンロード」フォルダから、いつもの場所（ホームフォルダ。Windows は `C:\Users\自分の名前` ）へドラッグする
- 同じ名前のファイルがすでにあると「置き換えますか」と聞かれる。自分の notebook を消さないように、**置き換えない**（スキップする）

[3] 開いて実行する

1. Jupyter のファイル一覧で、ダウンロードした ipynb をクリックして開く
2. 上のメニューの「Run」→「Run All Cells」で、全部のセルを上から実行する
3. グラフが出れば成功。数値を変えて、もう一度実行してみよう（もらったファイルなので、自由に書き換えてよい）

うまくいかないとき

起きたこと	原因	直し方
ファイル名の最後が `.ipynb.txt` や `.ipynb.json` になっている	ブラウザが名前を付け直した	Jupyter のファイル一覧で右クリック →「Rename」で、最後を `.ipynb` にする
ファイル名に `(1)` が付いている	同じファイルを2回ダウンロードした	古いほうを消すか、そのまま使ってよい
ダブルクリックしても開かない	ipynb は Jupyter から開くファイル	Jupyter のファイル一覧から開く
`FileNotFoundError`	notebook が読む Excel ファイルが、同じフォルダにない	`dsa2026data.xlsx` を notebook と同じ場所に置く（L-1 の [2]、L-4）
`ModuleNotFoundError: No module named 'japanize_matplotlib'`	相手のパソコンに入っていない	第 3 講のインストールのセルを実行する

人に渡すときの約束

- notebook が Excel などのデータを読むときは、**データのファイルも一緒に**渡す。ファイル名・シート名は変えない
- 渡す前に「Run」→「Run All Cells」で、**上から全部実行してエラーが出ない**ことを確かめる（途中のセルだけ実行した状態だと、相手のところで動かないことがある）
- 渡したい notebook は、Jupyter のファイル一覧で右クリック →「Download」でも取り出せる。メールなどで送る

発展

np.array・範囲を塗る発展は第 4 講 I-1・I-2。

K-6 スタイルシート

- `with plt.style.context('名前'):` の下に字下げして書くと、そのグラフだけ見た目がまとめて変わる
- 使える名前は `plt.style.available` で一覧できる
- 日本語が □ になるスタイルがある。そのときは `japanize_matplotlib.japanize()` を `with` の中で実行する

```
print(plt.style.available)
```

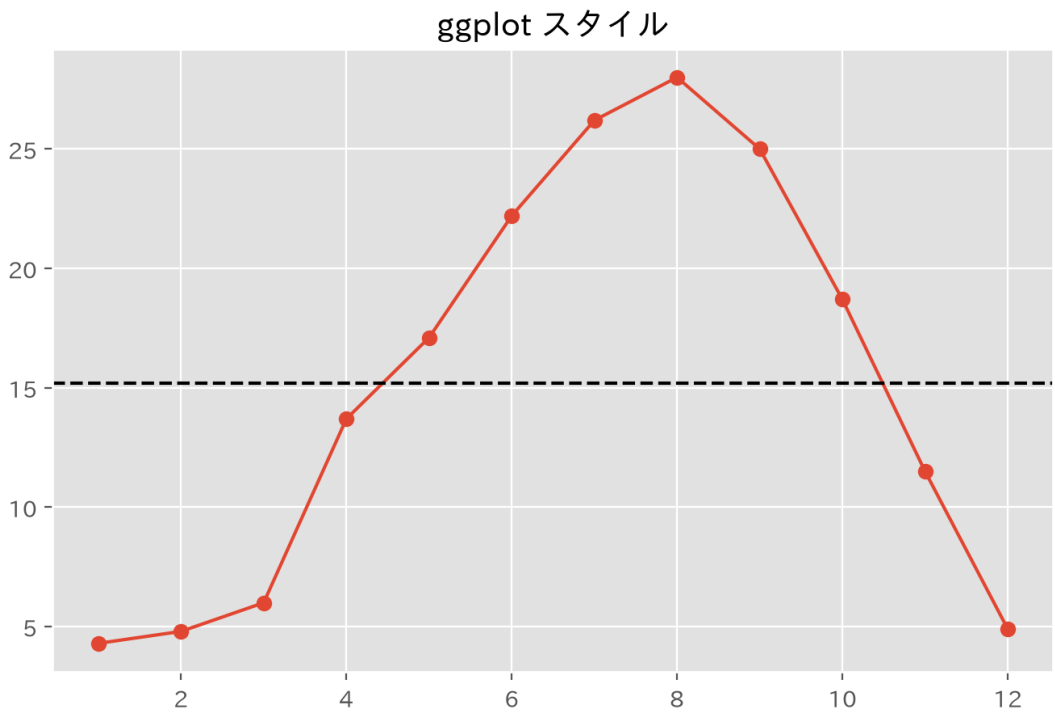
```
['Solarize_Light2', 'bmh', 'classic', 'dark_background', 'fast', 'fivethirtyeight', 'ggplot',
'grayscale', 'petroff10', 'petroff6', 'petroff8', 'seaborn-v0_8', 'seaborn-v0_8-bright', 'sea
born-v0_8-colorblind', 'seaborn-v0_8-dark', 'seaborn-v0_8-dark-palette', 'seaborn-v0_8-darkgr
id', 'seaborn-v0_8-deep', 'seaborn-v0_8-muted', 'seaborn-v0_8-notebook', 'seaborn-v0_8-pape
r', 'seaborn-v0_8-pastel', 'seaborn-v0_8-poster', 'seaborn-v0_8-talk', 'seaborn-v0_8-ticks',
'seaborn-v0_8-white', 'seaborn-v0_8-whitegrid', 'tableau-colorblind10']
```

```
import numpy as np

month = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12]
temp = [4.3, 4.8, 6.0, 13.7, 17.1, 22.2, 26.2, 28.0, 25.0, 18.7, 11.5, 4.9]

with plt.style.context('ggplot'):
    japanize_matplotlib.japanize()
```

```
    plt.figure(figsize=(8, 5))
    plt.plot(month, temp, marker='o')
    plt.axhline(y=np.mean(temp), color='k', linestyle='--')
    plt.title('ggplot スタイル')
    plt.show()
```



まとめ

やりたいこと	書き方
保存	`plt.savefig('名前.png', dpi=300, bbox_inches='tight')`（**`plt.show()` の前**）
背景を透明に	`transparent=True`
余白を詰める	`bbox_inches='tight'`
後から直せる形式	`plt.savefig('名前.svg')` → PowerPoint の「図形に変換」や Illustrator で直す
改行	文字の中に `\n`（`'1行目\n2行目'`）
画面の表示をきれいに	`%config InlineBackend.figure_format = 'retina'`
Excel を読む	`df = pd.read_excel('dsa2026data.xlsx', sheet_name='シート名')`（ファイルは notebook と同じ場所に置く）
最初の5行を見る	`df.head()`
列を取り出す	`df['列名']`（列名の一覧は `list(df.columns)`）
もらった ipynb を開く	Jupyter のファイル一覧から開き、「Run」→「Run All Cells」（データのファイルも同じフォルダに）
3つ目の列を色で（カラーマップ）	`plt.scatter(df['列1'], df['列2'], c=df['列3'], cmap='viridis')` → `plt.colorbar(label='名前')`

第 6 講：棒グラフ・円グラフ

第 5 講 L の「Excel を読む → 列を取り出す → 描く」の流れで、項目ごとの量を比べる**棒グラフ**（N）と、全体に対する割合を見せる**円グラフ**（O）を描く。どちらも、最初に完成形と部品表を見てから、部品を1つずつ学ぶ。

N. 棒グラフ

棒グラフは、**項目ごとの量**（月ごとの降水量、曜日ごとの売上など）を棒の高さで比べるグラフ。

N-0 完成形

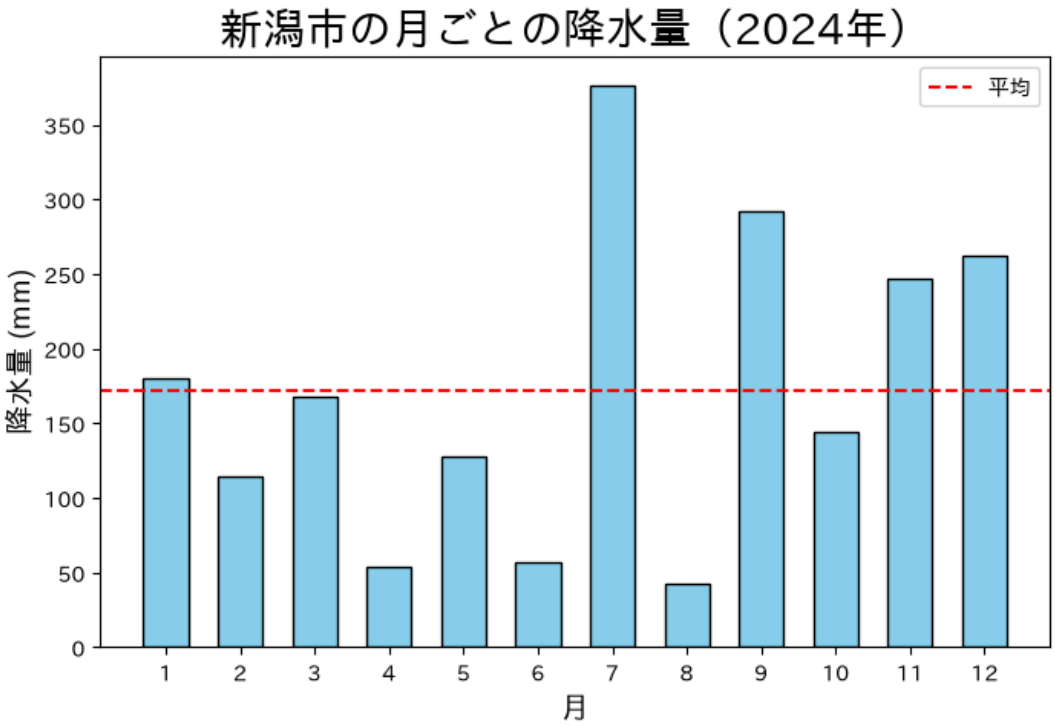
このグラフを目指す（新潟市の2024年の月ごとの降水量）。

1. **全部コピーして**自分の notebook のセルに貼り付け、実行する（`Shift + Enter`）
2. まずは**数値や文字を変えて**、グラフのどこが変わるかを確かめる
3. そのあと1行ずつ分解して、各部品（[0]〜[7]）の働きを学ぶ（N-1〜N-7）

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import japanize_matplotlib

df = pd.read_excel('dsa2026data.xlsx', sheet_name='降水量')  # Excel を読む
x = df['月']  # 列を取り出す
y = df['降水量']

plt.figure(figsize=(8, 5))  # [0] グラフの大きさ
plt.bar(x, y, color='skyblue', width=0.6, edgecolor='k')  # [1] 棒 [2] 色 [3] 幅 [4] 外枠線
plt.axhline(y=np.mean(y), color='r', linestyle='--', label='平均')  # [6] 平均の補助線
plt.xticks(x)  # [7] 横軸の目盛りを 1〜12 すべてに
plt.title('新潟市の月ごとの降水量（2024年）', fontsize=20)  # [7] タイトル
plt.xlabel('月', fontsize=14)  # [7] 横軸の名前
plt.ylabel('降水量（mm）', fontsize=14)  # [7] 縦軸の名前
plt.legend()  # [7] 凡例
plt.show()
```



変えてみよう（1か所変えたら、もう一度実行する）

- 色：`'skyblue'` → `'tan'`、`'g'`、`'pink'`
- 幅：`width=0.6` → `width=1`（すき間がなくなる）、`width=0.3`
- 網掛け：`edgecolor='k'` の後ろに `, hatch='//'` を足す。`'//'` → `'xx'`、`'..'`
- シート：`'降水量'` のまま、列を `df['降水量']` → `df['気温']` に（ラベルも直す）
- `plt.xticks(x)` の行頭に `#` を付けると、横軸の目盛りがどうなるか

番号	部品	書き方	学ぶところ
[0]	グラフの大きさ（基本設定）	`plt.figure(figsize=(8, 5))`	第 3 講
[1]	棒を描く	`plt.bar(x, y)`	N-1
[2]	棒の色	`color='skyblue'`	N-2
[3]	棒の幅	`width=0.6`	N-3
[4]	外枠線と網掛け	`edgecolor='k'`、`hatch='//'`	N-4
[5]	項目を増やす	x と y のリストを長くする	N-5
[6]	平均の補助線	`plt.axhline(y=np.mean(y))`	N-6
[7]	目盛り・タイトル・軸の名前・凡例	`plt.xticks`、`plt.title` など	第 3 講
	1本だけ色を変える	`color=['gray', 'gray', 'tomato', 'gray']`	N-2
	棒が多いときに1本だけ色を変える	`colorlist = ['c'] * len(x)` → `colorlist[5] = 'm'`	P-4（発展）
	横棒グラフ	`plt.barh(x, y)`	N-7

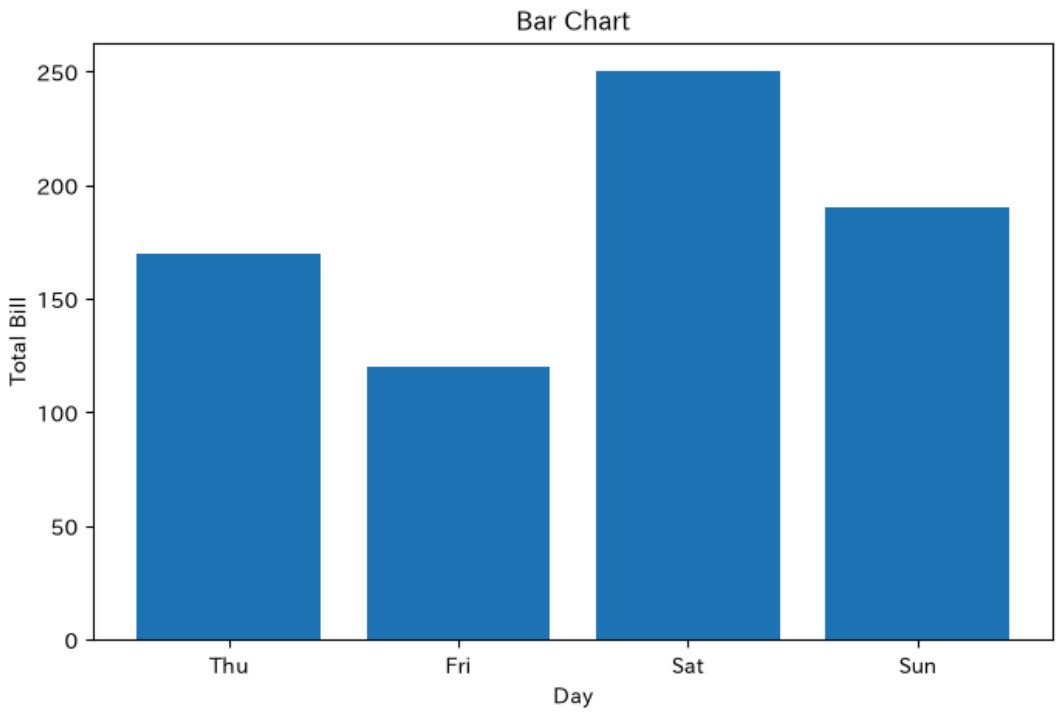
番号	部品	書き方	学ぶところ
	件数を数えて棒グラフ	`count = df['列名'].value_counts()` → `plt.bar(count.index, count)`	O-7

N-1 棒グラフの基本

- `plt.plot(x, y)` を `plt.bar(x, y)` に変えるだけ。x が**横軸の項目**、y が**棒の高さ**
- 横軸の項目は、数でも**文字**（`'Thu'` など）でもよい

```
x = ['Thu', 'Fri', 'Sat', 'Sun']
y = [170, 120, 250, 190]

plt.figure(figsize=(8, 5))
plt.bar(x, y)
plt.title('Bar Chart')
plt.xlabel('Day')
plt.ylabel('Total Bill')
plt.show()
```



折れ線グラフ・散布図との違い

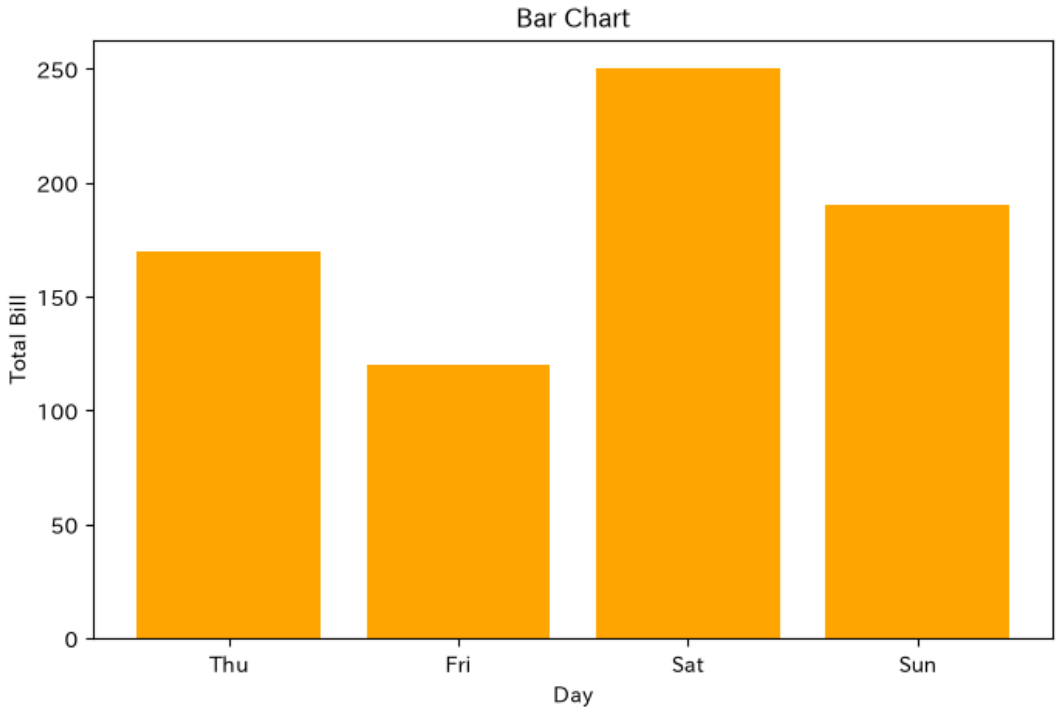
	折れ線グラフ	散布図	棒グラフ
関数	`plt.plot(x, y)`	`plt.scatter(x, y)`	`plt.bar(x, y)`
向いているもの	時間の流れ（順番に意味がある）	2つの量の関係	**項目ごとの量の比べ合い**
縦軸	0 から始めなくてよい	0 から始めなくてよい	**0 から始める**（棒の長さで比べるため）

N-2 棒の色

- 折れ線グラフ・散布図と同じく `color=`（短く `c=` は棒グラフでは使えないので `color=` と書く）

- 全部の棒が同じ色になる

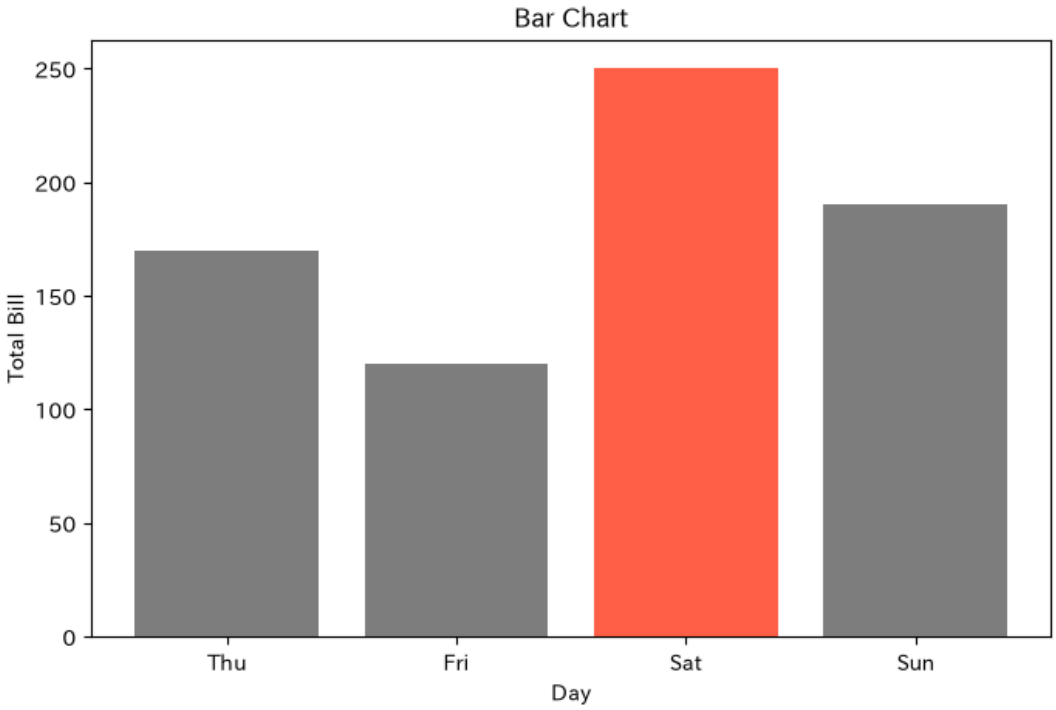
```
plt.figure(figsize=(8, 5))
plt.bar(x, y, color='orange')
plt.title('Bar Chart')
plt.xlabel('Day')
plt.ylabel('Total Bill')
plt.show()
```



1本だけ色を変える

- 前々回の散布図と同じく、`color=['gray', 'gray', 'tomato', 'gray']` のように**リスト**で書くと、棒ごとに色を変えられる
- リストの色は、x の項目の**順番どおり**に1本ずつ当てはまる（Thu → Fri → Sat → Sun）
- 目立たせたい棒だけ色を付け、他は灰色にすると、**どこを見てほしいか**が伝わる

```
plt.figure(figsize=(8, 5))
plt.bar(x, y, color=['gray', 'gray', 'tomato', 'gray'])  # 3番目（Sat）だけ赤に
plt.title('Bar Chart')
plt.xlabel('Day')
plt.ylabel('Total Bill')
plt.show()
```

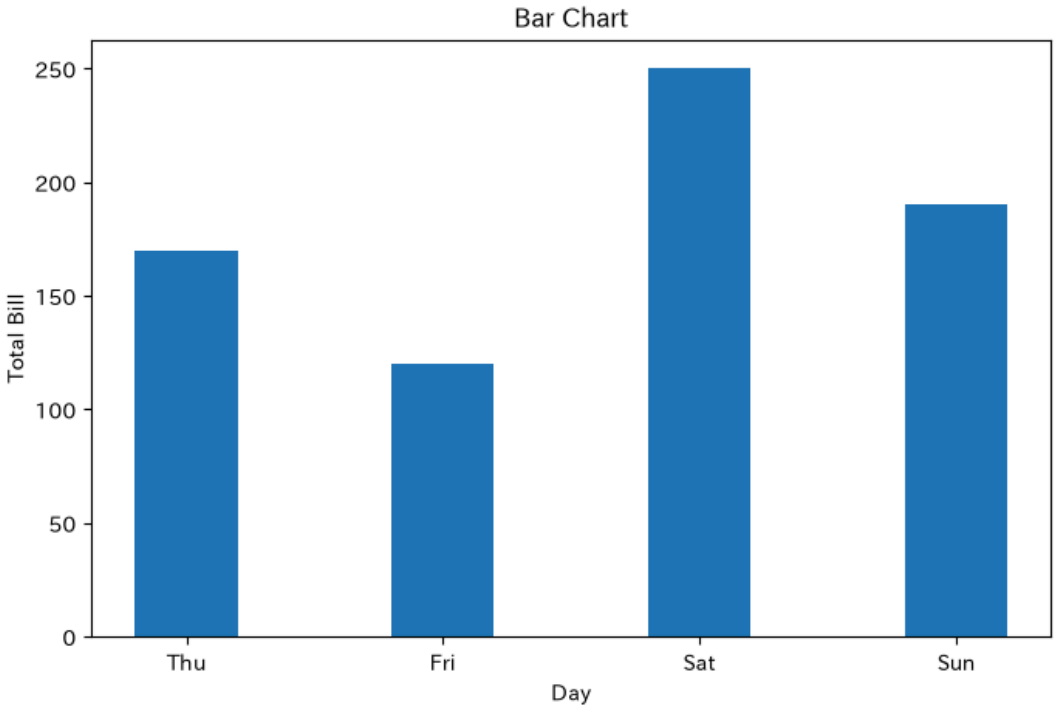


- 色の数は、棒の数（x の数）と**同じ**にする
- 色を変える棒を間違えないように、左から何番目かを数えてから書く
- 棒が多いときの書き方は P-4（発展）

N-3 棒の幅

- `width=` で棒の幅を変える。何も書かないと `0.8`。`1` にすると、すき間がなくなる

```
plt.figure(figsize=(8, 5))
plt.bar(x, y, width=0.4)
plt.title('Bar Chart')
plt.xlabel('Day')
plt.ylabel('Total Bill')
plt.show()
```



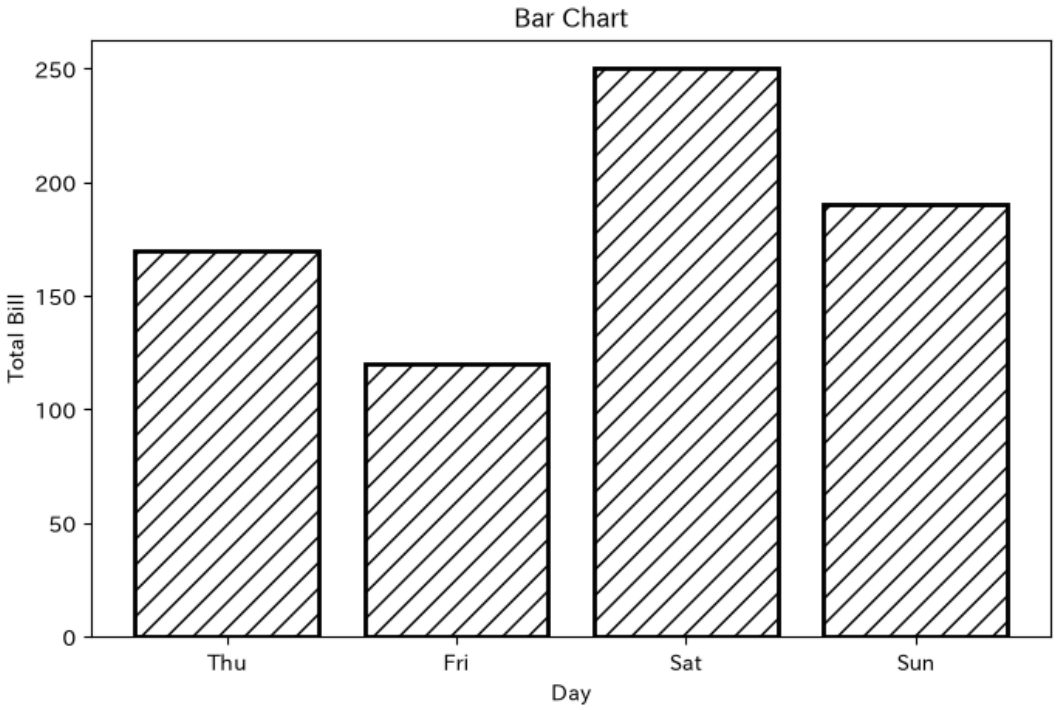
N-4 外枠線と網掛け

- `edgecolor=` で棒の**外枠線**の色、`linewidth=` でその太さ
- `hatch=` で**網掛け**（模様）。白黒で印刷しても区別できる

書き方	模様	書き方	模様
`'/'`	斜線	`'x'`	バツ
`'\\'`	逆向きの斜線	`'.'`	点
`'-'`	横線	`'o'`	丸
`'+'`	格子	`'*'`	星

- 同じ記号を重ねる（`'//'`、`'///'`）と、模様が細かくなる

```
plt.figure(figsize=(8, 5))
plt.bar(x, y, color='w', edgecolor='k', linewidth=2, hatch='//')
plt.title('Bar Chart')
plt.xlabel('Day')
plt.ylabel('Total Bill')
plt.show()
```



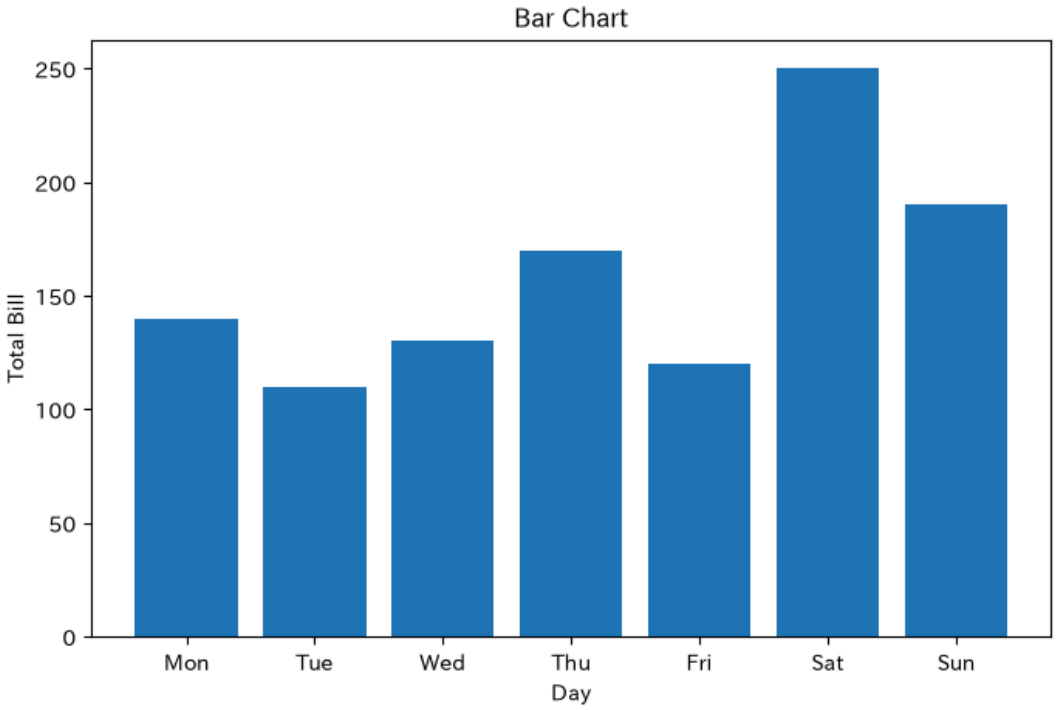
N-5 データを増やす

- 項目を増やすときは、x と y の**両方に**同じ数だけ足す（数が違うとエラー：`ValueError: shape mismatch`）
- 個数は `len(x)` と `len(y)` で確かめられる（第 2 講）

```
x7 = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
y7 = [140, 110, 130, 170, 120, 250, 190]
print(len(x7), len(y7))

plt.figure(figsize=(8, 5))
plt.bar(x7, y7)
plt.title('Bar Chart')
plt.xlabel('Day')
plt.ylabel('Total Bill')
plt.show()
```

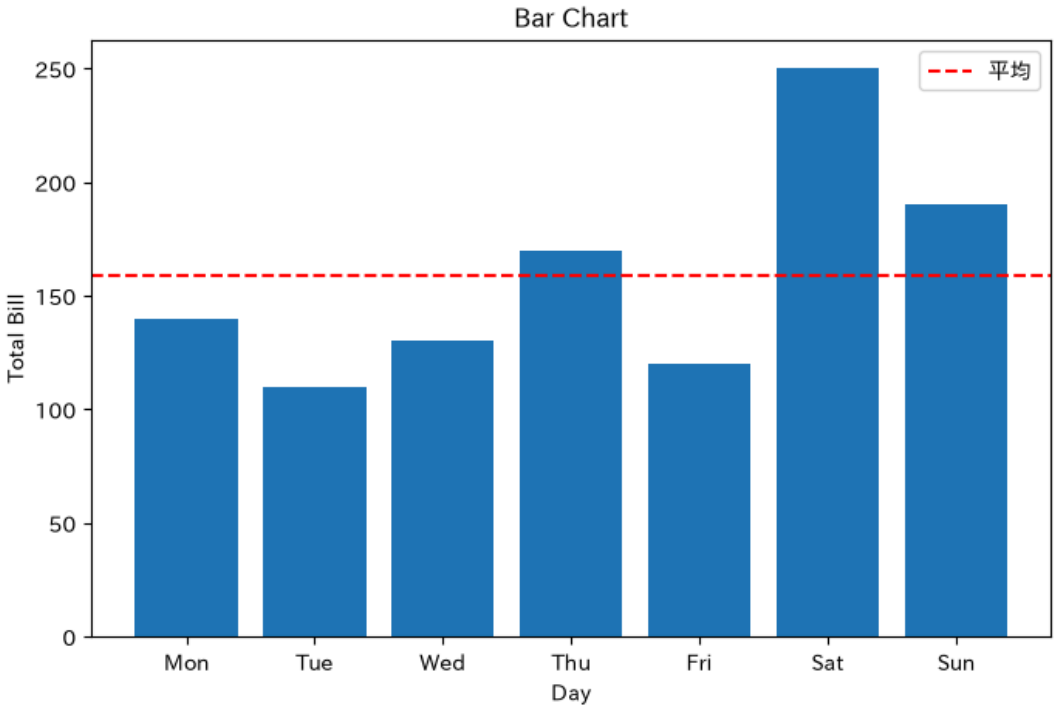
```
7 7
```



N-6 平均の補助線

- 第 4 講 G と同じ。平均は `np.mean(リスト)`、横の補助線は `plt.axhline(y=値)`
- 平均の値を**自分で計算して書き込まない**。`np.mean(y7)` と書けば、データを変えても線が自動で動く

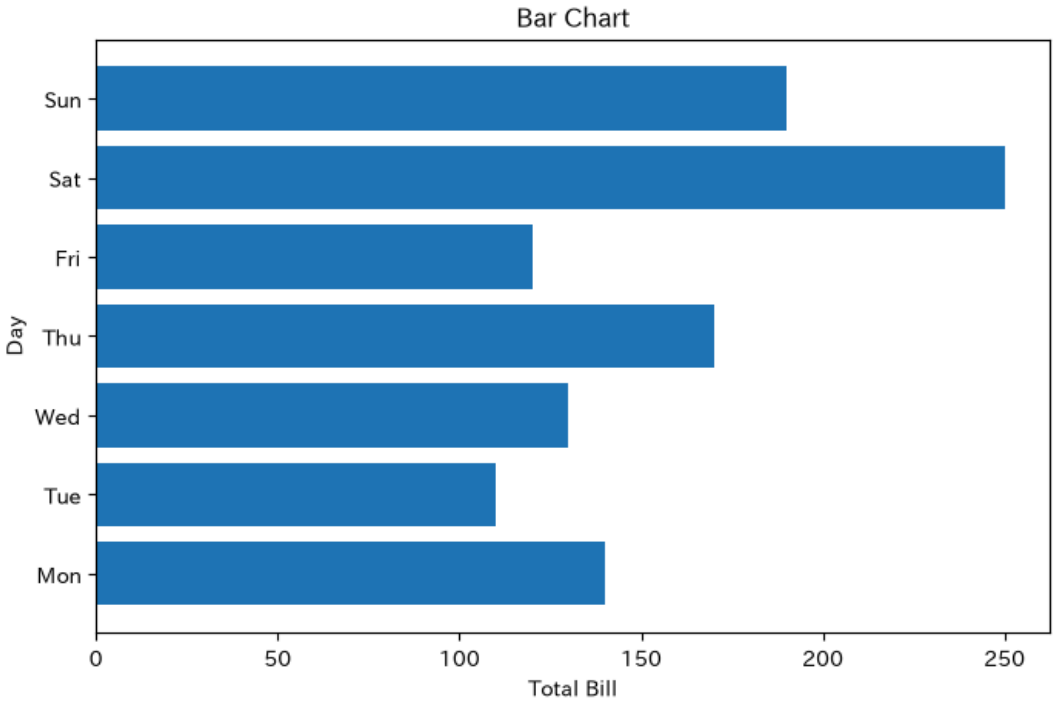
```
plt.figure(figsize=(8, 5))
plt.bar(x7, y7)
plt.axhline(y=np.mean(y7), color='r', linestyle='--', label='平均')
plt.title('Bar Chart')
plt.xlabel('Day')
plt.ylabel('Total Bill')
plt.legend()
plt.show()
```



N-7 横棒グラフ

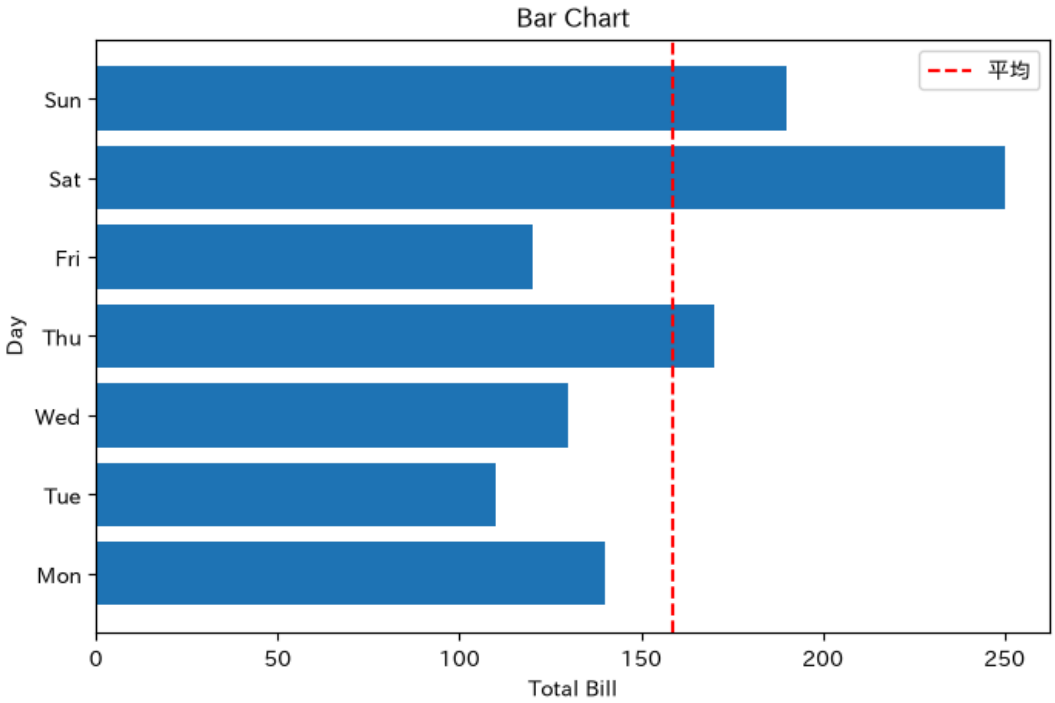
- `plt.bar` を `plt.barh` に変えると、棒が**横向き**になる（h は horizontal＝水平）
- 横向きになると、項目は**縦軸**、量は**横軸**に移る。だから `xlabel` と `ylabel` の中身を**入れ替える**
- 項目の名前が長いとき（都市名など）は、横棒グラフのほうが読みやすい

```
plt.figure(figsize=(8, 5))
plt.barh(x7, y7)
plt.title('Bar Chart')
plt.xlabel('Total Bill')
plt.ylabel('Day')
plt.show()
```



- 横棒グラフでは、量が横軸にあるので、平均の補助線は**縦の線** `plt.axvline(x=値)` になる

```
plt.figure(figsize=(8, 5))
plt.barh(x7, y7)
plt.axvline(x=np.mean(y7), color='r', linestyle='--', label='平均')
plt.title('Bar Chart')
plt.xlabel('Total Bill')
plt.ylabel('Day')
plt.legend()
plt.show()
```

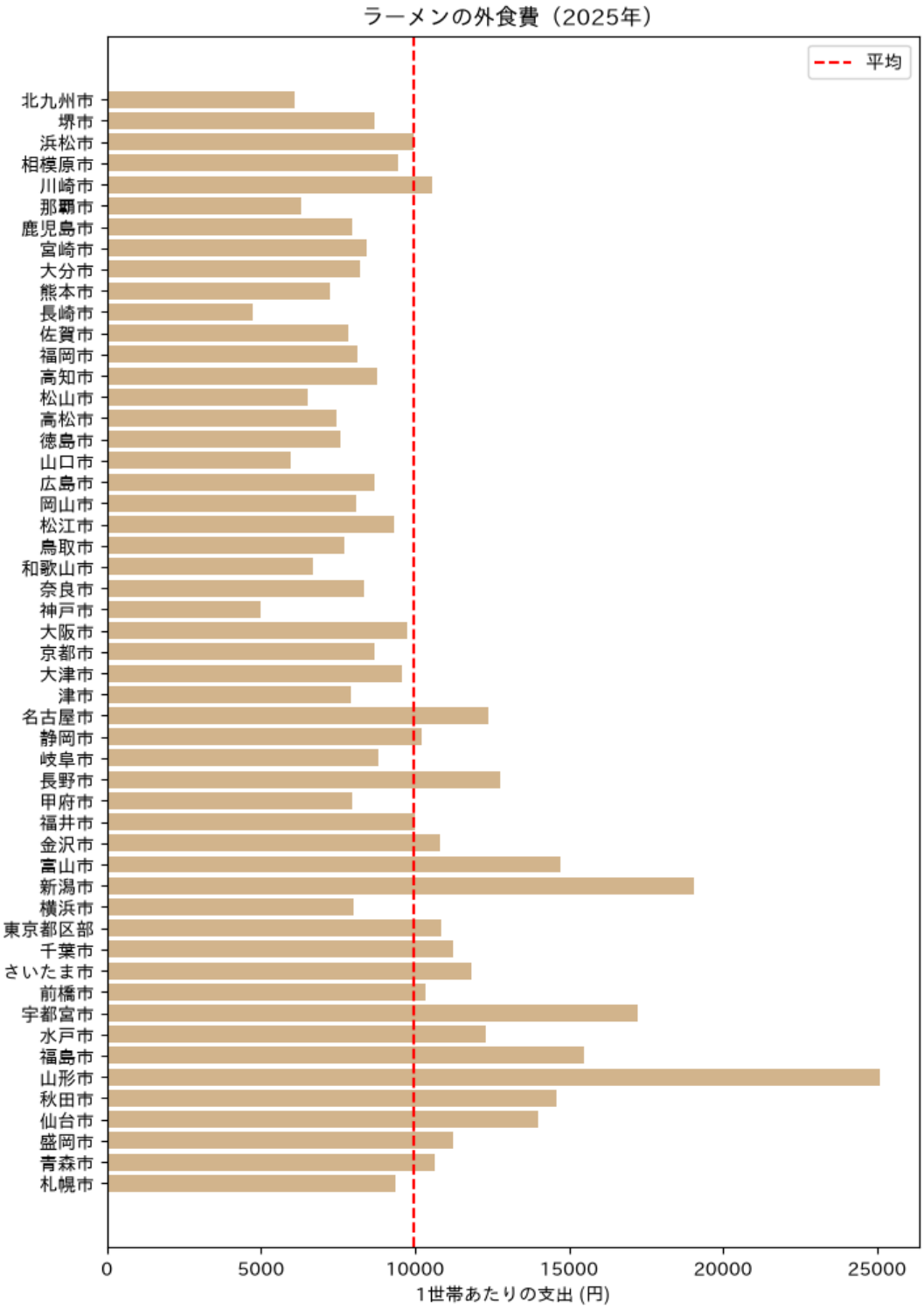


N-8 Excel のデータで棒グラフ

- `ramen` シートの52都市の外食費を、横棒グラフにする。都市名が長く、数も多いので横棒が向いている
- 棒が多いので、`figsize=(8, 12)` で**縦長**にする

```
df_ramen = pd.read_excel('dsa2026data.xlsx', sheet_name='ramen')

plt.figure(figsize=(8, 12))
plt.barh(df_ramen['city'], df_ramen['Y2025'], color='tan')
plt.axvline(x=np.mean(df_ramen['Y2025']), color='r', linestyle='--', label='平均')
plt.title('ラーメンの外食費（2025年）')
plt.xlabel('1世帯あたりの支出（円）')
plt.legend()
plt.show()
```



使ってみよう

- `'Y2025'` を `'Y2021'` に変えてみよう（2か所）
- `accident` シートを読み、`df['年']` と `df['死者数']` で棒グラフ（縦向き）を描いてみよう
- 多い順に並べたいときは P-1（発展）

O. 円グラフ

円グラフは、**全体に対する割合**（アンケートの回答の割合、予算の内訳など）を見せるグラフ。基本の形は次のとおり。

```
x = [100, 200, 300, 400, 500]
plt.pie(x)
plt.show()
```

O-0 完成形

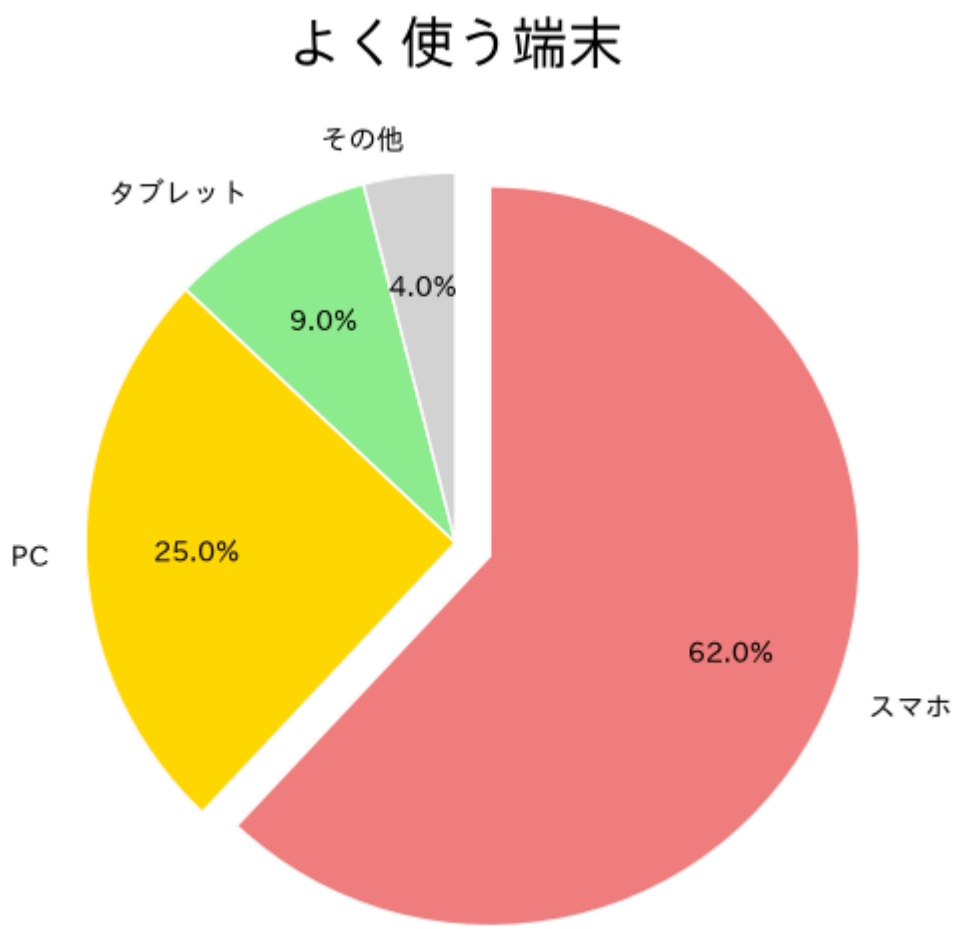
よく使う端末のアンケート結果（架空）。12時の位置から時計回りに、大きい順に並べている。

1. **全部コピーして**自分の notebook のセルに貼り付け、実行する
2. まずは**数値や文字を変えて**、グラフのどこが変わるかを確かめる
3. そのあと部品（[1]〜[7]）の働きを1つずつ学ぶ（O-1〜O-6）

```
import numpy as np
import matplotlib.pyplot as plt
import japanize_matplotlib

values = [62, 25, 9, 4]
labels = ['スマホ', 'PC', 'タブレット', 'その他']
colors = ['lightcoral', 'gold', 'lightgreen', 'lightgray']
shift = [0.1, 0, 0, 0]

plt.figure(figsize=(6, 6))  # [0] グラフの大きさ
plt.pie(values,  # [1] 円グラフ
        startangle=90, counterclock=False,  # [2] 12時から時計回り
        labels=labels,  # [3] ラベル
        colors=colors,  # [4] 色
        explode=shift,  # [5] 切り出し
        autopct='%.1f%%', pctdistance=0.7,  # [6] 割合の表示
        wedgeprops={'edgecolor': 'w'})  # 境目を白い線に
plt.title('よく使う端末', fontsize=20)  # [7] タイトル
plt.show()
```



上の `plt.pie(...)` は1つの命令なので、1行に並べて書くと次のようになる。

```
plt.pie(values, startangle=90, counterclock=False, labels=labels, colors=colors,
explode=shift, autopct='%.1f%%', pctdistance=0.7, wedgeprops={'edgecolor': 'w'})
```

- これでも同じグラフになるが、長くて読みにくい。そこで、見やすさのために `,` の後ろで**改行**してよい（`(` の中なら、次の行も続きとして読まれる）
- 改行すると1行に1つの部品を書けて、行の右にコメント（`# [3] ラベル` など）も付けられる

変えてみよう（1か所変えたら、もう一度実行する）

- 値：`62` → `40` に（割合が自動で計算し直される）
- `counterclock=False` を消すと、並ぶ向きがどうなるか
- 切り出し：`[0.1, 0, 0, 0]` → `[0, 0, 0, 0.2]`
- 割合：`'%.1f%%'` → `'%.0f%%'`（小数なし）

番号	部品	書き方	学ぶところ
[0]	グラフの大きさ（基本設定）	`plt.figure(figsize=(6, 6))`	第 3 講
[1]	円グラフを描く	`plt.pie(x)`	O-1
[2]	12時から時計回り	`startangle=90, counterclock=False`	O-1

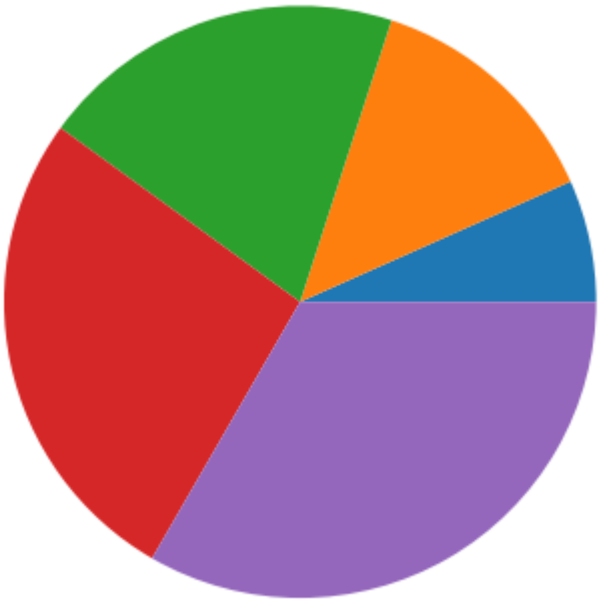
番号	部品	書き方	学ぶところ
[3]	ラベル	`labels=リスト`	O-2
[4]	色	`colors=リスト`	O-3
[5]	切り出し	`explode=リスト`	O-4
[6]	割合の表示	`autopct='%.1f%%'` 、`pctdistance=0.7`	O-5
	灰色だけで描く	`colors=['0.8', '0.6', …]`	O-6
	カラーマップ	`colors=plt.get_cmap('Pastel1').colors`	P-2（発展）
	境目の線	`wedgeprops={'edgecolor': 'w', 'linewidth': 3}`	O-8
	ラベルの位置と文字	`labeldistance=0.6` 、`textprops={'color': 'w', 'fontsize': 14}`	O-8
	ドーナツグラフ	`wedgeprops={'width': 0.4}`	O-8
	件数を数えて円グラフ	`count = df['列名'].value_counts()`	O-7
	二重のドーナツグラフ	`radius=0.7`	P-3（発展）
[7]	タイトル	`plt.title`	第 3 講

O-1 円グラフの基本と始まりの位置

- `plt.pie(値のリスト)` で描く。割合は**自動で計算**される（合計が 100 でなくてよい）
- 何も指定しないと、**3時の位置**から**反時計回り**に並ぶ

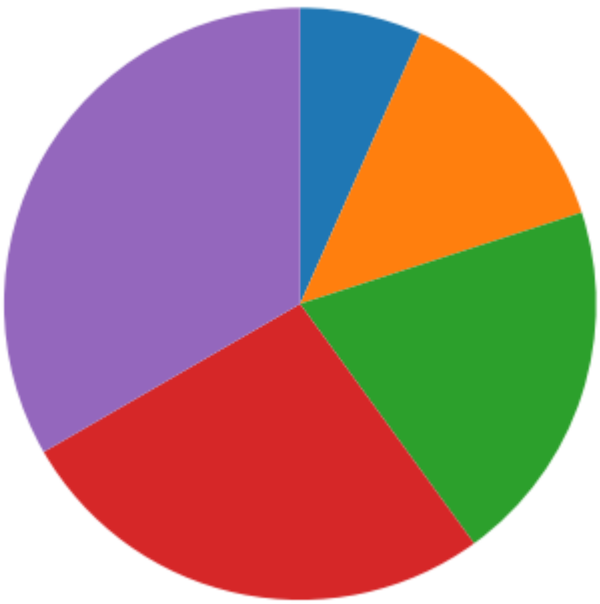
```
x = [100, 200, 300, 400, 500]

plt.pie(x)
plt.show()
```



- `startangle=90` で**12時の位置**から始まる（角度は3時の位置から反時計回りに数える。90度＝12時）
- `counterclock=False` で**時計回り**になる（counter clock＝反時計回り、を False＝しない）
- 時計と同じ「12時から時計回り」にすると読みやすい

```
plt.pie(x, startangle=90, counterclock=False)
plt.show()
```

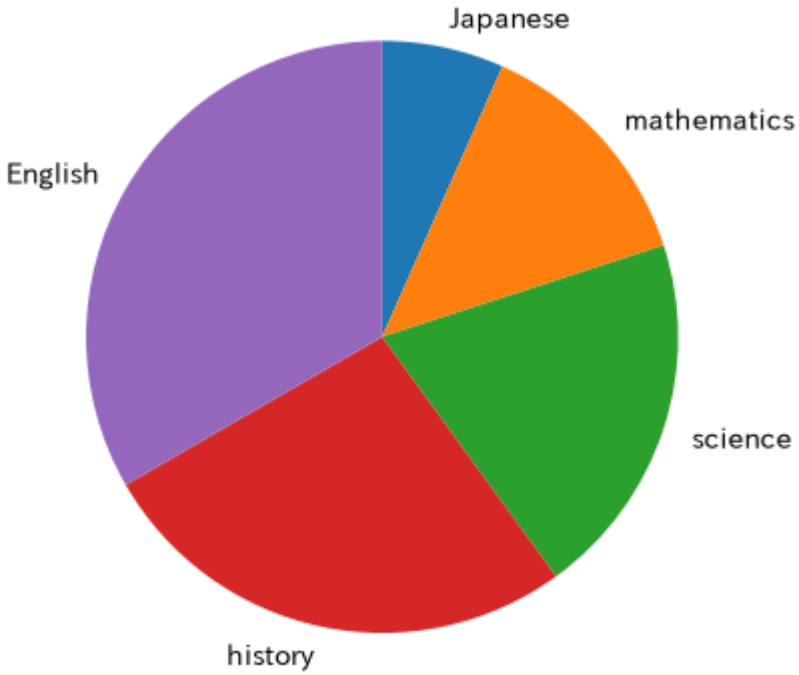


O-2 ラベル

- `labels=リスト` で、それぞれの部分に名前を付ける。値と**同じ個数・同じ順番**で書く

```
label = ['Japanese', 'mathematics', 'science', 'history', 'English']

plt.pie(x, startangle=90, counterclock=False, labels=label)
plt.show()
```



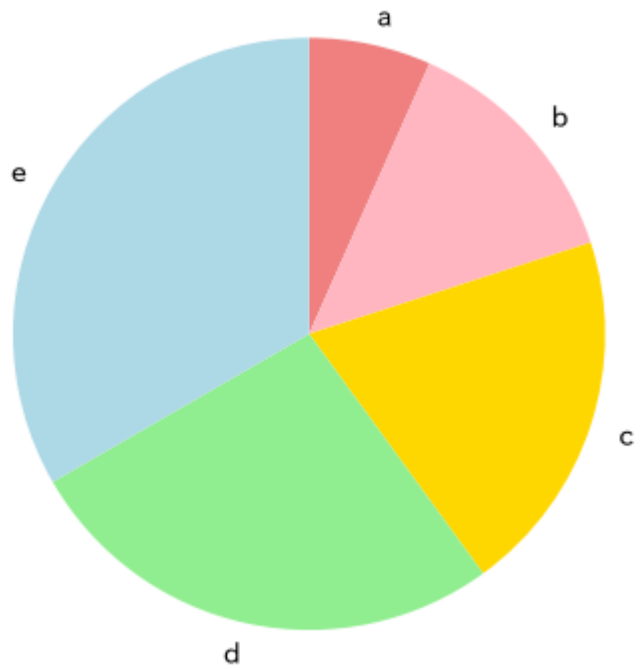
- 日本語のラベルも使える（`import japanize_matplotlib` を実行してあれば）
- 個数が違うと `ValueError` になる

O-3 色

- `colors=リスト` で、部分ごとに色を決める。棒グラフの `color=` と違い、**s が付く**ことに注意
- 色の名前は第 3 講の色の表（`'lightcoral'`、`'gold'` など）を使う

```
mylabel = ['a', 'b', 'c', 'd', 'e']
mycolor = ['lightcoral', 'lightpink', 'gold', 'lightgreen', 'lightblue']

plt.pie(x, startangle=90, counterclock=False, labels=mylabel, colors=mycolor)
plt.show()
```

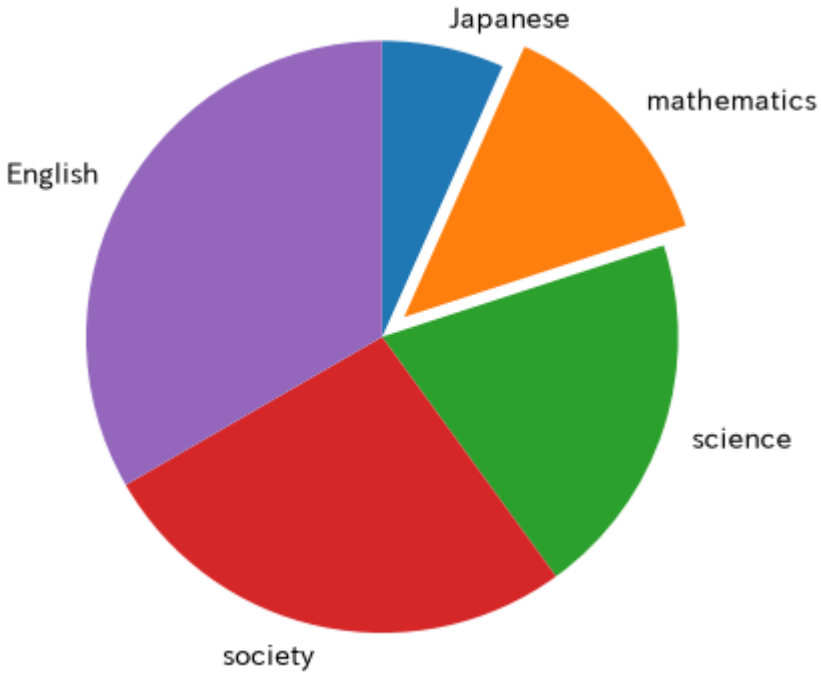


O-4 一部を切り出す

- `explode=リスト` で、部分ごとに中心から**どれだけ離すか**を決める。`0` は離さない
- 目立たせたい部分だけ `0.1` くらいにする

```
mylabel = ['Japanese', 'mathematics', 'science', 'society', 'English']
shift = [0, 0.1, 0, 0, 0]

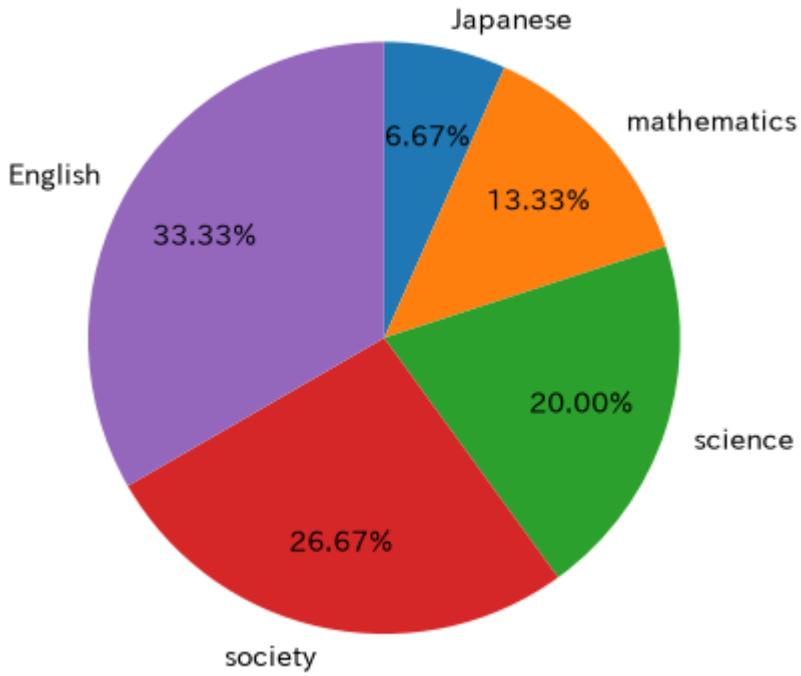
plt.pie(x, startangle=90, counterclock=False, labels=mylabel, explode=shift)
plt.show()
```



O-5 割合を表示する

- `autopct='%.1f%%'` で、各部分に**割合**（%）を書く。`.1f` は小数第1位まで、`.2f` は第2位まで
- 最後の `%%` は「% の記号を表示する」という意味（`%` を1つだけ書くとエラー）
- `pctdistance=` で、割合の文字を**中心からどれだけ離すか**を決める（`0` が中心、`1` が円の縁。何も書かないと `0.6`）

```
plt.pie(x, startangle=90, counterclock=False, labels=mylabel, autopct='%.2f%%',
pctdistance=0.7)
plt.show()
```

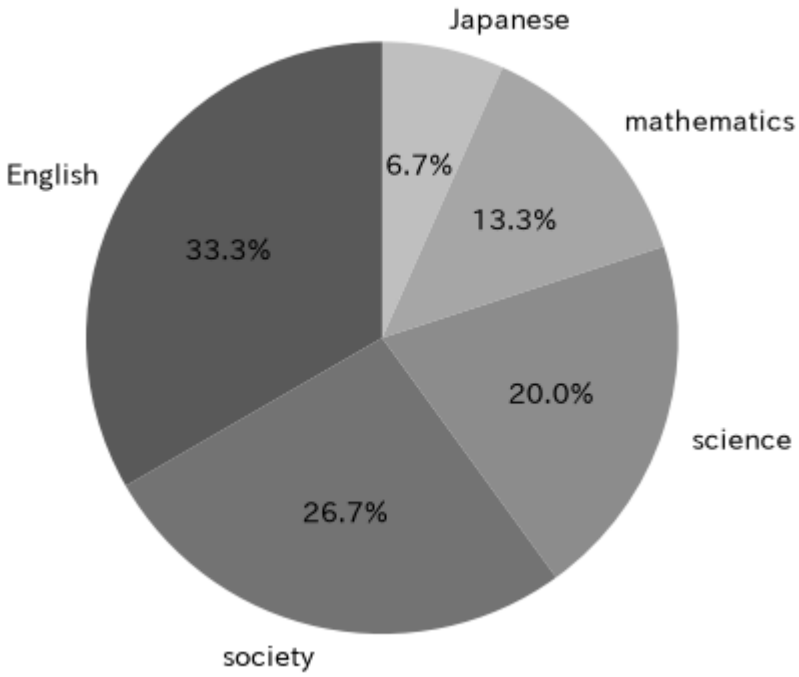


O-6 灰色だけで描く

- 色を `'0.75'` のように **0〜1 の数を文字列で**書くと、灰色になる（`'0'` が黒、`'1'` が白）
- 白黒で印刷する資料に使う

```
grayscales = ['0.75', '0.65', '0.55', '0.45', '0.35']

plt.pie(x, startangle=90, counterclock=False, labels=mylabel, autopct='%.1f%%',
colors=grayscales)
plt.show()
```



- `'0.75'` を `0.75`（引用符なし）にするとエラーになる。**文字列で**書くこと

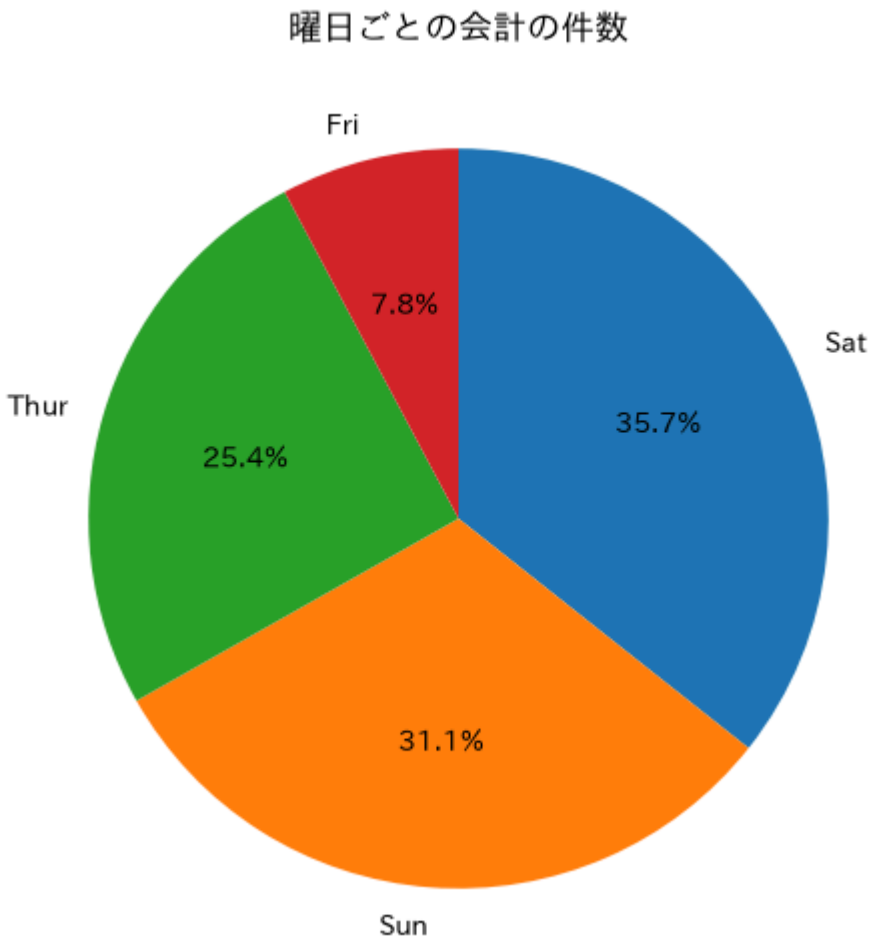
O-7 Excel のデータで円グラフ

- `tip_data` シートは、あるレストランの244件の会計の記録（食事代・チップ・曜日など）
- `df['列名'].value_counts()` で、**同じ値が何件あるか**を数えられる。曜日ごとの件数を円グラフにする

```
df_tip = pd.read_excel('dsa2026data.xlsx', sheet_name='tip_data')
count = df_tip['day'].value_counts()
count
```

```
day
Sat     87
Sun     76
Thur    62
Fri     19
Name: count, dtype: int64
```

```
plt.figure(figsize=(6, 6))
plt.pie(count, labels=count.index, startangle=90, counterclock=False, autopct='%.1f%%')
plt.title('曜日ごとの会計の件数')
plt.show()
```



- `value_counts()` は**件数の多い順**に並べてくれる。12時から時計回りに大きい順に並ぶ
- `count.index` は、数えた値の名前（ここでは曜日）
- 数えた結果は**棒グラフ**にも使える。`plt.bar(count.index, count)` と書く（横軸が名前、棒の高さが件数）

円グラフを使うときの注意

注意	理由
項目は5つくらいまで	多いと細くて読めない。少ないものは「その他」にまとめる
合計に意味があるときだけ使う	「全体のうち何 %」でないもの（月ごとの気温など）は棒グラフや折れ線グラフにする
立体（3D）にしない	手前の部分が大きく見えてしまう（第12講）
2つの円グラフを並べて比べない	角度の違いは読み取りにくい。比べるなら棒グラフ

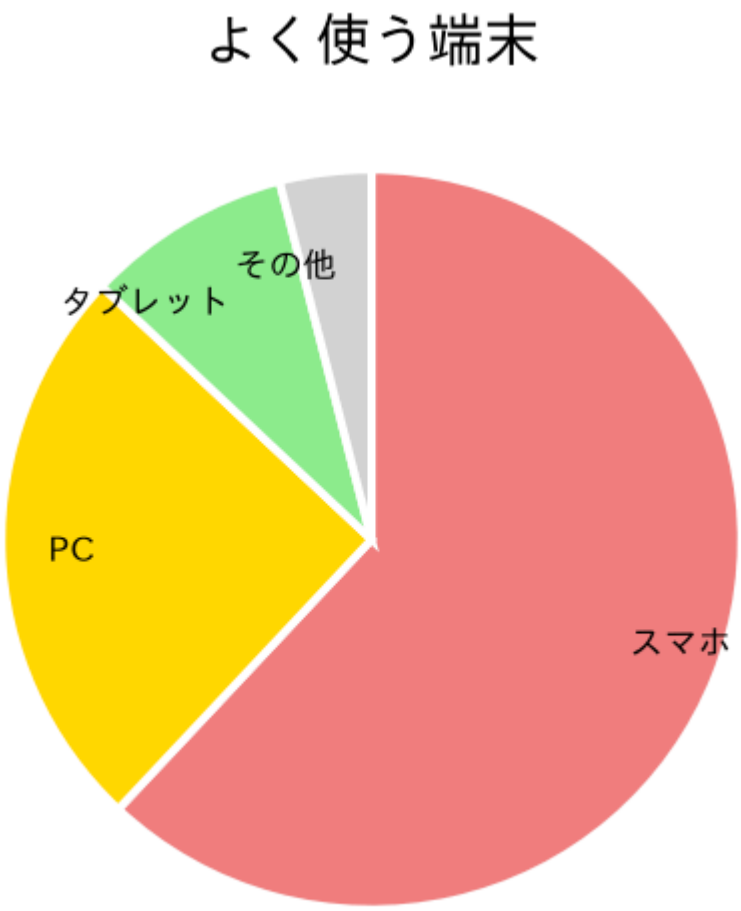
O-8 境目の線・ラベルの位置・ドーナツグラフ

円グラフの見た目は、次の3つでさらに細かく変えられる。どれも `{'名前': 値, '名前': 値}` の形（**辞書**）で書く。

書き方	変わるところ
`wedgeprops={'edgecolor': 'w', 'linewidth': 3}`	部分（扇形）の**境目の線**の色と太さ

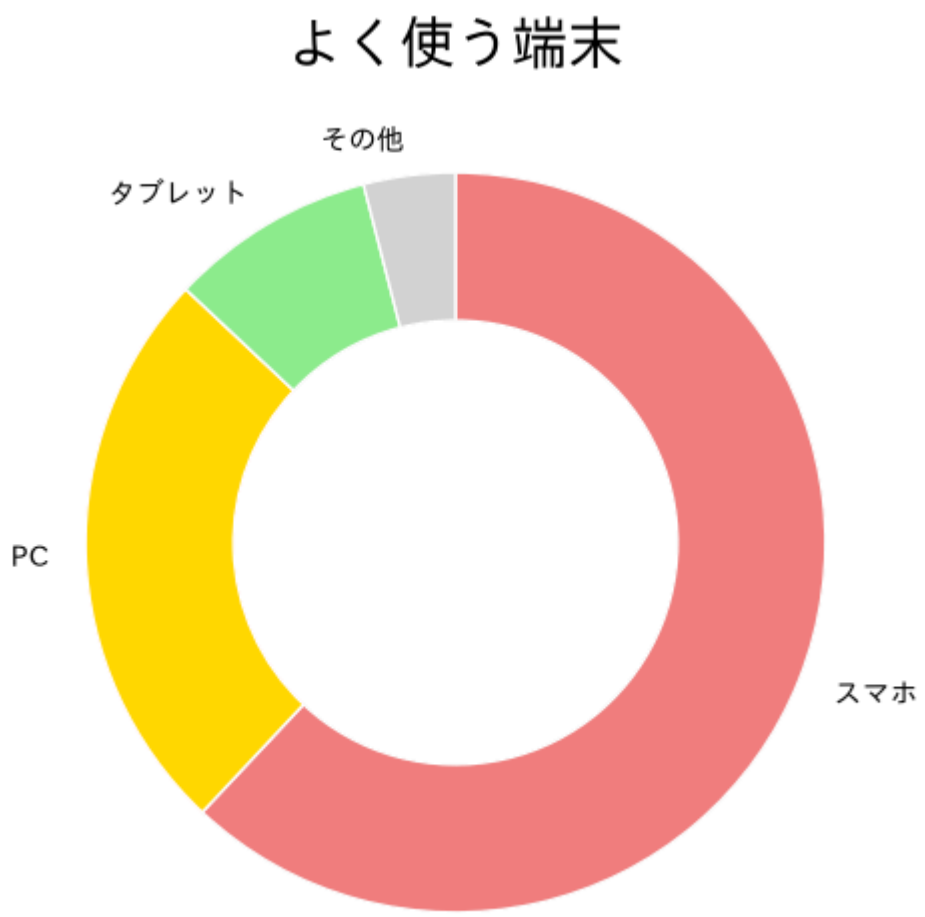
書き方	変わるところ
`labeldistance=0.6`	**ラベルの位置**（中心が 0、縁が 1。何も書かないと 1.1 で円の外側）
`textprops={'color': 'w', 'fontsize': 14}`	**ラベルの文字**の色と大きさ

```
plt.figure(figsize=(6, 6))
plt.pie(values, startangle=90, counterclock=False, labels=labels, colors=colors,
        wedgeprops={'edgecolor': 'w', 'linewidth': 3},  # 境目を太さ3の白い線に
        labeldistance=0.75,  # ラベルを円の内側に
        textprops={'color': 'k', 'fontsize': 12})  # ラベルの文字を黒・大きさ12に
plt.title('よく使う端末', fontsize=20)
plt.show()
```



- `values`・`labels`・`colors` は O-0 で作ったもの。O-0 を実行していないと `NameError` になる
- `wedgeprops=` に `'width': 0.4` を加えると、**真ん中が空いたドーナツグラフ**になる（輪の太さが 0.4。円の半径は 1）

```
plt.figure(figsize=(6, 6))
plt.pie(values, startangle=90, counterclock=False, labels=labels, colors=colors,
        wedgeprops={'width': 0.4, 'edgecolor': 'w'})  # 輪の太さ 0.4 のドーナツグラフ
plt.title('よく使う端末', fontsize=20)
plt.show()
```



- 真ん中が空いているので、そこに合計などを書くこともできる（`plt.text(0, 0, '全100人', ha='center', fontsize=16)`）

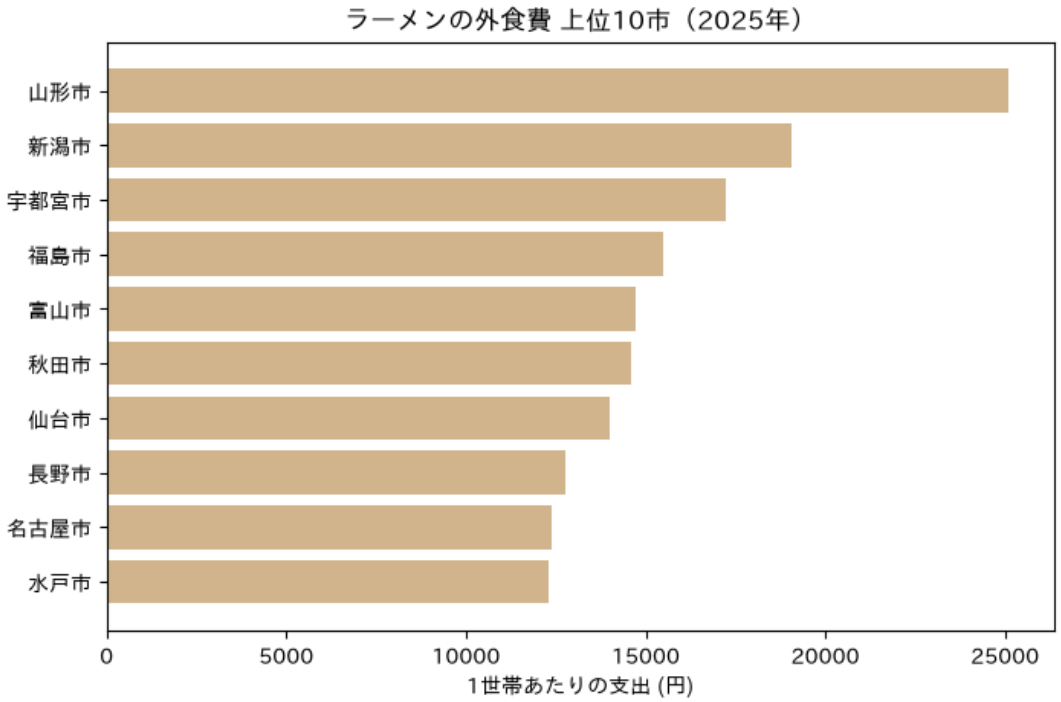
P. 発展

P-1 並べ替えて上位だけ描く

- `df.sort_values('列名', ascending=False)` で、その列の**大きい順**に並べ替える（`ascending=True` なら小さい順）
- `.head(10)` で上から10行だけ取り出す
- `barh` は**上から下へ**ではなく**下から上へ**並べるので、`plt.gca().invert_yaxis()` で上下を逆にすると1位が一番上になる

```
top10 = df_ramen.sort_values('Y2025', ascending=False).head(10)

plt.figure(figsize=(8, 5))
plt.barh(top10['city'], top10['Y2025'], color='tan')
plt.gca().invert_yaxis()
plt.title('ラーメンの外食費 上位10市（2025年）')
plt.xlabel('1世帯あたりの支出（円）')
plt.show()
```



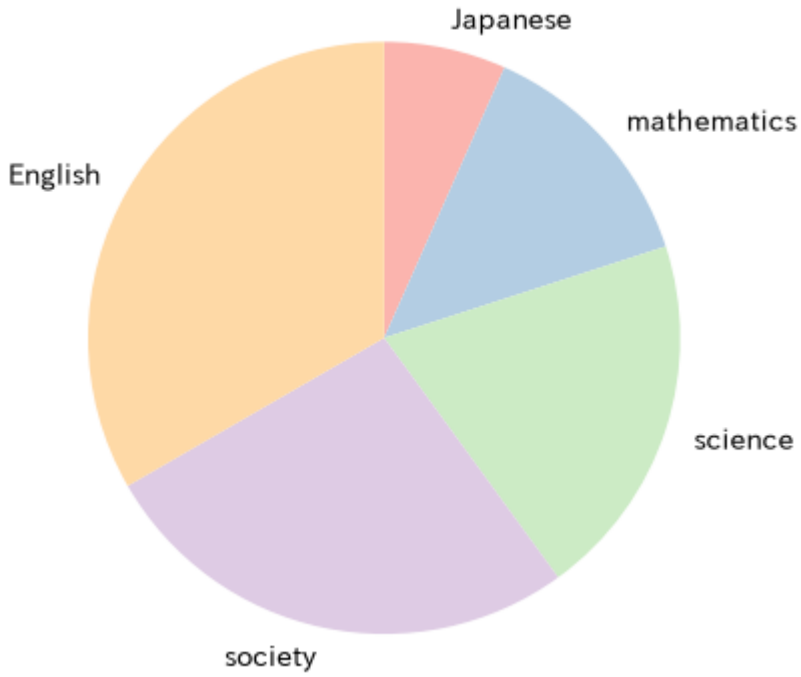
P-2 カラーマップで色をまとめて決める

- 色を1つずつ選ぶ代わりに、**カラーマップ**（決まった色の組み合わせ）から取り出せる
- `plt.get_cmap('Pastel1').colors` で、Pastel1 という色の組み合わせのリストが得られる
- 名前の例：`'Pastel1'`、`'Set2'`、`'tab10'`、`'Accent'`（色の組み合わせの一覧）

```
x = [100, 200, 300, 400, 500]
mylabel = ['Japanese', 'mathematics', 'science', 'society', 'English']

mycolor = plt.get_cmap('Pastel1').colors

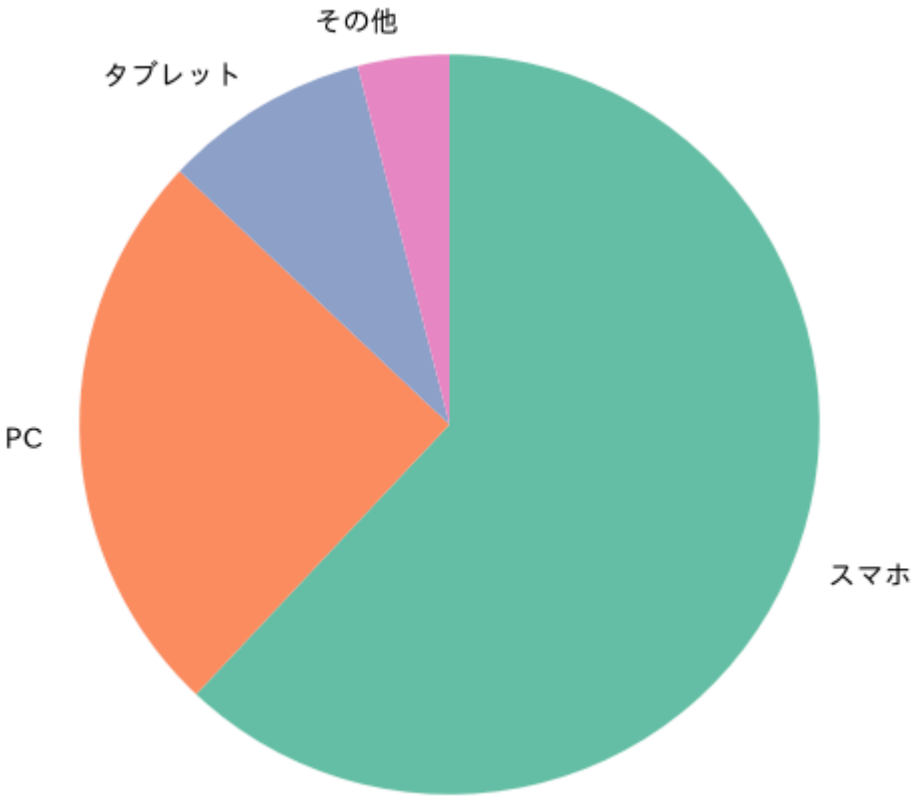
plt.pie(x, startangle=90, counterclock=False, labels=mylabel, colors=mycolor)
plt.show()
```



色の組み合わせを替える

- 何も指定しないときは **tab10**（1本目は青、2本目はオレンジ、3本目は緑…）
- `plt.pie` の括弧の中に `colors=` を書き足すと、そのグラフの色の組み合わせが替わる。そのグラフだけに効くので、次のグラフには影響しない
- `'Set2'` の部分は `'Pastel1'`、`'Dark2'` などに替えられる（色の組み合わせの一覧）
- 一覧の **Qualitative**（色どうしがはっきり違う組み合わせ）の中から選ぶ。第 5 講の散布図で `cmap=` に使った `'viridis'`・`'coolwarm'` などは、値の大きさに合わせて色が少しずつ変わる組み合わせで、散布図のように連続的に変わる値を色で表すときに使う。円グラフや棒グラフのように項目の違いをはっきり見せたいときは Qualitative、と使い分ける
- 名前の末尾に `_r` を付けると、色の順番が逆になる（例：`'Set2_r'`）

```
plt.figure(figsize=(6, 6))
plt.pie([62, 25, 9, 4], labels=['スマホ', 'PC', 'タブレット', 'その他'], startangle=90,
counterclock=False,
        colors=plt.get_cmap('Set2').colors)  # 色の組み合わせを Set2 に
plt.show()
```



- 棒グラフでは引数名が `color=`（s なし）になる。`plt.bar(x, y, color=plt.get_cmap('Set2').colors)` と書くと、棒が1本ずつ違う色になる
- このあとのグラフをまとめて替えたいときは、グラフを描く前に次の1行を実行する。それ以降のグラフ（棒・円・折れ線）がすべてその色の組み合わせになる。折れ線を何本も描くグラフの色を揃えたいときに便利

```
plt.rcParams['axes.prop_cycle'] = plt.cycler(color=plt.get_cmap('Set2').colors)  # 以降のグラフを Set2 に
```

- 元に戻すときは `'Set2'` を `'tab10'` にして実行する（`'tab10'` が最初から使われている色の組み合わせ）

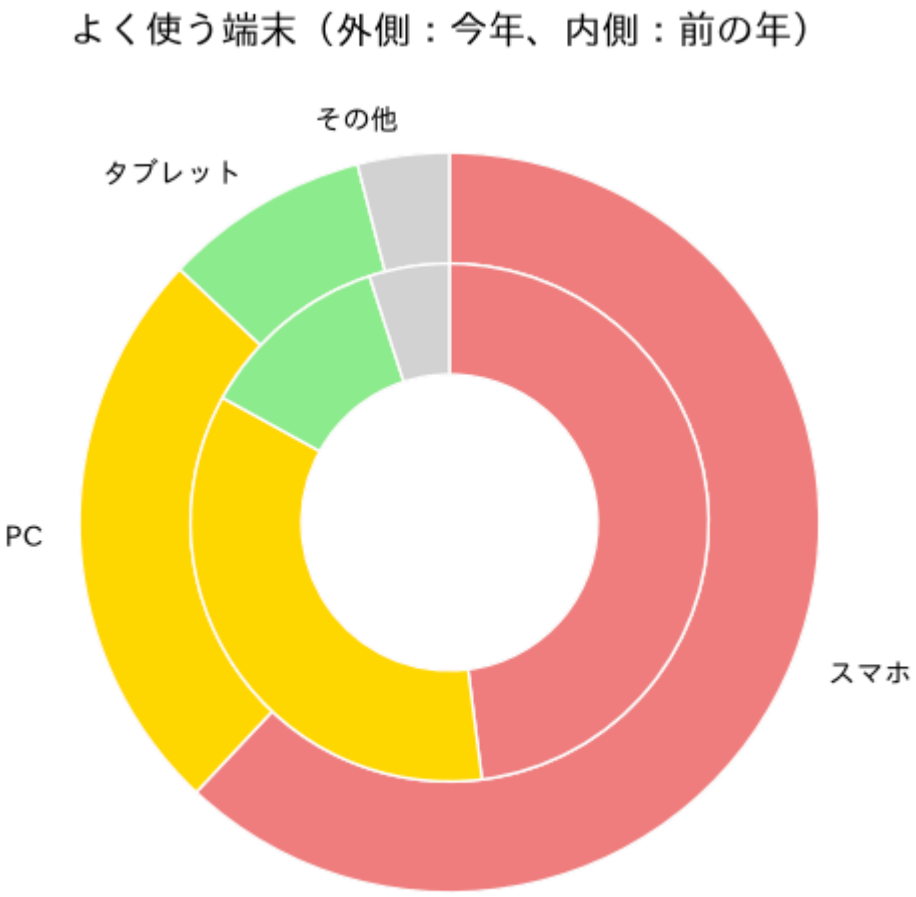
P-3 二重のドーナツグラフ

- `plt.pie` を2回書くと、同じ場所に重なって描かれる
- 2つ目に `radius=0.7`（半径 0.7）を付けると、1つ目の輪の内側に収まる。2つのデータ（去年と今年など）を比べるときに使う

```
values2 = [48, 35, 12, 5]  # 前の年の結果（架空）

plt.figure(figsize=(6, 6))
plt.pie(values, startangle=90, counterclock=False, labels=labels, colors=colors,
        wedgeprops={'width': 0.3, 'edgecolor': 'w'})  # 外側の輪（今年）
plt.pie(values2, startangle=90, counterclock=False, colors=colors, radius=0.7,
        wedgeprops={'width': 0.3, 'edgecolor': 'w'})  # 内側の輪（前の年）
```

```
plt.title('よく使う端末（外側：今年、内側：前の年）', fontsize=14)
plt.show()
```



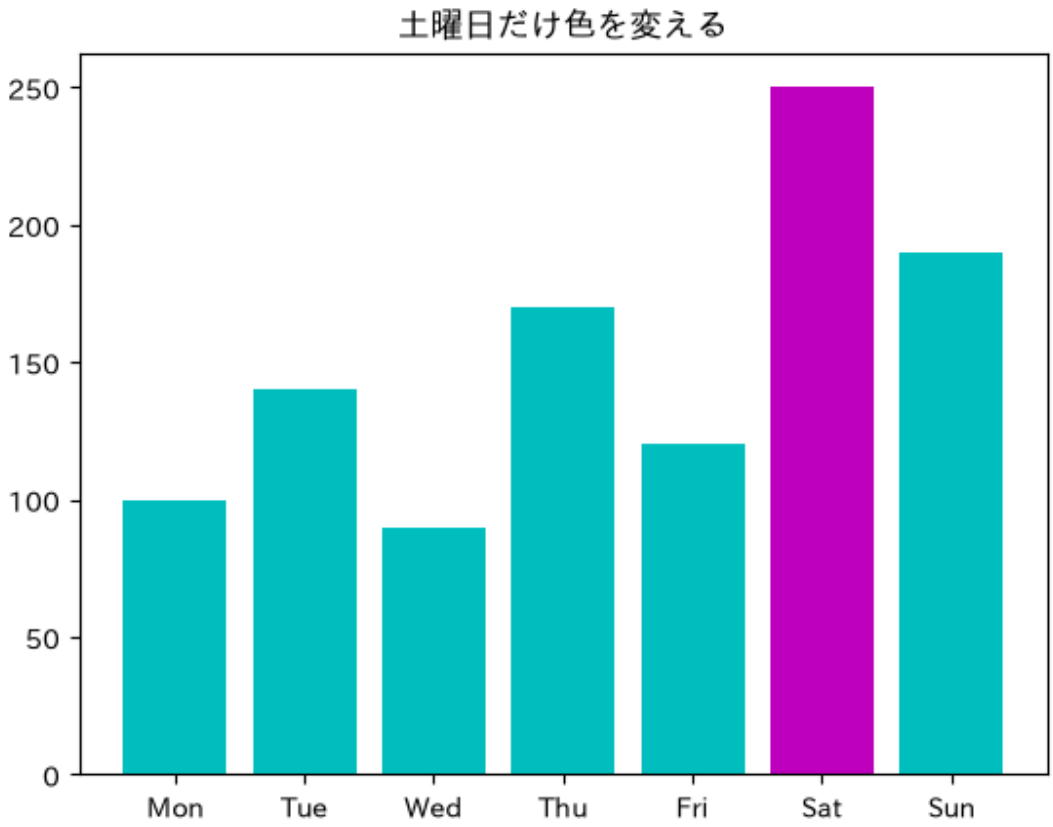
P-4 棒が多いときに1本だけ色を変える

- N-2 では色を1本ずつ書き並べたが、棒が多いと書くのが大変で、数も間違えやすい
- `['c'] * len(x)` で、`'c'`（シアン）を x の数だけ並べたリストを作れる（`['c'] * 3` は `['c', 'c', 'c']`）
- `colorlist[5] = 'm'` で、リストの**6番目**だけを `'m'`（マゼンタ）に置き換える。リストの番号は **0 から**数える（Mon が 0、Sat が 5）

```
x = ['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
y = [100, 140, 90, 170, 120, 250, 190]

colorlist = ['c'] * len(x)  # 'c' を7つ並べたリスト
colorlist[5] = 'm'          # 6番目（Sat）だけ 'm' に

plt.bar(x, y, color=colorlist)
plt.title('土曜日だけ色を変える')
plt.show()
```



- x の数が変わっても、`len(x)` が数え直すので、色のリストを書き直さなくてよい
- `colorlist[7] = 'm'` のように、ない番号（7つなら 0〜6 以外）を書くと `IndexError: list assignment index out of range`

まとめ

やりたいこと	書き方
以下は棒グラフ	
棒グラフ	`plt.bar(x, y)`
色・幅	`color='skyblue'` 、`width=0.6`
1本だけ色を変える	`color=['gray', 'gray', 'tomato', 'gray']` （棒の順に色を並べる）
棒が多いときに1本だけ	`colorlist = ['c'] * len(x)` → `colorlist[5] = 'm'` （番号は 0 から）
外枠線・網掛け	`edgecolor='k'` 、`hatch='//'`
横棒グラフ	`plt.barh(x, y)` （ `xlabel` と `ylabel` を入れ替える）
平均の補助線	縦の棒： `plt.axhline(y=np.mean(y))` 、横の棒： `plt.axvline(x=np.mean(y))`
以下は円グラフ	
円グラフ	`plt.pie(値のリスト)`
12時から時計回り	`startangle=90, counterclock=False`
ラベル・色	`labels=リスト` 、`colors=リスト` （s が付く）

やりたいこと	書き方
一部を切り出す	`explode=[0, 0.1, 0, …]`
割合を表示	`autopct='%.1f%%'` 、`pctdistance=0.7`
灰色	`colors=['0.8', '0.6', …]` （文字列で）
境目の線	`wedgeprops={'edgecolor': 'w', 'linewidth': 3}`
ラベルの位置・文字	`labeldistance=0.6` 、`textprops={'color': 'w', 'fontsize': 14}`
ドーナツグラフ	`wedgeprops={'width': 0.4}`
二重のドーナツグラフ	2つ目の `plt.pie` に `radius=0.7` （内側に収まる）
件数を数える	`df['列名'].value_counts()` （棒グラフは `plt.bar(count.index, count)` ）

第 7 講：値を数える・辞書・棒グラフの補足・箱ひげ図

第 6 講 O-7 の `value_counts()` を確かめ、グラフのオプションの指定にも使う**辞書**を学ぶ（Q）。棒グラフの補足（R）のあと、グループごとのばらつきを比べる**箱ひげ図**を描く（S）。

準備：この講で使うライブラリを読み込み、データを用意する。

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import japanize_matplotlib

df = pd.read_excel('dsa2026data.xlsx', sheet_name='exam')
df.head()
```

	name	国語	数学	理科	社会	英語
0	伊藤 涼介	94	76	62	81	62
1	上田 美咲	63	57	59	80	74
2	木村 太郎	55	66	82	78	66
3	佐藤 亜美	61	87	80	65	78
4	鈴木 恵美子	56	58	69	97	61

- 表が出れば準備完了。`FileNotFoundError` が出たら、`dsa2026data.xlsx` を notebook と同じ場所に置く（第 5 講 L-1）
- `exam` シートは、30人の5教科のテストの点数（名前は架空）

Q. 値を数える・辞書

第 6 講 O-7 では、`value_counts()` で曜日ごとの件数を数えた。何をしているのかをもう一度確かめてから、辞書の基礎を学ぶ。

Q-1 Excel の列の値を数える

- `df['列名'].value_counts()` で、その列に**同じ値が何件あるか**を数える
- `tip_data` シートは、あるレストランの244件の会計の記録（食事代・チップ・性別・曜日など）
- ここでは `smoker` 列（グループに喫煙者がいたか：Yes / No）の件数を数える

```
df_tip = pd.read_excel('dsa2026data.xlsx', sheet_name='tip_data')
count = df_tip['smoker'].value_counts()
count
```

```
smoker
No     151
Yes     93
Name: count, dtype: int64
```

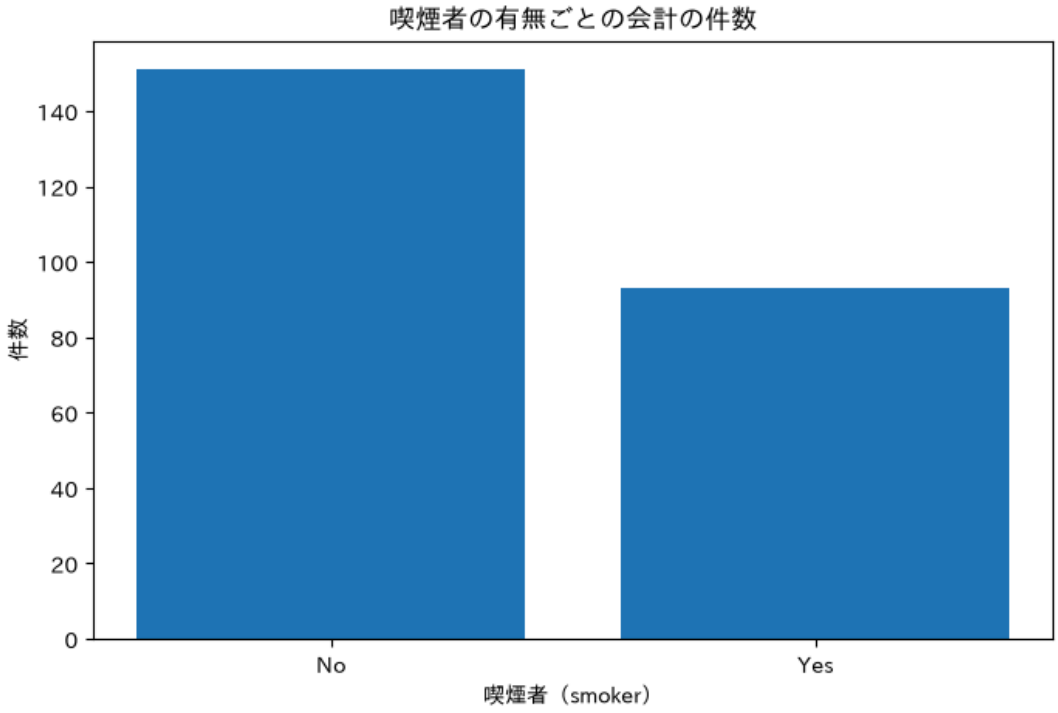
- 左の列（No, Yes）が **index**（数えた値の名前）、右の列が**件数**。件数の多い順に並ぶ
- `count.index` で名前だけ、`count.values` で件数だけを取り出せる

```
print(count.index)
print(count.values)
```

```
Index(['No', 'Yes'], dtype='str', name='smoker')
[151  93]
```

- 名前を横軸、件数を棒の高さにすれば棒グラフになる（`plt.bar(count.index, count)` と書いても同じ）

```
plt.figure(figsize=(8, 5))
plt.bar(count.index, count.values)
plt.title('喫煙者の有無ごとの会計の件数')
plt.xlabel('喫煙者（smoker）')
plt.ylabel('件数')
plt.show()
```



Q-2 辞書の基礎

- **辞書**は、「**キー**（名前）」と「**値**」を `:` でつないだ組を、`{ }` の中に並べたもの
- リストは番号（`[0]`, `[1]`, …）で取り出すが、辞書は**キー**で取り出す
- 例：在籍番号（キー）と名前（値）の組（架空のデータ）

```
students = {'S001': '新潟 太郎', 'S002': '古町 花子', 'S003': '万代 次郎'}

print(students['S002'])
print(len(students))
```

```
古町 花子
3
```

- `辞書['キー']` で、そのキーの値を取り出す
- `len(辞書)` で、組の個数が分かる
- `辞書['キー'] = 値` で、組を**追加**する。すでにあるキーなら、値が**上書き**される

```
students['S004'] = '白山 さくら'
students['S001'] = '新潟 一郎'

print(students)
```

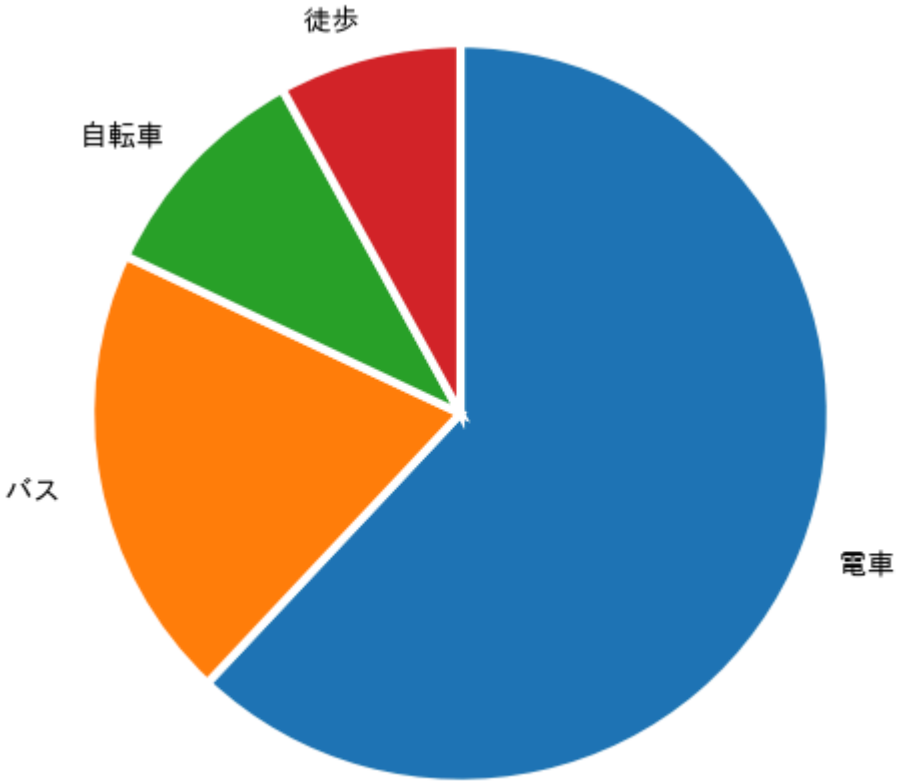
```
{'S001': '新潟 一郎', 'S002': '古町 花子', 'S003': '万代 次郎', 'S004': '白山 さくら'}
```

- 同じキーは2つ持てない。`'S001'` は追加ではなく上書きになった
- ないキーを指定すると `KeyError: 'S005'` というエラーになる（エラー文の最後に、見つからなかったキーが出る）
- Q-1 の `value_counts()` の結果も「名前と件数の組」なので、辞書と同じように `count['No']` で取り出せる（今の `count` は `smoker` 列を数えたもの）

Q-3 オプションの辞書

- 第 6 講の円グラフの `wedgeprops={'edgecolor': 'w'}` は、「扇の部品（wedge）の設定（props）」を辞書で渡していた。キーが**設定の名前**、値が**設定値**
- 設定が2つ以上あるときは、`,` で区切って並べる

```
plt.figure(figsize=(6, 6))
plt.pie([62, 20, 10, 8], labels=['電車', 'バス', '自転車', '徒歩'], startangle=90,
counterclock=False,
        wedgeprops={'edgecolor': 'w', 'linewidth': 3})
plt.show()
```



- `'linewidth': 3` で境目の線を太くした
- このあとの箱ひげ図（S-4）の `boxprops={'facecolor': 'lightblue'}` も同じ形（箱の部品の設定）
- 第 8 講 V-3 の `arrowprops={'arrowstyle': '->'}` も同じ形（矢印の部品の設定）

R. 棒グラフの補足

棒グラフを描くときによく使う操作を4つ補足する。データは Q-1 で読み込んだ `df_tip`（tip_data シート）。R-1 で曜日ごとの件数 `count` を数え直す。

R-1 横軸の名前を書き換える

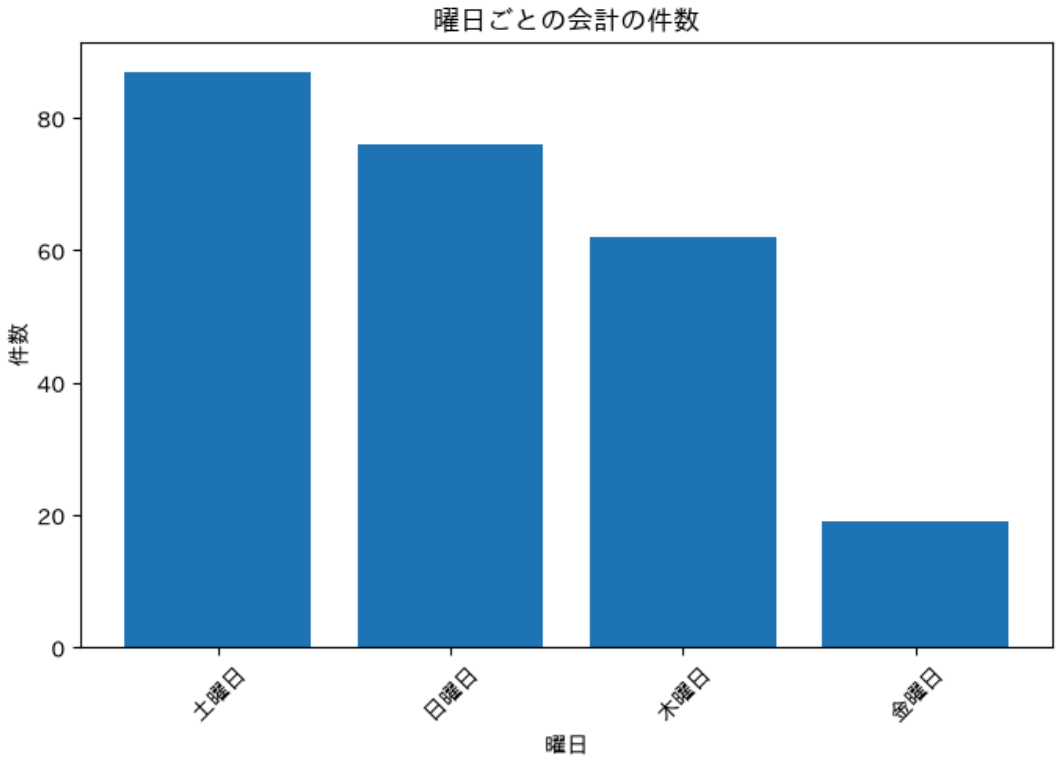
- `plt.xticks(元の名前, 新しい名前)` で、横軸の名前を書き換える。2つのリストは**同じ順番・同じ個数**にする
- 文字が重なるときは `rotation=45`（45度傾ける）

```
count = df_tip['day'].value_counts()
print(count.index)

plt.figure(figsize=(8, 5))
plt.bar(count.index, count.values)
plt.xticks(count.index, ['土曜日', '日曜日', '木曜日', '金曜日'], rotation=45)
plt.title('曜日ごとの会計の件数')
```

```
plt.xlabel('曜日')
plt.ylabel('件数')
plt.show()
```

```
Index(['Sat', 'Sun', 'Thur', 'Fri'], dtype='str', name='day')
```



- `count.index` は Sat, Sun, Thur, Fri の順（件数の多い順）。だから新しい名前も「土曜日, 日曜日, 木曜日, 金曜日」の順に書く
- `rotation=45` で、名前を45度傾けた。長い名前が隣と重なるときに使う
- 箱ひげ図の項目の名前（S-3）も同じ `plt.xticks` で付ける

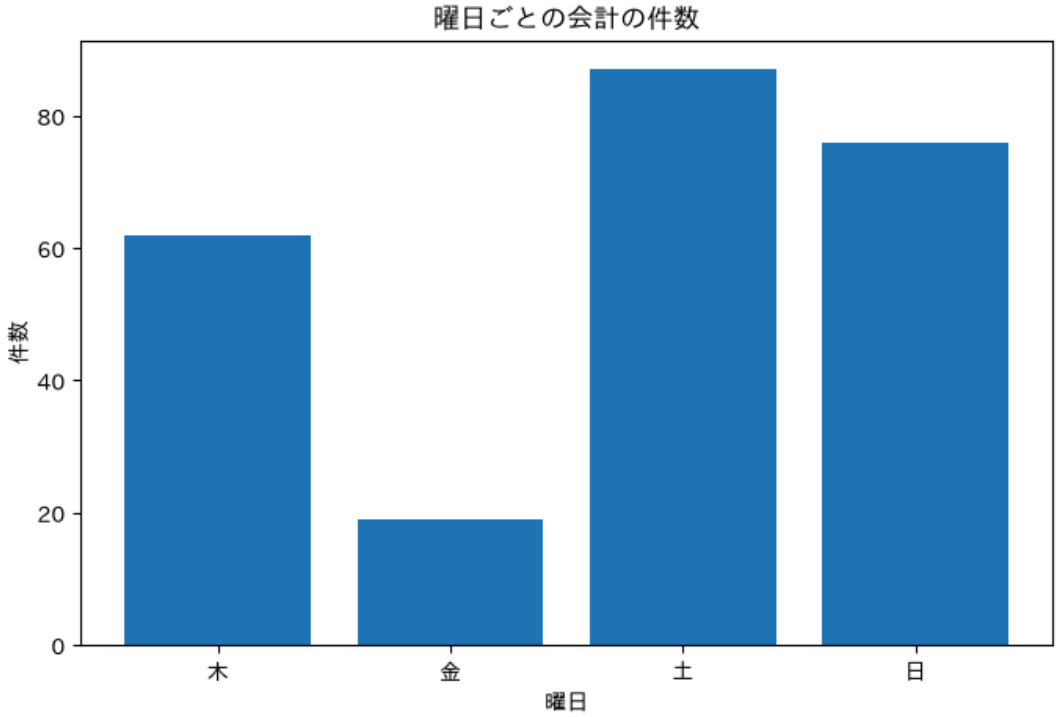
R-2 項目の順番を決める

- `value_counts()` は件数の**多い順**に並べる。曜日のように**決まった順番**で並べたいときは、後ろに `.reindex([順番のリスト])` を付ける
- ここでは曜日の順（木・金・土・日）に並べる

```
count = df_tip['day'].value_counts().reindex(['Thur', 'Fri', 'Sat', 'Sun'])
count
```

```
day
Thur    62
Fri     19
Sat     87
Sun     76
Name: count, dtype: int64
```

```
plt.figure(figsize=(8, 5))
plt.bar(count.index, count.values)
plt.xticks(count.index, ['木', '金', '土', '日'])
plt.title('曜日ごとの会計の件数')
plt.xlabel('曜日')
plt.ylabel('件数')
plt.show()
```

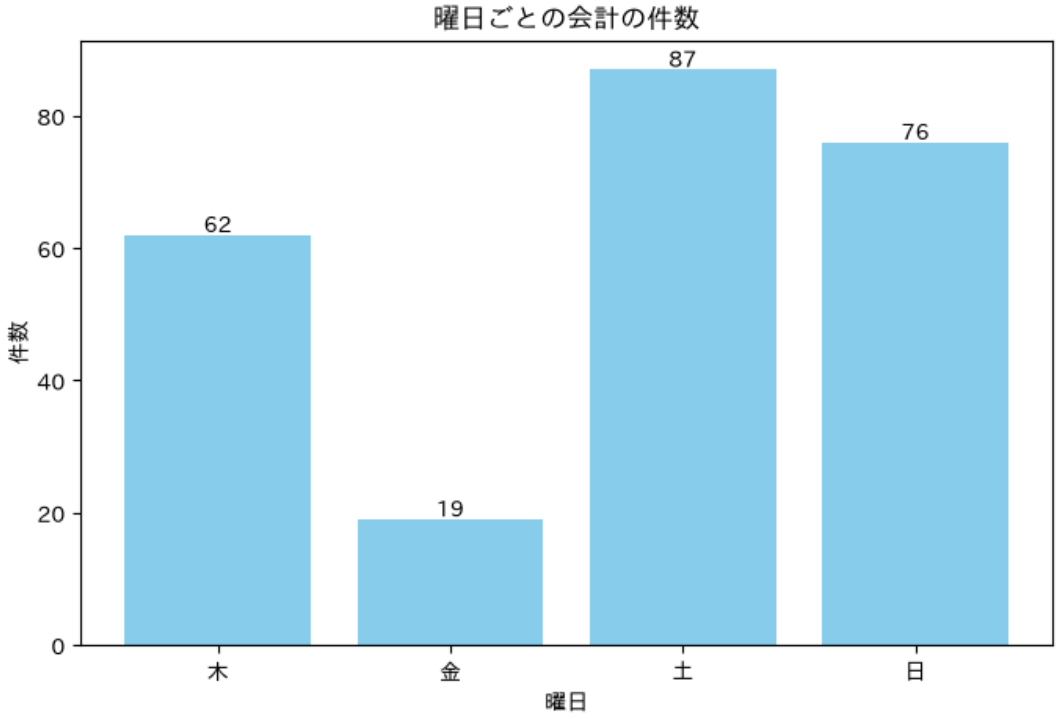


- `reindex` のリストには、`count.index` にある名前を**つづりも同じに**書く（`'Thu'` と書くと、その棒は件数なし（NaN）になる）
- ほかの並べ方：件数の少ない順は `.sort_values()`、名前の順（アルファベット順・数の順）は `.sort_index()`
- 人数（`size`）のように**数**の項目は、横軸の位置が数で決まるので、並べ替えなくても 1, 2, 3, … の順に並ぶ

R-3 棒の上に値を書く

- `plt.bar(...)` の結果を変数（ここでは `b`）に入れ、`plt.bar_label(b)` と書くと、棒の上に高さの値が書かれる

```
plt.figure(figsize=(8, 5))
b = plt.bar(count.index, count.values, color='skyblue')
plt.bar_label(b)
plt.xticks(count.index, ['木', '金', '土', '日'])
plt.title('曜日ごとの会計の件数')
plt.xlabel('曜日')
plt.ylabel('件数')
plt.show()
```



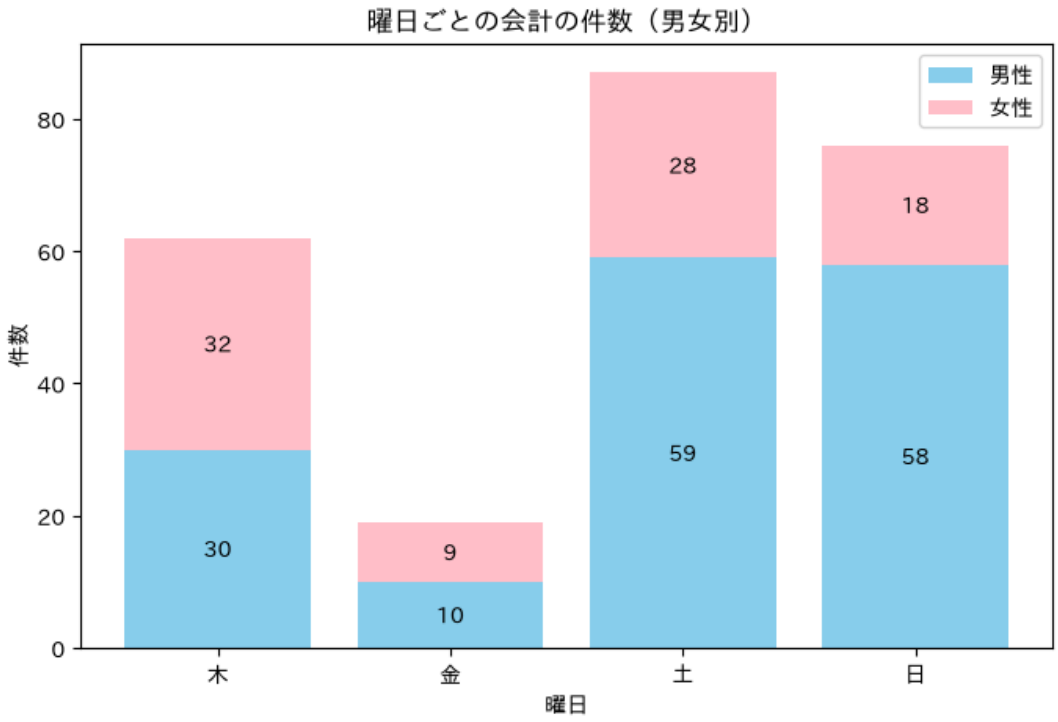
- 小数の桁をそろえるときは `plt.bar_label(b, fmt='%.1f')`（小数第1位まで）。`'%.1f'` は円グラフの `autopct='%.1f%%'`（第 6 講）と同じ書き方
- 棒の**中**に書くときは `label_type='center'`（R-4 で使う）

R-4 積み上げ棒グラフ

- 曜日ごとの件数を、男性と女性に分けて**積み上げる**
- 2回目の `plt.bar` に `bottom=1回目の高さ` を付けると、その上に積まれる。`label=` と `plt.legend()` で凡例を付ける
- 男女別の件数は、ここでは数えた結果を手で書く

```
day = ['Thur', 'Fri', 'Sat', 'Sun']
male = [30, 10, 59, 58]
female = [32, 9, 28, 18]

plt.figure(figsize=(8, 5))
b1 = plt.bar(day, male, color='skyblue', label='男性')
b2 = plt.bar(day, female, bottom=male, color='pink', label='女性')
plt.bar_label(b1, label_type='center')
plt.bar_label(b2, label_type='center')
plt.xticks(day, ['木', '金', '土', '日'])
plt.title('曜日ごとの会計の件数（男女別）')
plt.xlabel('曜日')
plt.ylabel('件数')
plt.legend()
plt.show()
```



- `bottom=male` を消すと、女性の棒が男性の棒に**重なって**描かれる（積み上がらない）
- 男女を足した高さは、R-2 の曜日ごとの件数と同じになる（木：30 + 32 = 62）

応用：株価の動きを描く（ローソク足）

- 株価のグラフでよく見る**ローソク足**は、1日の値動きを1本で表す。1日の株価には4つの値がある

値	意味
始値（はじめね）	その日の最初の値段
終値（おわりね）	その日の最後の値段
高値（たかね）	その日で一番高い値段
安値（やすね）	その日で一番安い値段

- 1本は「**胴体**（始値と終値の間の太い棒）」と「**ひげ**（高値から安値までの細い線）」でできている。終値が始値より高い日（上がった日）を**陽線**、低い日（下がった日）を**陰線**といい、色を分ける（日本では陽線を赤、陰線を青や黒にすることが多い）
- 専用の命令がなくても、**積み上げ棒の** `bottom=` と、縦線・色分けの組み合わせで描ける

部品	書き方	もとにした考え
[1] ひげ	`plt.vlines(x, 下の値, 上の値)` で縦線を引く	新しい命令（縦の線を引くだけ）
[2] 胴体	`plt.bar(x, 終値 - 始値, bottom=始値)`	積み上げ棒の `bottom=`（上の R-4）
[3] 色分け	`np.where(条件, 合うときの値, 合わないときの値)` で色のリストを作る	条件で分ける（`np.where` は第13講 AL-4 も参照）

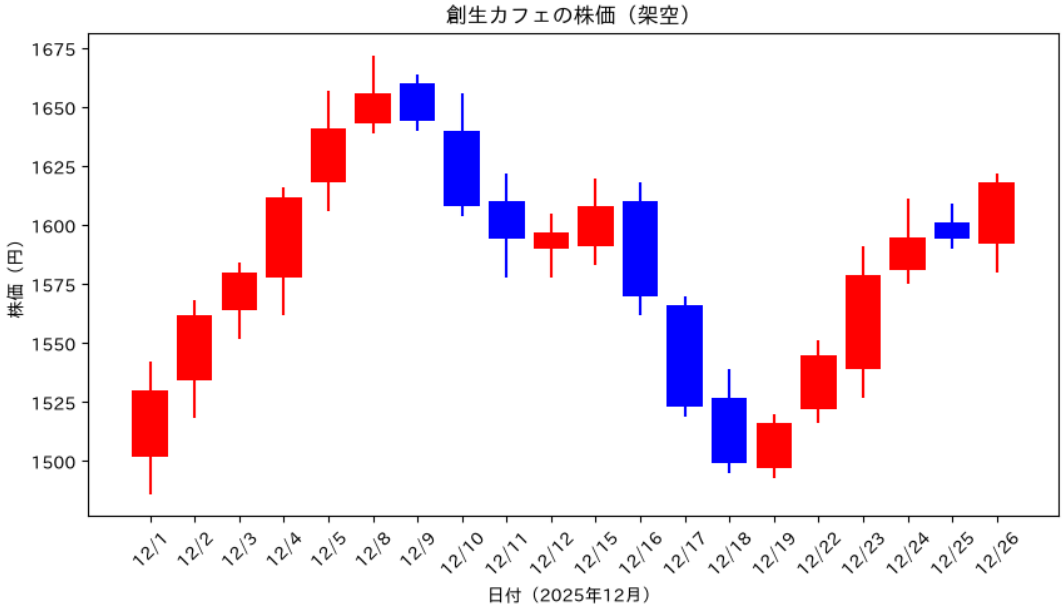
- 下がった日は `終値 - 始値` がマイナスになり、棒は `bottom`（始値）から**下向き**に伸びる。それでちょうど胴体になる

- 使うのは、第11講 AU-1 の `dsa2026review.xlsx` の `cafe_stock` シート（架空のデータ）。notebook と同じ場所に置く。日付は `'12/1'` のような**文字**にしてあるので、土日のすき間はできずに詰めて並ぶ

```
df_stock = pd.read_excel('dsa2026review.xlsx', sheet_name='cafe_stock')

up = df_stock['終値'] >= df_stock['始値']
colors = np.where(up, 'red', 'blue')

plt.figure(figsize=(10, 5))
plt.vlines(df_stock['日付'], df_stock['安値'], df_stock['高値'], color=colors)
plt.bar(df_stock['日付'], df_stock['終値'] - df_stock['始値'], bottom=df_stock['始値'],
color=colors)
plt.xticks(rotation=45)
plt.xlabel('日付（2025年12月）')
plt.ylabel('株価（円）')
plt.title('創生カフェの株価（架空）')
plt.show()
```



- `up` は、行ごとに「上がったか」の `True` ／ `False` が並んだもの。`np.where(up, 'red', 'blue')` で、`True` の行は `'red'`、`False` の行は `'blue'` になる
- ひげ（`vlines`）を先に描くと、胴体（`bar`）がその上に重なる
- 12/1〜12/8 は赤が続いて上がり、12/9〜12/11 と 12/16〜12/18 は青が続いて下がり、12/19 からまた上がっている
- 株価の他に、**1日の気温（最高・最低）**や、テストの点数の**最高・最低・平均**なども同じ形で描ける

S. 箱ひげ図

箱ひげ図は、いくつかのグループの**ばらつき**を並べて比べるグラフ。点数が**どのあたりに集まっているか**を、5つの数にまとめて表す。

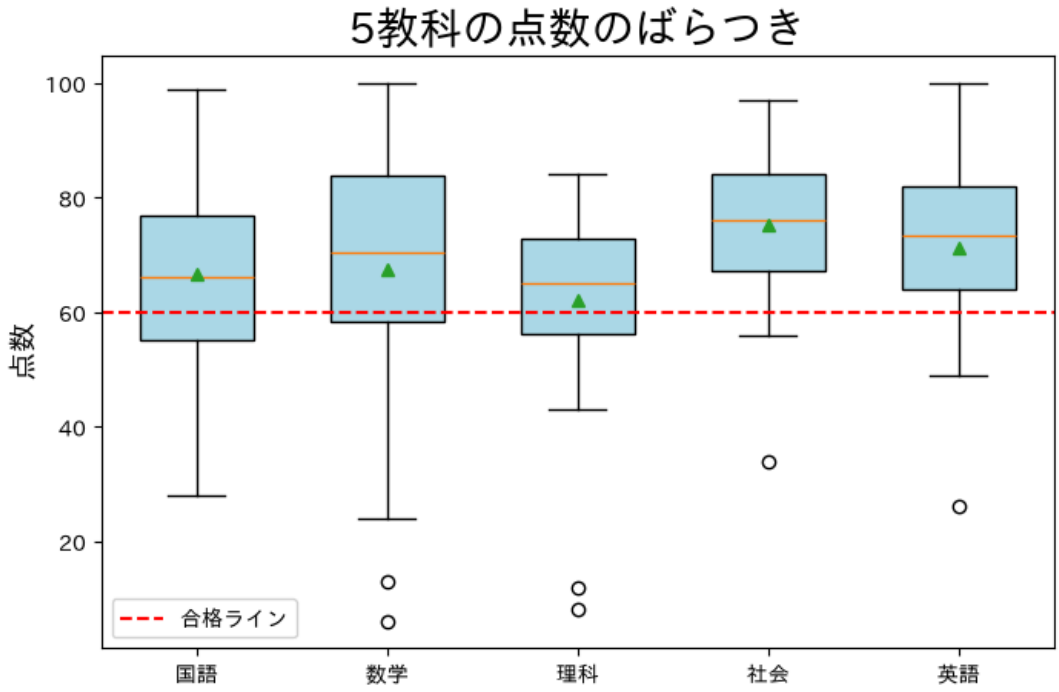
S-0 完成形

このグラフを目指す（30人の5教科の点数のばらつき）。

1. **全部コピーして**自分の notebook のセルに貼り付け、実行する（`Shift + Enter`）
2. まずは**数値や文字を変えて**、グラフのどこが変わるかを確かめる
3. そのあと、箱ひげ図の読み方（S-1）を確かめてから、各部品（[0]〜[7]）の働きを学ぶ（S-2〜S-7）

```
df = pd.read_excel('dsa2026data.xlsx', sheet_name='exam')  # Excel を読む

plt.figure(figsize=(8, 5))  # [0] グラフの大きさ
plt.boxplot([df['国語'], df['数学'], df['理科'], df['社会'], df['英語']],  # [1] 箱ひげ図 [6] Excel の列を並べる
            widths=0.6, patch_artist=True, boxprops={'facecolor': 'lightblue'},  # [3] 箱の幅と塗りつぶし
            showmeans=True)  # [4] 平均
plt.xticks([1, 2, 3, 4, 5], ['国語', '数学', '理科', '社会', '英語'])  # [2] 項目の名前
plt.axhline(y=60, color='r', linestyle='--', label='合格ライン')  # [5] 補助線
plt.title('5教科の点数のばらつき', fontsize=20)  # [7] タイトル
plt.ylabel('点数', fontsize=14)  # [7] 縦軸の名前
plt.legend()  # [7] 凡例
plt.show()
```



- `plt.boxplot(...)` は1つの命令。長いので `,` の後ろで**改行**している（`(` の中なら、次の行も続きとして読まれる。第 6 講 O-0）

変えてみよう（1か所変えたら、もう一度実行する）

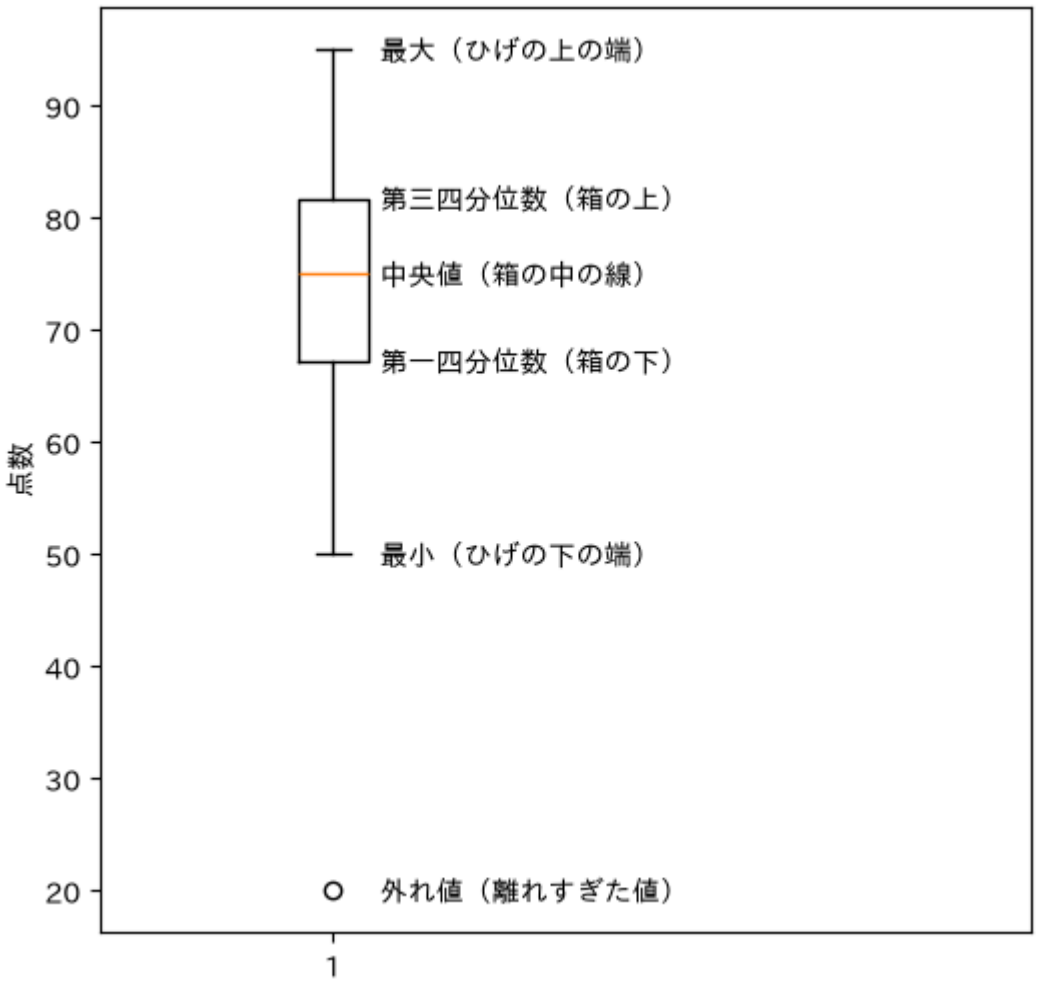
- 色：`'lightblue'` → `'pink'`、`'lightgreen'`
- 幅：`widths=0.6` → `widths=0.3`、`widths=1`
- `showmeans=True` を `showmeans=False` にすると、何が消えるか
- 合格ライン：`y=60` → `y=70`
- `plt.xticks(...)` の行頭に `#` を付けると、横軸の下がどうなるか

番号	部品	書き方	学ぶところ
[0]	グラフの大きさ（基本設定）	`plt.figure(figsize=(8, 5))`	第 3 講
[1]	箱ひげ図を描く	`plt.boxplot([データ1, データ2, …])`	S-2
[2]	項目の名前	`plt.xticks([1, 2, 3], ['名前1', '名前2', '名前3'])`	S-3
[3]	箱の幅と塗りつぶし	`widths=0.6` 、`patch_artist=True, boxprops={'facecolor': '色'}`	S-4
[4]	平均を表示する	`showmeans=True`	S-5
[5]	補助線	`plt.axhline(y=60, label='合格ライン')`	S-6
[6]	Excel の列を並べる	`[df['国語'], df['数学'], …]`	S-7
[7]	タイトル・軸の名前・凡例	`plt.title` 、`plt.legend()` など	第 3 講
	箱ごとに色を変える	`bp['boxes']` と for 文	T-1（発展）

S-1 箱ひげ図の読み方

```
data = [50, 52, 58, 61, 63, 65, 67, 68, 70, 71, 72, 73, 74, 75, 75,
        76, 77, 78, 79, 80, 81, 82, 84, 85, 86, 88, 90, 92, 95, 20]

plt.figure(figsize=(6, 6))
plt.boxplot(data)
plt.text(1.1, np.max(data[:-1]), '最大（ひげの上の端）', va='center')
plt.text(1.1, np.percentile(data, 75), '第三四分位数（箱の上）', va='center')
plt.text(1.1, np.median(data), '中央値（箱の中の線）', va='center')
plt.text(1.1, np.percentile(data, 25), '第一四分位数（箱の下）', va='center')
plt.text(1.1, np.min(data[:-1]), '最小（ひげの下の端）', va='center')
plt.text(1.1, 20, '外れ値（離れすぎた値）', va='center')
plt.xlim(0.5, 2.5)
plt.ylabel('点数')
plt.show()
```



部分	意味
箱の中の線	**中央値**（大きさの順に並べて真ん中の値）
箱	第一四分位数から第三四分位数まで（四分位範囲、第 4 講 G-2）。真ん中の半分（50%）の人が入る範囲。箱が**長いほどばらつきが大きい**
ひげ	外れ値を除いた最小から最大まで
○	**外れ値**（ほかから離れすぎた値）。箱の長さの 1.5 倍より外にある値

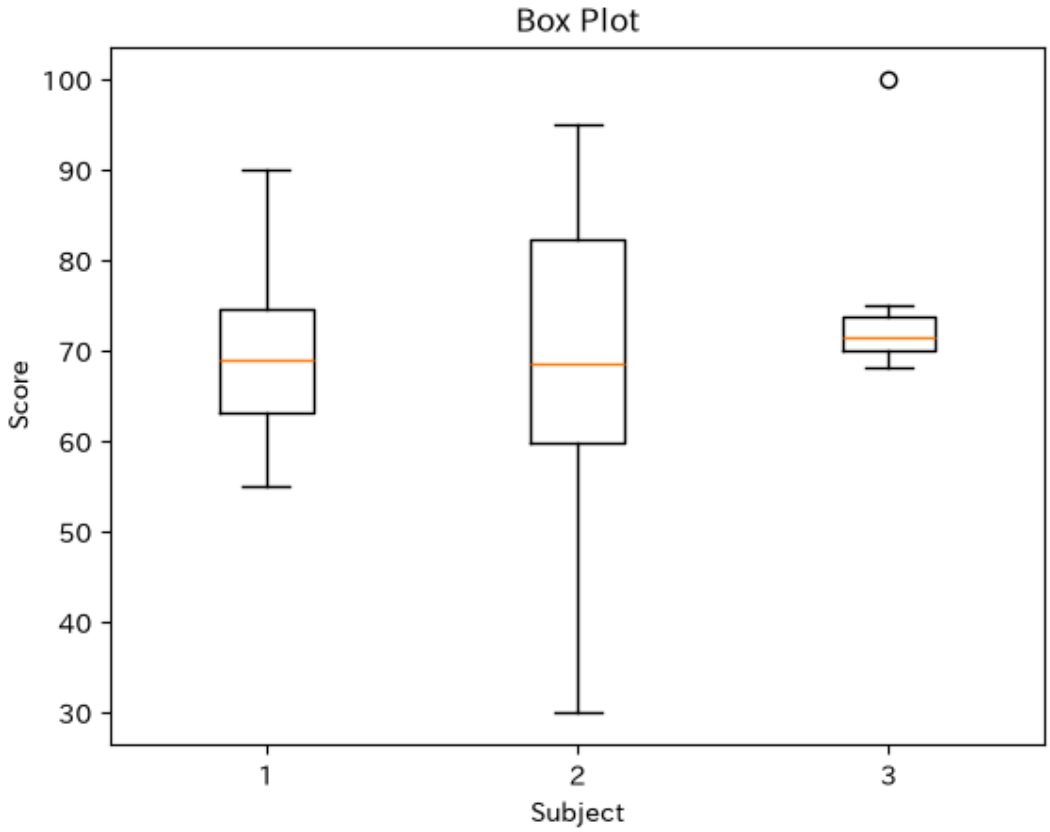
- 上の図の文字は `plt.text`（第 4 講 H-10・第 8 講 V-2）で書いている。ここでは読み方だけ分かればよい

S-2 箱ひげ図の基本

- `plt.boxplot([データ1, データ2, …])` で、グループごとの箱が**左から順に**並ぶ
- データは**リストのリスト**（`[ ]` の中にリストを並べる）で渡す

```
japanese = [62, 75, 58, 81, 70, 66, 90, 55, 73, 68]
math = [45, 88, 62, 95, 30, 71, 84, 59, 77, 66]
english = [70, 72, 68, 75, 74, 69, 71, 73, 100, 70]

plt.boxplot([japanese, math, english])
plt.title('Box Plot')
plt.xlabel('Subject')
plt.ylabel('Score')
plt.show()
```



- 数学（2番目）は箱が長い＝点数がばらばら。英語（3番目）は箱が短い＝みんな同じくらい。英語の 100 点は外れ値（○）
- 箱が1つだけなら `plt.boxplot(japanese)` でもよい

グラフに渡すもの：リストとリストのリスト

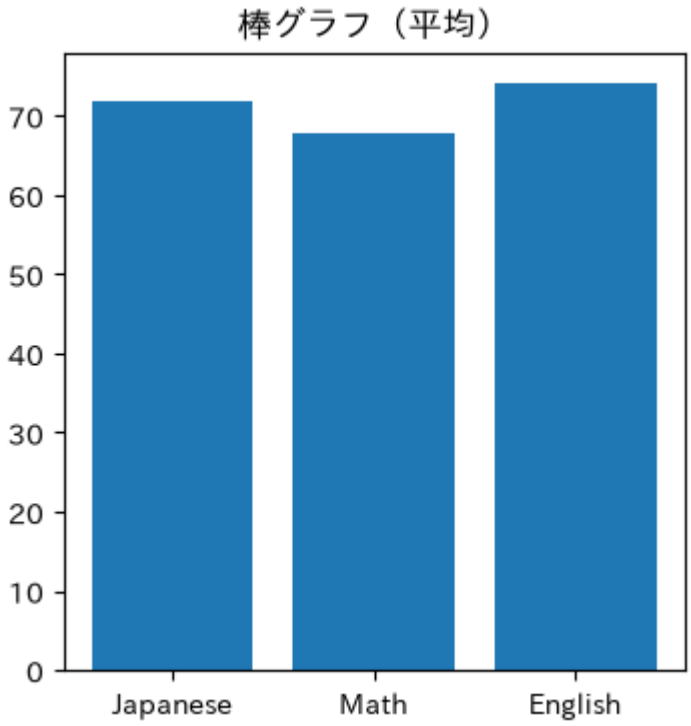
箱ひげ図は `plt.boxplot([japanese, math, english])` と書く。`[ ]` の中にリストが入っていて、これまでのグラフと形が違う。ここで整理する。

基本のグラフ：x と y にリストを1つずつ渡す

- `plt.plot(x, y)`、`plt.scatter(x, y)`、`plt.bar(x, y)` は、x のリストと y のリストを渡す
- x と y の**同じ位置の値が組**になって、点や棒が1つ決まる。つまり、**点や棒1つに、数が1つ**

```
x = ['Japanese', 'Math', 'English']
y = [71.8, 67.7, 74.2]

plt.figure(figsize=(4, 4))
plt.bar(x, y)
plt.title('棒グラフ（平均）')
plt.show()
```



- `'Japanese'` と `71.8`、`'Math'` と `67.7`、`'English'` と `74.2` が組になって、棒が3本描かれる

箱ひげ図：箱1つに、リスト1つ

- 箱を1つ描くには、中央値や箱の範囲を求めるために、**その教科の全員の点数**が必要。数1つではなく、**リスト1つ**を渡す
- 箱を3つ描くには、リストが3つ必要。この3つを `[ ]` で1つにまとめて渡す。リストを並べたリストなので、**リストのリスト**と呼ぶ

```
scores = [japanese, math, english]

print(scores)
print(len(scores))
print(scores[0])
```

```
[[62, 75, 58, 81, 70, 66, 90, 55, 73, 68], [45, 88, 62, 95, 30, 71, 84, 59, 77, 66], [70, 72, 68, 75, 74, 69, 71, 73, 100, 70]]
3
[62, 75, 58, 81, 70, 66, 90, 55, 73, 68]
```

- `scores` の中身は `[[国語の10人], [数学の10人], [英語の10人]]`。`[ ]` が二重になっている
- `len(scores)` は 3。人数の 30 ではなく、**教科の数**（リストが3つ）
- `scores[0]` で、最初のリスト（国語）を取り出せる

グラフ	書き方	渡すもの
折れ線・散布図・棒グラフ	`plt.plot(x, y)` など	x のリストと y のリスト
箱ひげ図	`plt.boxplot([データ1, データ2])`	データの**リストのリスト**

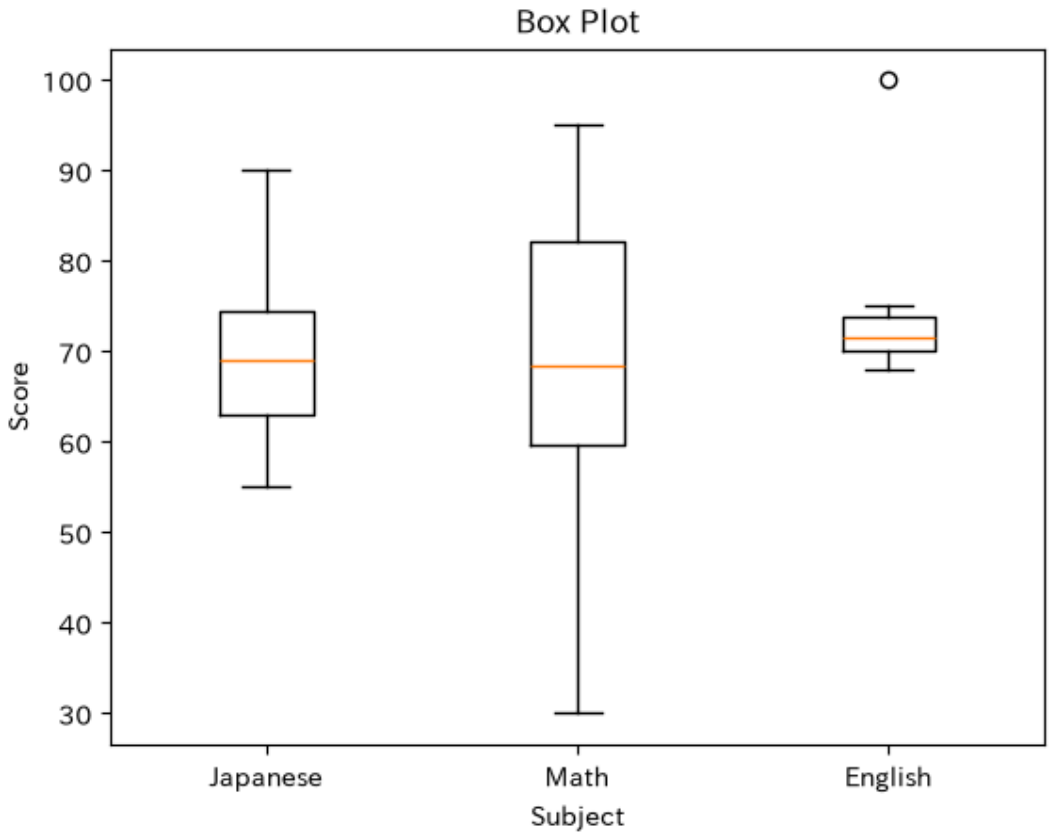
- 箱ひげ図には x を渡さない。箱は左から 1, 2, … の位置に並ぶ。名前は次の S-3 で付ける
- **注意**：`[ ]` を忘れて `plt.boxplot(japanese, math)` と書くと、エラーにならずに、**箱が1つだけ**描かれることがある。箱の数がおかしいときは `[ ]` を確かめる

- `[ ]` の中には、Excel の列も並べられる（S-7）

S-3 項目の名前

- 横軸の下には `1, 2, 3` と番号が付く。`plt.xticks([1, 2, 3], ['名前1', '名前2', '名前3'])` で名前に置き換える
- 番号と名前は**同じ個数**にする

```
plt.boxplot([japanese, math, english])
plt.xticks([1, 2, 3], ['Japanese', 'Math', 'English'])
plt.title('Box Plot')
plt.xlabel('Subject')
plt.ylabel('Score')
plt.show()
```



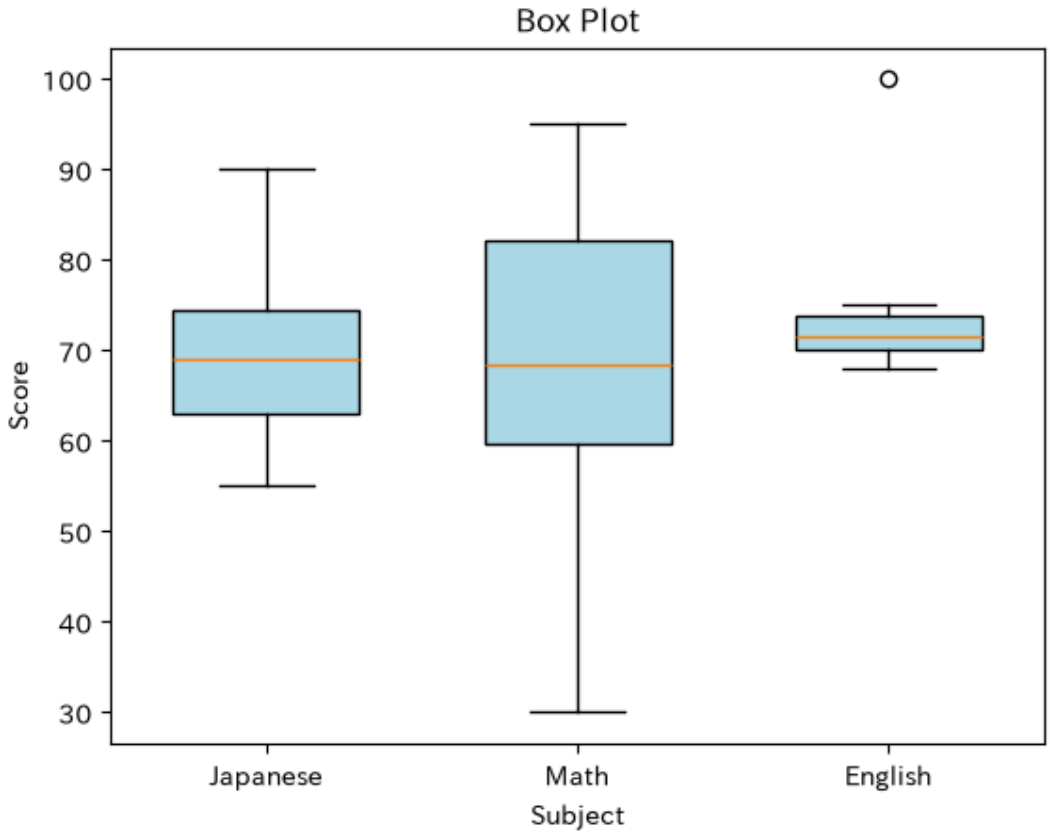
- `plt.boxplot(..., tick_labels=['Japanese', 'Math', 'English'])` と書いても同じ（matplotlib 3.9 より古いと `labels=`）。どの環境でも動くので、この授業では `plt.xticks` を使う

S-4 箱の幅と塗りつぶし

- `widths=0.6` で箱の幅を変える（何も書かないと、箱の数に合わせて 0.15〜0.5 の間で決まる。`1` で隣の箱とくっつく）
- `patch_artist=True` で箱の中を**塗りつぶす**（何も書かないと白抜き）。塗る色は `boxprops={'facecolor': '色'}`

```
plt.boxplot([japanese, math, english], widths=0.6, patch_artist=True, boxprops={'facecolor':
'lightblue'})
plt.xticks([1, 2, 3], ['Japanese', 'Math', 'English'])
```

```
plt.title('Box Plot')
plt.xlabel('Subject')
plt.ylabel('Score')
plt.show()
```

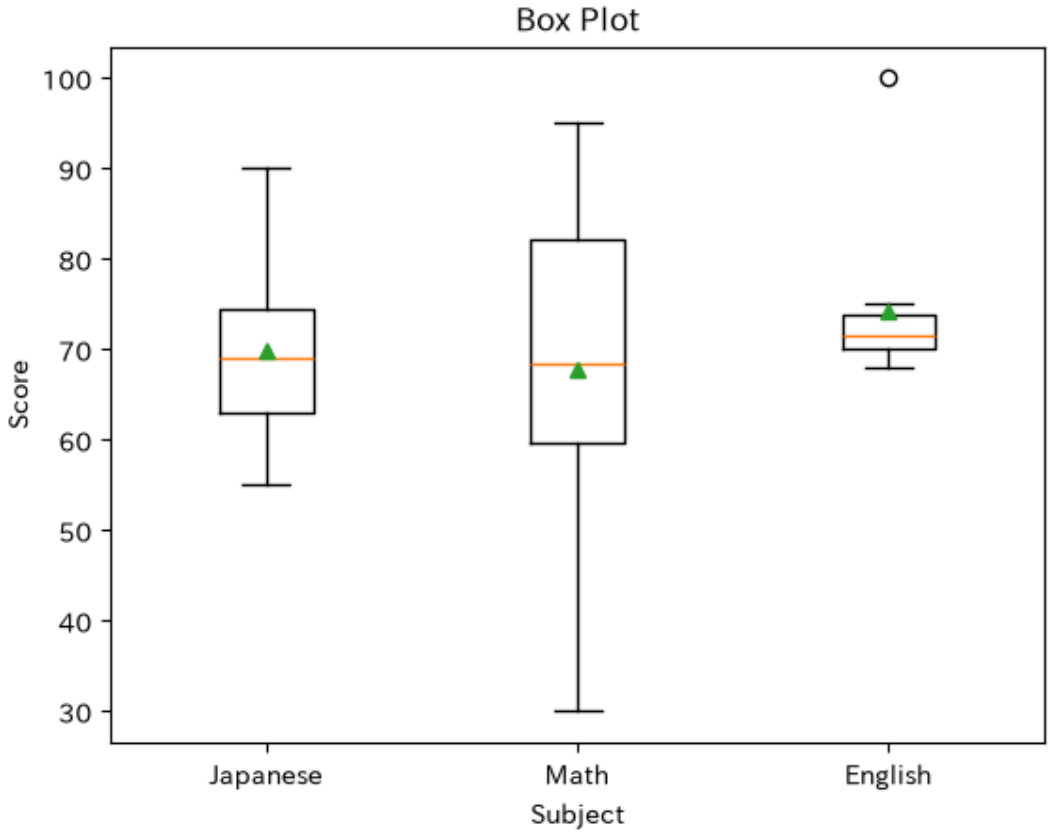


- `boxprops={'facecolor': 'lightblue'}` は Q-3 のオプションの辞書。「箱の部品の、中の色は lightblue」と読む

S-5 平均を表示する

- 箱の中の線は**中央値**で、平均ではない。`showmeans=True` で**平均**を ▲ で表示する
- 外れ値があると、平均は中央値から外れ値の側へずれる

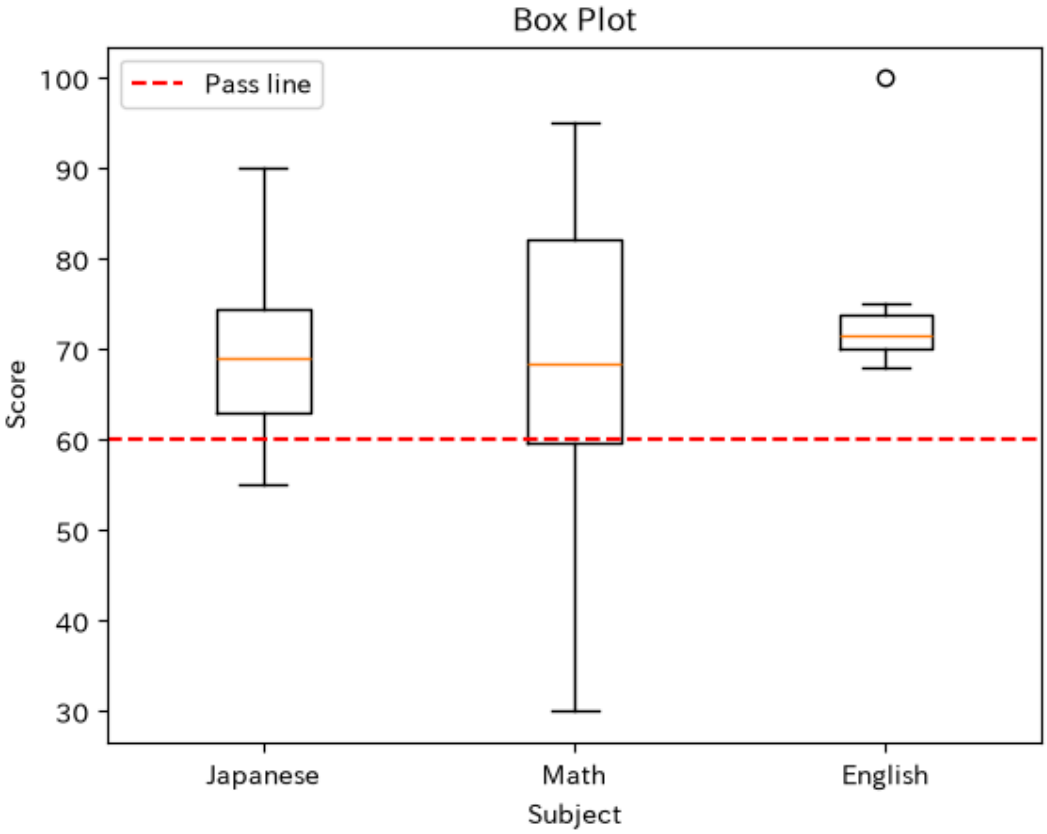
```
plt.boxplot([japanese, math, english], showmeans=True)
plt.xticks([1, 2, 3], ['Japanese', 'Math', 'English'])
plt.title('Box Plot')
plt.xlabel('Subject')
plt.ylabel('Score')
plt.show()
```



S-6 補助線を引く

- 第 4 講の `plt.axhline(y=値)` が箱ひげ図でも使える。合格ラインや全体の平均を引く

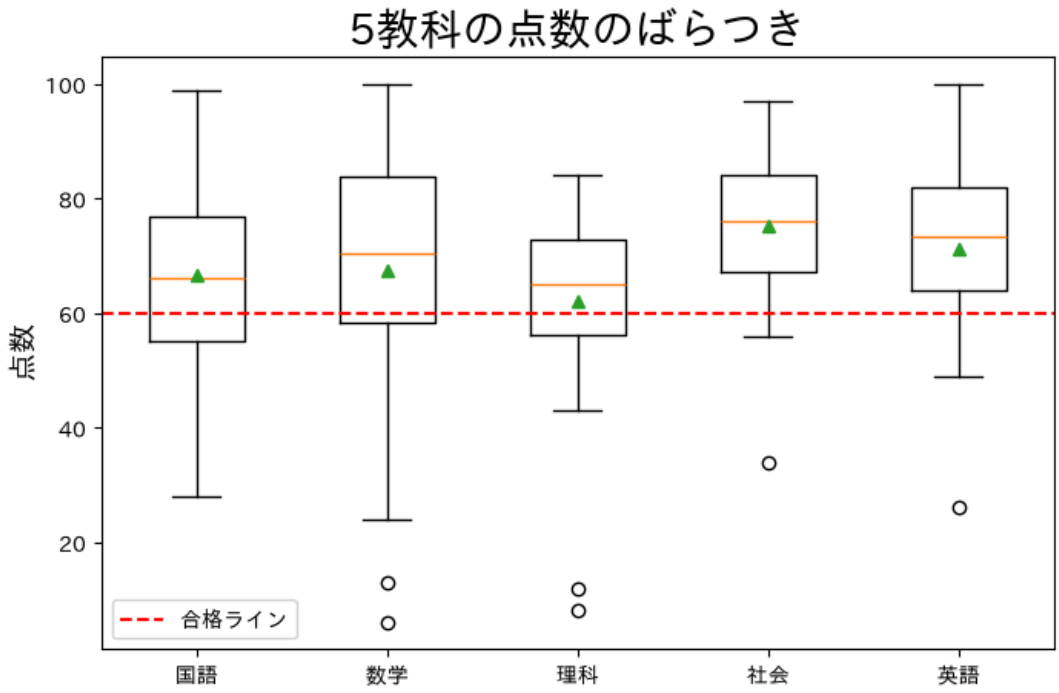
```
plt.boxplot([japanese, math, english])
plt.axhline(y=60, color='r', linestyle='--', label='Pass line')
plt.xticks([1, 2, 3], ['Japanese', 'Math', 'English'])
plt.title('Box Plot')
plt.xlabel('Subject')
plt.ylabel('Score')
plt.legend()
plt.show()
```



S-7 Excel のデータで箱ひげ図

- `exam` シートの5教科を並べる。`df['国語']` などを**リストの中に**並べて渡す
- `df['国語']` は1列分のデータで、リスト1つと同じように使える。箱1つに1列で、S-2 のリストのリストと同じ形になる

```
plt.figure(figsize=(8, 5))
plt.boxplot([df['国語'], df['数学'], df['理科'], df['社会'], df['英語']], showmeans=True)
plt.xticks([1, 2, 3, 4, 5], ['国語', '数学', '理科', '社会', '英語'])
plt.axhline(y=60, color='r', linestyle='--', label='合格ライン')
plt.title('5教科の点数のばらつき', fontsize=20)
plt.ylabel('点数', fontsize=14)
plt.legend()
plt.show()
```



使ってみよう

- 社会は箱が短く、数学は長い。点数のばらつきが大きいのはどの教科か
- 理科の下にある ○ は誰の点数か、`df` の表で探してみよう

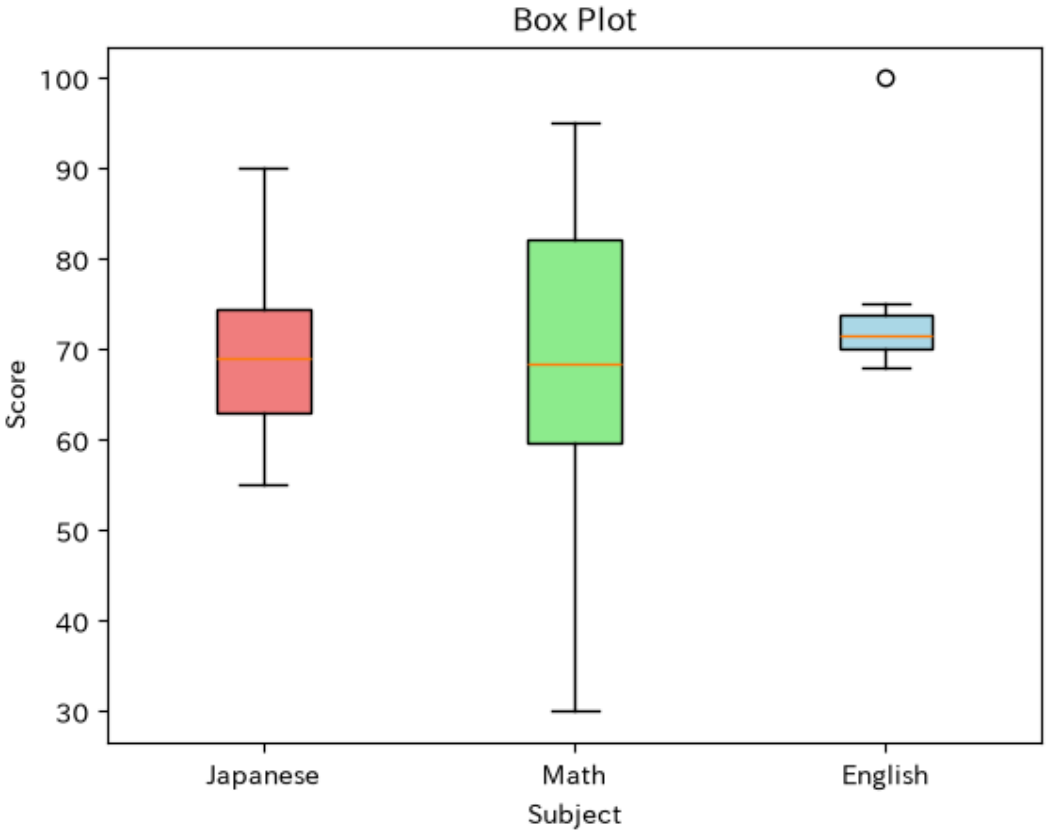
T. 発展

T-1 箱ごとに色を変える

- `patch_artist=True` を付けて描いた結果を変数（ここでは `bp`）に入れると、`bp['boxes']` で箱を1つずつ取り出せる。`bp` は Q-2 の辞書で、`'boxes'` がキー
- for 文（第12講）で、箱ごとに色を塗る

```
bp = plt.boxplot([japanese, math, english], patch_artist=True)
mycolor = ['lightcoral', 'lightgreen', 'lightblue']
for box, c in zip(bp['boxes'], mycolor):
    box.set_facecolor(c)

plt.xticks([1, 2, 3], ['Japanese', 'Math', 'English'])
plt.title('Box Plot')
plt.xlabel('Subject')
plt.ylabel('Score')
plt.show()
```



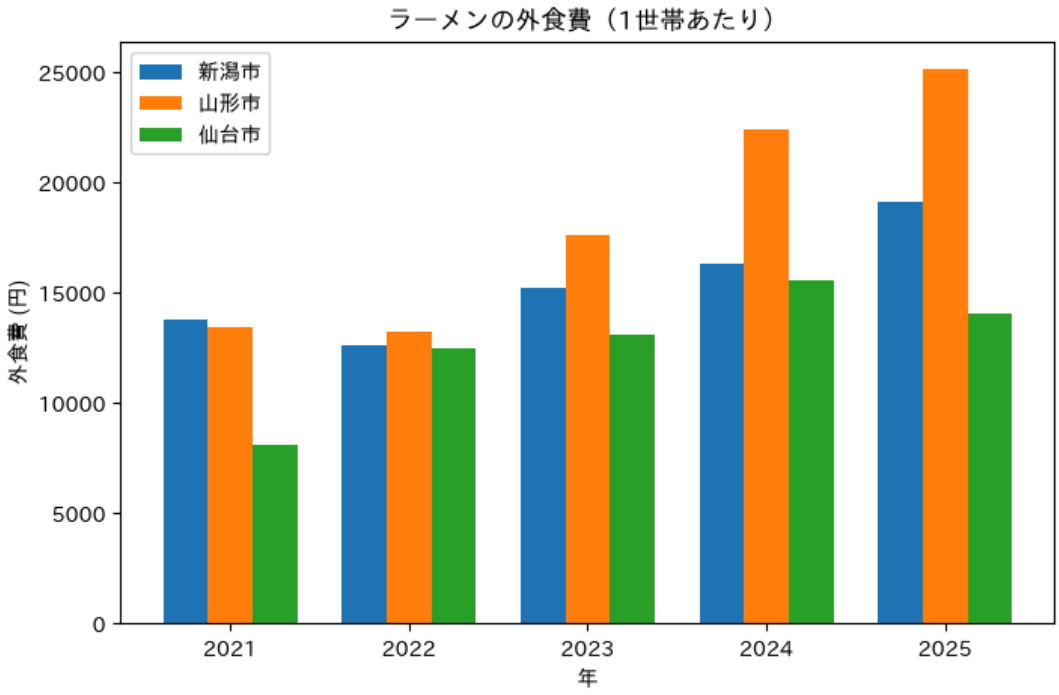
T-2 棒を横に並べる

- 新潟市・山形市・仙台市のラーメンの外食費（ramen シート）を、年ごとに横に並べて比べる。数値はリストで書いておく
- `np.arange(len(year))` で、棒を置く位置 0, 1, 2, 3, 4 を作る
- 3本の棒を `- w`・そのまま・`+ w` だけずらして置き、`width=w` で幅を細くする（リストには `- w` ができないので、np を使う）
- 横軸の名前は R-1 と同じく `plt.xticks(位置, 名前)` で付ける
- R-4 の積み上げは「上に重ねる」、こちらは「横に並べる」
- 変えてみよう：`x - w` を `x` にすると、棒が重なる

```
year = ['2021', '2022', '2023', '2024', '2025']
niigata = [13734, 12573, 15224, 16292, 19073]
yamagata = [13434, 13196, 17593, 22389, 25102]
sendai = [8067, 12480, 13074, 15534, 14013]

x = np.arange(len(year))   # 0, 1, 2, 3, 4（棒を置く位置）
w = 0.25                   # 棒の幅

plt.figure(figsize=(8, 5))
plt.bar(x - w, niigata, width=w, label='新潟市')
plt.bar(x, yamagata, width=w, label='山形市')
plt.bar(x + w, sendai, width=w, label='仙台市')
plt.xticks(x, year)
plt.title('ラーメンの外食費（1世帯あたり）')
plt.xlabel('年')
plt.ylabel('外食費（円）')
plt.legend()
plt.show()
```



まとめ

やりたいこと	書き方
以下は値を数える・辞書	
値を数える	`count = df['列名'].value_counts()`
名前・件数を取り出す	`count.index`（名前）、`count.values`（件数）
辞書を作る	`d = {'キー1': 値1, 'キー2': 値2}`
取り出す・追加する	`d['キー']`、`d['キー'] = 値`（あるキーなら上書き）
オプションの辞書	`wedgeprops={'edgecolor': 'w', 'linewidth': 3}`
以下は棒グラフの補足	
横軸の名前を書き換える	`plt.xticks(count.index, ['新しい名前', …])`、傾けるなら `rotation=45`
項目の順番	`.reindex(['Thur', 'Fri', 'Sat', 'Sun'])`、`.sort_values()`、`.sort_index()`
棒の上に値	`b = plt.bar(...)` → `plt.bar_label(b)`（`fmt='%.1f'`、`label_type='center'`）
積み上げ	2回目の `plt.bar` に `bottom=1回目の高さ`
ローソク足	ひげは `plt.vlines(x, 安値, 高値)`、胴体は `plt.bar(x, 終値 - 始値, bottom=始値)`、色は `np.where(終値 >= 始値, 'red', 'blue')`
以下は箱ひげ図	

やりたいこと	書き方
箱ひげ図	`plt.boxplot([データ1, データ2, …])`（データの**リストのリスト**を渡す。箱1つにリスト1つ）
項目の名前	`plt.xticks([1, 2, 3], ['名前1', '名前2', '名前3'])`
箱の幅・塗りつぶし	`widths=0.6`、`patch_artist=True, boxprops={'facecolor': '色'}`
平均を表示	`showmeans=True`
補助線	`plt.axhline(y=60)`

第 8 講：複合グラフと注釈

棒グラフと折れ線グラフを重ね、左右に y 軸を持つ**複合グラフ**を描く（Z）。大きな数の単位の変え方・指数化・対数目盛もここで学ぶ。最後に、グラフに文字や矢印を書く**注釈**を学ぶ（V。矢印の指定には第 7 講 Q-3 の辞書を使う）。

準備：この講で使うライブラリを読み込み、データを用意する。

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import japanize_matplotlib

df = pd.read_excel('dsa2026data.xlsx', sheet_name='降水量')
df
```

	月	降水量	気温
0	1	180.0	4.3
1	2	114.5	4.8
2	3	168.0	6.0
3	4	54.0	13.7
4	5	128.0	17.1
5	6	57.0	22.2
6	7	376.5	26.2
7	8	42.0	28.0
8	9	292.0	25.0
9	10	144.5	18.7
10	11	246.5	11.5
11	12	262.5	4.9

- 表が出れば準備完了。`FileNotFoundError` が出たら、`dsa2026data.xlsx` を notebook と同じ場所に置く（第 5 講 L-1）

Z. 複合グラフ・両 y 軸

Z-0 完成形

降水量（棒グラフ）と気温（折れ線グラフ）を1枚に重ね、左に降水量、右に気温の y 軸を付けたもの。大雨の7月には矢印で、最高気温の8月には文字で説明を書いている。

1. **全部コピーして**自分の notebook のセルに貼り付け、実行する
2. まずは**数値や文字を変えて**、グラフのどこが変わるかを確かめる
3. そのあと部品（[1]〜[5]）の働きを1つずつ学ぶ（Z-1〜Z-2、V-1〜V-3）

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import japanize_matplotlib

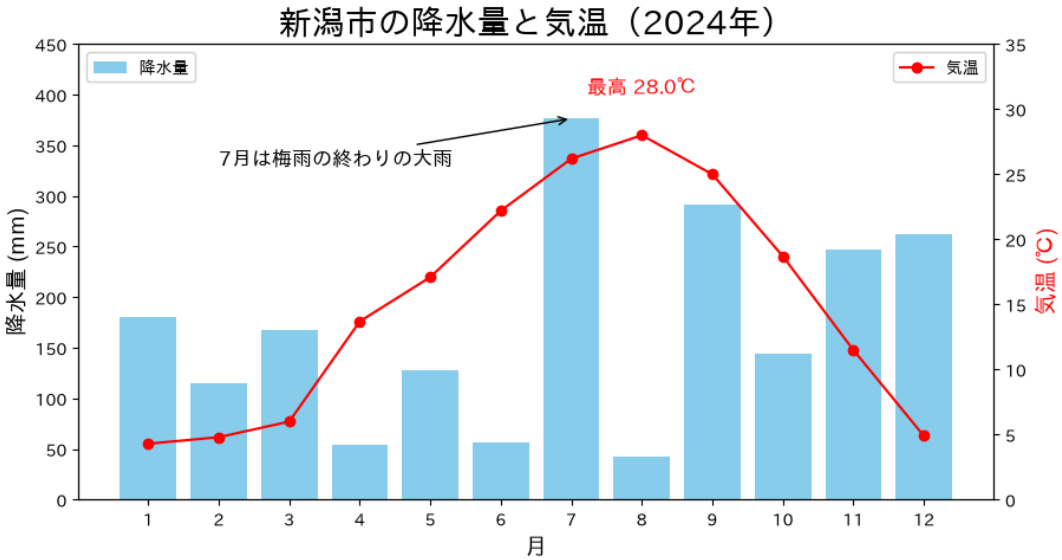
df = pd.read_excel('dsa2026data.xlsx', sheet_name='降水量')

plt.figure(figsize=(10, 5))  # [0] グラフの大きさ

plt.bar(df['月'], df['降水量'], color='skyblue', label='降水量')  # [1] 左の y 軸に棒グラフ
plt.ylabel('降水量 (mm)', fontsize=14)
plt.ylim(0, 450)
plt.annotate('7月は梅雨の終わりの大雨', xy=(7, 376.5), xytext=(2, 330), fontsize=12,
             arrowprops={'arrowstyle': '->'})  # [4] 矢印付きの注釈
plt.xlabel('月', fontsize=14)
plt.xticks(range(1, 13))
plt.title('新潟市の降水量と気温（2024年）', fontsize=20)
plt.legend(loc='upper left')

plt.twinx()  # [2] 右にもう1本 y 軸を作る
plt.plot(df['月'], df['気温'], color='r', marker='o', label='気温')  # [3] 右の y 軸に折れ線グラフ
plt.ylabel('気温 (℃)', color='r', fontsize=14)
plt.ylim(0, 35)
plt.text(8, 31, '最高 28.0℃', color='r', ha='center', va='bottom', fontsize=12)  # [5] 文字を書く
plt.legend(loc='upper right')

plt.show()
```



変えてみよう（1か所変えたら、もう一度実行する）

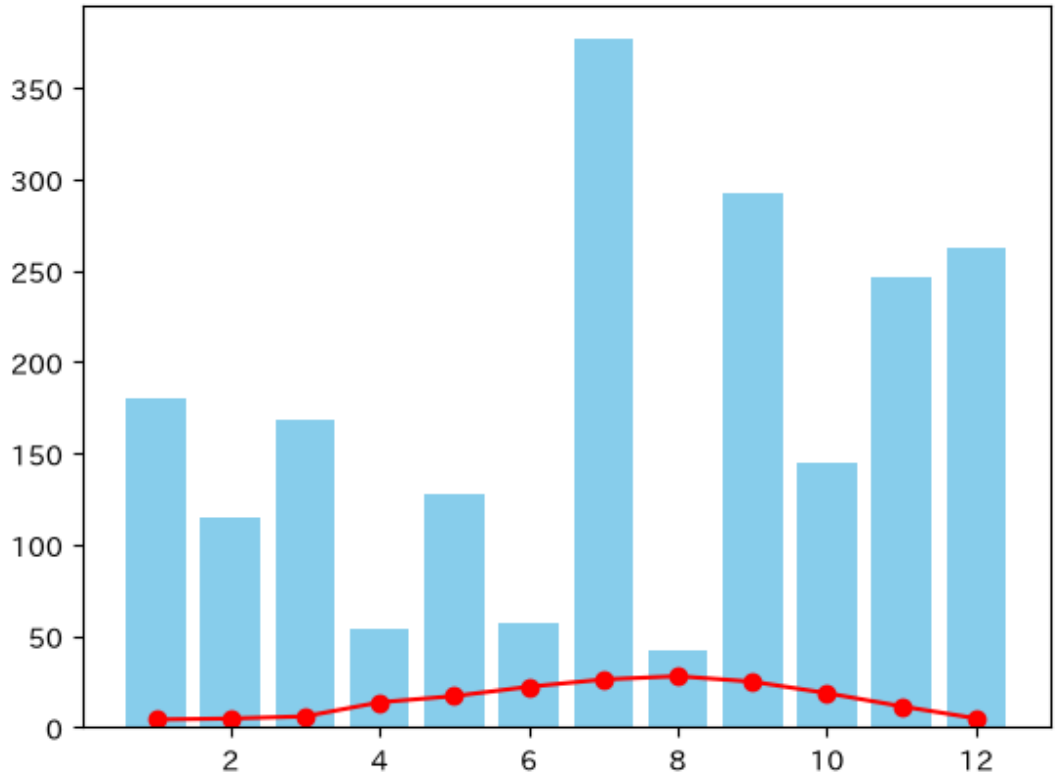
- `plt.twinx()` の行を消すと、気温の線がどうなるか（Z-1）
- `xytext=(2, 330)` → `xytext=(9, 420)`（説明の文字が動く。矢印の先は動かない）
- `plt.text(8, 31, …)` の `31` → `33`
- `color='r'` → `color='orange'`

番号	部品	書き方	学ぶところ
[0]	グラフの大きさ（書かなくてもよい）	`plt.figure(figsize=(10, 5))`	
[1]	左の y 軸に棒グラフ	`plt.bar(…)` 、`plt.ylabel('降水量（mm）')`	Z-2
[2]	右にもう1本 y 軸を作る	`plt.twinx()`	Z-2
[3]	右の y 軸に折れ線グラフ	`plt.plot(…)` 、`plt.ylabel('気温（℃）', color='r')`	Z-2
[4]	矢印付きの注釈	`plt.annotate('文字', xy=(…), xytext=(…), arrowprops={…})`	V-3
[5]	文字を書く	`plt.text(横の位置, 縦の位置, '文字')`	V-1・V-2

Z-1 2つのグラフを重ねる

- 降水量（棒グラフ）と気温（折れ線グラフ）を1枚に描きたい。そのまま重ねると、単位（mm と ℃）も大きさも違うので、気温の線が下につぶれる

```
plt.bar(df['月'], df['降水量'], color='skyblue')
plt.plot(df['月'], df['気温'], color='r', marker='o')
plt.show()
```



Z-2 右にもう1本 y 軸を作る（ひな形）

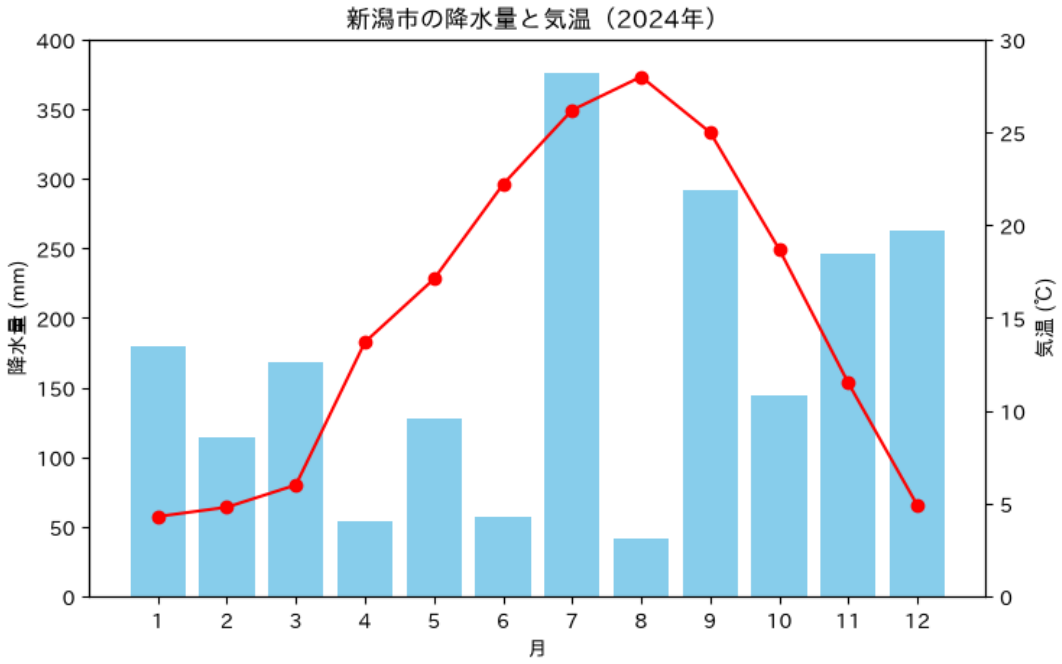
- `plt.twinx()` で、**右側にもう1本 y 軸**を作る。`plt.twinx()` の**前**に書いたものは左の y 軸、**後**に書いたものは右の y 軸に付く
- **下のセルはひな形**。列名・ラベル・色を変えれば、そのまま使える

```
plt.figure(figsize=(8, 5))

# 左の y 軸（棒グラフ）と、両方に共通する設定
plt.bar(df['月'], df['降水量'], color='skyblue', label='降水量')
plt.ylabel('降水量 (mm)')
plt.ylim(0, 400)
plt.xlabel('月')
plt.title('新潟市の降水量と気温（2024年）')
plt.xticks(range(1, 13))

# 右の y 軸を追加（折れ線グラフ）
plt.twinx()
plt.plot(df['月'], df['気温'], color='r', marker='o', label='気温')
plt.ylabel('気温 (℃)')
plt.ylim(0, 30)

plt.show()
```



- 横軸の設定（`xlabel`・`xticks`）と `title` は、`plt.twinx()` の**前**に書く
- どちらの軸がどのグラフか分かるように、**y 軸の名前に単位**を書く
- `plt.ylim(0, …)` は、`plt.twinx()` の前に書けば左、後に書けば右の y 軸の範囲になる。**左右とも 0 から始める**のが基本。書かないと、気温の軸は 3℃ くらいから始まり、1月の気温が 0℃ 近くに見えてしまう
- 凡例は、左右で別々になる。`plt.legend(loc='upper left')` と `plt.legend(loc='upper right')` のように、位置を分けて書く（Z-0）

右の y 軸の色を変える

- 右の y 軸の名前・目盛り・枠線を、折れ線と同じ色にすると、どちらの軸を読めばよいかが一目で分かる
- `plt.twinx()` の**後**に書くと、右の y 軸の色が変わる

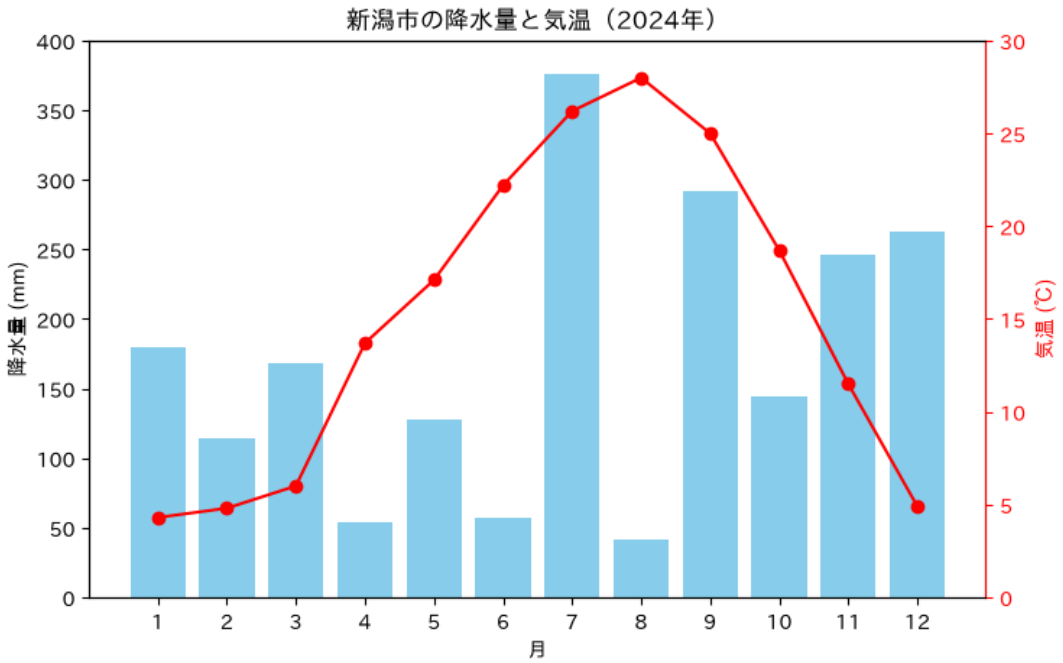
```
plt.figure(figsize=(8, 5))

plt.bar(df['月'], df['降水量'], color='skyblue', label='降水量')
plt.ylabel('降水量 (mm)')
plt.ylim(0, 400)
```

```
plt.xlabel('月')
plt.title('新潟市の降水量と気温（2024年）')
plt.xticks(range(1, 13))

plt.twinx()
plt.plot(df['月'], df['気温'], color='r', marker='o', label='気温')
plt.ylabel('気温（℃）', color='r')          # 軸の名前の色
plt.ylim(0, 30)
plt.tick_params(axis='y', colors='r')      # 目盛りと目盛りの数字の色
plt.gca().spines['right'].set_color('r')  # 右の枠線の色

plt.show()
```



- 軸の名前の色は `plt.ylabel(…, color='r')`、目盛り（短い線と数字）の色は `plt.tick_params(axis='y', colors='r')`
- `tick_params` では `colors`（**s が付く**）と書く。`color` だと短い線だけが赤くなり、数字は黒のまま
- 数字だけを赤にするなら `plt.yticks(color='r')` でもよい
- 右の枠線（軸の縦線）の色は `plt.gca().spines['right'].set_color('r')`。`plt.gca()` は「今のグラフ」、`spines['right']` は「右の枠線」という意味。左なら `'left'`
- 左の y 軸の名前と目盛りも同じように、`plt.twinx()` の**前**に書けば色を変えられる（例：`plt.tick_params(axis='y', colors='tab:blue')`）。ただし、**左の枠線だけは `plt.twinx()` の後**に書く（下の「twinx を使うときの枠線」）

twinx を使うときの枠線（`plt.gca()` を書く場所）

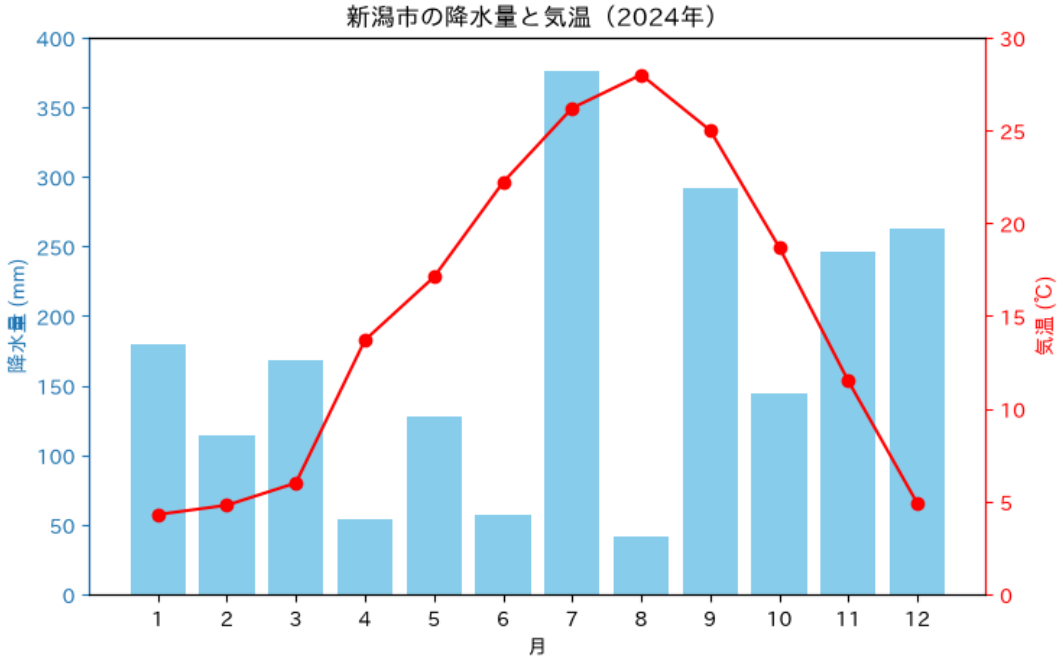
- `plt.twinx()` を実行すると、右の y 軸を持つ**新しいグラフが、元のグラフの上に重ねられる**
- 新しいグラフにも上下左右の4本の枠線（黒）がある。`plt.twinx()` の**前**に左の枠線を青にしても、その上に新しいグラフの**黒い左の枠線が重なって隠れる**ので、色が変わらないように見える
- `plt.twinx()` の後の `plt.gca()` は、上に重なった新しいグラフを指す。そこで、左右の枠線の色は**どちらも `plt.twinx()` の後**（`plt.show()` の直前）に書く

```
plt.figure(figsize=(8, 5))
```

```
plt.bar(df['月'], df['降水量'], color='skyblue', label='降水量')
plt.ylabel('降水量 (mm)', color='tab:blue')
plt.ylim(0, 400)
plt.tick_params(axis='y', colors='tab:blue')          # 左の目盛り：twinx の前
plt.gca().spines['left'].set_color('tab:blue')        # ×ここに書いても、上に重なる黒い枠線に隠れる
plt.xlabel('月')
plt.title('新潟市の降水量と気温（2024年）')
plt.xticks(range(1, 13))

plt.twinx()
plt.plot(df['月'], df['気温'], color='r', marker='o', label='気温')
plt.ylabel('気温 (℃)', color='r')
plt.ylim(0, 30)
plt.tick_params(axis='y', colors='r')                 # 右の目盛り：twinx の後
plt.gca().spines['right'].set_color('r')              # 右の枠線
plt.gca().spines['left'].set_color('tab:blue')        # ○左の枠線も twinx の後に書く

plt.show()
```



- `×` の行を消しても、グラフは変わらない（隠れているだけなので）。`○` の行を消すと、左の枠線が黒に戻る
- 名前（`ylabel`）と目盛り（`tick_params`）は、`plt.twinx()` の前は左、後は右に効く。**枠線（`spines`）だけは、左右とも `plt.twinx()` の後**
- 枠線の上を消す `plt.gca().spines['top'].set_visible(False)` も同じで、`plt.twinx()` の前と後の**両方**に書かないと、片方の上の枠線が残る

Z-3 単位を変える

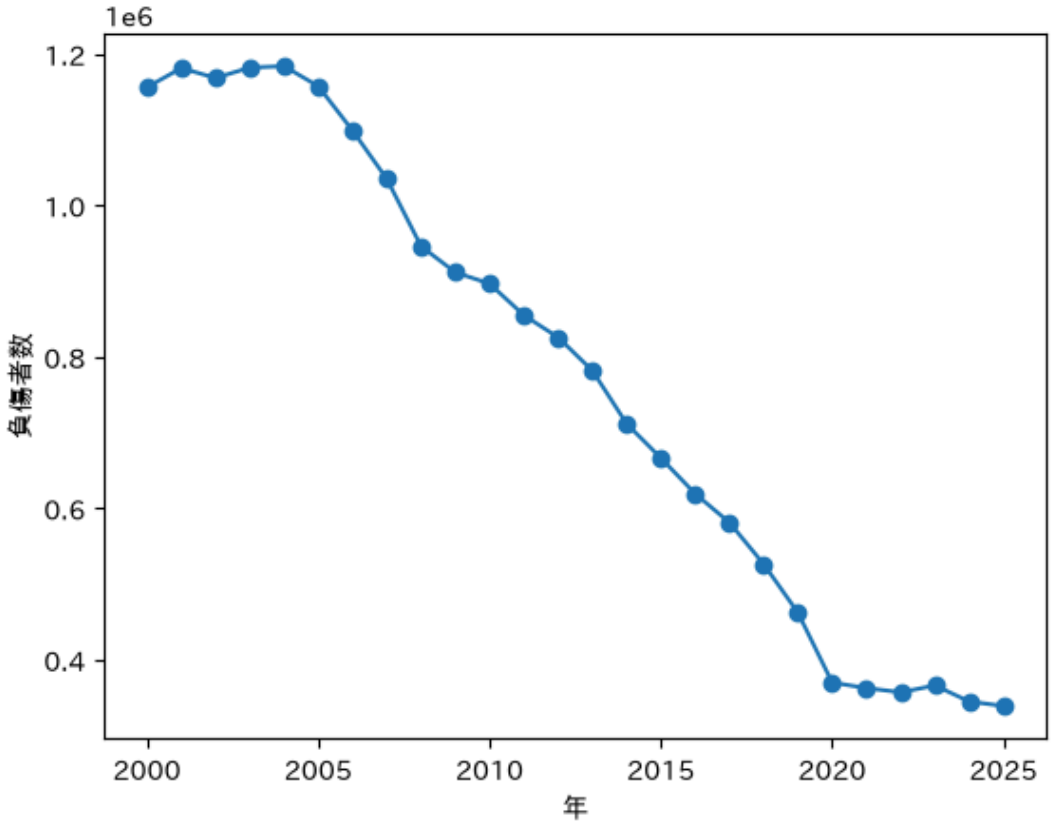
- `accident` シートは、全国の交通事故件数・死者数・負傷者数など（2000〜2025年）

```
df_acc = pd.read_excel('dsa2026data.xlsx', sheet_name='accident')
df_acc.head()
```

	年	交通事故件数	死者数	負傷者数	重傷者数
0	2000	931950	9073	1155707	80105
1	2001	947253	8757	1181039	79677
2	2002	936950	8396	1168029	78297
3	2003	948281	7768	1181681	75112
4	2004	952720	7436	1183617	72817

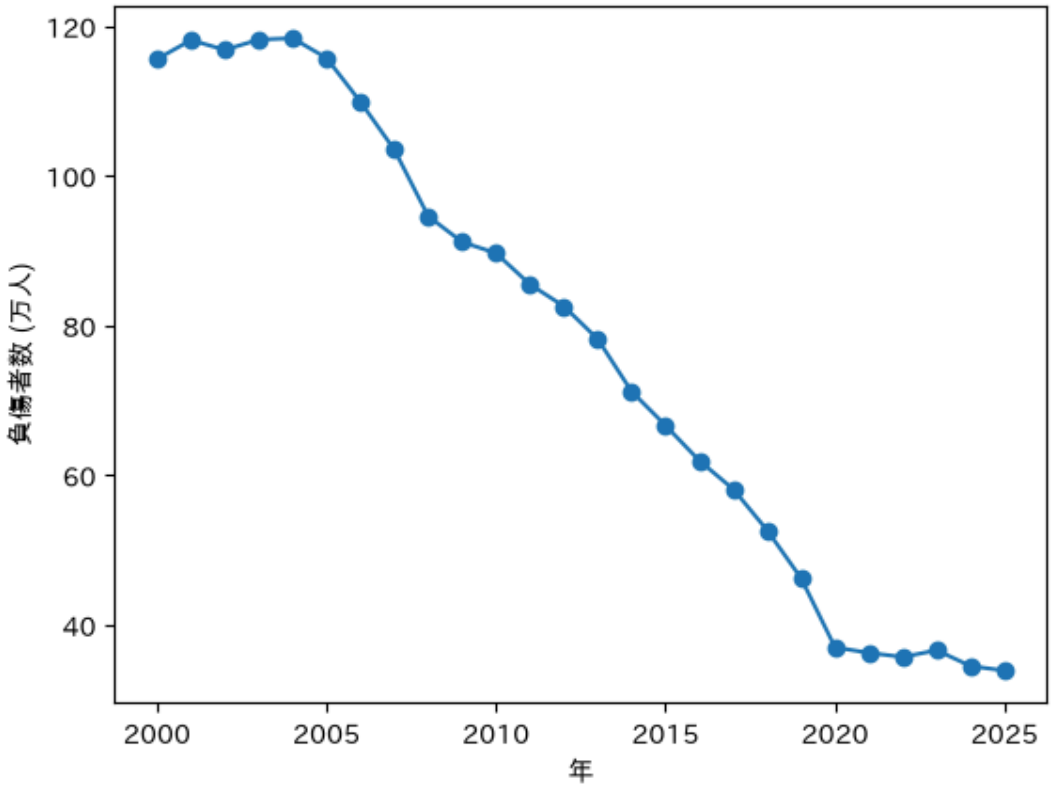
- 負傷者数を、そのまま折れ線グラフにしてみる

```
plt.plot(df_acc['年'], df_acc['負傷者数'], marker='o')
plt.xlabel('年')
plt.ylabel('負傷者数')
plt.show()
```



- 縦軸の左上に `1e6` と出る。これは「目盛りの数 × 10 の 6 乗（100万）」という意味。目盛りの `1.0` は 1.0 × 100万 ＝ 100万人
- このままでは読みにくい。データを割って**単位を変える**と読みやすくなる。`df_acc['負傷者数'] / 10000` で「万人」になる
- 単位を変えたら、**軸の名前にも単位**を書く

```
plt.plot(df_acc['年'], df_acc['負傷者数'] / 10000, marker='o')
plt.xlabel('年')
plt.ylabel('負傷者数（万人）')
plt.show()
```



- 目盛りが 40〜120 になり、「2004年は約118万人、2025年は約34万人」と読める
- 100万より小さい数でも、交通事故件数（最大 約95万件）のように桁が多いと読みにくい。同じように `/ 10000` で「万件」にするとよい
- ほかの例：円 → 千円（`/ 1000`）、人 → 千人（`/ 1000`）、g → kg（`/ 1000`）

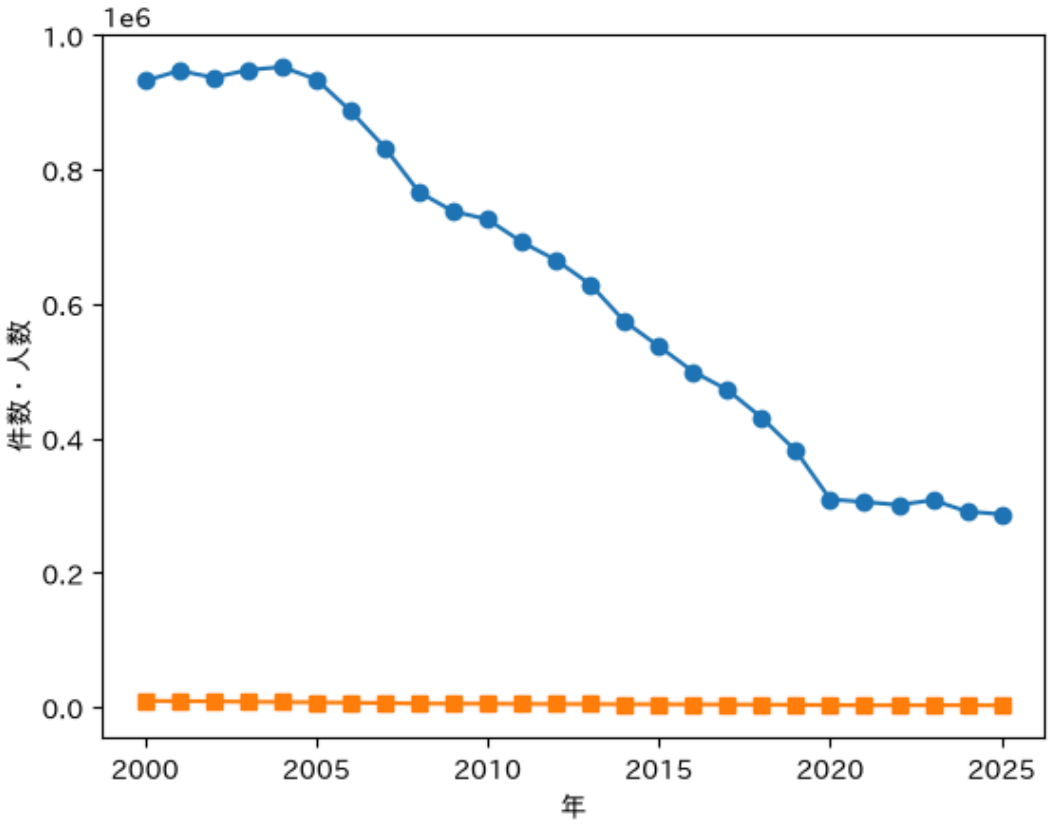
使ってみよう

- 交通事故件数（万件、棒グラフ）と死者数（人、折れ線グラフ）の複合グラフを、Z-2 のひな形を直して描いてみよう

折れ線2本で両 y 軸を使う

- 複合グラフは「棒グラフ＋折れ線グラフ」だけではない。**折れ線2本**でも、左右に y 軸を作れる
- 交通事故件数（約29万〜95万件）と死者数（約2,500〜9,000人）は、単位も大きさも違う（約100倍）。1本の y 軸で描くと、死者数の線が下に張り付く

```
plt.plot(df_acc['年'], df_acc['交通事故件数'], marker='o')
plt.plot(df_acc['年'], df_acc['死者数'], marker='s')
plt.xlabel('年')
plt.ylabel('件数・人数')
plt.show()
```



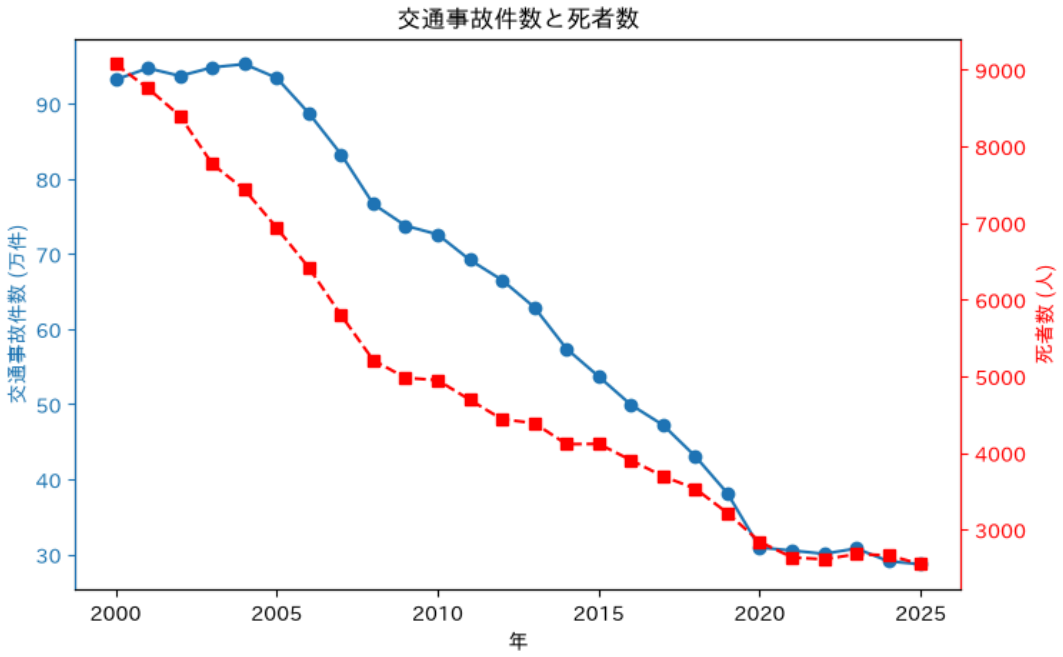
- Z-2 のひな形の `plt.bar` を `plt.plot` に変えれば、折れ線2本の両 y 軸になる

```
plt.figure(figsize=(8, 5))

plt.plot(df_acc['年'], df_acc['交通事故件数'] / 10000, color='tab:blue', marker='o')
plt.ylabel('交通事故件数（万件）', color='tab:blue')
plt.tick_params(axis='y', colors='tab:blue')
plt.xlabel('年')
plt.title('交通事故件数と死者数')

plt.twinx()
plt.plot(df_acc['年'], df_acc['死者数'], color='r', marker='s', linestyle='--')
plt.ylabel('死者数（人）', color='r')
plt.tick_params(axis='y', colors='r')
plt.gca().spines['right'].set_color('r')
plt.gca().spines['left'].set_color('tab:blue')

plt.show()
```



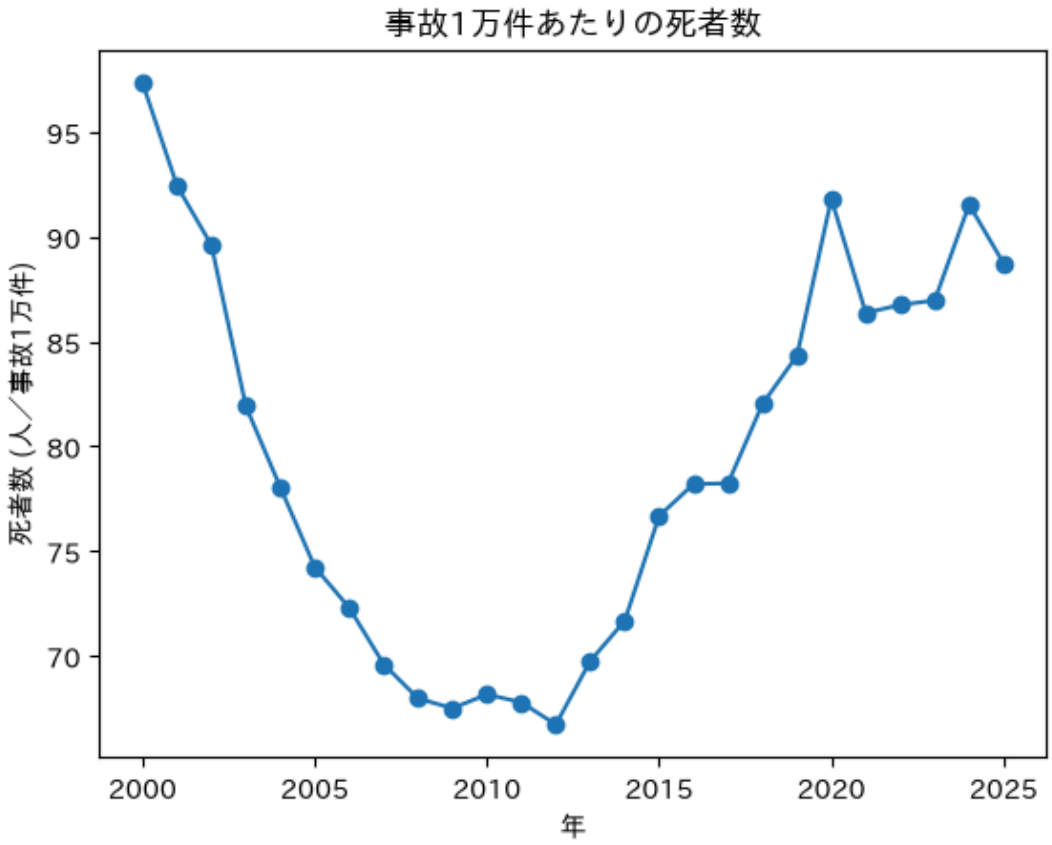
- 折れ線どうしは見分けにくいので、**色**に加えて**マーカー**（`'o'` と `'s'`）や**線の種類**（`linestyle='--'`）も変える
- 左右の軸を線と同じ色にすると、凡例がなくても、どちらの軸を読めばよいかが分かる
- 左の枠線の色も `plt.twinx()` **の後**に `plt.gca().spines['left']` で変える（右のグラフが左の枠線の上に重なって描かれるため）
- **注意**：左右で目盛りが違うので、2本の線が**交わる位置や、上下の位置関係には意味がない**。このグラフでは2020年ごろに交わるが、「事故件数と死者数が同じになった」わけではない。目盛りの決め方（`plt.ylim`）しだいで、交わったり離れたりする

割り算して比を見る

- 上の両 y 軸のグラフからは、どちらも減っていることは分かるが、それ以上は読み取りにくい
- 死者数を交通事故件数で**割る**と、「事故1件あたりの死者数」になる。小さい数になるので `* 10000` して「**事故1万件あたりの死者数**」にする

```
ratio = df_acc['死者数'] / df_acc['交通事故件数'] * 10000

plt.plot(df_acc['年'], ratio, marker='o')
plt.xlabel('年')
plt.ylabel('死者数（人／事故1万件）')
plt.title('事故1万件あたりの死者数')
plt.show()
```



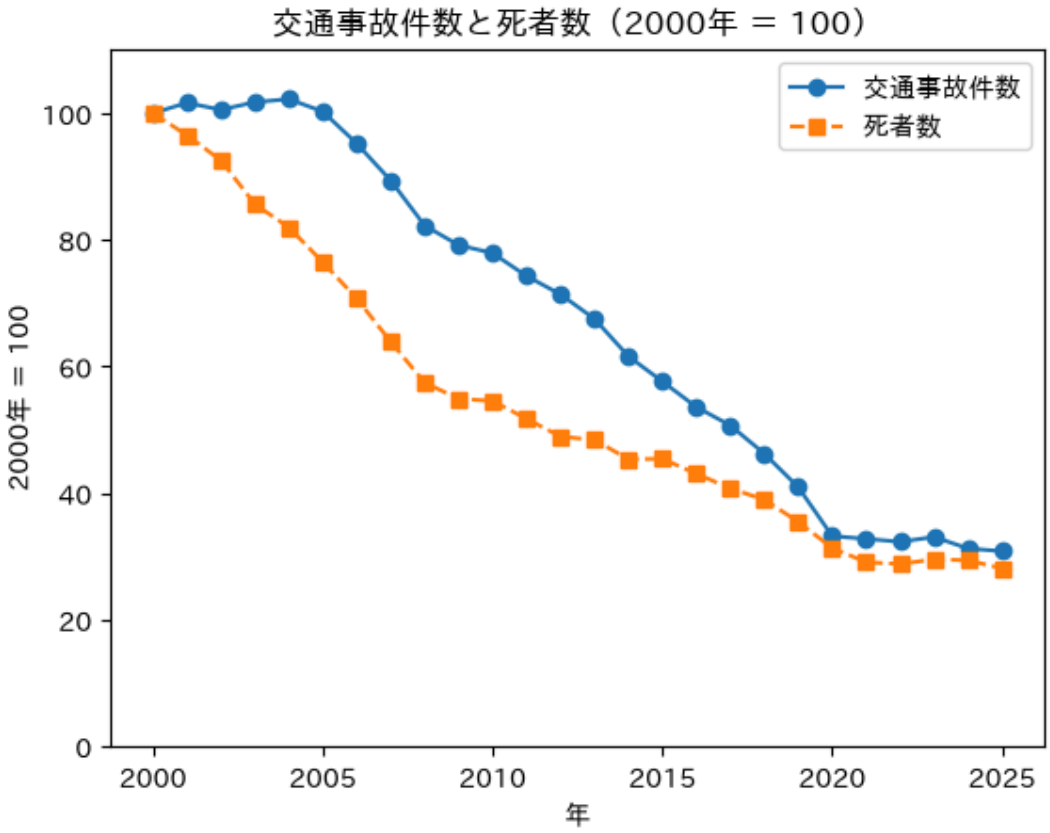
- 2012年ごろまでは下がり、そのあとは上がっている（**U字形**）。事故の件数は減ったが、起きた事故1件あたりの重さは、2012年ごろから**むしろ増えている**
- 2本の線を並べただけでは見えなかったことが、**割り算して比にする**と見えてくる。2つの量の関係を見たいときは、両 y 軸よりも比を描くほうが正確に伝わる
- 比にすると単位が変わる。軸の名前に「人／事故1万件」のように、**何あたりか**を書く
- ほかの例：人口あたり（1人あたりGDP）、面積あたり（人口密度）、負傷者のうち重傷者の割合（%）

指数化（基準の年 ＝ 100）

- それぞれの列を**最初の年（2000年）の値で割って 100 倍**すると、「2000年を 100 としたとき、いくつか」になる。これを**指数化**という
- `df_acc['死者数'][0]` は、死者数の列の**最初の行**（2000年）の値
- 単位が消えるので、単位の違う2本を **y 軸1本**で比べられる

```
idx_acc = df_acc['交通事故件数'] / df_acc['交通事故件数'][0] * 100
idx_death = df_acc['死者数'] / df_acc['死者数'][0] * 100

plt.plot(df_acc['年'], idx_acc, marker='o', label='交通事故件数')
plt.plot(df_acc['年'], idx_death, marker='s', linestyle='--', label='死者数')
plt.xlabel('年')
plt.ylabel('2000年 ＝ 100')
plt.ylim(0, 110)
plt.title('交通事故件数と死者数（2000年 ＝ 100）')
plt.legend()
plt.show()
```



- 2025年は、交通事故件数が約31、死者数が約28。どちらも2000年の**3割ほど**に減った
- 2012年ごろまでは死者数のほうが速く減り（2012年：事故件数 約71、死者数 約49）、そのあとは2本の差が縮まっている。これが、比のグラフの U字形の理由
- 両 y 軸とちがい、2本が同じ目盛りなので、**線の上下や交わりにも意味がある**
- どの年を基準にするかで見え方が変わる。基準の年を軸の名前やタイトルに書く

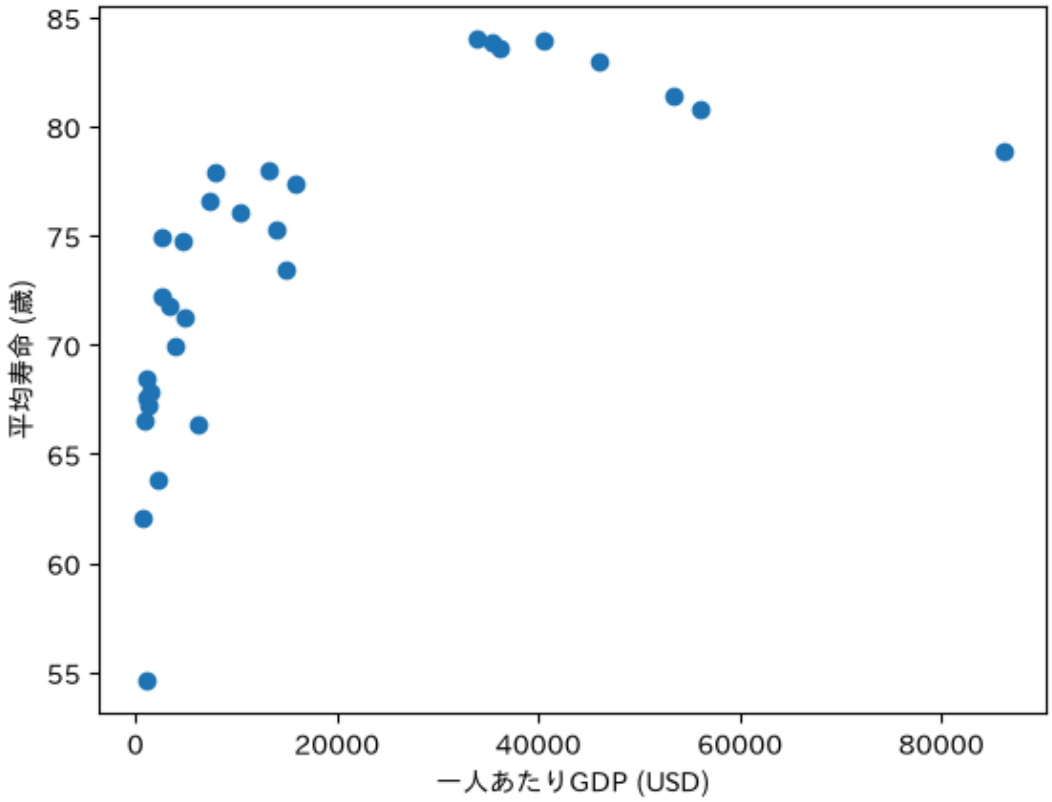
方法	書き方	向いているとき
両 y 軸	`plt.twinx()`	それぞれの**量そのもの**（件数・人数）も読ませたい
比	`列A / 列B`	2つの量の**関係**（1件あたり、1人あたり）を見たい
指数化	`列 / 列[0] * 100`	2つの量の**変化の速さ**を比べたい

Z-4 対数目盛（データはそのまま）

- `worldbank` シートは、人口の多い30か国の一人あたり GDP（ドル）と平均寿命（2024年）
- 一人あたり GDP は、約700ドルから約86,000ドルまで、**100倍以上**の差がある。そのまま散布図にしてみる

```
df_wb = pd.read_excel('dsa2026data.xlsx', sheet_name='worldbank')

plt.scatter(df_wb['一人あたりGDP'], df_wb['出生時平均寿命'])
plt.xlabel('一人あたりGDP（USD）')
plt.ylabel('平均寿命（歳）')
plt.show()
```

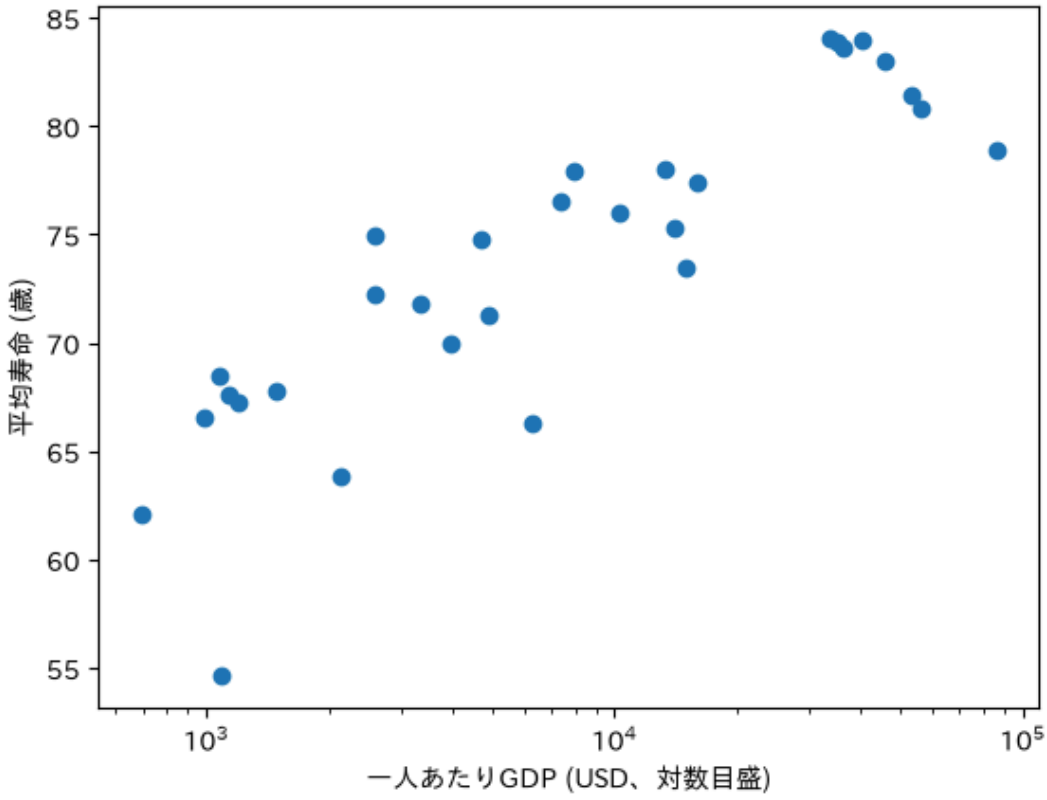


- 多くの点が左に集まり、GDP の低い国どうしの違いが見えない

対数目盛：表示だけを変える

- `plt.xscale('log')` を1行足すと、**データはそのまま**で、横軸の目盛りだけが対数（1目盛りごとに10倍）になる

```
plt.scatter(df_wb['一人あたりGDP'], df_wb['出生時平均寿命'])
plt.xscale('log')
plt.xlabel('一人あたりGDP（USD、対数目盛）')
plt.ylabel('平均寿命（歳）')
plt.show()
```



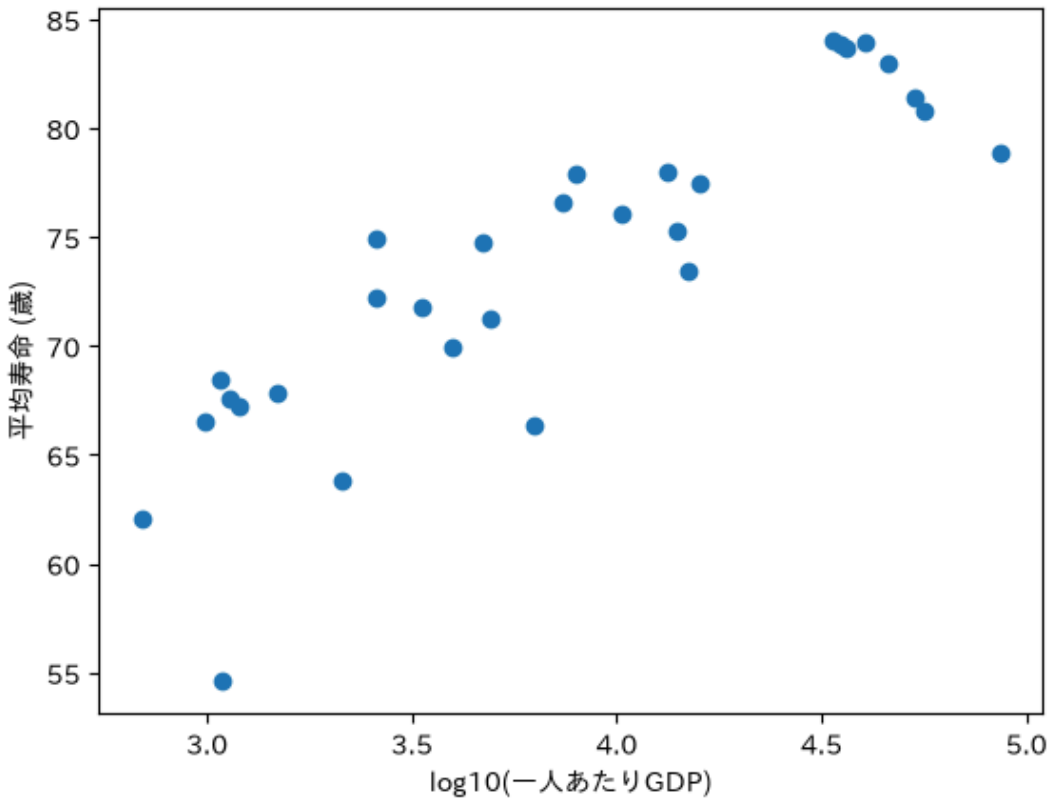
- 点が広がり、GDP の低い国どうしの違いも見える。目盛りは 10^3（1000ドル）、10^4（1万ドル）、10^5（10万ドル）と**元の単位のまま**読める。間の小さい目盛りは 2000、3000、…、9000 ドル
- 縦軸を対数目盛にするときは `plt.yscale('log')`
- 対数目盛にしたら、軸の名前に「対数目盛」と書く（等間隔だと思われないように）
- 0 やマイナスの値は、対数目盛には描けない

	単位を変える（Z-3）	対数目盛（Z-4）
書き方	`df['列名'] / 10000`	`plt.xscale('log')`・`plt.yscale('log')`
データ	割った値に変わる	**そのまま**
目盛り	等間隔（10、20、30 …）	10倍ごとに同じ間隔（10、100、1000 …）
向いているとき	桁が多くて読みにくい	値の差が何十倍・何百倍もある

（参考）データの対数を取る方法

- `np.log10(…)` で、データそのものを「10 の何乗か」に変えても、点は同じように広がる

```
plt.scatter(np.log10(df_wb['一人あたりGDP']), df_wb['出生時平均寿命'])
plt.xlabel('log10(一人あたりGDP)')
plt.ylabel('平均寿命（歳）')
plt.show()
```



- ただし、横軸の目盛りが 3.0、3.5、4.0 … になる。3 は 1000 ドル、4 は 1万ドルのことだが、目盛りを見てもすぐには何ドルか分からない
- **データそのものを変えてしまう**ので、表の数とグラフの数が食い違う。グラフで見せるだけなら、対数目盛（`plt.xscale('log')`）のほうがよい

V. 注釈：グラフに文字を書く

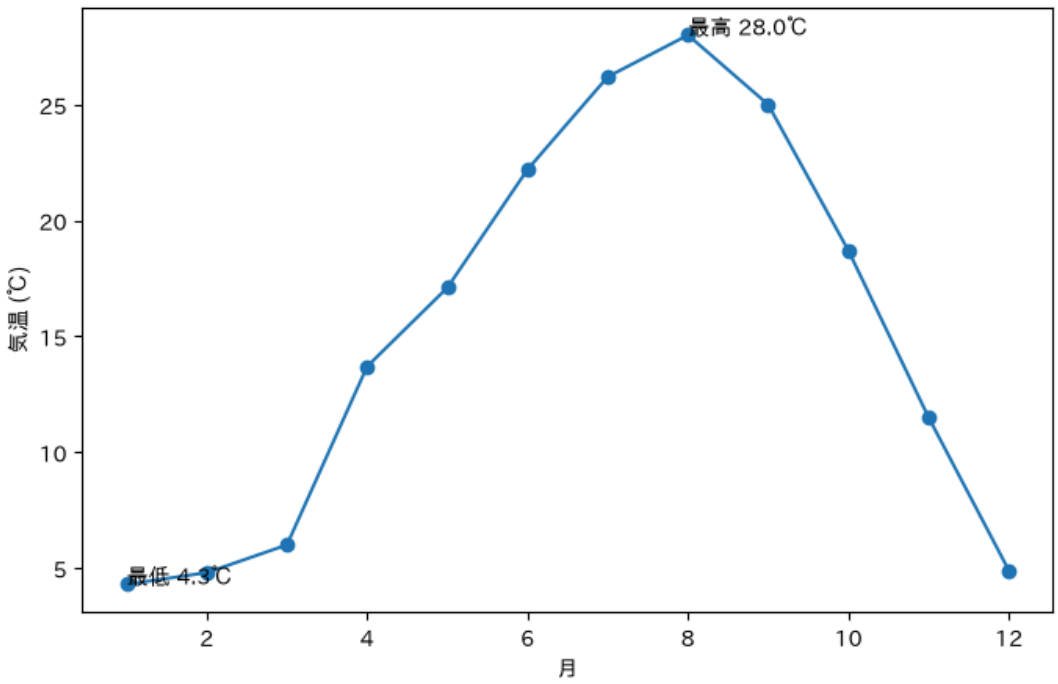
グラフの伝えたいところに文字や矢印を書くと、見る人に「どこを見ればよいか」が伝わる。

V-1 plt.text の基本

第 4 講 H-10 で、散布図の点に1つだけ名前を付けた。ここでくわしく学ぶ。

- `plt.text(x, y, '文字')` で、**グラフの目盛りで** (x, y) の位置に文字を書く
- 文字の代わりに数（変数）を渡してもよい

```
plt.figure(figsize=(8, 5))
plt.plot(df['月'], df['気温'], marker='o')
plt.text(8, 28, '最高 28.0℃')
plt.text(1, 4.3, '最低 4.3℃')
plt.xlabel('月')
plt.ylabel('気温 (℃)')
plt.show()
```

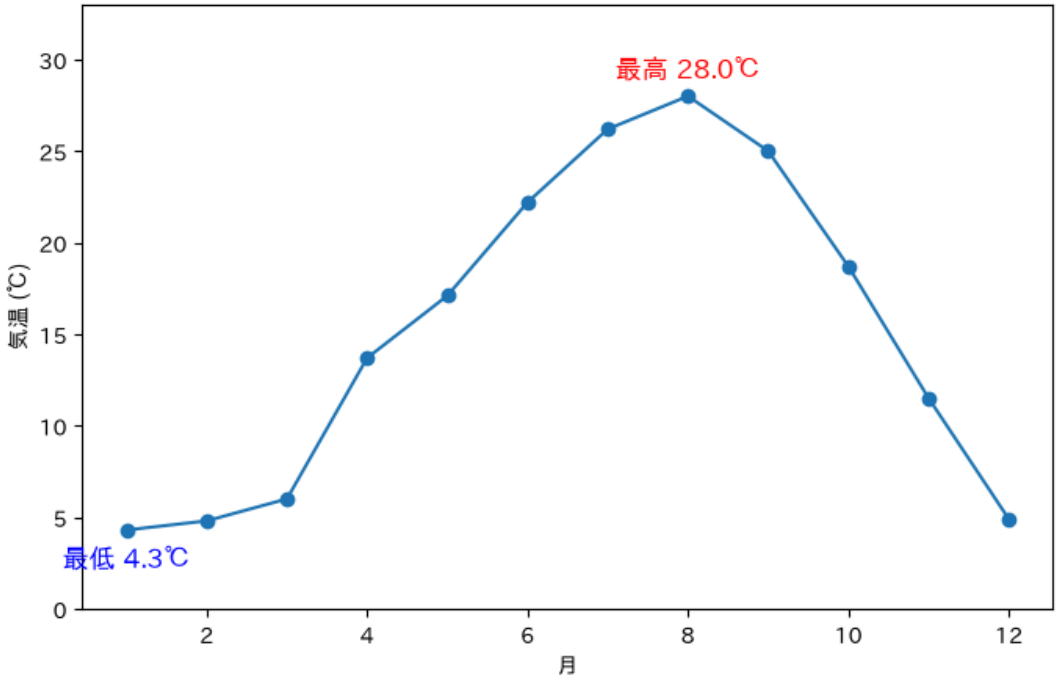


- 点と文字が重なって読みにくい。位置を少しずらす（V-2）

V-2 文字の大きさ・色・位置

書き方	意味
`fontsize=12`	文字の大きさ
`color='r'`	文字の色
`ha='center'`	横の位置合わせ：`'left'`（指定した点から右へ書く。何も書かないとこれ）、`'center'`（点が文字の中央）、`'right'`
`va='bottom'`	縦の位置合わせ：`'bottom'`（点の上に書く）、`'center'`、`'top'`（点の下に書く）

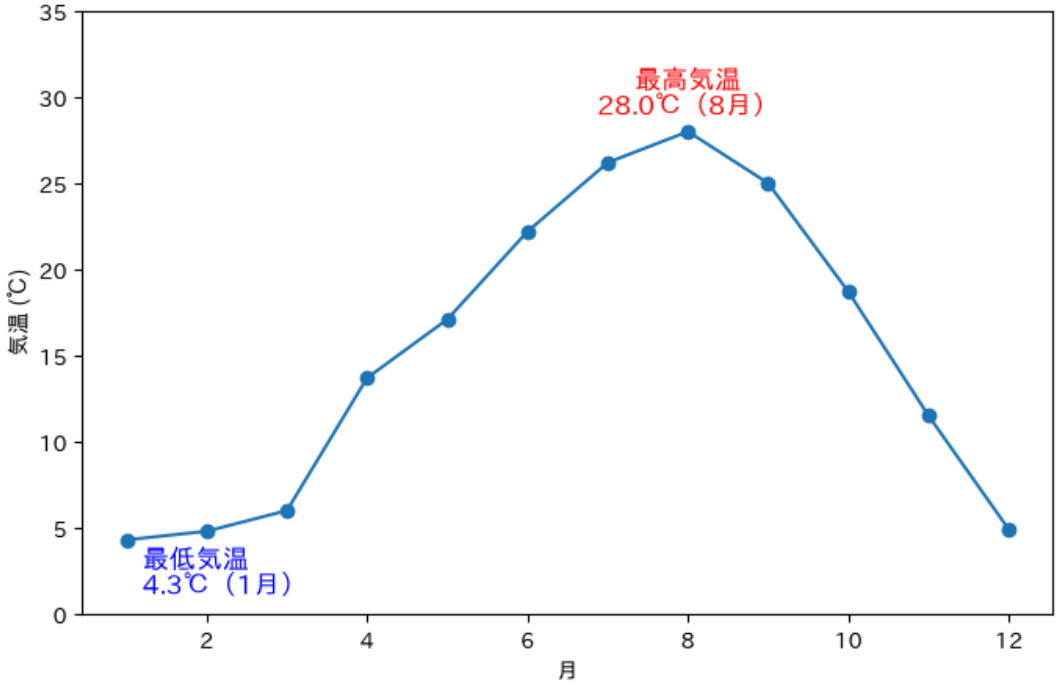
```
plt.figure(figsize=(8, 5))
plt.plot(df['月'], df['気温'], marker='o')
plt.text(8, 28.8, '最高 28.0℃', ha='center', va='bottom', color='r', fontsize=12)
plt.text(1, 3.5, '最低 4.3℃', ha='center', va='top', color='b', fontsize=12)
plt.ylim(0, 33)
plt.xlabel('月')
plt.ylabel('気温 (℃)')
plt.show()
```



複数行の注釈

- 第 5 講 K-3 で紹介した `\n`（改行）を使うと、注釈を2行以上にできる

```
plt.figure(figsize=(8, 5))
plt.plot(df['月'], df['気温'], marker='o')
plt.text(8, 28.8, '最高気温\n28.0℃（8月）', ha='center', va='bottom', color='r', fontsize=12)
plt.text(1.2, 4, '最低気温\n4.3℃（1月）', ha='left', va='top', color='b', fontsize=12)
plt.ylim(0, 35)
plt.xlabel('月')
plt.ylabel('気温（℃）')
plt.show()
```



- `'…\n…'` で2行になる。`ha='center'` で各行が中央にそろう（最低気温のように `ha='left'` なら左そろえ）

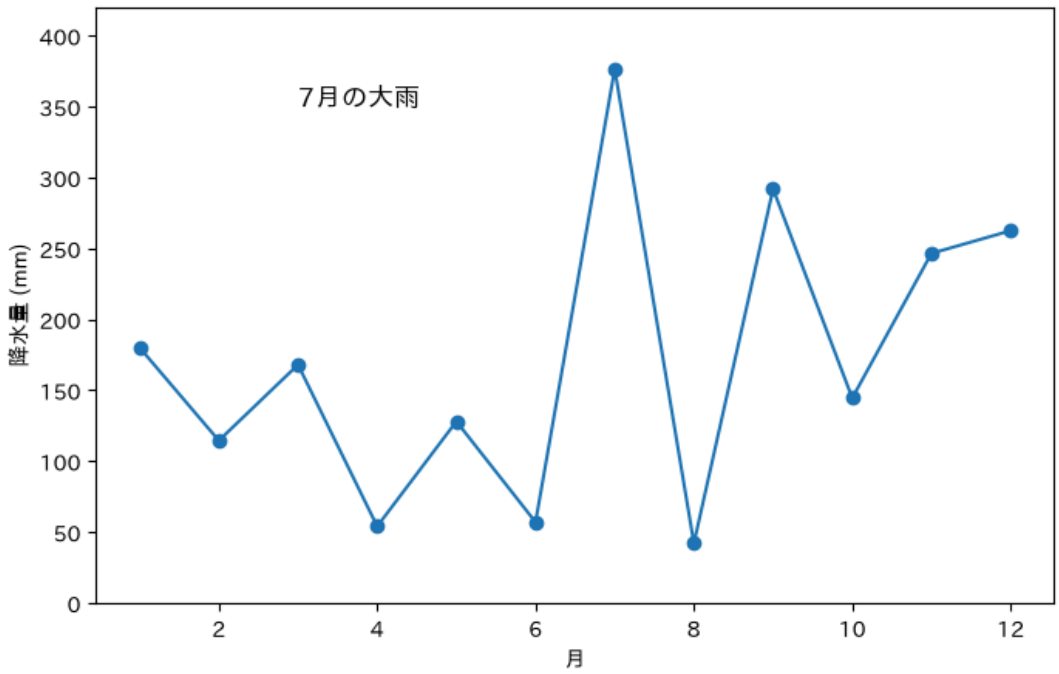
- `va='bottom'` は文字のかたまり全体の下端を点の位置に合わせる。行が増えると上に伸びるので、`plt.ylim` で上を空ける
- 行の間隔は `linespacing=1.5` などで変えられる
- 複合グラフでは、`plt.twinx()` の**後**に書いた `plt.text` は、**右の y 軸**の目盛りで位置を決める（Z-0 の [5]）
- 棒の上に値を書くだけなら、第 7 講 R-3 の `plt.bar_label` が簡単

V-3 矢印付きの注釈

	`plt.text`	`plt.annotate`
指定する位置	1つ：文字を置く場所	2つ：指したい点 `xy` と、文字を置く場所 `xytext`
矢印	なし	`arrowprops={…}` で付く
向いているとき	点のすぐそばに書けるとき（V-1・V-2 の最高・最低気温）	点のまわりが混んでいて、空いているところに文字を置きたいとき

- 7月の大雨の説明を、空いている左上（3, 350）に `plt.text` で書いてみる

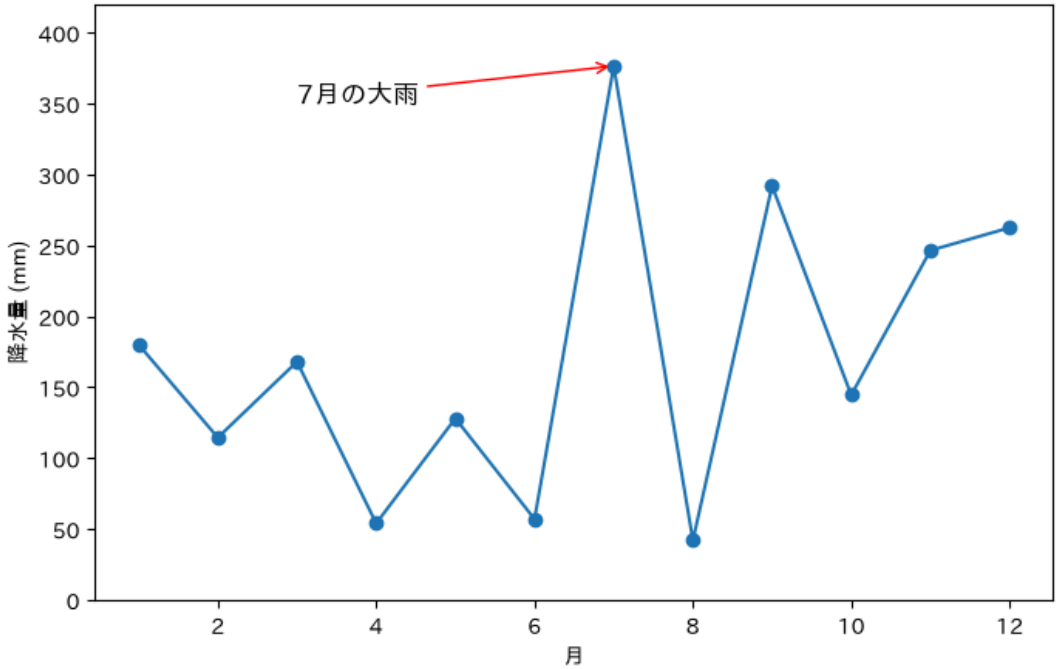
```
plt.figure(figsize=(8, 5))
plt.plot(df['月'], df['降水量'], marker='o')
plt.text(3, 350, '7月の大雨', fontsize=12)
plt.ylim(0, 420)
plt.xlabel('月')
plt.ylabel('降水量 (mm)')
plt.show()
```



- 文字は読めるが、**どの点の説明か分からない**。`plt.text` は文字の位置しか指定できない
- `plt.annotate` なら、同じ場所に文字を置いたまま、矢印で7月の点を指せる
- `plt.annotate('文字', xy=(指したい点), xytext=(文字の位置), arrowprops={'arrowstyle': '->'})` で、矢印付きの注釈を書ける
- `xy` が**矢印の先**、`xytext` が**文字を置く場所**。どちらもグラフの目盛りで書く

- `arrowprops` の `{ }` は 第 7 講 Q-3 の**辞書**。`'color': 'r'` を足すと矢印が赤になる

```
plt.figure(figsize=(8, 5))
plt.plot(df['月'], df['降水量'], marker='o')
plt.annotate('7月の大雨', xy=(7, 376.5), xytext=(3, 350), fontsize=12,
             arrowprops={'arrowstyle': '->', 'color': 'r'})
plt.ylim(0, 420)
plt.xlabel('月')
plt.ylabel('降水量 (mm)')
plt.show()
```



変えてみよう

- `'arrowstyle': '->'` → `'-|>'`（矢印の形が変わる）
- `xytext=(3, 350)` → `xytext=(10, 400)`

使ってみよう

- Z-3 の負傷者数（万人）の折れ線グラフに、いちばん多い 2004 年（約118.4万人）を指す矢印と説明を書いてみよう

まとめ

やりたいこと	書き方
以下は複合グラフ	
右に y 軸を作る	`plt.twinx()`（前に書いたものは左、後に書いたものは右）
左右の y 軸の範囲	`plt.ylim(0, 400)`（`plt.twinx()` の前は左、後は右。0 から始める）

やりたいこと	書き方
凡例を左右に分ける	`plt.legend(loc='upper left')` 、`plt.legend(loc='upper right')`
軸の名前の色	`plt.ylabel('気温 (℃)', color='r')`
目盛りの色	`plt.tick_params(axis='y', colors='r')` （ `colors` に s が付く）
右の枠線の色	`plt.gca().spines['right'].set_color('r')`
左の枠線の色	`plt.gca().spines['left'].set_color('tab:blue')` （左の枠線も `plt.twinx()` **の後**に書く。前に書くと、重なった右のグラフの枠線に隠れる）
以下は単位・比・目盛り	
`1e6` の意味	目盛りの数 × 100万
単位を変える	`df['列名'] / 10000` （万）、`df['列名'] / 1000` （千）。軸の名前にも単位を書く
比（何あたり）	`df['列A'] / df['列B'] * 10000` （列B 1万あたり）。軸の名前に「人／事故1万件」のように何あたりかを書く
指数化（基準の年 ＝ 100）	`df['列名'] / df['列名'][0] * 100` 。単位の違う2本を y 軸1本で比べられる
対数目盛（データはそのまま）	`plt.xscale('log')` （横軸）、`plt.yscale('log')` （縦軸）。`np.log10` でデータを変えない
以下は注釈	
文字を書く	`plt.text(x, y, '文字')`
位置・大きさ・色	`ha='center'` 、`va='bottom'` 、`fontsize=12` 、`color='r'`
複数行の注釈	`plt.text(x, y, '1行目\n2行目')`
矢印付きの注釈	`plt.annotate('文字', xy=(矢印の先), xytext=(文字の位置), arrowprops={'arrowstyle': '->'})`

第 9 講：ヒストグラムとエラーバー

1つの量の分布を見る**ヒストグラム**（AA）と、平均とばらつきを比べる**エラーバー**（AB）を学ぶ。AB-7 で、第 7 講の箱ひげ図との使い分けを整理する。

準備：この講で使うライブラリを読み込み、データを用意する。

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import japanize_matplotlib
```

AA. ヒストグラム

ヒストグラムは、データを**区間に分けて、それぞれの区間にいくつ入るかを数えた**グラフ。点数が**どのあたりに集まっているか**（分布）を見るときに使う。

	棒グラフ	ヒストグラム
関数	`plt.bar(x, y)`	`plt.hist(データ)`
渡すもの	項目（x）と高さ（y）の**2つ**	データ**1つだけ**（数えるのは matplotlib）
横軸	項目（曜日、都市など）	**数値の区間**（0〜10点、10〜20点 …）
縦軸	量	**個数**（人数など）
棒のすき間	あける	**あけない**（区間がつながっているため）

- **文字**のデータ（曜日）は、同じ値ごとに `value_counts()` で数えた（第 7 講）
- **数値**のデータ（点数など）は、同じ値がほとんどないので、そのままでは数えにくい。**区間（0〜10点、10〜20点、…）ごとに数える**のがヒストグラム。数えるのは `plt.hist` がやってくれる

AA-0 完成形（ヒストグラム）

このグラフを目指す（30人の数学の点数の分布）。

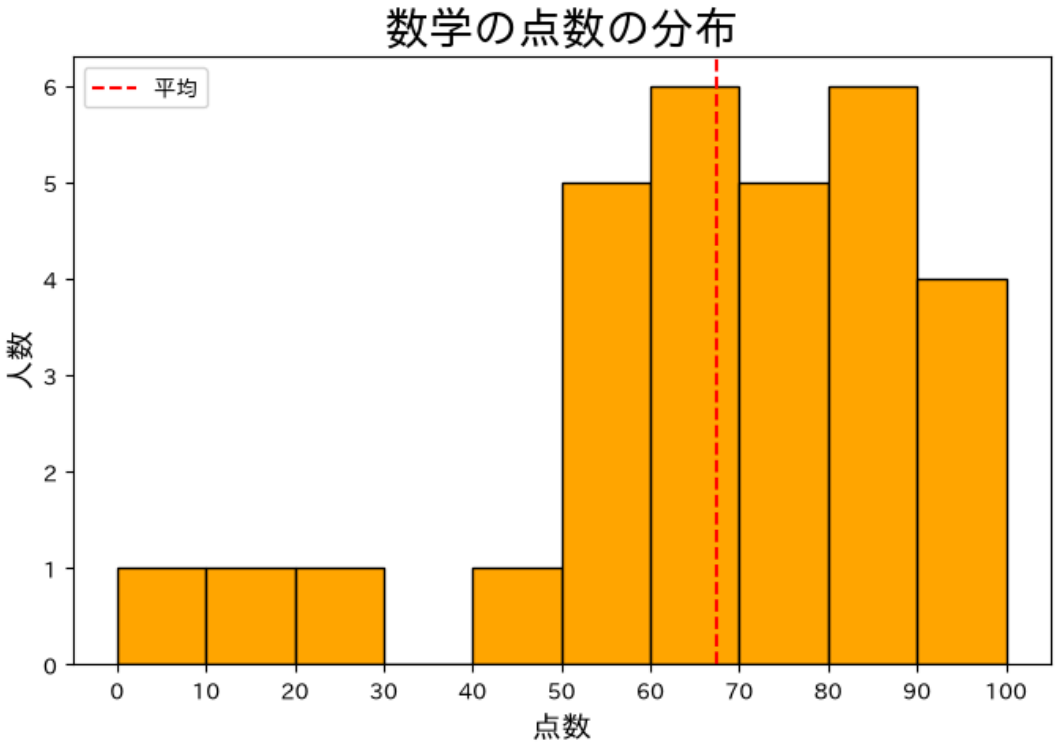
1. **全部コピーして**自分の notebook のセルに貼り付け、実行する（`Shift + Enter`）
2. まずは**数値や文字を変えて**、グラフのどこが変わるかを確かめる
3. そのあと1行ずつ分解して、各部品（[0]〜[7]）の働きを学ぶ（AA-1〜AA-6）

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import japanize_matplotlib

df = pd.read_excel('dsa2026data.xlsx', sheet_name='exam')  # Excel を読む
data = df['数学']  # 列を取り出す

plt.figure(figsize=(8, 5))  # [0] グラフの大きさ
```

```
plt.hist(data, bins=10, range=(0, 100), color='orange', edgecolor='k')  # [1] ヒストグラム [2] 区切りの数 [3] 区切りの範囲 [4] 色と枠線
plt.axvline(x=np.mean(data), color='r', linestyle='--', label='平均')  # [5] 平均の補助線
plt.xticks(range(0, 101, 10))  # [7] 横軸の目盛りを 0, 10, …, 100 に
plt.title('数学の点数の分布', fontsize=20)  # [7] タイトル
plt.xlabel('点数', fontsize=14)  # [7] 横軸の名前
plt.ylabel('人数', fontsize=14)  # [7] 縦軸の名前
plt.legend()  # [7] 凡例
plt.show()
```



変えてみよう（1か所変えたら、もう一度実行する）

- 列：`df['数学']` → `df['社会']`、`df['英語']`（タイトルも直す）
- 区切りの数：`bins=10` → `bins=5`（20点刻み）、`bins=20`（5点刻み）
- `, range=(0, 100)` を消すと、区切りの位置がどうなるか
- 色：`'orange'` → `'skyblue'`、`'pink'`
- `, edgecolor='k'` を消すと、棒の区切りがどう見えるか

番号	部品	書き方	学ぶところ
[0]	グラフの大きさ（基本設定）	`plt.figure(figsize=(8, 5))`	第 3 講
[1]	ヒストグラムを描く	`plt.hist(データ)`	AA-1
[2]	区切りの数	`bins=10`	AA-2
[3]	区切りの範囲	`range=(0, 100)`	AA-4
[4]	色と枠線	`color='orange'`、`edgecolor='k'`	AA-3
[5]	平均の補助線	`plt.axvline(x=np.mean(データ))`	AA-5
[6]	2つのグループを重ねる	`plt.hist` を2回、`alpha=0.5`	AA-6
[7]	目盛り・タイトル・軸の名前・凡例	`plt.xticks`、`plt.title` など	第 3 講・AA-4

番号	部品	書き方	学ぶところ
	重ねずに並べる	`plt.hist([データ1, データ2])`	AC-2（発展）
	横向きにする	`orientation='horizontal'`	AC-2（発展）

AA-1 ヒストグラムの基本：まず手で数えてみる

例は、小波秀雄『統計学入門』第1章の**男子高校生100人の体重**（表1.1）。`dsa2026data.xlsx` の `weight` シートに入っている。

[1] データを見る：いちばん軽い人は 43.6 kg、いちばん重い人は 64.6 kg

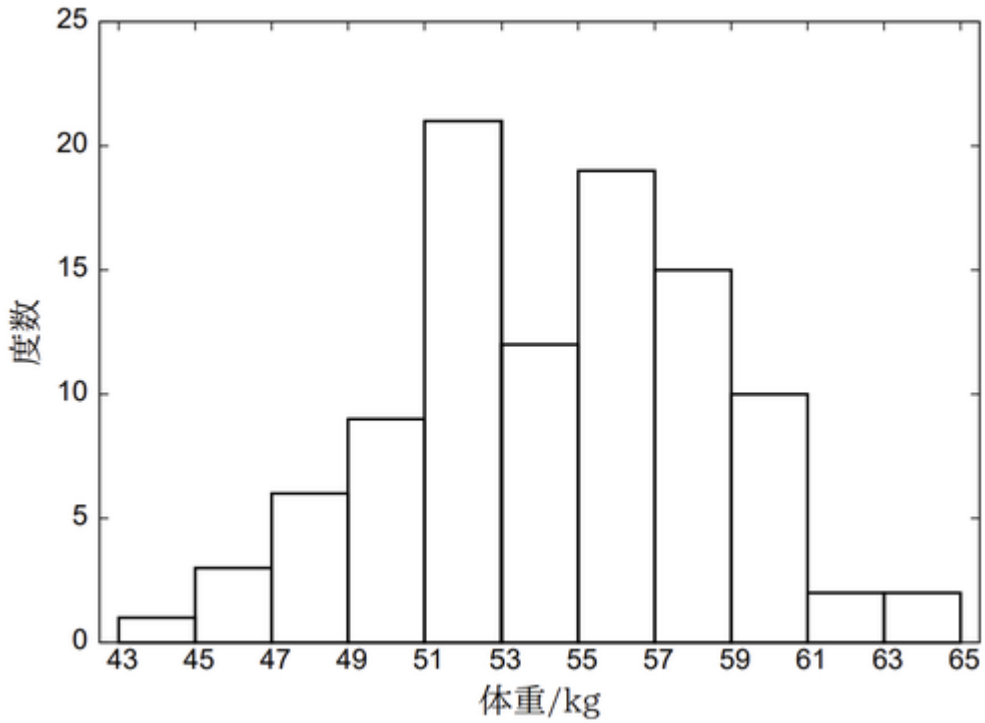
[2] 区間を決める：43〜65 kg を **2 kg ずつ**に分ける → 43〜45、45〜47、…、63〜65 の **11区間**

[3] 区間ごとに人数を数える（表1.2）

区間 (kg)	43〜45	45〜47	47〜49	49〜51	51〜53	53〜55	55〜57	57〜59	59〜61	61〜63	63〜65
人数	1	3	6	9	21	12	19	15	10	2	2

- 区間の境目ちょうどの値は、**右側の区間**に入れる。たとえば 49.0 kg の2人は「47〜49」ではなく「49〜51」で数える（いちばん右の区間だけは、右端の 65 も含む）

[4] 数えた人数を、すき間のない棒にする（図1.4）

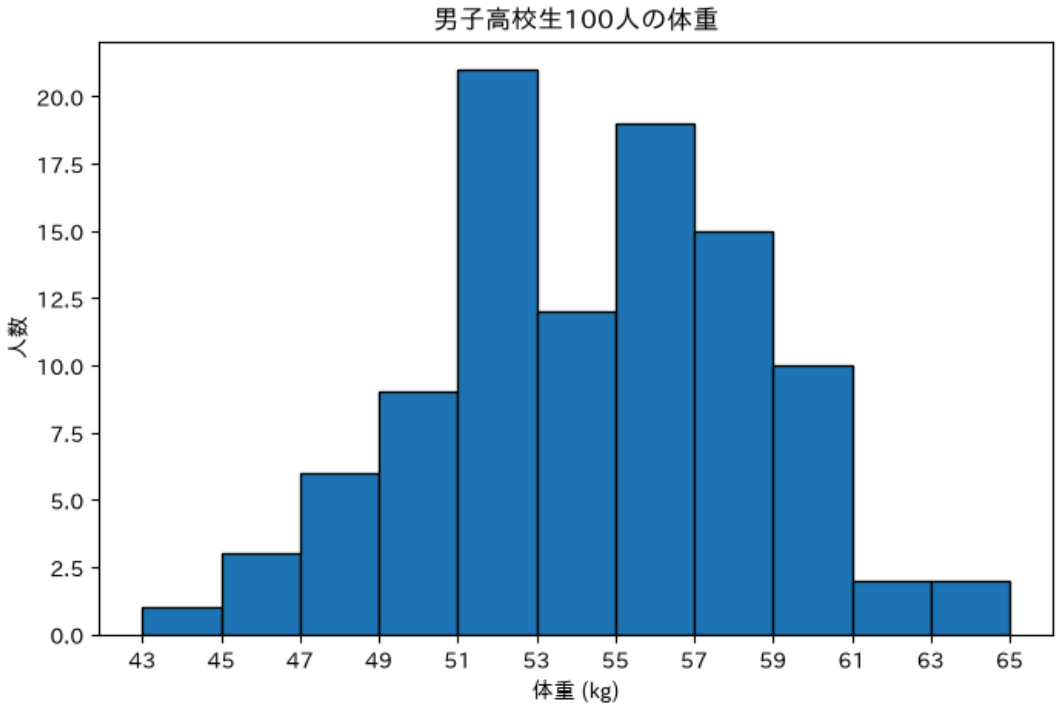


[3] の数えるところは、`plt.hist` が代わりにやってくれる。`bins=11` が [2] の「11区間」、`range=(43, 65)` が「43〜65 kg」にあたる。

```
df_weight = pd.read_excel('dsa2026data.xlsx', sheet_name='weight')

plt.figure(figsize=(8, 5))
plt.hist(df_weight['体重'], bins=11, range=(43, 65), edgecolor='k')
```

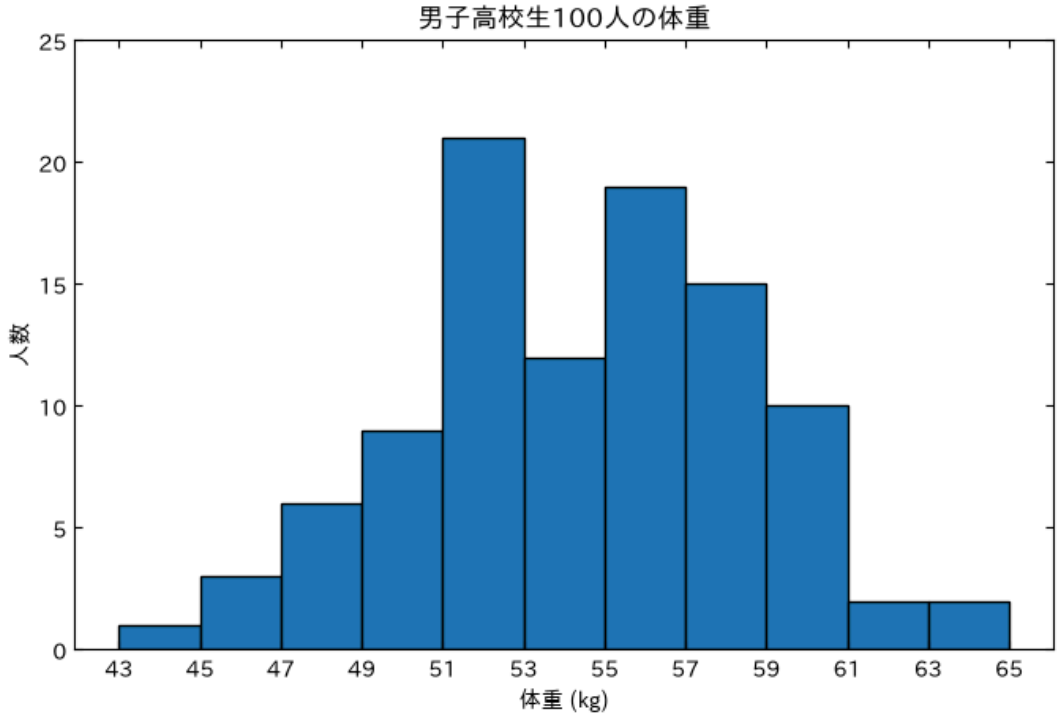
```
plt.xticks(range(43, 66, 2))
plt.title('男子高校生100人の体重')
plt.xlabel('体重 (kg)')
plt.ylabel('人数')
plt.show()
```



（参考）テキストの図1.4 と同じ目盛りにする

- `plt.yticks(range(0, 26, 5))` ：縦軸の目盛りを 0, 5, ..., 25 の整数にする
- `plt.tick_params(direction='in', top=True, right=True)` ：目盛りの線を内側に向け（ `direction='in'` ）、上と右にも付ける（ `top=True` 、 `right=True` ）

```
plt.figure(figsize=(8, 5))
plt.hist(df_weight['体重'], bins=11, range=(43, 65), edgecolor='k')
plt.xticks(range(43, 66, 2))
plt.yticks(range(0, 26, 5))
plt.tick_params(direction='in', top=True, right=True)
plt.title('男子高校生100人の体重')
plt.xlabel('体重 (kg)')
plt.ylabel('人数')
plt.show()
```



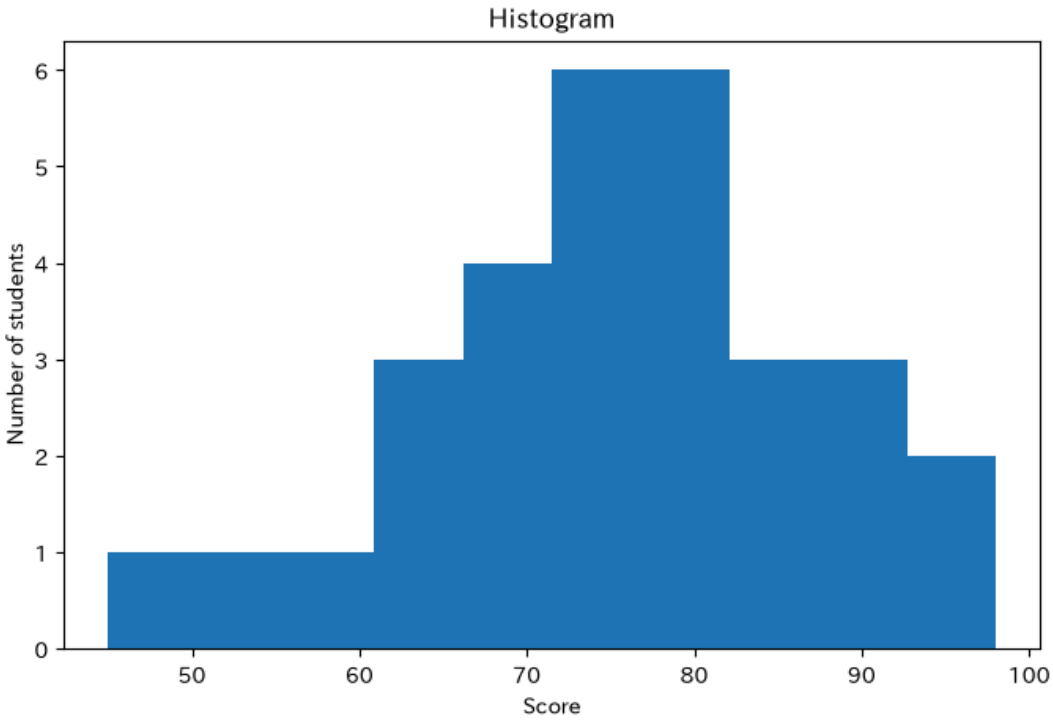
- 棒の高さが [3] の表の人数と同じになっていることを確かめよう（51〜53 kg が 21人でいちばん多い）
- `plt.xticks(range(43, 66, 2))` は、横軸の目盛りを区間の境目（43, 45, ..., 65）にする
- matplotlib も、境目ちょうどの値は右側の区間に入れる。だから表1.2 と同じ人数になる

自動の区切り

30人の点数のリスト `score` で試す。区間の決め方を書かないと、matplotlib が**データの最小値から最大値まで**を自動で10等分する。

```
score = [45, 52, 58, 61, 63, 65, 67, 68, 70, 71, 72, 73, 74, 75, 75,
         76, 77, 78, 79, 80, 81, 82, 84, 85, 86, 88, 90, 92, 95, 98]

plt.figure(figsize=(8, 5))
plt.hist(score)
plt.title('Histogram')
plt.xlabel('Score')
plt.ylabel('Number of students')
plt.show()
```

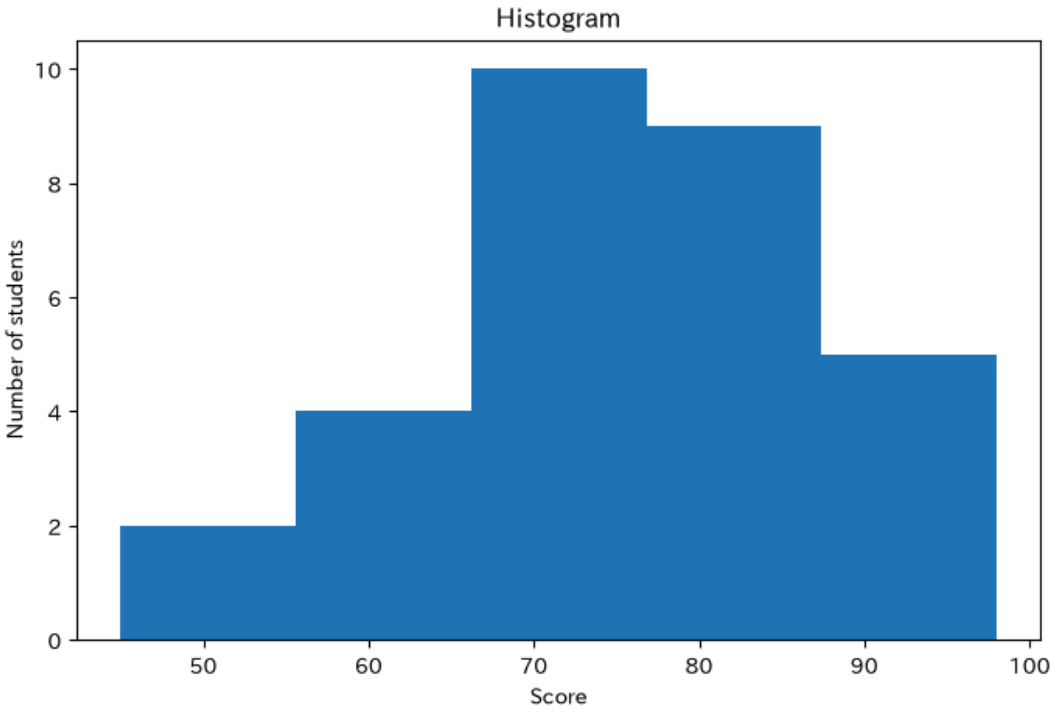


- 区切りは 45〜98 点を10等分した半端な値（50.3、55.6 …）になる。0〜100 点を10点刻みにする方法は AA-4

AA-2 区切りの数

- `bins=` で**区間の数**を決める（bin＝箱）。何も書かないと `10`
- 少なすぎると形が分からず、多すぎるとガタガタになる。データの数が30くらいなら 5〜10 が目安

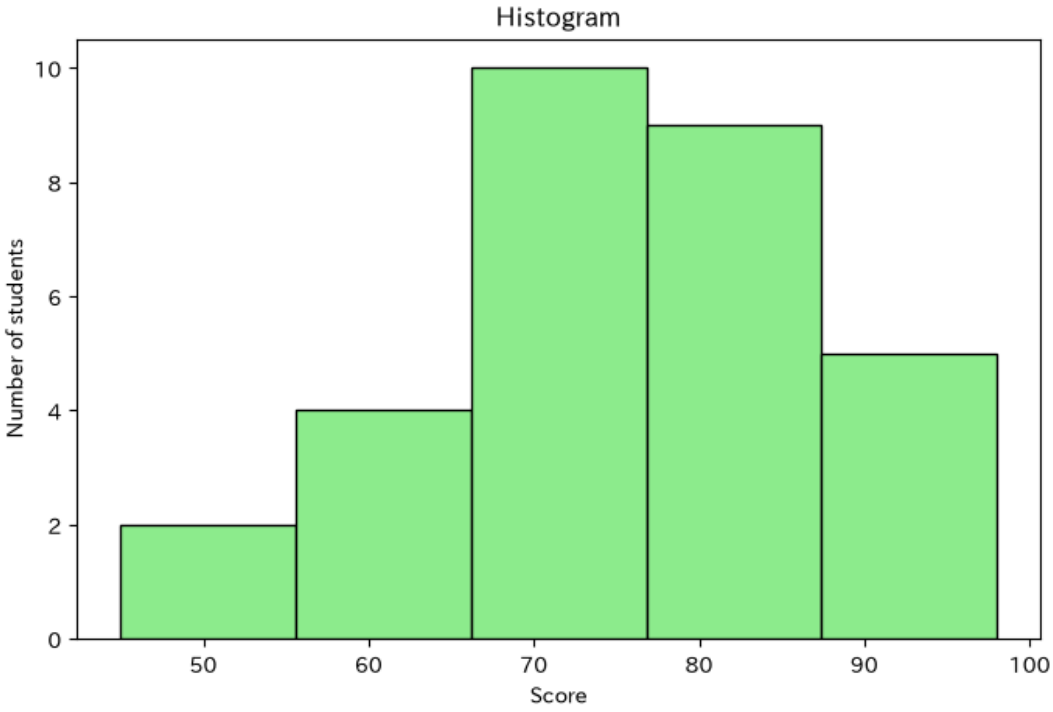
```
plt.figure(figsize=(8, 5))
plt.hist(score, bins=5)
plt.title('Histogram')
plt.xlabel('Score')
plt.ylabel('Number of students')
plt.show()
```



AA-3 色と枠線

- 棒グラフと同じく `color=` 、 `edgecolor=` 。ヒストグラムは棒がくっついているので、**枠線を付けると区切りが見やすい**

```
plt.figure(figsize=(8, 5))
plt.hist(score, bins=5, color='lightgreen', edgecolor='k')
plt.title('Histogram')
plt.xlabel('Score')
plt.ylabel('Number of students')
plt.show()
```



- 上の図は、棒の区切りと横軸の数字がずれて見える。2つは別々に決まるため

    - 棒の区切り：データの最小値 45 点から最大値 98 点までを `bins=5` で5等分した **45、55.6、66.2、76.8、87.4、98**（1区間 10.6 点）
    - 横軸の数字：matplotlib が区切りとは関係なく、切りのよい値（50、60、70 …）を自動で並べたもの
- 区切りと数字を揃える方法は、次の AA-4

AA-4 区切りの範囲

- 何も書かないと、区間は**データの最小値から最大値まで**になる（上の図は 45〜98 点）。区切りが 55.6 点のような半端な値になる
- `range=(0, 100)` と書くと 0〜100 点を区切る。`bins=10` と合わせると、**10点刻み**（0〜10, 10〜20, …）になる
- `(0, 100)` のように、いくつかの値を `,` で区切って ( ) で囲んだものを**タプル**という
    - `(下限, 上限)` のように**順番に意味がある**。`(100, 0)` と逆に書くとエラーになる
    - リスト `[0, 100]` と似ているが、タプルは**あとから中身を変えられない**。「下限と上限」「横と縦の大きさ」のような決まった組を渡すときに使う
    - `figsize=(8, 5)`（横 8、縦 5）も同じタプル。`[0, 100]` とリストで書いても動くが、この授業では ( ) で書く

かっこの使い分け

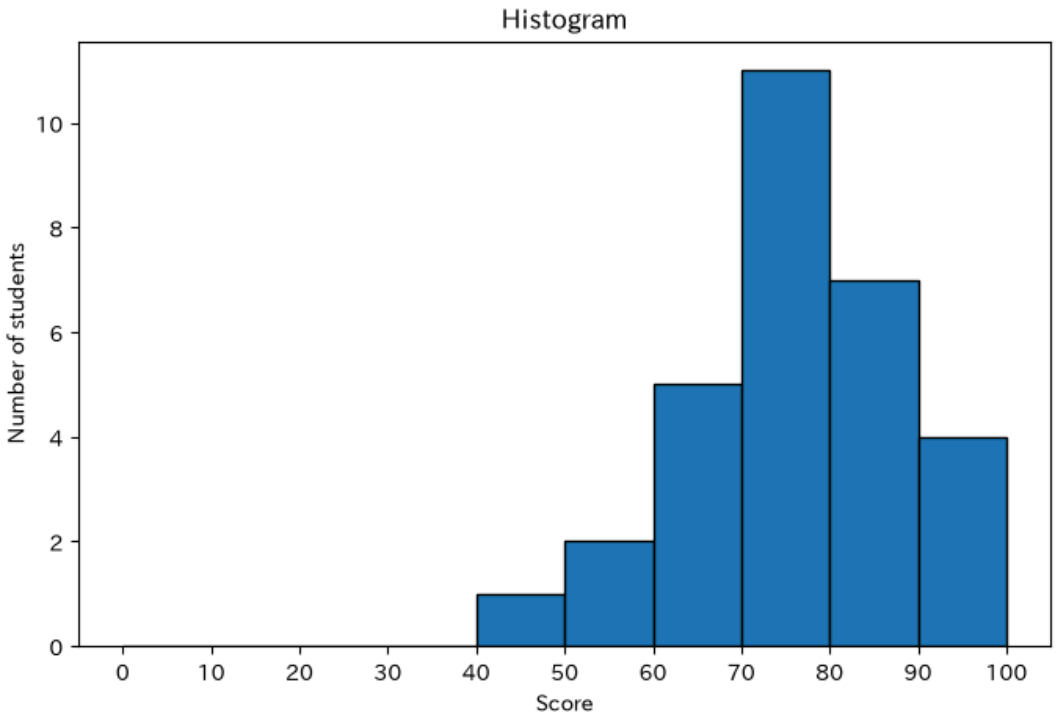
これまでに出てきたかっこは、次の3種類。**かっこの直前に何があるか**で見分ける。

かっこ	直前にあるもの	使う場面	例
`(` `)`	**関数の名前**	関数に値を渡す	`plt.hist(データ)`、`np.mean(score)`、`print(x)`
`(` `)`	`=` や `,`	値の組（タプル）	`range=(0, 100)`、`figsize=(8, 5)`
`[` `]`	`=` や `,`	値を並べる（リスト）	`score = [45, 52, 58]`
`[` `]`	**変数の名前**	中から1つ取り出す	`df['数学']`、`score[0]`、`patches[7]`
`{` `}`	`=` や `,`	名前と値の組（辞書：第 7 講）	`{'月': 120, '火': 135}`

- 迷ったら、「**関数の後ろは ( )、変数の後ろは [ ]**」と覚える。`df('数学')` や `plt.hist[データ]` はエラーになる
- かっこの中にかっこが入ってもよい。`plt.hist(df['体重'], range=(42, 66))` は、「`plt.hist( … )` の中に、`df[ … ]` と `( … )` が入っている」と外側から読む
- 開いたかっこは必ず閉じる。カーソルをかっこの横に置くと、対になるかっこに色が付く
- かっこは**半角**で書く。全角の `（ ）` はエラーになる

```
plt.figure(figsize=(8, 5))
plt.hist(score, bins=10, range=(0, 100), edgecolor='k')
plt.xticks(range(0, 101, 10))
plt.title('Histogram')
plt.xlabel('Score')
```

```
plt.ylabel('Number of students')
plt.show()
```



- `plt.xticks(range(0, 101, 10))` は、横軸の目盛りを 0, 10, 20, ..., 100 にする（なくてもよい）
- 名前は同じ range だが、別のもの。`range=(0, 100)`（= あり）は `plt.hist` の**区切りの範囲**の指定、`range(0, 101, 10)`（= なし）は 0 から 10 おきに数を並べる **Python の命令**（最後の 101 は含まないので、100 まで並ぶ）
- 10点刻みのように切りのよい値にならないときの目盛りの揃え方は、発展の AC-1

区切りを切りのよい数にする条件

点数は 0〜100 点と範囲が決まっているが、体重や来客数のように**範囲が決まっていない**データでは、`range=` を自分で決める。次の3つを満たすと、区切りが切りのよい数になる。

1. **全部のデータが入る**：下限は最小値以下、上限は最大値以上にする（2つ重ねるときは、2つを合わせた最小値・最大値で考える）
2. **刻みが切りのよい数**：刻み ＝（上限 − 下限）÷ `bins` が 1, 2, 5, 10, 20, 50 ... になる
3. **下限と上限が刻みの倍数**：刻みが 10 なら、区切りが 80, 90, 100 ... のように揃う

決める手順（AA-1 の体重の例）

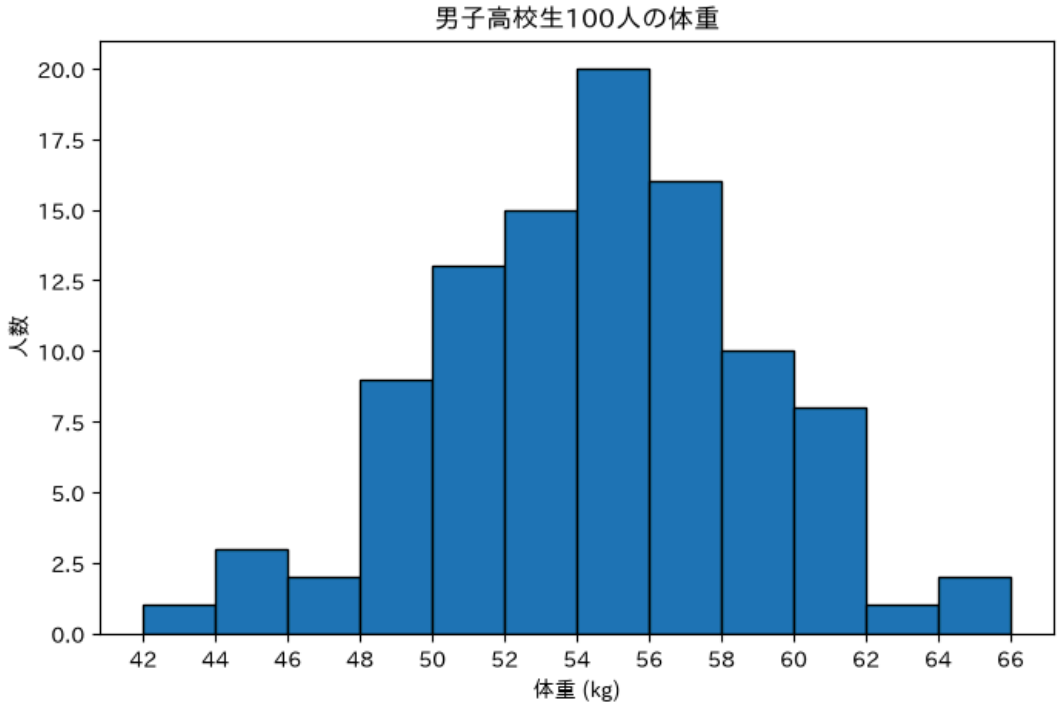
1. 最小値と最大値を調べる（`np.min`、`np.max`：第 4 講）→ 43.6 kg と 64.6 kg
2. 刻みを決める → 2 kg
3. 下限は 43.6 を 2 の倍数に**切り下げて** 42、上限は 64.6 を**切り上げて** 66
4. `bins` ＝（66 − 42）÷ 2 ＝ **12** → `bins=12, range=(42, 66)`

```
print(np.min(df_weight['体重']), np.max(df_weight['体重']))
```

```
43.6 64.6
```

```
plt.figure(figsize=(8, 5))
plt.hist(df_weight['体重'], bins=12, range=(42, 66), edgecolor='k')
plt.xticks(range(42, 67, 2))
```

```
plt.title('男子高校生100人の体重')
plt.xlabel('体重 (kg)')
plt.ylabel('人数')
plt.show()
```

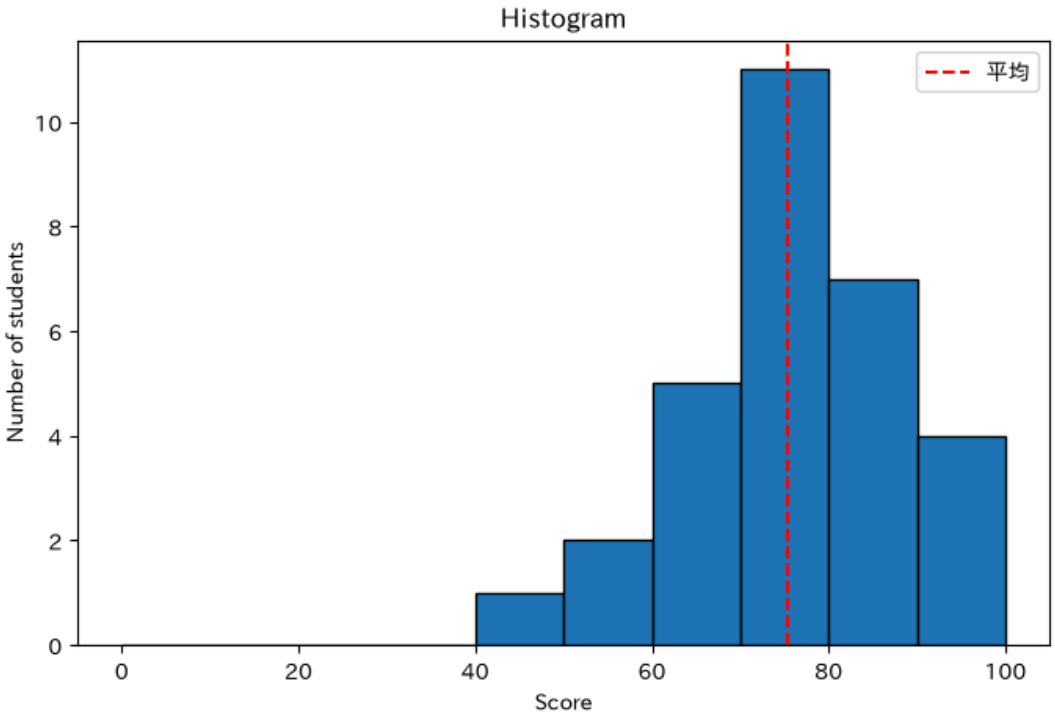


- AA-1 は教科書（表1.2）に合わせて 43〜65 kg にした。こちらも条件1・2は満たしている
- 刻みを 5 kg にすると、下限 40、上限 65、`bins=5` になる。区間が少なくなりすぎないかも見て決める

AA-5 平均の補助線

- ヒストグラムでは点数が**横軸**にあるので、平均は**縦の線** `plt.axvline(x=np.mean(データ))`（第 6 講 N-7 の横棒グラフのときと同じ）

```
plt.figure(figsize=(8, 5))
plt.hist(score, bins=10, range=(0, 100), edgecolor='k')
plt.axvline(x=np.mean(score), color='r', linestyle='--', label='平均')
plt.title('Histogram')
plt.xlabel('Score')
plt.ylabel('Number of students')
plt.legend()
plt.show()
```

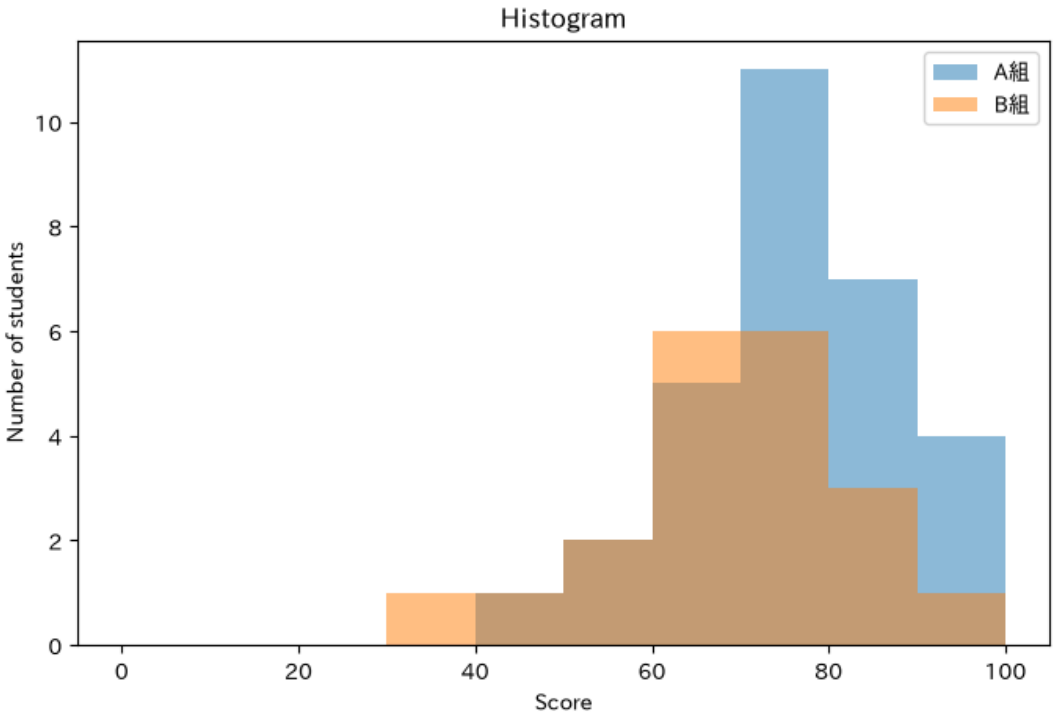


AA-6 2つのグループを重ねる

- 第 4 講の散布図と同じく、`plt.hist` を**2回**書くと重なる
- 後ろが隠れないように `alpha=0.5`（透明度）、どちらがどちらか分かるように `label=` と `plt.legend()`
- 2つの `bins=` と `range=` は**同じ値**にする（区間がずれると比べられない）

```
score_b = [38, 47, 55, 58, 60, 62, 64, 66, 67, 69,
           70, 71, 72, 74, 76, 78, 80, 83, 87, 91]

plt.figure(figsize=(8, 5))
plt.hist(score, bins=10, range=(0, 100), alpha=0.5, label='A組')
plt.hist(score_b, bins=10, range=(0, 100), alpha=0.5, label='B組')
plt.title('Histogram')
plt.xlabel('Score')
plt.ylabel('Number of students')
plt.legend()
plt.show()
```



使ってみよう

- `exam` シートの `'数学'` と `'社会'` のヒストグラムを、上のように重ねてみよう。平均は近いのに、分布の形はかなり違う

```
df_exam = pd.read_excel('dsa2026data.xlsx', sheet_name='exam')
```

AB. エラーバー

AB-0 完成形（エラーバー）

5教科の**平均点**を点で、**平均 ± 標準偏差**を上下の線（エラーバー）で示したもの。

1. **全部コピー**して自分の notebook のセルに貼り付け、実行する
2. まずは**色や大きさを変えて**、グラフのどこが変わるかを確かめる
3. そのあと部品（[1]〜[3]）の働きを1つずつ学ぶ（AB-1〜AB-5）

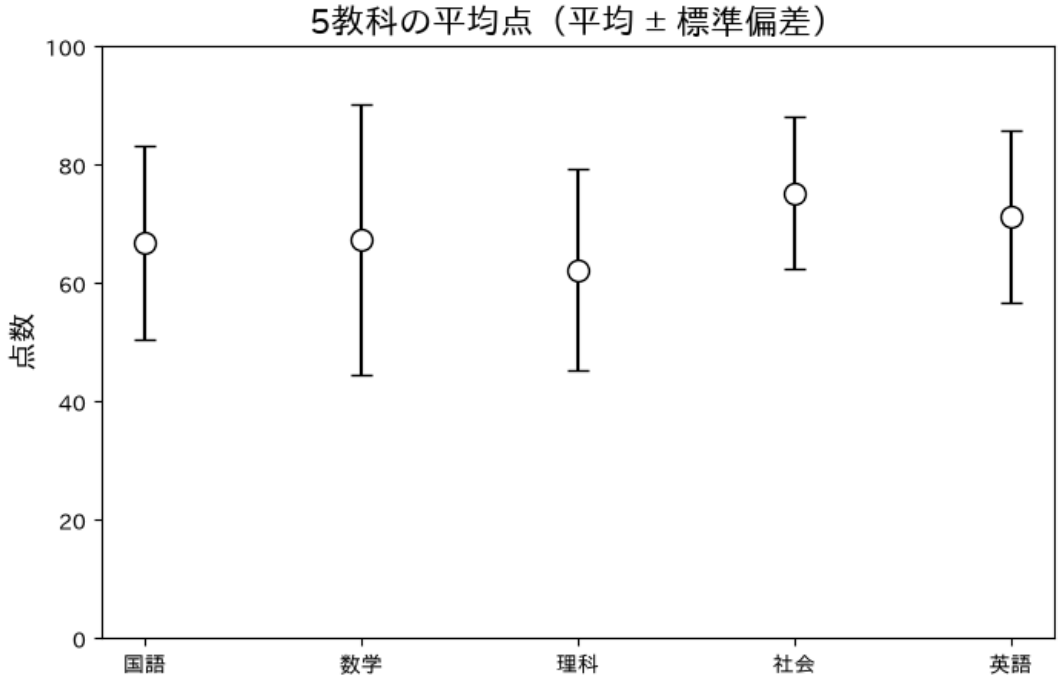
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import japanize_matplotlib

df = pd.read_excel('dsa2026data.xlsx', sheet_name='exam')
subjects = ['国語', '数学', '理科', '社会', '英語']

means = [np.mean(df['国語']), np.mean(df['数学']), np.mean(df['理科']),  # [1] 平均のリスト
         np.mean(df['社会']), np.mean(df['英語'])]
stds = [np.std(df['国語']), np.std(df['数学']), np.std(df['理科']),  # [1] 標準偏差のリスト
        np.std(df['社会']), np.std(df['英語'])]

plt.figure(figsize=(8, 5))
```

```
plt.errorbar(subjects, means, yerr=stds,  # [2] エラーバー
             fmt='o', markersize=10, capsize=5,  # [3] 見た目
             color='w', markeredgecolor='k', ecolor='k')
plt.ylim(0, 100)
plt.title('5教科の平均点（平均 ± 標準偏差）', fontsize=16)
plt.ylabel('点数', fontsize=14)
plt.show()
```



変えてみよう（1か所変えたら、もう一度実行する）

- `capsize=5` → `capsize=15`、`0`（線の端の横棒の長さ）
- `fmt='o'` → `fmt='s'`（四角）、`fmt='o-'`（点を線でつなぐ）
- `color='w'` → `color='orange'`（点の中の色）

番号	部品	書き方	学ぶところ
[1]	平均・標準偏差のリスト	`[np.mean(df['国語']), np.mean(df['数学']), …]`	AB-3・AB-5
[2]	エラーバー	`plt.errorbar(x, 平均, yerr=標準偏差)`	AB-4
[3]	見た目	`fmt`、`markersize`、`capsize`、`color`、`markeredgecolor`、`ecolor`	AB-4

AB-1 平均だけでは分からないこと

エラーバーは、平均とばらつきを**点1つと縦の線1本**で表したもの。いくつかのグループ（クラス、教科など）の平均とばらつきを並べて比べるときに使う。

- 下の2クラスは、**平均が同じ**（68点）。でも、点数の散らばり方は同じとは限らない
- 散らばりの大きさを1つの数で表したものが**標準偏差**（第 4 講 G-3）

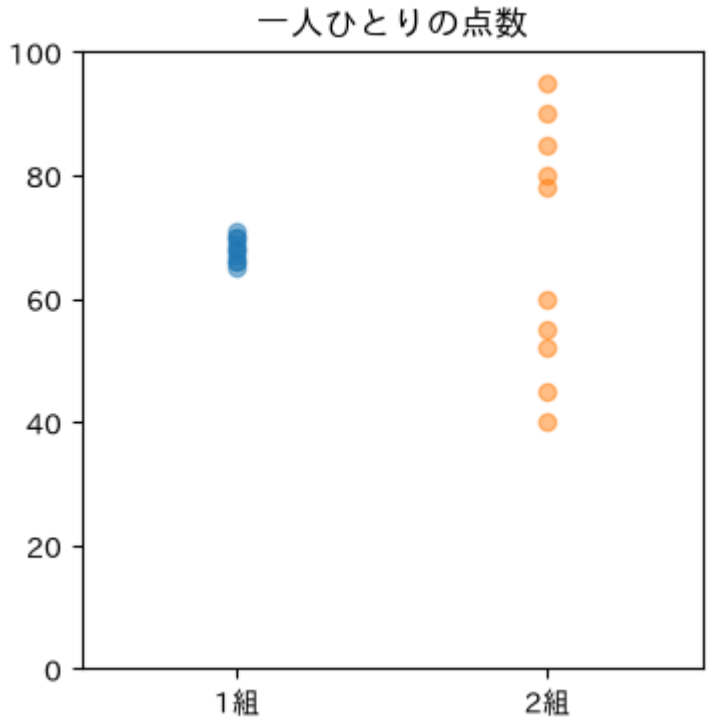
```
group1 = [66, 68, 70, 67, 69, 68, 71, 65, 70, 66]
group2 = [40, 95, 55, 85, 60, 90, 45, 80, 52, 78]
```

```
m = [np.mean(group1), np.mean(group2)]
s = [np.std(group1), np.std(group2)]
print('平均', m)
print('標準偏差', s)
```

```
平均 [np.float64(68.0), np.float64(68.0)]
標準偏差 [np.float64(1.8973665961010275), np.float64(18.83613548475377)]
```

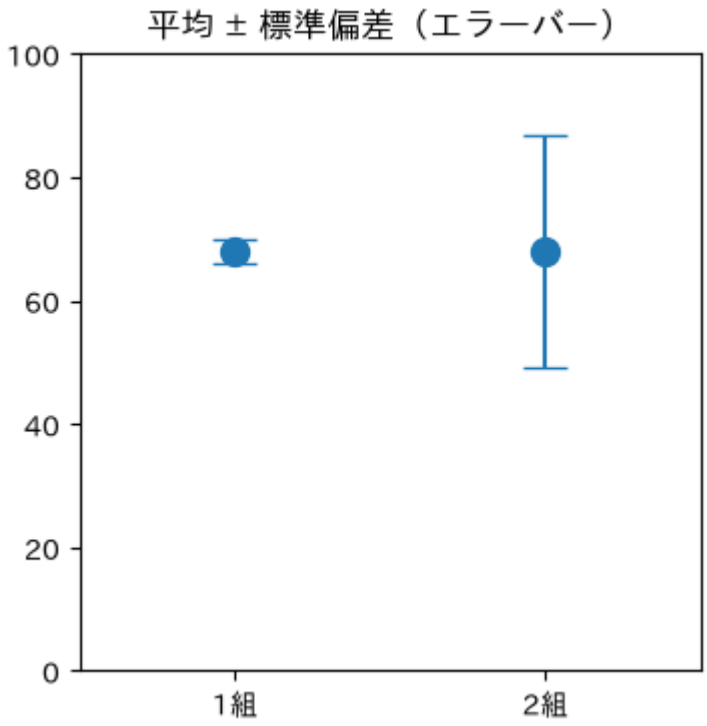
- まず、一人ひとりの点数を散布図で描く。横の位置は、クラスの名前を人数分（10個）並べたリスト
- `['1組'] * len(group1)` は「`'1組'` を group1 の人数分（10個）並べたリスト」という意味（リストに `* 数` を付けると、その数だけくり返して並べる。`len(リスト)` はリストの個数）

```
plt.figure(figsize=(4, 4))
plt.scatter(['1組'] * len(group1), group1, alpha=0.5)
plt.scatter(['2組'] * len(group2), group2, alpha=0.5)
plt.xlim(-0.5, 1.5)
plt.ylim(0, 100)
plt.title('一人ひとりの点数')
plt.show()
```



- 同じデータを、平均 ± 標準偏差の**エラーバー**で描く（書き方は AB-4）

```
plt.figure(figsize=(4, 4))
plt.errorbar(['1組', '2組'], m, yerr=s, fmt='o', capsize=8, markersize=10)
plt.xlim(-0.5, 1.5)
plt.ylim(0, 100)
plt.title('平均 ± 標準偏差（エラーバー）')
plt.show()
```



- エラーバーは、平均の点から上下に**標準偏差**の長さの線を引いたもの
- 平均が同じでも、ばらつきが大きいとエラーバーが**長く**（2組）、小さいと**短く**（1組）なる
- 山が1つで左右対称なデータ（正規分布に近いデータ）では、エラーバーの範囲（平均 ± 標準偏差）に**全体のおよそ3分の2**の人が入る。偏りや外れ値があるデータでは、この読み方は当てはまらない（AB-7）
- **注意**：エラーバーの長さが何を表すかは、図によって**いろいろ**ある
  - 標準偏差（データの散らばり）、標準誤差（平均のぶれ。AC-4）、95% 信頼区間（平均がありそうな範囲。AC-4）など
  - 標準誤差は標準偏差よりずっと短くなるので、同じデータでも見た目の印象が大きく変わる
  - Excel や seaborn などのツールでは、初期設定が標準偏差とは限らない
- だから、**描くとき**はタイトルや軸の名前に「平均 ± 標準偏差」のように**何を表すかを書く。読むとき**は、図の説明で**何を表しているかを確かめてから**読む。この授業では標準偏差で描く

AB-2 リストを np.array にする

- np.array にすると、全部の要素にまとめて計算できる（下の復習）。平均・標準偏差は `np.mean(変数)`・`np.std(変数)` で求める（AB-3）

- 2クラス（各100人）の、同じ教科のテストの点数

```
classA_score = [77,69,63,48,66,65,69,72,67,72,71,63,77,66,64,61,57,58,65,64,72,66,67,
                65,64,75,85,73,66,61,71,49,67,76,61,51,66,67,65,52,76,72,73,58,78,58,
                60,52,57,73,63,69,81,67,64,61,60,58,75,48,66,67,59,67,58,78,88,64,69,
                80,72,72,84,75,57,77,80,63,69,54,57,68,64,64,52,66,74,75,65,79,72,66,
                77,68,52,63,59,77,71,68]

classB_score = [57,76,75,62,66,57,70,67,78,67,55,72,79,61,56,76,72,69,71,76,69,71,73,
                67,67,71,58,65,74,69,77,74,75,77,65,73,84,66,63,53,56,71,63,63,68,70,
                77,69,73,62,64,57,79,79,64,57,66,83,64,61,68,57,48,69,67,57,58,71,73,
                74,68,56,71,58,63,80,67,60,68,67,55,59,51,63,71,75,71,61,95,67,71,68,
                72,61,67,73,60,65,81,64]
```

```
classA_score_np = np.array(classA_score)
classB_score_np = np.array(classB_score)

print(classA_score_np)
print(classB_score_np)
```

```
[77 69 63 48 66 65 69 72 67 72 71 63 77 66 64 61 57 58 65 64 72 66 67 65
 64 75 85 73 66 61 71 49 67 76 61 51 66 67 65 52 76 72 73 58 78 58 60 52
 57 73 63 69 81 67 64 61 60 58 75 48 66 67 59 67 58 78 88 64 69 80 72 72
 84 75 57 77 80 63 69 54 57 68 64 64 52 66 74 75 65 79 72 66 77 68 52 63
 59 77 71 68]
[57 76 75 62 66 57 70 67 78 67 55 72 79 61 56 76 72 69 71 76 69 71 73 67
 67 71 58 65 74 69 77 74 75 77 65 73 84 66 63 53 56 71 63 63 68 70 77 69
 73 62 64 57 79 79 64 57 66 83 64 61 68 57 48 69 67 57 58 71 73 74 68 56
 71 58 63 80 67 60 68 67 55 59 51 63 71 75 71 61 95 67 71 68 72 61 67 73
 60 65 81 64]
```

- `print( )` の中には、変換後の変数名（`classA_score_np` など）を入れる
- リストは `[77, 69, 63, …]` と `,` 区切りで、np.array は `[77 69 63 …]` と空白区切りで表示される

（復習）np.array でできること

np.array の便利なところは、平均や標準偏差を求められることだけではない。第 4 講 I-1（発展）で見たように、**全部の要素にまとめて**計算できる。

やりたいこと	書き方
全部の要素に同じ計算をする	`x + 5`、`x * 2`
同じ位置の要素どうしの和・積	`x + y`、`x * y`
正規化（最小を 0、最大を 1 にそろえる）	`(x - np.min(x)) / (np.max(x) - np.min(x))`

```
x = np.array([60, 80, 70, 90, 50])
y = np.array([70, 60, 90, 80, 40])

print(x + 5)
print(x + y)
print(x * y)
print((x - np.min(x)) / (np.max(x) - np.min(x)))
```

```
[65 85 75 95 55]
[130 140 160 170  90]
[4200 4800 6300 7200 2000]
[0.25 0.75 0.5  1.   0.  ]
```

- `x + y` は 60 + 70、80 + 60、… と、**同じ位置どうし**を計算する。`x * y` も同じ
- 正規化すると、いちばん小さい値（50）が 0、いちばん大きい値（90）が 1 になる。満点や単位が違うデータを、同じものさしで比べたいときに使う
- リストのままだと、`+` は2つのリストを**つなげる**だけで、`*` はエラーになる。`[60, 80] + [70, 60]` は `[60, 80, 70, 60]`

Excel から読んだ列（Series）も同じように使える

- `df['列名']` で取り出した列は、**Series**（シリーズ）という形式になる。リストでも np.array でもないが、**np.array と同じように**計算できる
- `np.array` に変えなくても、そのまま和・積・正規化ができ、`np.mean()` や `np.std()` にも渡せる

```
df_exam = pd.read_excel('dsa2026data.xlsx', sheet_name='exam')
math_score = df_exam['数学']
eng_score = df_exam['英語']

print(type(math_score))
print((math_score + eng_score).head())
print(((math_score - np.min(math_score)) / (np.max(math_score) -
np.min(math_score))).head())
print('平均', np.mean(math_score))
print('標準偏差', np.std(math_score))
```

```
<class 'pandas.Series'>
0    138
1    131
2    132
3    165
4    119
dtype: int64
0    0.744681
1    0.542553
2    0.638298
3    0.861702
4    0.553191
Name: 数学, dtype: float64
平均 67.36666666666666
標準偏差 22.77642543410962
```

- Series を `print` すると、左に行番号、最後に `dtype` が表示される（`.head()` は最初の5行）
- この授業のエラーバーは `np.std(df['列名'])` で計算する（AB-5）

AB-3 平均と標準偏差を求める

```
mean_A = np.mean(classA_score_np)
std_A = np.std(classA_score_np)
mean_B = np.mean(classB_score_np)
std_B = np.std(classB_score_np)

print('classA 平均', mean_A, '標準偏差', std_A)
print('classB 平均', mean_B, '標準偏差', std_B)
```

```
classA 平均 66.72 標準偏差 8.369085971598093
classB 平均 67.49 標準偏差 7.963033341635584
```

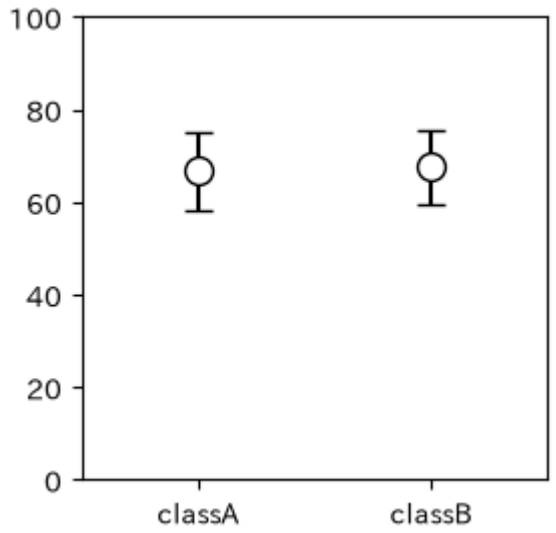
- `np.mean( )`・`np.std( )` には、np.array に変える前のリスト（`classA_score`）を渡しても同じ値になる

AB-4 エラーバーを描く

- `plt.errorbar(横の位置, 平均, yerr=エラーバーの長さ)` で描く。横の位置・平均・長さは、**同じ個数のリスト**にする
- `fmt='o'` で、点だけを描く（何も書かないと、点が線でつながる）
- 見た目のオプション：`markersize`（点の大きさ）、`capsize`（線の端の横棒の長さ）、`color`（点の中の色）、`markeredgecolor`（点のふちの色）、`ecolor`（線の色）

```
myclass = ['classA', 'classB']
mean_score = [mean_A, mean_B]
errorbar = [std_A, std_B]

plt.figure(figsize=(3, 3))
plt.errorbar(myclass, mean_score, yerr=errorbar, fmt='o', capsize=5, markersize=10,
             color='w', markeredgecolor='k', ecolor='k')
plt.xlim(-0.5, 1.5)
plt.ylim(0, 100)
plt.show()
```



- 横軸が文字（`'classA'` など）のとき、位置は左から `0, 1` になる。`plt.xlim(-0.5, 1.5)` で左右に余白を作っている

AB-5 5教科のエラーバー

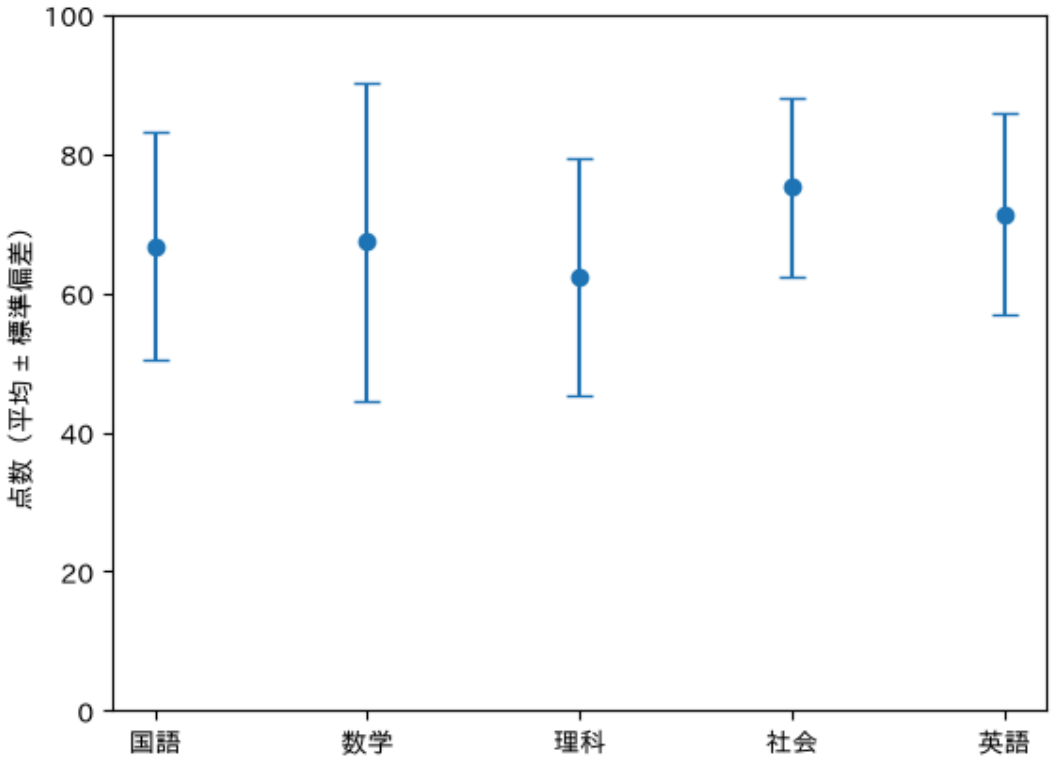
- `exam` シートの5教科で、教科ごとに平均と標準偏差を計算し、リストにまとめる（AB-0 の [1]）
- 5教科なので、`np.mean` と `np.std` を5回ずつ書く。同じ処理をくり返す書き方（for 文、第12講）を使うと、まとめて書ける
- `(` や `[` の中なら、`,` の後ろで改行して次の行に続けて書ける

```
df = pd.read_excel('dsa2026data.xlsx', sheet_name='exam')
subjects = ['国語', '数学', '理科', '社会', '英語']

means = [np.mean(df['国語']), np.mean(df['数学']), np.mean(df['理科']),
         np.mean(df['社会']), np.mean(df['英語'])]
stds = [np.std(df['国語']), np.std(df['数学']), np.std(df['理科']),
        np.std(df['社会']), np.std(df['英語'])]
print(means)
print(stds)
```

```
plt.errorbar(subjects, means, yerr=stds, fmt='o', capsize=5)
plt.ylim(0, 100)
plt.ylabel('点数（平均 ± 標準偏差）')
plt.show()
```

```
[np.float64(66.76666666666667), np.float64(67.36666666666666), np.float64(62.23333333333333
4), np.float64(75.23333333333333), np.float64(71.36666666666666)]
[np.float64(16.3354080314988), np.float64(22.77642543410962), np.float64(17.03659459973018),
np.float64(12.914290103946437), np.float64(14.554457125644442)]
```

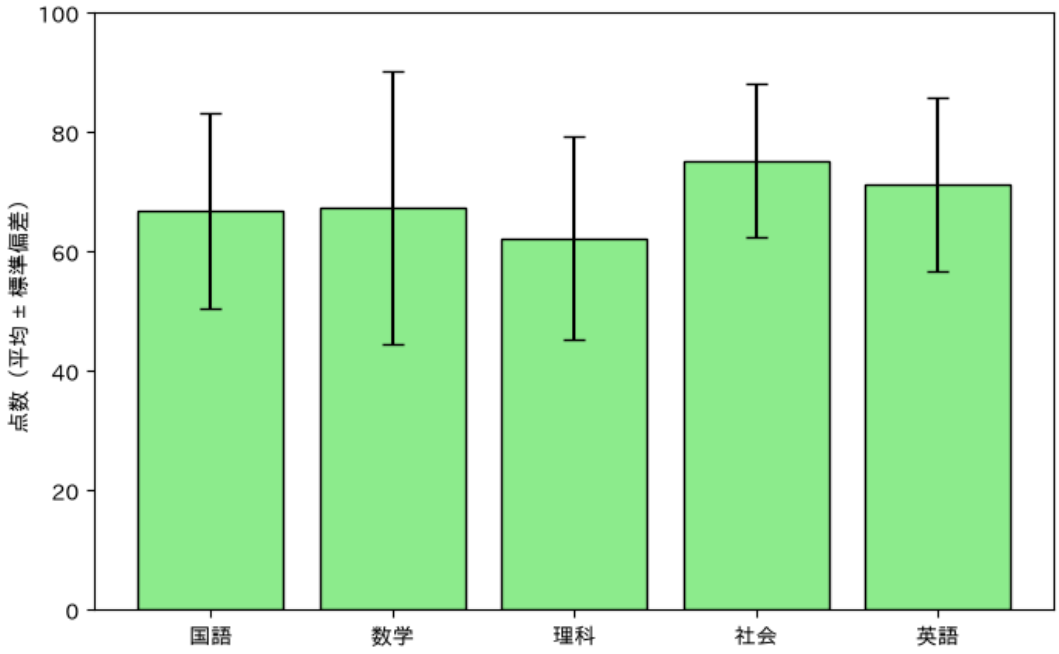


- 数学はエラーバーが長い＝点数のばらつきが大きい（第 7 講の箱ひげ図でも、数学の箱は長かった）

AB-6 棒グラフにエラーバーを付ける

- 棒グラフでは、`plt.bar(…, yerr=標準偏差, capsize=5)` と書くだけでエラーバーが付く

```
plt.figure(figsize=(8, 5))
plt.bar(subjects, means, yerr=stds, capsize=5, color='lightgreen', edgecolor='k')
plt.ylim(0, 100)
plt.ylabel('点数（平均 ± 標準偏差）')
plt.show()
```



使ってみよう

- AB-3 で求めた classA・classB の平均（`mean_A`、`mean_B`）と標準偏差（`std_A`、`std_B`）で、棒グラフにエラーバーを付けてみよう
- 棒の色は `color=['lightcoral', 'lightblue']` のように、リストで1本ずつ変えられる（第 6 講）

AB-7 エラーバーと箱ひげ図の使い分け

エラーバーも箱ひげ図（第 7 講）も、グループごとのばらつきを比べるグラフ。違いは、**何を代表の値にするか**。

- エラーバー：**平均**と**標準偏差**の2つの数で表す
- 箱ひげ図：**中央値**・箱（真ん中の半分の人）・ひげ・外れ値の**5つほどの数**で表す

グラフに渡すもの（第 7 講 S-2 の整理に、エラーバー・ヒストグラムを加える）

- 折れ線・散布図・棒グラフは、x と y の**同じ位置の値が組**になって、点や棒が1つ決まる（点や棒1つに、数が1つ）
- エラーバーも同じ形。`plt.errorbar(x, y, yerr=…)` の x、y、yerr は、どれも**グループの数だけ値が並んだリスト**（AB-1）
- 箱ひげ図は、箱1つを描くのに**そのグループ全員の点数**（リスト1つ）が必要。箱が2つならリストが2つ。それを `[ ]` でまとめた**リストのリスト**を渡す（第 7 講 S-2）

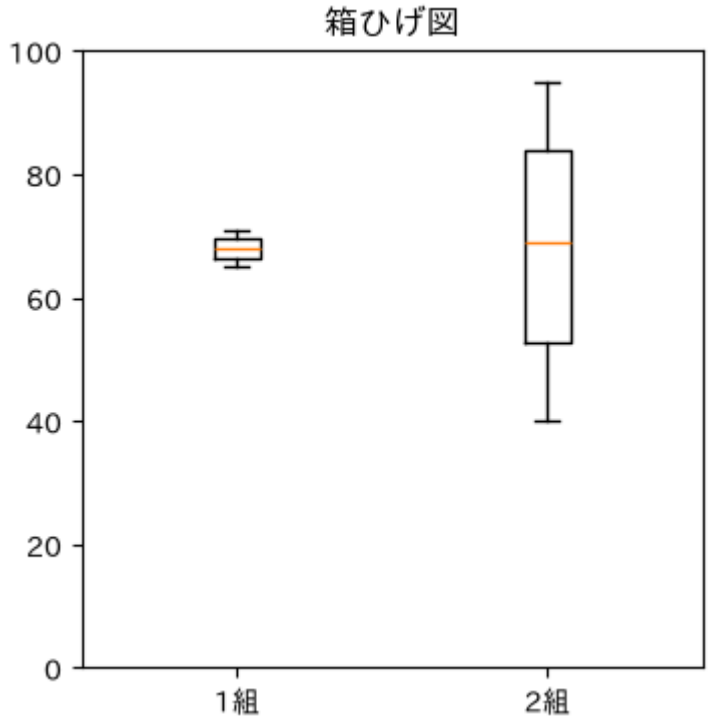
グラフ	書き方	渡すもの
折れ線・散布図・棒グラフ	`plt.plot(x, y)` など	x のリストと y のリスト
エラーバー	`plt.errorbar(x, y, yerr=…)`	x、y、yerr のリスト
ヒストグラム	`plt.hist(データ)`	データのリスト1つ
箱ひげ図	`plt.boxplot([データ1, データ2])`	データの**リストのリスト**

- 箱ひげ図には x を渡さない。箱は左から 1, 2, … の位置に並ぶ。名前は `plt.xticks([1, 2], ['1組', '2組'])` で付ける

- **注意**：`[ ]` を忘れて `plt.boxplot(group1, group2)` と書くと、エラーにならずに、**箱が1つだけ**描かれることがある。箱の数がおかしいときは `[ ]` を確かめる
- `[ ]` の中には、Excel の列も並べられる：`plt.boxplot([df['国語'], df['数学']])`

まず、AB-1 の1組・2組（平均はどちらも 68 点）を、エラーバーと同じ大きさ・同じ縦軸で、箱ひげ図に描く。

```
plt.figure(figsize=(4, 4))
plt.boxplot([group1, group2])
plt.xticks([1, 2], ['1組', '2組'])
plt.ylim(0, 100)
plt.title('箱ひげ図')
plt.show()
```



- AB-1 のエラーバーと見比べると、どちらも「1組は狭く集まり、2組は広く散らばっている」ことを表している
- このように、外れ値がなく左右がほぼ対称なデータでは、エラーバーと箱ひげ図は**同じこと**を伝える

違いがはっきり出るのは、**ほかから離れた値**（**外れ値**）があるとき。下の3組は、AB-1 の1組の最後の1人だけが**0点**（欠席して白紙で出した）にしたもの。

```
group3 = [66, 68, 70, 67, 69, 68, 71, 65, 70, 0]  # 1組の最後の1人だけ 0点

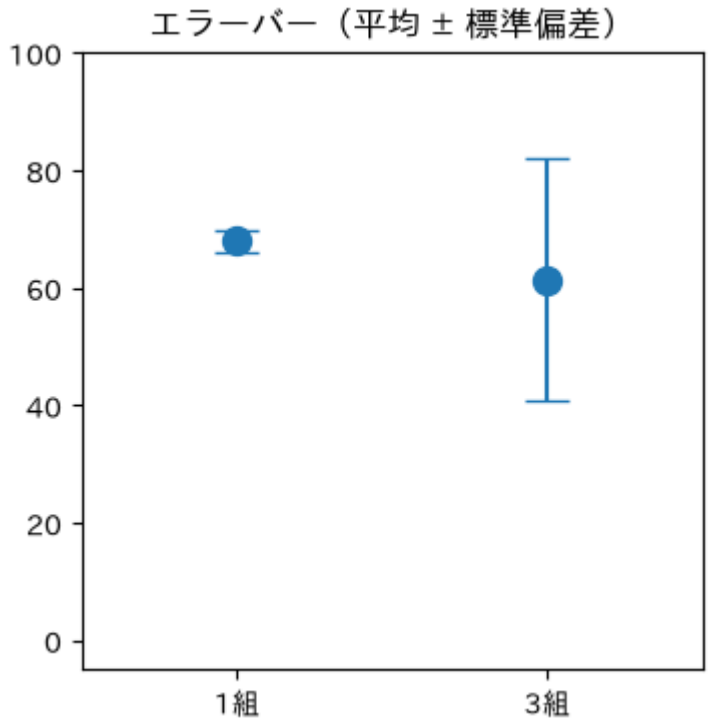
print('1組 平均', np.mean(group1), '標準偏差', np.std(group1), '中央値', np.median(group1))
print('3組 平均', np.mean(group3), '標準偏差', np.std(group3), '中央値', np.median(group3))
```

```
1組 平均 68.0 標準偏差 1.8973665961010275 中央値 68.0
3組 平均 61.4 標準偏差 20.543612145871524 中央値 68.0
```

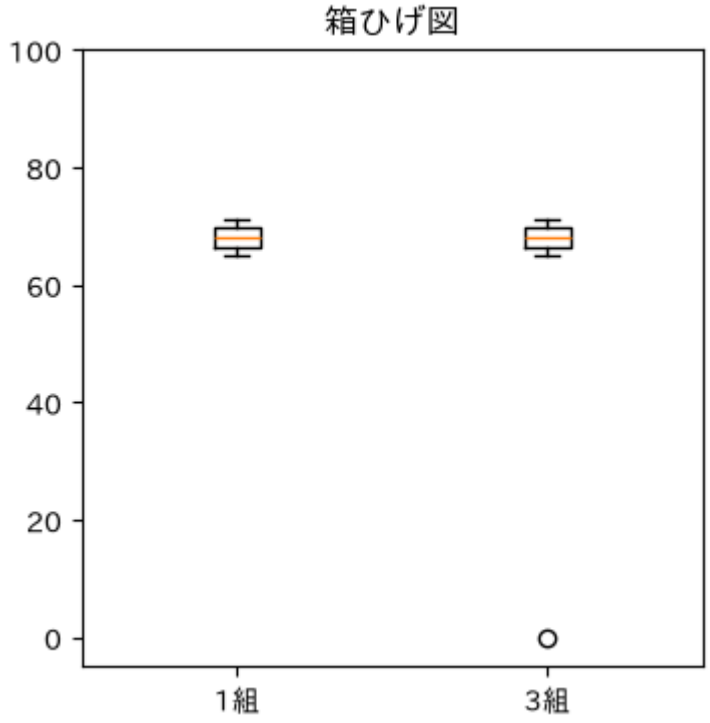
- 0点の1人のせいで、3組の平均は 68 → 61.4 点に下がり、標準偏差は 1.9 → 20.5 点に大きくなる
- 中央値（真ん中の人の点数）は、どちらも 68 点のまま

同じデータを、エラーバーと箱ひげ図で描いて比べる。

```
plt.figure(figsize=(4, 4))
plt.errorbar(['1組', '3組'], [np.mean(group1), np.mean(group3)],
             yerr=[np.std(group1), np.std(group3)], fmt='o', capsize=8, markersize=10)
plt.xlim(-0.5, 1.5)
plt.ylim(-5, 100)
plt.title('エラーバー（平均 ± 標準偏差）')
plt.show()
```




```
plt.figure(figsize=(4, 4))
plt.boxplot([group1, group3])
plt.xticks([1, 2], ['1組', '3組'])
plt.ylim(-5, 100)
plt.title('箱ひげ図')
plt.show()
```



- エラーバーでは、3組は「40〜80点に広く散らばっている」ように見える。実際は、9人が 65〜71 点に集まっていて、1人だけが 0 点
- 箱ひげ図では、3組の箱は1組とほぼ同じで、0 点の1人は**外れ値の点**として別に描かれる。「ほとんどの人は同じくらいで、1人だけ離れている」ことが分かる

	エラーバー	箱ひげ図
真ん中の印	平均	中央値
線・箱の長さ	標準偏差（1つの数）	箱は真ん中の半分の人、ひげは最小〜最大
外れ値があると	平均も線の長さも**引っぱられる**	箱と中央値はほとんど変わらず、外れ値は**点で別に描かれる**
分かること	**平均**の違い	データの**形**（偏り・外れ値）
向いている場面	平均を比べたい／グループが多い／棒グラフに付けたい	散らばり方そのものを見たい／偏りや外れ値がありそう

使い分けの覚え方

- 「**平均を比べたいならエラーバー、データの形を見たいなら箱ひげ図**」
- 迷ったら、まず箱ひげ図（かヒストグラム）で形を見る。左右がほぼ対称で外れ値がなければ、エラーバーにまとめてもよい
- エラーバーは2つの数だけで表すので、すっきりして見える。そのぶん、外れ値や偏りは**隠れてしまう**

分野によって、よく使うグラフが違う

物理・化学などの自然科学の論文では、箱ひげ図はあまり見かけず、エラーバーが多い。理由は、**ばらつきが何を意味するか**が違うから。

- **自然科学の実験**：同じ量を何回か測ると、測るたびに少しずつずれる。このばらつきは**測定の誤差**（ノイズ）で、知りたいのは平均（本当の値に近いもの）と、その確からしさ
  - 結果を「値 ± 誤差」で書くので、エラーバーがそのまま合う
  - 測る回数が 3〜5 回くらいと少ないことが多い。3点で箱ひげ図を描いても、箱や外れ値は意味を持たない
  - 測定の誤差は、正規分布に近くなることが多い。そのため、平均と標準偏差の2つの数で十分に表せる
  - 横軸が温度や時間のような連続した量で、折れ線や散布図の各点に誤差を付けて描くことが多い。エラーバーは各点に小さく付けられるが、箱ひげ図は付けにくい
- **社会科学・教育・医学・生物学など**：人や生き物の**一人ひとり（1匹ずつ）の違い**そのものが知りたいことが多い。データの数も多い
  - 偏りや外れ値にも意味があるので、分布の形が見える箱ひげ図がよく使われる

まとめ：「ばらつきが**測定の誤差**ならエラーバー、ばらつき**そのもの（個人差）**を見たいなら箱ひげ図」

使ってみよう

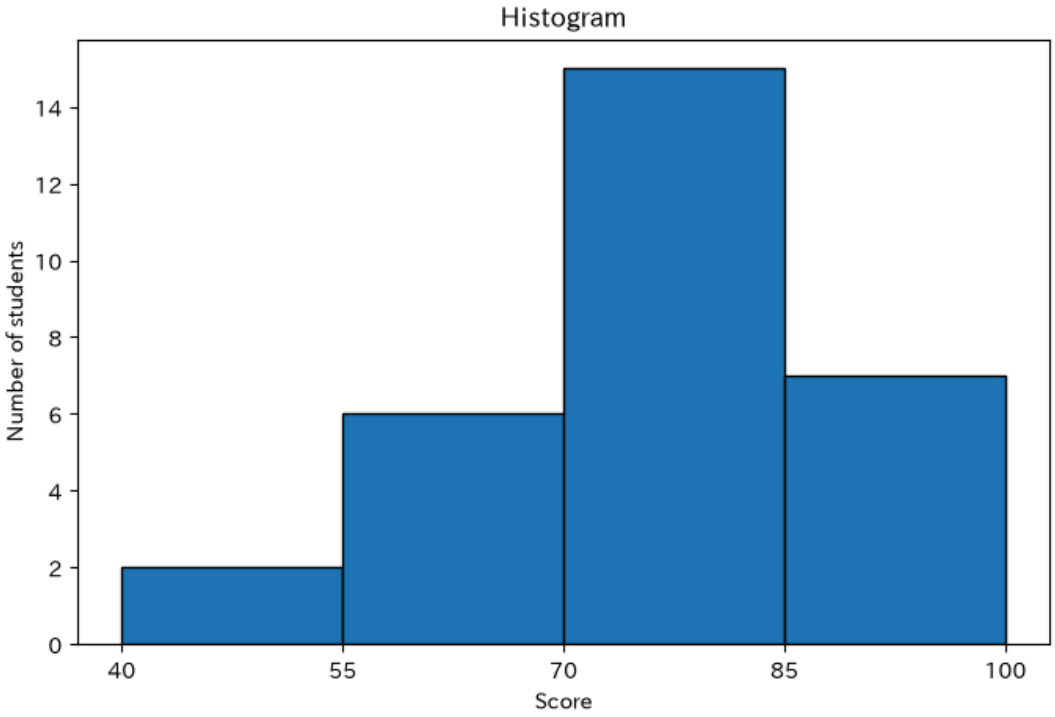
- `exam` シートの5教科（AB-5）を、`plt.boxplot` でも描いて、エラーバーと見比べよう（第 7 講の書き方）

AC. 発展

AC-1 区切りを15点刻みにする

- 10点刻みのように切りのよい値にならないときも、考え方は同じ。**1区間の幅** ＝（**終わり** − **始め**）÷ **`bins`** を計算して、その幅で目盛りを並べる
- 下の例は 40〜100 点を4つに分ける。幅は (100 − 40) ÷ 4 = **15点**なので、目盛りは `range(40, 101, 15)` で 40, 55, 70, 85, 100
- 幅が 12.5 点のように小数になると `range()` では並べられない（`range()` は整数だけ）。割り切れる `bins` を選ぶとよい

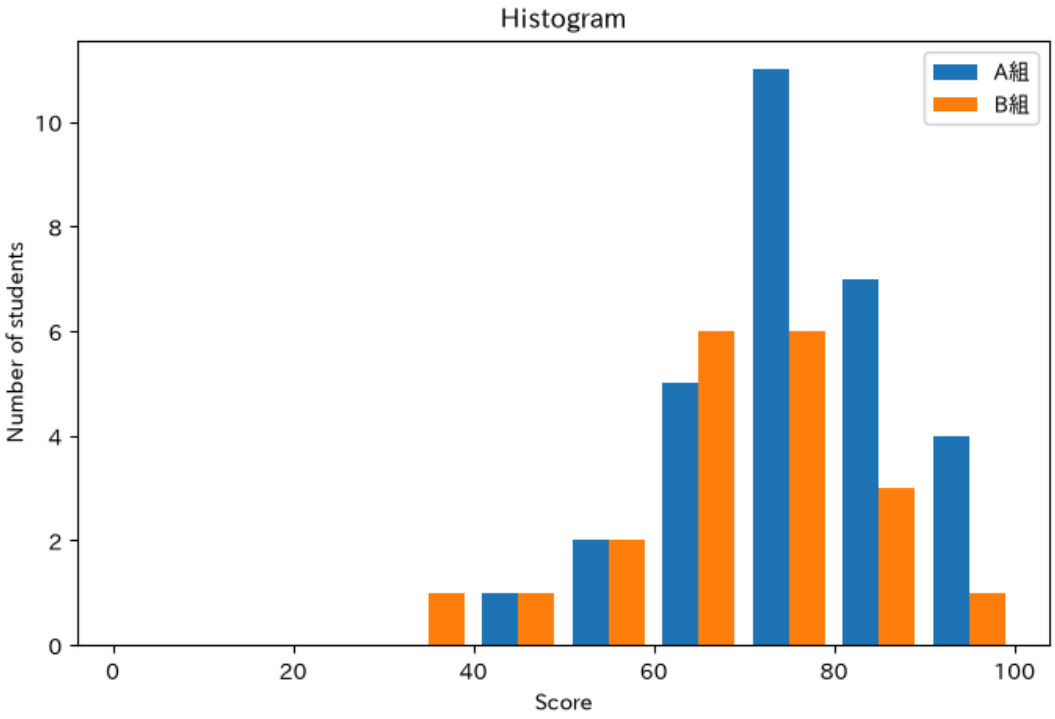
```
plt.figure(figsize=(8, 5))
plt.hist(score, bins=4, range=(40, 100), edgecolor='k')  # 40〜100点を4つに分ける → 15点刻み
plt.xticks(range(40, 101, 15))  # 目盛りも15点刻み：40, 55, 70, 85, 100
plt.title('Histogram')
plt.xlabel('Score')
plt.ylabel('Number of students')
plt.show()
```



AC-2 ヒストグラムを並べる・横向きにする

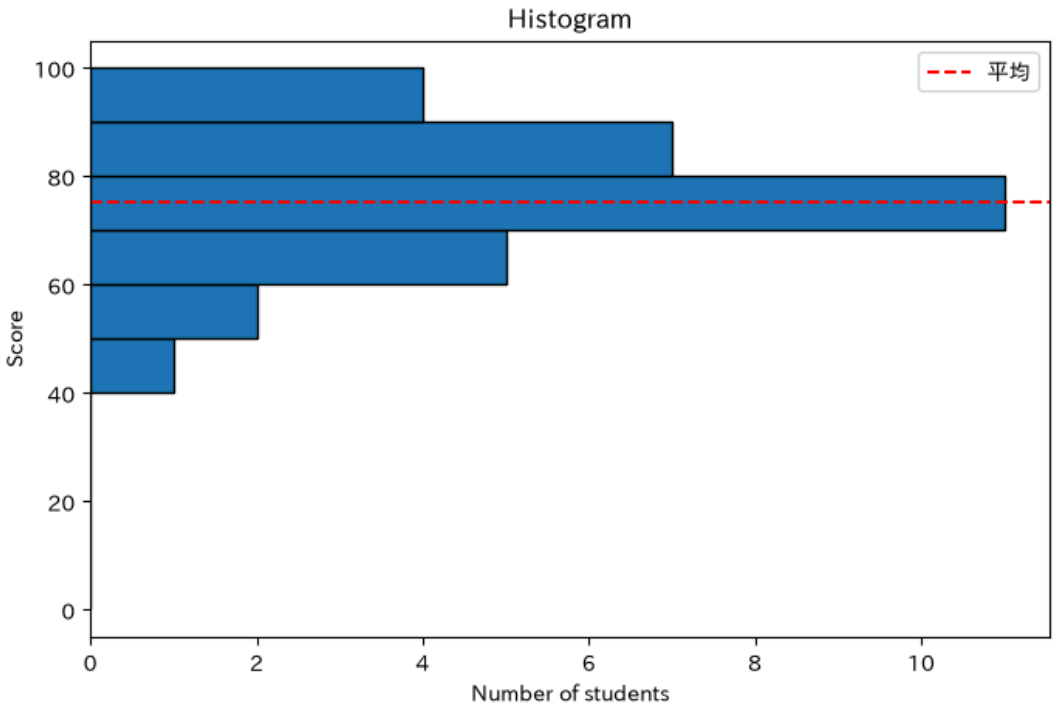
- `plt.hist([データ1, データ2])` のように**リストにまとめて**渡すと、重ねずに**横に並べて**描く。`label=` もリストで渡す

```
plt.figure(figsize=(8, 5))
plt.hist([score, score_b], bins=10, range=(0, 100), label=['A組', 'B組'])
plt.title('Histogram')
plt.xlabel('Score')
plt.ylabel('Number of students')
plt.legend()
plt.show()
```



- `orientation='horizontal'` で横向きになる。横棒グラフ（第 6 講 N-7）と同じく `xlabel` と `ylabel` を入れ替え、平均の線は**横の線** `plt.axhline`

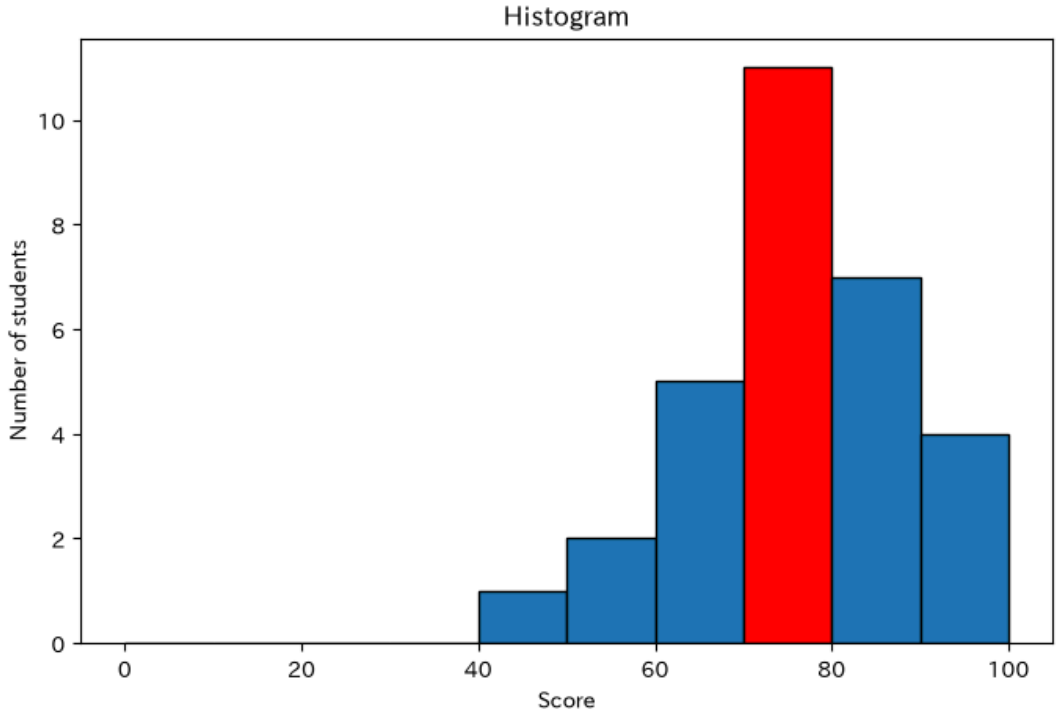
```
plt.figure(figsize=(8, 5))
plt.hist(score, bins=10, range=(0, 100), edgecolor='k', orientation='horizontal')
plt.axhline(y=np.mean(score), color='r', linestyle='--', label='平均')
plt.title('Histogram')
plt.xlabel('Number of students')
plt.ylabel('Score')
plt.legend()
plt.show()
```



AC-3 ヒストグラムの1本だけ色を変える

- 棒グラフ（第 6 講 P-4）では `color=` に色のリストを渡せたが、`plt.hist` の `color=` は**全部の棒に同じ色**しか付けられない
- そこで、描いた棒をあとから1本取り出して色を塗り替える
- `plt.hist` は描いた結果を**3つ**返す。`n, bins, patches = plt.hist(...)` のように、`,` で区切った3つの変数で受け取る
  - `n`：各区間の人数
  - `bins`：区切りの位置
  - `patches`：棒（左から順に入っている）
- `patches` はリストと同じく先頭が `[0]`。10点刻みなら 0〜10点が `[0]`、10〜20点が `[1]` … 70〜80点が `[7]`
- `patches[7].set_facecolor('r')` で、その棒だけ色が変わる（`facecolor` は塗りの色。枠線の色は `edgecolor`）

```
plt.figure(figsize=(8, 5))
n, bins, patches = plt.hist(score, bins=10, range=(0, 100), edgecolor='k')
patches[7].set_facecolor('r')  # 左から8本目（70〜80点）だけ赤
plt.title('Histogram')
plt.xlabel('Score')
plt.ylabel('Number of students')
plt.show()
```

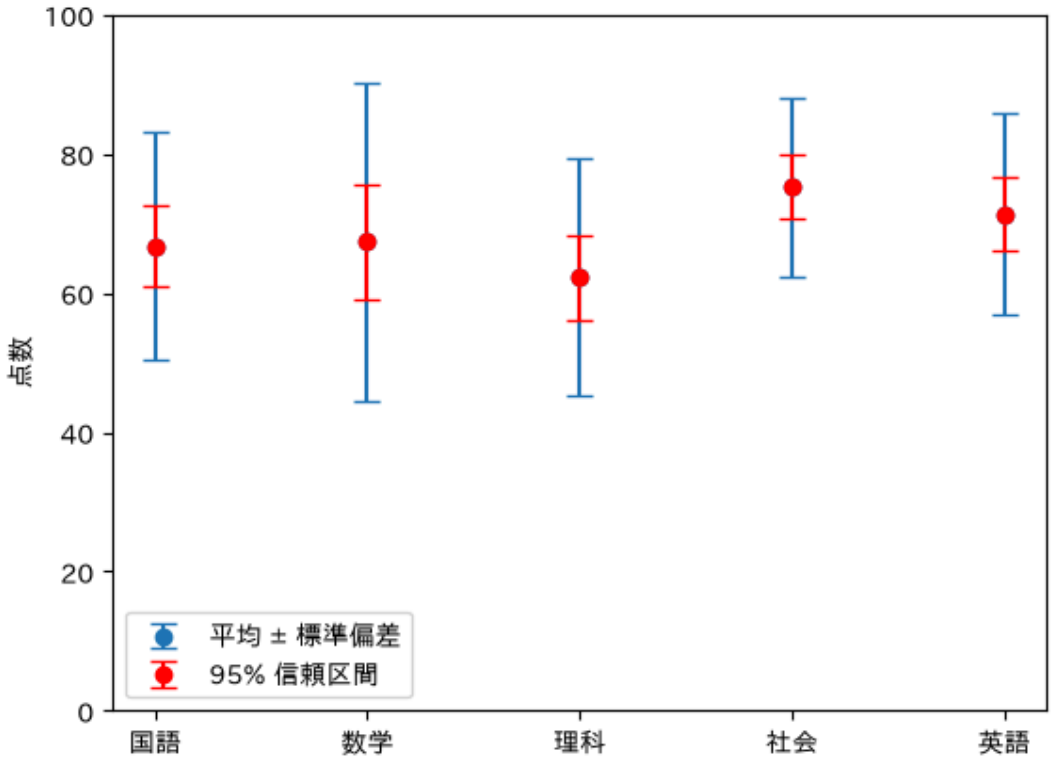


AC-4 標準誤差と信頼区間

- **標準偏差**：一人ひとりの点数のばらつき
- **標準誤差** ＝ 標準偏差 ÷ √人数：**平均値そのもの**のぶれ（もう一度同じ人数でテストをしたら、平均はどれくらい変わりそうか）。人数が多いほど小さくなる
- 平均 ± 1.96 × 標準誤差 が、おおよそ **95% 信頼区間**

```
n = len(df)
ses = 1.96 * np.array(stds) / np.sqrt(n)

plt.errorbar(subjects, means, yerr=stds, fmt='o', capsize=5, label='平均 ± 標準偏差')
plt.errorbar(subjects, means, yerr=ses, fmt='o', capsize=5, color='r', label='95% 信頼区間')
plt.ylim(0, 100)
plt.ylabel('点数')
plt.legend()
plt.show()
```



- `np.array(stds)` にすると、`1.96 * …` や `/ np.sqrt(n)` が5教科ぶん**まとめて**計算される（リストのままではできない）
- 信頼区間は標準偏差より短い。2つの教科の信頼区間が重なっていなければ、「平均点に差がある」と考える目安になる

AC-5 関数とメソッド

- この授業では、平均を `np.mean(x)` のように**関数**で求めてきた。`関数名(データ)` の形で、データを `( )` の中に渡す
- 同じ計算を `x.mean()` と書くこともできる。データの後ろに `.` を付けて、そのデータが持っている機能を呼び出す書き方で、**メソッド**という
- np.array や Series（Excel の列）は、平均・標準偏差・最大・最小などのメソッドを持っている

関数	メソッド	意味
`np.mean(x)`	`x.mean()`	平均
`np.std(x)`	`x.std()`	標準偏差
`np.max(x)`	`x.max()`	最大

関数	メソッド	意味
`np.min(x)`	`x.min()`	最小

```
x = np.array([60, 80, 70, 90, 50])

print(np.mean(x), x.mean())
print(np.max(x), x.max())
print((x - np.min(x)) / (np.max(x) - np.min(x)))
print((x - x.min()) / (x.max() - x.min()))
```

```
70.0 70.0
90 90
[0.25 0.75 0.5  1.   0.  ]
[0.25 0.75 0.5  1.   0.  ]
```

- 関数でもメソッドでも、結果は同じ。ネットや AI の回答にはメソッドの書き方もよく出てくるので、読めるようにしておくとよい
- リストはこのメソッドを持っていないので、`[60, 80].mean()` はエラーになる。関数の `np.mean([60, 80])` は、リストのままでも使える
- AC-3 の `patches[7].set_facecolor('r')` もメソッド。`patches[7]`（棒1本）が持っている「塗りの色を変える」機能を呼び出している

注意：Series の `.std()` だけは値が少し違う

```
df_exam = pd.read_excel('dsa2026data.xlsx', sheet_name='exam')
math_score = df_exam['数学']

print('np.std(math_score) ', np.std(math_score))
print('math_score.std()   ', math_score.std())
```

```
np.std(math_score)  22.77642543410962
math_score.std()    23.16579424471576
```

- Series の `.std()` は「データの数 − 1」で割り（数学は 23.2）、`np.std()` や np.array の `.std()` は「データの数」で割る（数学は 22.8）
- この授業のエラーバーは、関数の `np.std()` で計算する

まとめ

やりたいこと	書き方
以下はヒストグラム	
ヒストグラム	`plt.hist(データ)`
区間の数・範囲	`bins=10` 、`range=(0, 100)`
色・枠線	`color='lightgreen'` 、`edgecolor='k'`
平均の補助線	`plt.axvline(x=np.mean(データ))`

やりたいこと	書き方
2つのグループを重ねる	`plt.hist` を2回、`alpha=0.5` 、`label=` と `plt.legend()`
以下はエラーバー	
np.array にする	`np.array(リスト)` 。平均は `np.mean(変数)` 、標準偏差は `np.std(変数)`
エラーバー	`plt.errorbar(x, 平均のリスト, yerr=標準偏差のリスト, fmt='o', capsize=5)`
見た目	`markersize=10` 、`color='w'` 、`markeredgecolor='k'` 、`ecolor='k'`
棒グラフに付ける	`plt.bar(x, 平均のリスト, yerr=標準偏差のリスト, capsize=5)`
エラーバーと箱ひげ図	平均を比べるならエラーバー、データの形（偏り・外れ値）を見るなら箱ひげ図 `plt.boxplot([データ1, データ2])`

- ヒストグラムは**1つの量の分布**、箱ひげ図は**いくつかのグループのばらつき比べ**、エラーバーは**平均とばらつき**を並べて比べるときに使う

第10講：遊び心のあるグラフ・グラフを並べる

ここまでのグラフに、画像や記号のマーカー・背景の画像・ドーナツグラフやゲージなどの工夫を加えて、**記憶に残る**グラフにする（AH）。次に、`plt.subplot` で**複数のグラフを1枚に並べる**（AI）。AJ は発展（手書き風・アニメーション・plotly・スライダー）。

準備：AH で使う画像 `sweets_montblanc.png` を、notebook と同じ場所（ホームフォルダ）に置く。置けないときは、次のセルを実行すると GitHub からダウンロードできる（Windows・Mac 共通）。

- 先頭の `!` は、Python ではなくパソコンのコマンドを動かす印。`curl` はインターネットからファイルを取ってくるコマンド（`-sO` は、途中経過を出さず、元と同じ名前で保存する）

```
!curl -sO https://raw.githubusercontent.com/CreativeStudies-
NGT/lectures/main/dsa/sweets_montblanc.png
```

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import japanize_matplotlib

from matplotlib.offsetbox import OffsetImage, AnnotationBbox
```

- 2行目の `from matplotlib.offsetbox import …` は、画像をマーカーにするための道具（AH-1 で使う）

AH. 遊び心のあるグラフ

AH-0 完成形

月ごとのモンブランの売上を、**モンブランの画像**をマーカーにして描いたもの。

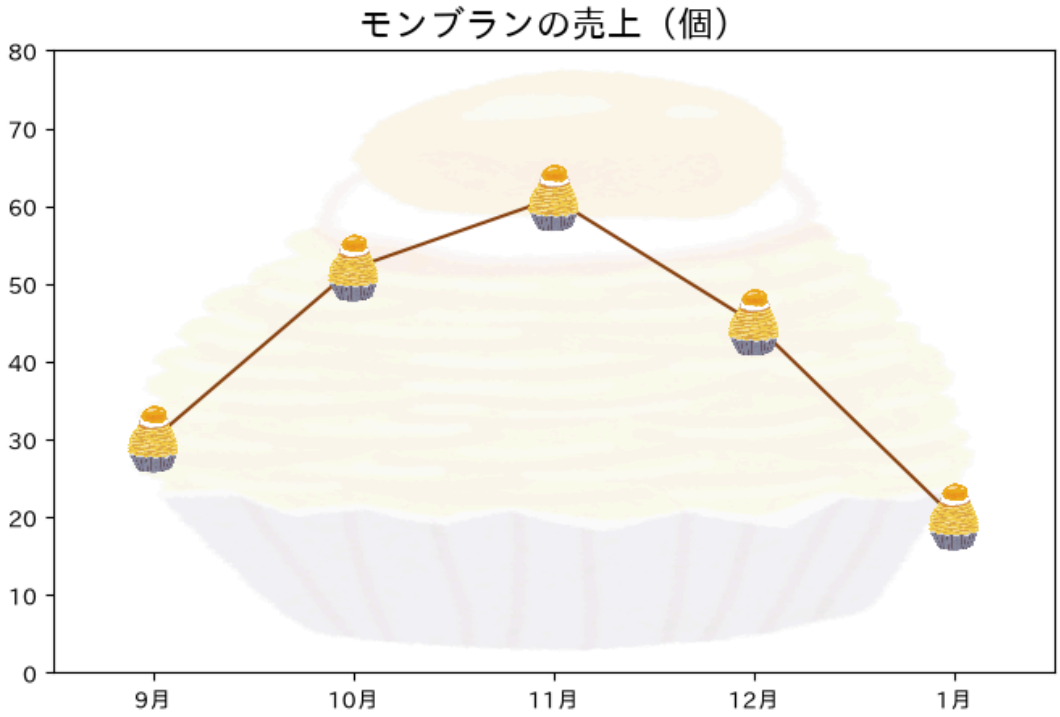
1. **全部コピーして**自分の notebook のセルに貼り付け、実行する
2. まずは**数や大きさを変えて**、グラフのどこが変わるかを確かめる
3. そのあと部品（[1]〜[3]）の働きを1つずつ学ぶ（AH-1〜AH-3）

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import japanize_matplotlib
from matplotlib.offsetbox import OffsetImage, AnnotationBbox

img = plt.imread('sweets_montblanc.png')  # [1] 画像を読む
y = [30, 52, 61, 45, 20]

plt.figure(figsize=(8, 5))
plt.plot([0, 1, 2, 3, 4], y, color='saddlebrown', zorder=1)
for i in range(len(y)):  # [2] 画像のマーカー
    box = AnnotationBbox(OffsetImage(img, zoom=0.08), (i, y[i]), frameon=False)
    plt.gca().add_artist(box)
plt.imshow(img, extent=[-0.5, 4.5, 0, 80], alpha=0.1, aspect='auto')  # [3] 背景
plt.xticks([0, 1, 2, 3, 4], ['9月', '10月', '11月', '12月', '1月'])
```

```
plt.xlim(-0.5, 4.5)
plt.ylim(0, 80)
plt.title('モンブランの売上（個）', fontsize=16)
plt.show()
```



変えてみよう（1か所変えたら、もう一度実行する）

- `zoom=0.08` → `zoom=0.15`（画像の大きさ）
- `alpha=0.1` → `alpha=0.4`（背景の濃さ）
- `y` の数を変える（例：`[30, 52, 61, 45, 20]` → `[10, 20, 70, 60, 30]`）

番号	部品	書き方	学ぶところ
[1]	画像を読む	`img = plt.imread('画像ファイル名')`	AH-1
[2]	画像のマーカー	`for` で1つずつ `AnnotationBbox(OffsetImage(img, zoom=…), (x, y))`	AH-1
[3]	背景の画像	`plt.imshow(img, extent=[…], alpha=0.1, aspect='auto')`	AH-3

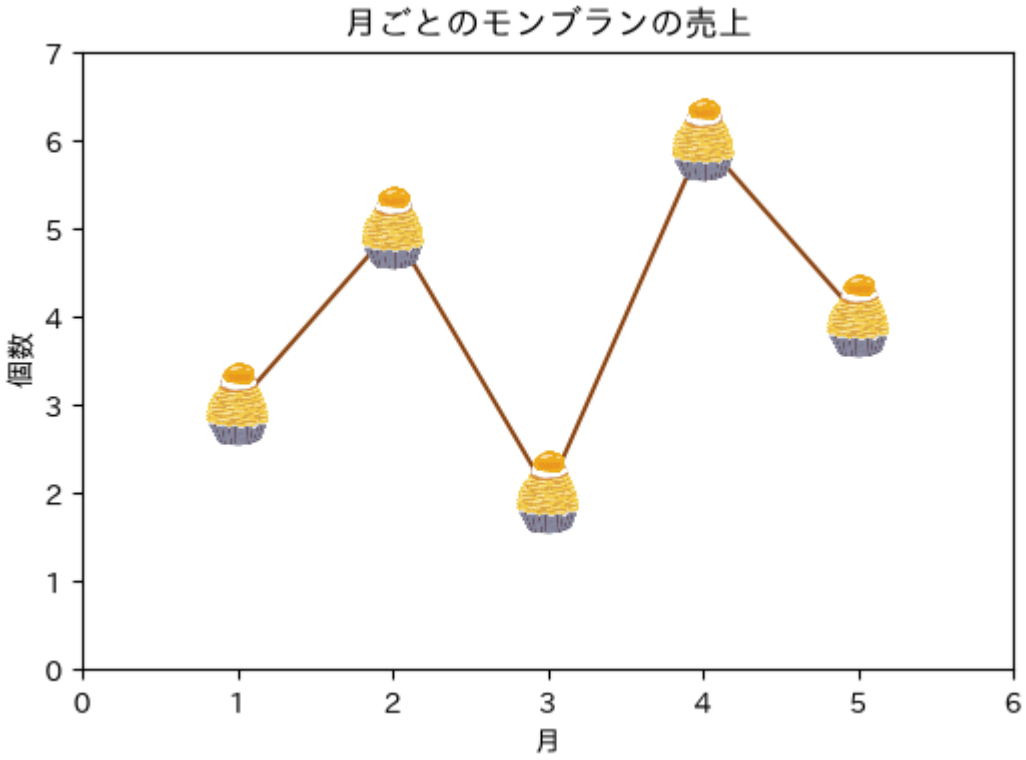
AH-1 画像をマーカーにする

- 下のセルは**ひな形**。写して使えばよく、1行ずつ覚えなくてよい
- 変えるのは `画像ファイル名` ・ `x` ・ `y` ・ `zoom`（画像の大きさ）だけ
- 画像は、背景が透明な png（「いらすとや」の画像など）を使うときれいに見える

```
img = plt.imread('sweets_montblanc.png')
x = [1, 2, 3, 4, 5]
y = [3, 5, 2, 6, 4]

plt.figure(figsize=(6, 4))
plt.plot(x, y, color='saddlebrown', zorder=1)
for i in range(len(x)):
    box = AnnotationBbox(OffsetImage(img, zoom=0.08), (x[i], y[i]), frameon=False)
    plt.gca().add_artist(box)
plt.xlim(0, 6)
```

```
plt.ylim(0, 7)
plt.title('月ごとのモンブランの売上')
plt.xlabel('月')
plt.ylabel('個数')
plt.show()
```



- `for i in range(len(x)):` は **for 文**（くり返し）。くわしくは第12講。ここでは写して使えばよい
  - `range(len(x))` は 0, 1, 2, 3, 4（点の数だけ）。`i` をこの順に変えながら、その下の字下げした2行をくり返す
  - つまり、1つ目の点、2つ目の点、…と、画像を1枚ずつ置いている
- `(x[i], y[i])` は i 番目の点の位置
- `plt.imread('画像ファイル名')` で画像を読み込み、`img` という名前を付けておく
  - `FileNotFoundError` と出るときは、画像がこの notebook と同じフォルダにない。ダウンロードした notebook は「ダウンロード」フォルダで開かれていることが多いので、notebook と画像を両方ともいつもの場所（ホームフォルダ）に置く
- 画像は3段階で置く：`OffsetImage(img, zoom=0.08)` で画像を貼れる部品にする → `AnnotationBbox(部品, (x, y), frameon=False)` で置く場所を決めた箱に入れる（`frameon=False` は箱の枠線を消す）→ `plt.gca().add_artist(box)` でいまのグラフに貼る（`plt.gca()` は「いま描いているグラフ」）
- `zorder=` は重なりの順番（数が大きいほど上、第 4 講 H-6）。線を `zorder=1`（小さい数）にして、画像より下に描かれるようにしている
- 画像は `plt.xlim`・`plt.ylim` の範囲を自動で広げない。範囲は自分で指定する

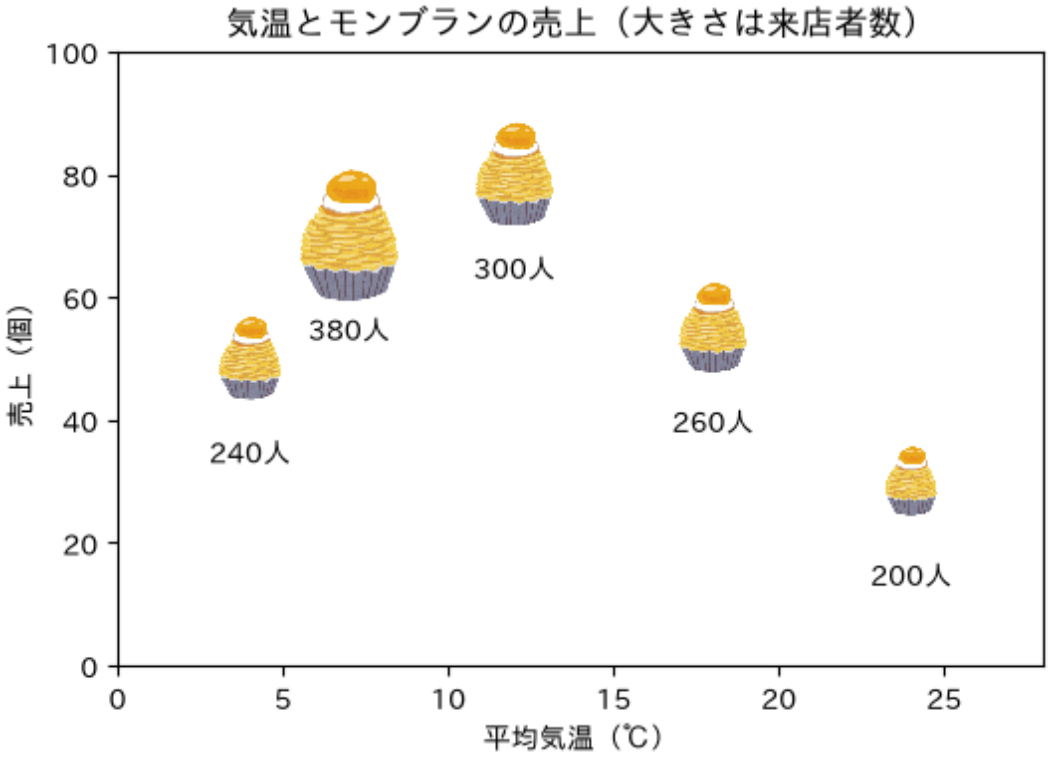
散布図にして、3つ目の値で画像の大きさを変える

- 第 4 講 H-3 のバブルチャートでは、3つ目の値（年齢）を `s=` に渡して点の大きさを変えた。画像のマーカーでも同じことができる
- 下は、月ごとの気温（x）とモンブランの売上（y）に、**来店者数**を画像の大きさとして加えたもの
- `plt.plot` で線を引かないので、点だけの散布図になる

- 大きさは `zoom=` で決める。来店者数をそのまま渡すと大きすぎるので、`visitors[i] / 3000` で小さくしている（380人で `zoom` が約0.13）

```
img = plt.imread('sweets_montblanc.png')
temp = [24, 18, 12, 7, 4]  # 平均気温（9月〜1月）
sales = [30, 55, 80, 70, 50]  # モンブランの売上（個）
visitors = [200, 260, 300, 380, 240]  # 来店者数（人）

plt.figure(figsize=(6, 4))
for i in range(len(temp)):
    box = AnnotationBbox(OffsetImage(img, zoom=visitors[i] / 3000), (temp[i], sales[i]),
frameon=False)
    plt.gca().add_artist(box)
    plt.text(temp[i], sales[i] - 17, str(visitors[i]) + '人', ha='center')
plt.xlim(0, 28)
plt.ylim(0, 100)
plt.title('気温とモンブランの売上（大きさは来店者数）')
plt.xlabel('平均気温（℃）')
plt.ylabel('売上（個）')
plt.show()
```

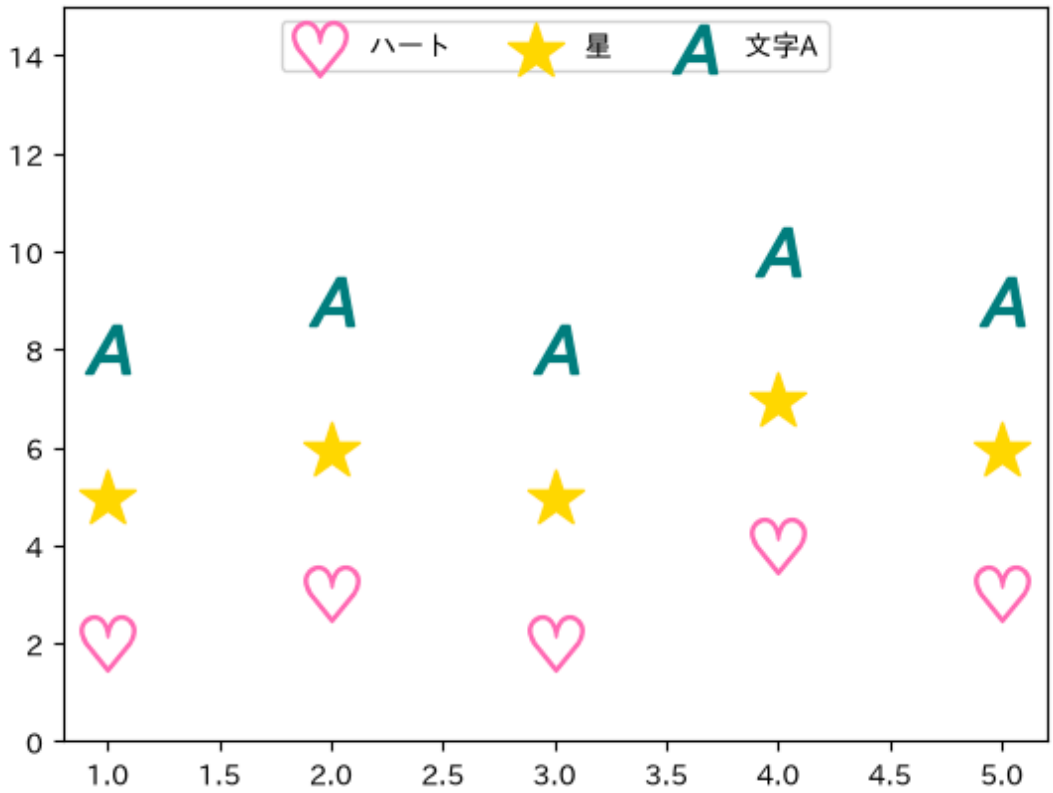


- AH-1 のひな形との違いは、`zoom=0.08`（全部同じ大きさ）を `zoom=visitors[i] / 3000`（点ごとに違う大きさ）にしたところ。`i` 番目の画像に、`i` 番目の来店者数が使われる
- `plt.text(x, y, 文字, ha='center')` で、画像の下に来店者数を書いている。`str(数)` は数を文字にする命令で、`+ '人'` で「人」をつなげている。`ha='center'` は文字を x の位置の真ん中にそろえる（第 8 講で学ぶ）
- `zoom=` は画像の**幅（と高さ）**の倍率。`s=`（面積）とちがい、値が2倍なら幅も2倍になるので、大きさの差が `s=` より目立つ
- `3000` を `2000` や `5000` に変えて、大きさがどう変わるか確かめよう

AH-2 記号をマーカーにする

- 第 3 講のハートのように、`$` で囲んだ記号をマーカーにできる
- `'$\\star$'`（星）、`'$\\clubsuit$'`（クラブ）、`'$\\bigodot$'`（二重丸）など。文字そのもの `'$A$'` も使える

```
x = [1, 2, 3, 4, 5]
plt.scatter(x, [2, 3, 2, 4, 3], marker='$\\heartsuit$', s=400, color='hotpink', label='ハート')
plt.scatter(x, [5, 6, 5, 7, 6], marker='$\\star$', s=400, color='gold', label='星')
plt.scatter(x, [8, 9, 8, 10, 9], marker='$A$', s=300, color='teal', label='文字A')
plt.ylim(0, 15)
plt.legend(loc='upper center', ncol=3)
plt.show()
```



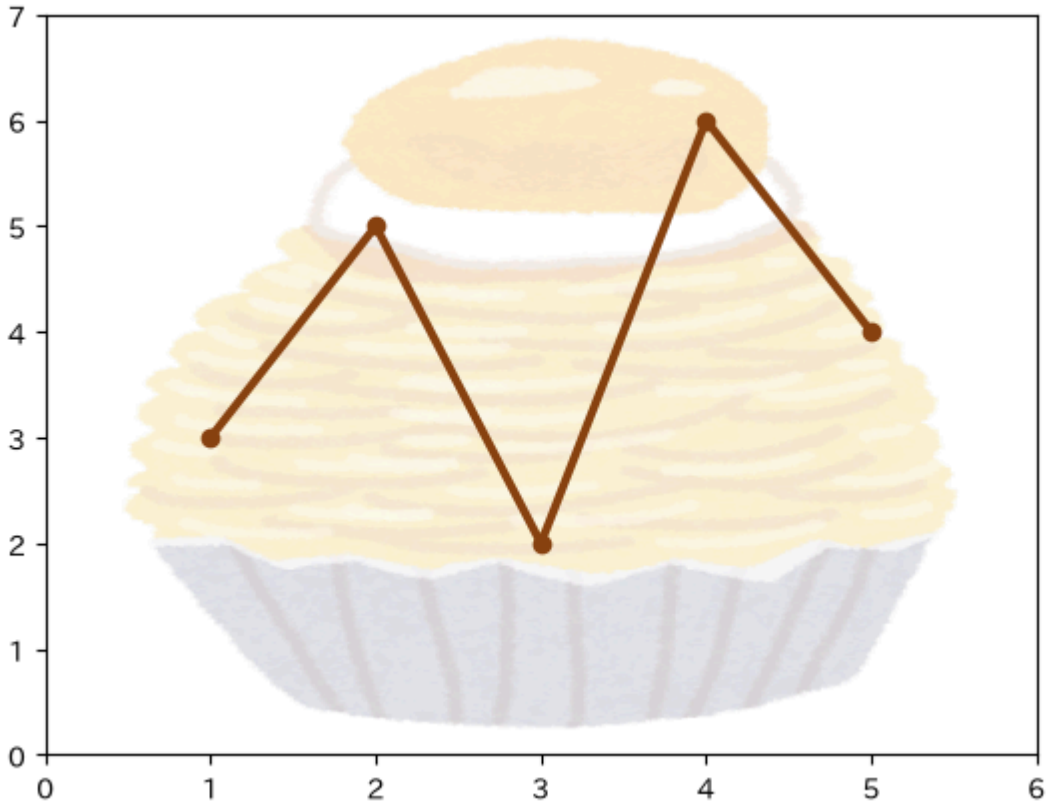
- `\\` は `\` を書くための書き方（Python の文字列の中では `\` を2つ重ねる）
- `s=` はマーカーの大きさ（散布図では `s`、折れ線では `markersize`）

AH-3 背景に画像を敷く

- `imshow` は image show（画像を見せる）の略。`plt.imread` で読み込んだ画像を、グラフの中に描く
- `plt.imshow(画像, extent=[x最小, x最大, y最小, y最大])` で、グラフの範囲いっぱいに画像を敷ける
- `alpha=` で薄くする。`aspect='auto'` を付けないと、画像の縦横の比に合わせてグラフがつぶれる

```
img = plt.imread('sweets_montblanc.png')
x = [1, 2, 3, 4, 5]
y = [3, 5, 2, 6, 4]
plt.imshow(img, extent=[0, 6, 0, 7], alpha=0.25, aspect='auto')  # 画像を x 0〜6、y 0〜7 の範囲に薄く敷く
plt.plot(x, y, marker='o', color='saddlebrown', linewidth=3)
```

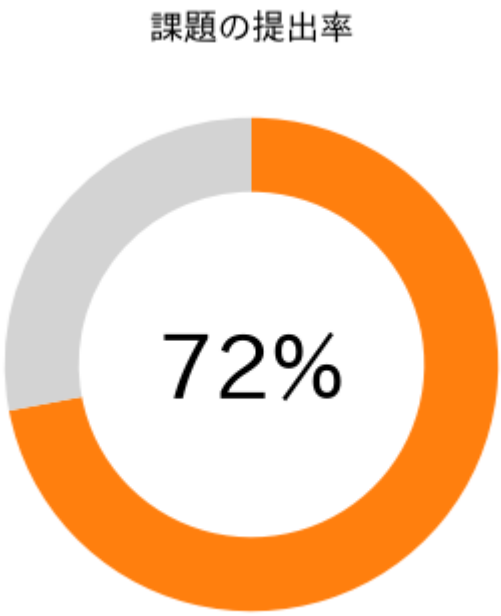
```
plt.xlim(0, 6)
plt.ylim(0, 7)
plt.show()
```



AH-4 ドーナツグラフ・ゲージ

- 第 6 講の円グラフ `plt.pie` に**1つ足すだけ**で、真ん中に穴のあいた**ドーナツグラフ**になる
- 穴に大きく数字を書くと、「何%か」がひと目で伝わる

```
plt.figure(figsize=(4, 4))
plt.pie([72, 28], colors=['tab:orange', 'lightgray'], startangle=90, counterclock=False,  # [1] 12時から時計回り
        wedgeprops={'width': 0.3})  # [2] 輪の太さ（円の半径を 1 としたとき）
plt.text(0, 0, '72%', ha='center', va='center', fontsize=32)  # [3] 真ん中の穴に数字
plt.title('課題の提出率')
plt.show()
```

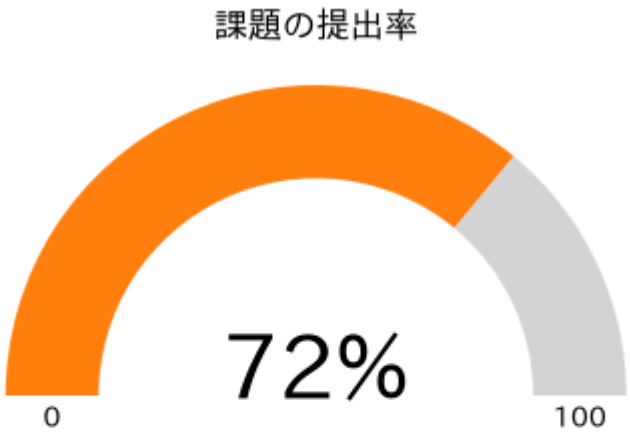


番号	何をしているか
[1]	`startangle=90` で12時の位置から、`counterclock=False` で時計回りに塗る
[2]	`wedgeprops={'width': 0.3}` で、外側から 0.3 の太さの輪だけを残す（`{ }` は辞書、第 7 講）
[3]	円グラフの真ん中は (0, 0)。`ha`・`va` は文字の横・縦をそろえる位置（`'center'` で真ん中）

半分にするとゲージになる

- 車のスピードメーターのような**半円のゲージ**も作れる
- しかけは、**合計（72 + 28 = 100）と同じ大きさの白い部分を足して下半分にし、`plt.ylim` で切り落とす**こと

```
plt.figure(figsize=(5, 3))
plt.pie([72, 28, 100], colors=['tab:orange', 'lightgray', 'white'], startangle=180,
counterclock=False,  # [1] 9時から時計回り
        wedgeprops={'width': 0.3})
plt.ylim(-0.2, 1.1)  # [2] 下半分（白い部分）を切り落とす
plt.text(0, 0.1, '72%', ha='center', va='center', fontsize=32)
plt.text(-0.85, -0.1, '0', ha='center')  # [3] 両端に目盛り
plt.text(0.85, -0.1, '100', ha='center')
plt.title('課題の提出率')
plt.show()
```



番号	何をしているか
[1]	`startangle=180` で9時（左端）から、時計回りに上を通って右へ塗る。3つ目の `100` が見えない下半分
[2]	円は縦が -1〜1。`plt.ylim(-0.2, 1.1)` で、上半分と目盛りの文字のところだけを見せる
[3]	輪の真ん中は半径 0.85（1 − 0.3 ÷ 2）なので、`x` を ±0.85 にすると両端の下に文字が来る

変えてみよう

- `72` と `28` を `40` と `60` に（文字の `'72%'` も直す）
- `'width': 0.3` → `0.1`（細い輪）
- `'tab:orange'` → `'tab:green'`

使ってみよう

- 好きな画像を同じフォルダに置き、AH-1 のひな形の `'sweets_montblanc.png'` を書き換えてみよう
  - 形式：**PNG**（`.png`）と **JPEG**（`.jpg` / `.jpeg`）が使える。iPhone の写真（`.heic`）は読めないので、PNG か JPEG に変換してから使う
  - マーカーにするなら、**背景が透明な PNG** がおすすめ（JPEG は透明にできないので、白い四角の背景ごと表示される）
  - 大きさ：モンブランの画像は 335×400 ピクセルで `zoom=0.08`。スマホの写真（4000 ピクセルくらい）は `zoom` を 0.005 くらいから試すか、先に画像を小さくしておく
  - ファイル名は、半角英数字（スペースなし）にしておくと間違いが少ない
- データは自分で考えた数でよい（例：1週間に飲んだコーヒーの杯数）

AI. 複数のグラフを並べる

AI-1 横に並べる・縦に並べる

- 比べたいグラフは、横に並べると見比べやすい
- `plt.subplot(縦の数, 横の数, 何番目)` のあとに書いたものが、その位置のグラフに描かれる
- 番号は左上から 1, 2, 3, … と数える。最後に `plt.tight_layout()` で重なりを自動で調整する

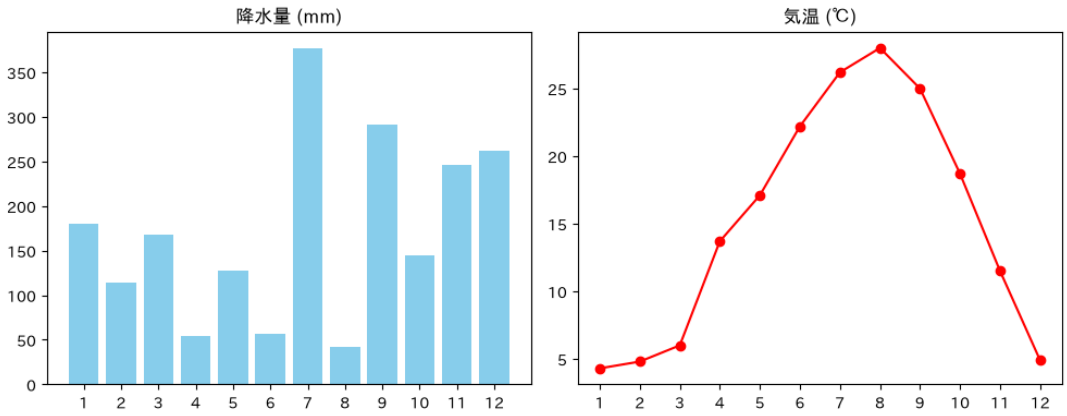
```
df_rain = pd.read_excel('dsa2026data.xlsx', sheet_name='降水量')

plt.figure(figsize=(10, 4))

plt.subplot(1, 2, 1)
plt.bar(df_rain['月'], df_rain['降水量'], color='skyblue')
plt.title('降水量 (mm)')
plt.xticks(range(1, 13))

plt.subplot(1, 2, 2)
plt.plot(df_rain['月'], df_rain['気温'], color='r', marker='o')
plt.title('気温 (℃)')
plt.xticks(range(1, 13))

plt.tight_layout()
plt.show()
```



- 第 8 講の複合グラフ（`plt.twinx()`）は1つのグラフに重ねる書き方、`plt.subplot` は別々のグラフを並べる書き方
- 縦に並べるときは `plt.subplot(2, 1, 1)`、`plt.subplot(2, 1, 2)`
- `range(1, 13)` は 1, 2, …, 12（終わりの 13 は含まない）。`plt.xticks` に渡して、1月〜12月の全部に目盛りを付けている
- 第11講では、誤解を招くグラフと正しいグラフをこの書き方で並べて比べる

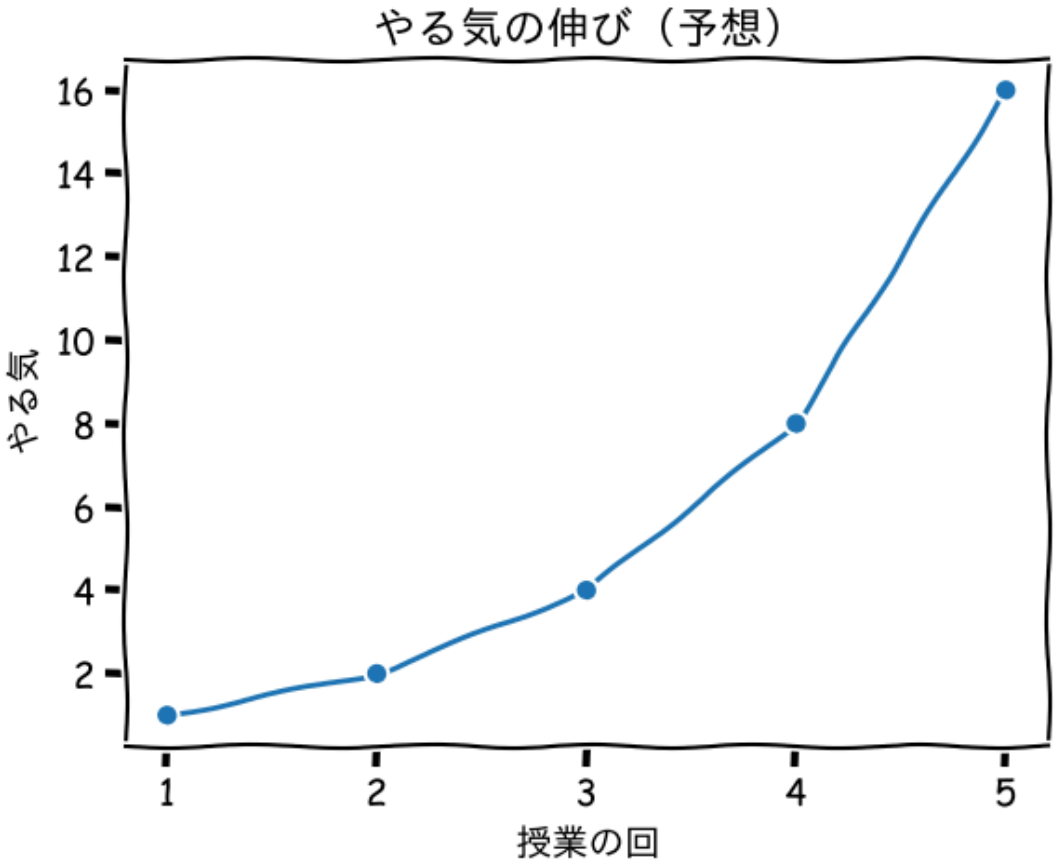
AJ. 発展

AJ-1 手書き風のグラフ

- `with plt.xkcd():` の下に字下げして書くと、手書き風になる

- **2行目のフォントの指定は必ず写す。**手書き風にすると文字が英語用のフォントに切り替わり、日本語が □ になるため
  - `'Comic Sans MS'`（英数字を手書き風に）→ 無い文字は `'IPAexGothic'`（japanize_matplotlib に入っている日本語フォント）で表示される
- 字下げした部分だけが手書き風になる。次のセルからは元に戻る

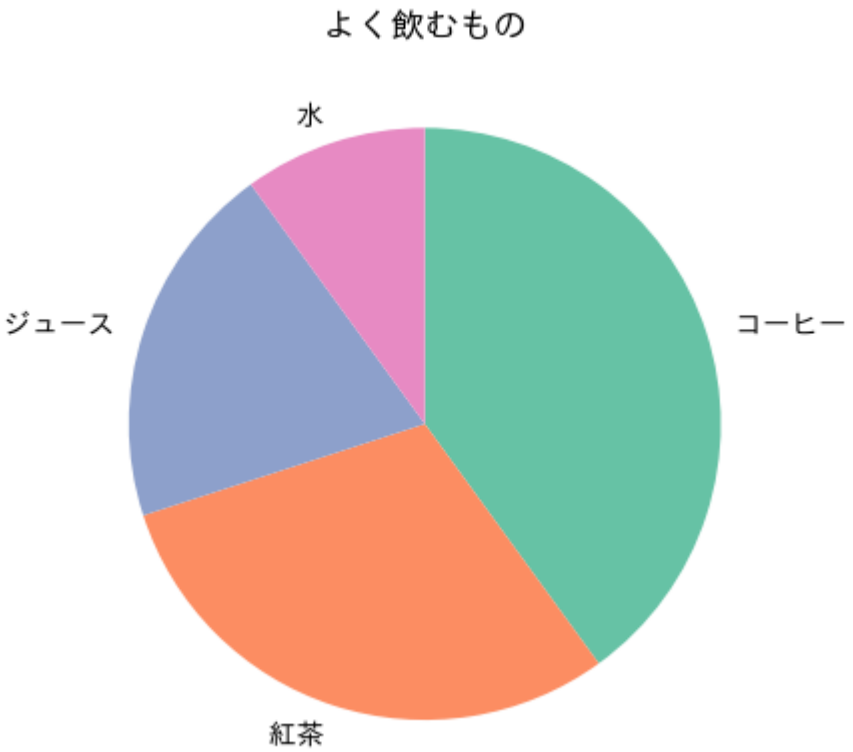
```
with plt.xkcd():
    plt.rcParams['font.family'] = ['Comic Sans MS', 'IPAexGothic']
    plt.plot([1, 2, 3, 4, 5], [1, 2, 4, 8, 16], marker='o')
    plt.title('やる気の伸び（予想）')
    plt.xlabel('授業の回')
    plt.ylabel('やる気')
    plt.show()
```



- Mac で `Comic Sans MS` が無いときは、英数字も `IPAexGothic` で表示される（エラーにはならない）
- `with ○○:` は、**字下げした部分の間だけ**設定を変え、終わったら元に戻す書き方。第 5 講 K-6 の `with plt.style.context('ggplot'):`（スタイルシート）も同じ仕組み
- 色の組み合わせ（カラーマップ）も、`with mpl.rc_context(...):` でそのグラフの間だけ変えられる（第 6 講 P-2 では `colors=` で指定した）

```
import matplotlib as mpl
from cycler import cycler

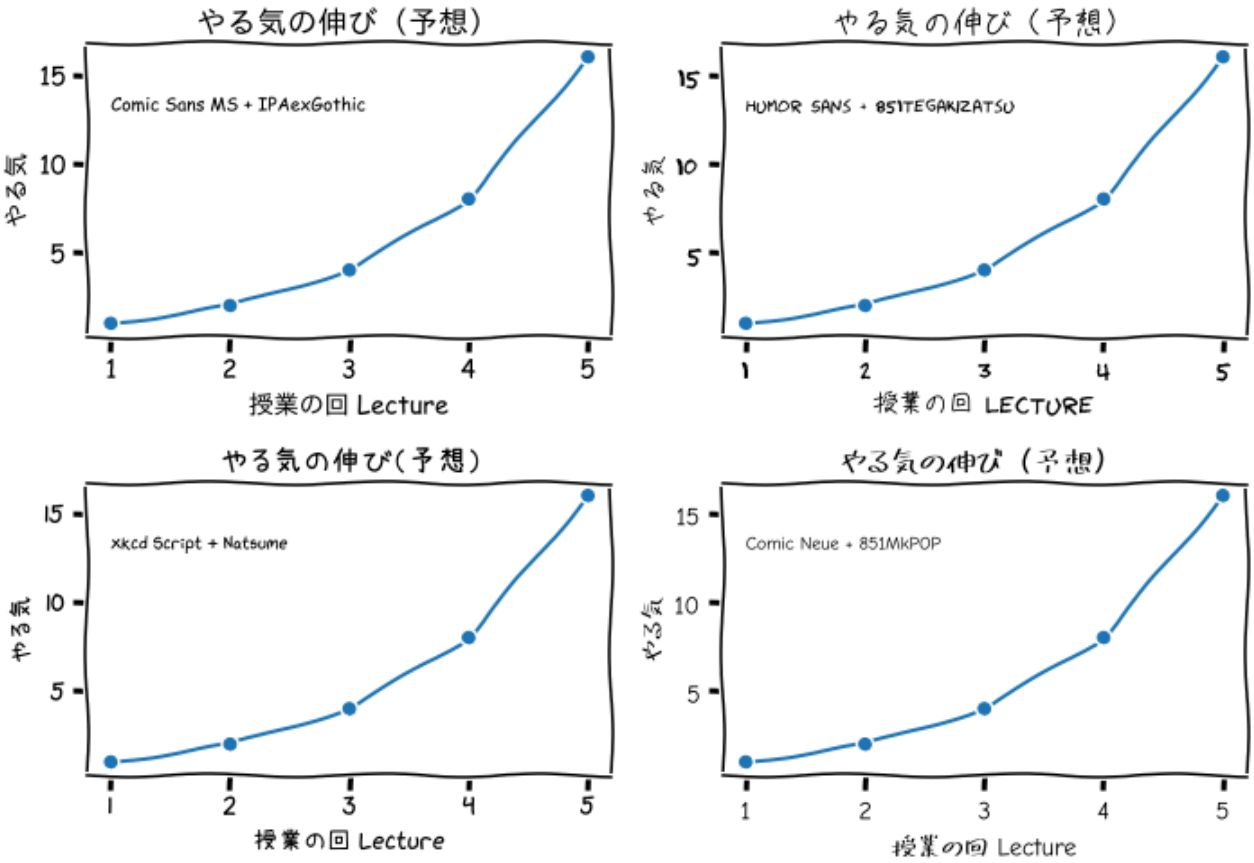
with mpl.rc_context({'axes.prop_cycle': cycler(color=plt.get_cmap('Set2').colors)}):
    plt.pie([40, 30, 20, 10], labels=['コーヒー', '紅茶', 'ジュース', '水'],
counterclock=False, startangle=90)
    plt.title('よく飲むもの')
    plt.show()
```



- `cycler(color=色のリスト)` で、グラフが順に使う色を決める。`plt.get_cmap('Set2').colors` は Set2 の色のリスト
- `colors=` を書くと、そちらが優先される（カラーマップの色にならない）。カラーマップを使うときは `colors=` を消す
- 使うには `import matplotlib as mpl` と `from cycler import cycler` が必要

AJ-2 手書き風フォントを入れると（紹介）

- 手書き風のフォントを使うと、文字ももっと手書きらしくできる。使い方は2通りある（インストールせずに使う／パソコンにインストールして使う）
- 下の図は、無料で配布されているフォントを入れたパソコンで描いたもの（左上が AJ-1 の組み合わせ）



種類	フォント	`font.family` に書く名前
英数字	Humor Sans（xkcd の漫画の文字）	`'Humor Sans'`
英数字	xkcd Script	`'xkcd Script'`
英数字	Comic Neue	`'Comic Neue'`
日本語	851手書き雑フォント	`'851tegakizatsu'`
日本語	なつめもじ	`'Natsume'`
日本語	851マカポップ	`'851MkPOP'`

インストールせずに使う（手軽な方法）

- フォントのファイルをダウンロードして、notebook の中で読み込む。インストールもキャッシュの削除も要らない
- 例は、英数字が Comic Neue または xkcd Script、日本語が 851マカポップの組み合わせ
- 下のセルでファイルをダウンロードする（合わせて約 4.7MB）。**1回実行すればよい**。ファイルは notebook と同じフォルダに保存される
  - うまくいかないときは、ComicNeue-Regular.ttf、xkcd-script.ttf、851MkPOP_101.ttf をブラウザでダウンロードして、notebook と同じフォルダに置く

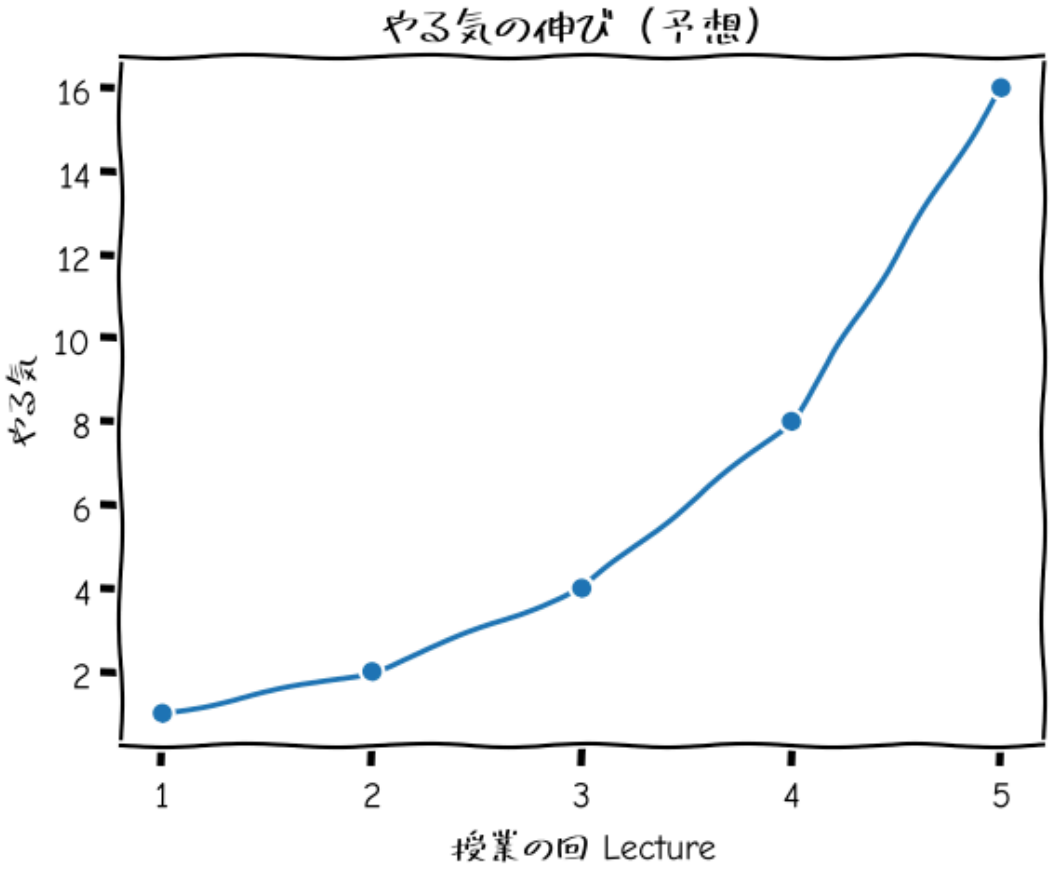
```
!curl -sLO https://github.com/google/fonts/raw/main/ofl/comicneue/ComicNeue-Regular.ttf
!curl -sLO https://github.com/ipython/xkcd-font/raw/master/xkcd-script/font/xkcd-script.ttf
!curl -sLO https://pm85122.onamae.jp/851MkPOP_101.ttf
```

- `addfont('ファイル名')` でフォントを読み込む。インストールしていないので、**Kernel → Restart のたびに、もう一度実行する**（ダウンロードはやり直さなくてよい）
- `FileNotFoundError` と出るときは、ファイルが notebook と同じフォルダにない

```
from matplotlib import font_manager

font_manager.fontManager.addfont('ComicNeue-Regular.ttf')
font_manager.fontManager.addfont('851MkPOP_101.ttf')

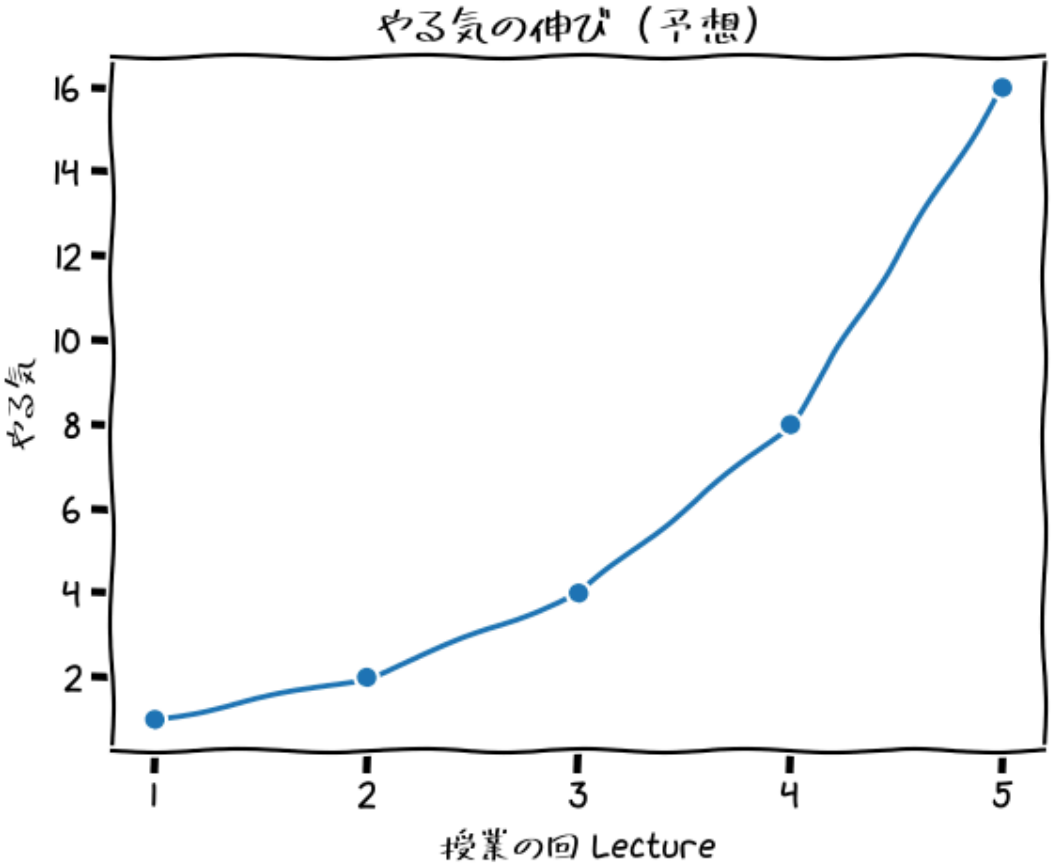
with plt.xkcd():
    plt.rcParams['font.family'] = ['Comic Neue', '851MkPOP']
    plt.plot([1, 2, 3, 4, 5], [1, 2, 4, 8, 16], marker='o')
    plt.title('やる気の伸び（予想）')
    plt.xlabel('授業の回 Lecture')
    plt.ylabel('やる気')
    plt.show()
```



- xkcd Script ＋ 851マカポップにすると、次のようになる。851マカポップは上のセルで読み込んであるので、xkcd Script だけ読み込む

```
font_manager.fontManager.addfont('xkcd-script.ttf')

with plt.xkcd():
    plt.rcParams['font.family'] = ['xkcd Script', '851MkPOP']
    plt.plot([1, 2, 3, 4, 5], [1, 2, 4, 8, 16], marker='o')
    plt.title('やる気の伸び（予想）')
    plt.xlabel('授業の回 Lecture')
    plt.ylabel('やる気')
    plt.show()
```



- 画像に保存するとき（`plt.savefig`）の注意
  - `dpi=` を大きくすると、線の揺れが小さくなり、notebook で見るより直線に近くなる。揺れの大きさがピクセル数で決まっているため（notebook の表示はおよそ 100 dpi）。揺れも `dpi ÷ 100` 倍にすると、notebook と同じくらいの揺れになる
  - `transparent=True`（背景を透明にする）で保存すると、線・枠・文字のまわりの白いふち取りが目立つ。`plt.rcParams['path.effects'] = []` で消せる
  - どちらも `with plt.xkcd(…):` の中に書く。360 dpi で保存する例：

```
with plt.xkcd(scale=3.6, length=360):  # 揺れを 3.6 倍（360 ÷ 100）にする
    plt.rcParams['font.family'] = ['xkcd Script', '851MkPOP']
    plt.rcParams['path.effects'] = []  # 白いふち取りを消す
    plt.plot([1, 2, 3, 4, 5], [1, 2, 4, 8, 16], marker='o')
    plt.title('やる気の伸び（予想）')
    plt.savefig('yaruki.png', dpi=360, transparent=True, bbox_inches='tight')
    plt.show()
```

パソコンにインストールして使う（例：851マカポップ）

1. 851MkPOP_101.ttf をダウンロードする（約 4.5MB）。ほかのフォントは、フォント名で検索すると配布ページが見つかる
2. ダウンロードしたファイルを開いて「インストール」（Windows は右クリック →「インストール」、Mac はダブルクリック →「Font Book」で「インストール」）
3. matplotlib が覚えているフォントの一覧（キャッシュ）を消す。下のセルで出てくるフォルダ（Windows は `C:\Users\自分の名前\.matplotlib`、Mac はホームフォルダの `.matplotlib`）を開き、`fontlist-` で始まる `.json` ファイル（例：`fontlist-v3.11.0.json`）を**全部**消す

    - `.` で始まるフォルダは隠しフォルダ。Windows はエクスプローラーの「表示」→「隠しファイル」にチェック、Mac は Finder で `Command` + `Shift` + `.`（ピリオド）
4. Kernel → Restart して、もう一度 `import` から実行する

```
import matplotlib
print(matplotlib.get_cachedir())
```

- 書く名前はファイル名と違うことがある。入れたフォントの名前は、下のセルで一覧を出して探す
    - `{f.name for f in …}` は内包表記という書き方（発展）。今は写して実行すればよい。`sorted` は名前を ABC 順に並べる

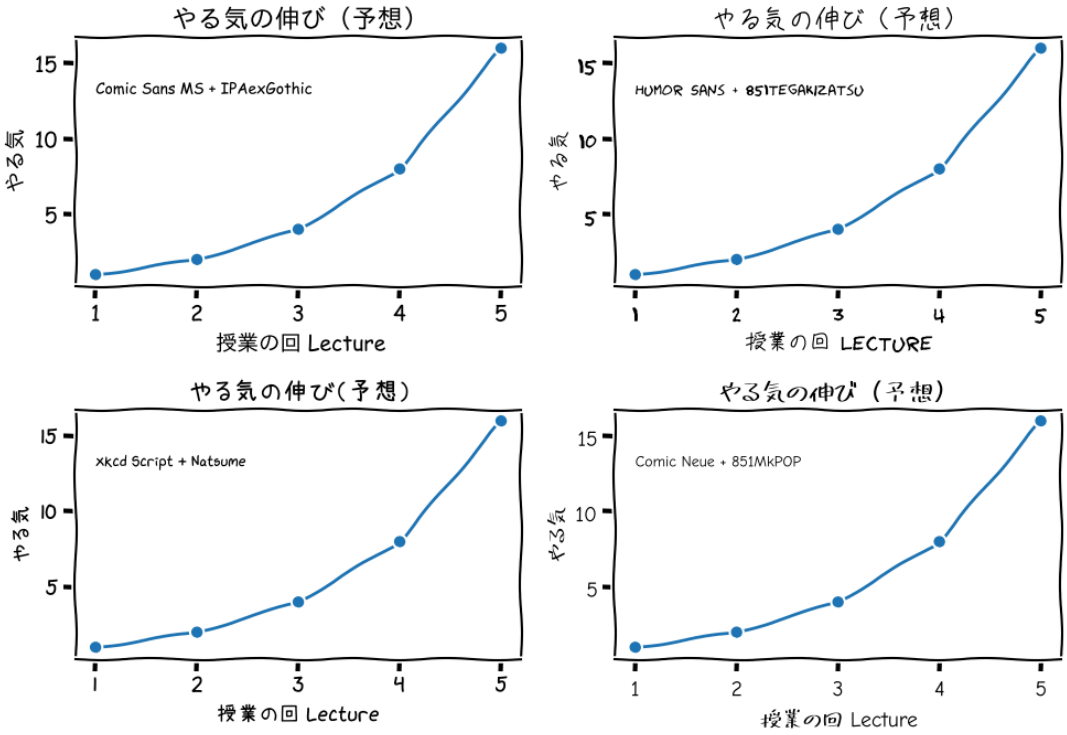
```
from matplotlib import font_manager
names = sorted({f.name for f in font_manager.fontManager.ttflist})
print(names)
```

['$\x82i\x82r\x83S\x83V\x83b\x83N', '$\x82i\x82r\x96¾\x92©', '%Century Oldstyle', '%CenturyOldst', '&CenturyOldst', '&Gothic720', '851MkPOP', '851tegakizatsu', '???????W3', 'Adobe Devanagari', 'Agency FB', 'Algerian', 'Arial', 'Arial Black', 'Arial Narrow', 'Arial Rounded MT Bold', 'Arphic Gansinkaisho4 Heavy JIS', 'Arphic Gothic4 Medium JIS', 'Arphic Gothic4 SuperBold JIS', 'Arphic Kaisho4 Medium JIS', 'Arphic Kaisho4JIS', 'Arphic Kanteiryu4 Heavy JIS', 'Arphic Kyokashotai4 Medium JIS', 'Arphic Kyokashotai4JIS', 'Arphic Marker-Tai4 Extra JIS', 'Arphic Match-Tai4 Bold JIS', 'Arphic Mincho4 Light JIS', 'Arphic Mincho4 Ultra JIS', 'Arphic PGansinkaisho4 Heavy JIS', 'Arphic PGothic4 Medium JIS', 'Arphic PGothic4 SuperBold JIS', 'Arphic PKaisho4 Medium JIS', 'Arphic PKaisho4JIS', 'Arphic PKanteiryu4 Heavy JIS', 'Arphic PKyokashotai4 Medium JIS', 'Arphic PKyokashotai4JIS', 'Arphic PMarker-Tai4 Extra JIS', 'Arphic PMatch-Tai4 Bold JIS', 'Arphic PMincho4 Light JIS', 'Arphic PMincho4 Ultra JIS', 'Arphic PPenkaisho4 Light JIS', 'Arphic PPenkaisho4JIS', 'Arphic PRoman-Mincho4 Ultra JIS', 'Arphic PRound-Gothic4 Extra JIS', 'Arphic PRound-Gothic4 Medium JIS', 'Arphic PYuyuGothictai4 Extra JIS', 'Arphic Penkaisho4 Light JIS', 'Arphic Penkaisho4JIS', 'Arphic Roman-Mincho4 Ultra JIS', 'Arphic Round-Gothic4 Extra JIS', 'Arphic Round-Gothic4 Medium JIS', 'Arphic YuyuGothictai4 Extra JIS', 'BIZ UDGothic', 'BIZ UDMincho', 'BIZ UDMincho Medium', 'BIZ UDPGothic', 'BIZ UDPMincho', 'BIZ UDPMincho Medium', 'Bahnschrift', 'Baskerville Old Face', 'Bauhaus 93', 'Bell MT', 'Berlin Sans FB', 'Berlin Sans FB Demi', 'Bernard MT Condensed', 'Blackadder ITC', 'Bodoni MT', 'Bodoni MT Black', 'Bodoni MT Condensed', 'Bodoni MT Poster Compressed', 'Book Antiqua', 'Bookman Old Style', 'Bookshelf Symbol 7', 'Bradley Hand ITC', 'Britannic Bold', 'Broadway', 'Brush Script MT', 'Calibri', 'Calibri Light', 'Californian FB', 'Calisto MT', 'Cambria', 'Cambria Math', 'Candara', 'Candara Light', 'Cascadia Code', 'Cascadia Mono', 'Castellar', 'Centaur', 'Century', 'Century Gothic', 'Century Schoolbook', 'CenturyOldst', 'Chiller', 'Colonna MT', 'Comic Neue', 'Comic Neue Light', 'Comic Sans MS', 'Consolas', 'Constantia', 'Cooper Black', 'Copperplate Gothic Bold', 'Copperplate Gothic Light', 'Corbel', 'Corbel Light', 'Courier New', 'Curlz MT', 'DFGothic-EB', 'DFGyoSho-Lt', 'DFHSGothic-W5', 'DFHSMincho-W3', 'DFHSMincho-W7', 'DHPGothic-EB', 'DHPGyoSho-Lt', 'DHPHSGothic-W5', 'DHPHSMincho-W3', 'DHPHSMincho-W7', 'DejaVu Sans', 'DejaVu Sans Display', 'DejaVu Sans Mono', 'DejaVu Serif', 'DejaVu Serif Display', 'Dubai', 'Dubai Light', 'Dubai Medium', 'EPSON ????', 'EPSON ?????', 'EPSON ???????', 'EPSON ????????', 'EPSON GOTHIC', 'EPSON GOTHIC W6', 'EPSON GOTHIC W7', 'EPSON PGOTHIC', 'EPSON PGOTHIC W6', 'EPSON PGOTHIC W7', 'EPSON \x8aÛ\x83S\x83V\x83b\x83N\x91Ì\x82l', 'EPSON \x8b³\x89È\x8f\x91\x91Ì\x82l', 'EPSON \x8ds\x8f\x91\x91Ì\x82l', 'EPSON \x90³\x9e²\x8f\x91\x91Ì\x82l', 'EPSON \x91¾\x8ap\x83S\x83V\x83b\x83N\x91Ì\x82a', 'EPSON \x91¾\x8aÛ\x83S\x83V\x83b\x83N\x91Ì\x82a', 'EPSON \x91¾\x8ds\x8f\x91\x91Ì\x82a', 'EPSON \x91¾\x96¾\x92©\x91Ì\x82a', 'EPSON 丸ゴシック体M', 'EPSON 太丸ゴシック体B', 'EPSON 太明朝体B', 'EPSON 太行書体B', 'EPSON 太角ゴシック体B', 'EPSON 教科書体M', 'EPSON 正楷書体M', 'EPSON 行書体M', 'Ebrima', 'Edwardian Script ITC', 'Elephant', 'Embassy JS', 'Engravers MT', 'Eras Bold ITC', 'Eras Demi ITC', 'Eras Light ITC', 'Eras Medium ITC', 'Euclid', 'Euclid Extra', 'Euclid Fraktur', 'Euclid Math One', 'Euclid Math Two', 'Euclid Symbol', 'Felix Titling', 'Fences', 'Footlight MT Light', 'Forte', 'Fraktur JS', 'Franklin Gothic Book', 'Franklin Gothic Demi', 'Franklin Gothic Demi Cond', 'Franklin Gothic Heavy', 'Franklin Gothic Medium', 'Franklin Gothic Medium Cond', 'Freestyle Script', 'French Script MT', 'Gabriola', 'Gadugi', 'Garamond', 'Georgia', 'Gigi', 'Gill Sans MT', 'Gill Sans MT Condensed', 'Gill Sans MT Ext Condensed Bold', 'Gill Sans Ultra Bold', 'Gill Sans Ultra Bold Condensed', 'Gloucester MT Extra Condensed', 'Gothic720', 'Goudy Old Style', 'Goudy Stout', 'HGGothicE', 'HGGothicM', 'HGGyoshotai', 'HGKyokashotai', 'HGMaruGothicMPRO', 'HGMinchoB', 'HGMinchoE', 'HGPGothicE', 'HGPGothicM', 'HGPGyoshotai', 'HGPKyokashotai', 'HGPMinchoB', 'HGPMinchoE', 'HGPSoeiKakugothicUB', 'HGPSoeiKakupoptai', 'HGPSoeiPresenceEB', 'HGSGothicE', 'HGSGothicM', 'HGSGyoshotai', 'HGSKyokashotai', 'HGSMinchoB', 'HGSMinchoE', 'HGSSoeiKakugothicUB', 'HGSSoeiKakupoptai', 'HGSSoeiPresenceEB', 'HGSeikaishotaiPRO', 'HGSoeiKakugothicUB', 'HGSoeiKakupoptai', 'HGSoeiPresenceEB', 'Haettenschweiler', 'Harlow Solid Italic', 'Harrington', 'High Tower Text', 'Humor Sans', 'IPAexGothic', 'IPAexMincho', 'Impact', 'Imprint MT Shadow', 'Informal Roman', 'Ink Free', 'JSHSMinchoU-W3', 'JSSGothic-Md', 'JSSMincho-Lt', 'Javanese Text', 'Jokerman', 'Juice ITC', 'JustEditMark', 'JustHalfMark', 'JustHalfMarkG', 'JustKanaMark', 'JustKanaMarkG', 'JustOubunMark', 'JustOubunMarkG', 'JustUnitMark', 'JustUnitMarkG', 'JustWabunMark', 'JustWabunMarkG', 'Kristen ITC', 'Kunstler Script', 'Last Resort High-Efficiency', 'Leelawadee', 'Leelawadee UI', 'Leelawadee UI Semilight', 'Lohit Telugu', 'Lucida Bright', 'Lucida Calligraphy', 'Lucida Console', 'Lucida Fax', 'Lucida Handwriting', 'Lucida Sans', 'Lucida Sans Typewriter', 'Lucida Sans Unicode', 'MS Gothic', 'MS Mincho', 'MS Outlook', 'MS PGothic', 'MS PMincho', 'MS Reference Sans Serif', 'MS Reference Specialty', 'MS UI Gothic', 'MT Extra', 'MT Extra Tiger', 'MV Boli', 'Magneto', 'Maiandra GD', 'Malgun Gothic', 'Malg

un Gothic Semilight', 'Matura MT Script Capitals', 'Meiryo', 'Meiryo UI', 'Microsoft Himalaya', 'Microsoft JhengHei', 'Microsoft JhengHei Light', 'Microsoft JhengHei UI', 'Microsoft JhengHei UI Light', 'Microsoft New Tai Lue', 'Microsoft PhagsPa', 'Microsoft Sans Serif', 'Microsoft Tai Le', 'Microsoft Uighur', 'Microsoft YaHei', 'Microsoft YaHei Light', 'Microsoft YaHei UI', 'Microsoft YaHei UI Light', 'Microsoft Yi Baiti', 'MingLiU-ExtB', 'MingLiU_HKSCS-ExtB', 'MingLiU_MSCS-ExtB', 'Mistral', 'Modern No. 20', 'Mongolian Baiti', 'Monotype Corsiva', 'Myanmar Text', 'NSimSun', 'Natsume', 'Niagara Engraved', 'Niagara Solid', 'Nirmala Text', 'Nirmala Text Semilight', 'Nirmala UI', 'Nirmala UI Semilight', 'Noto Sans JP', 'Noto Serif JP', 'OCR A Extended', 'OCRB', 'Old English Text MT', 'Onyx', 'PMingLiU-ExtB', 'Palace Script MT', 'Palatino Linotype', 'Papyrus', 'Parchment', 'Perpetua', 'Perpetua Titling MT', 'Playbill', 'Poor Richard', 'Pristina', 'Rage Italic', 'Ravie', 'Rockwell', 'Rockwell Condensed', 'Rockwell Extra Bold', 'STIXGeneral', 'STIXNonUnicode', 'STIXSizeFiveSym', 'STIXSizeFourSym', 'STIXSizeOneSym', 'STIXSizeThreeSym', 'STIXSizeTwoSym', 'Sans Serif Collection', 'Script MT Bold', 'Segoe Fluent Icons', 'Segoe MDL2 Assets', 'Segoe Print', 'Segoe Script', 'Segoe UI', 'Segoe UI Black', 'Segoe UI Emoji', 'Segoe UI Historic', 'Segoe UI Light', 'Segoe UI Semibold', 'Segoe UI Semilight', 'Segoe UI Symbol', 'Segoe UI Variable', 'Showcard Gothic', 'SimSun', 'SimSun-ExtB', 'SimSun-ExtG', 'Sitka', 'Sitka Text', 'Snap ITC', 'Stencil', 'Sylfaen', 'Symbol', 'Symbol Tiger', 'Symbol Tiger Expert', 'Tahoma', 'Tempus Sans ITC', 'Tera Special', 'Tiger', 'Tiger Expert', 'Times New Roman', 'Trebuchet MS', 'Tw Cen MT', 'Tw Cen MT Condensed', 'Tw Cen MT Condensed Extra Bold', 'UD Digi Kyokasho N', 'UD Digi Kyokasho NK', 'UD Digi Kyokasho NP', 'Ubuntu Mono', 'Verdana', 'Viner Hand ITC', 'Vivaldi', 'Vladimir Script', 'Webdings', 'Wide Latin', 'Wingdings', 'Wingdings 2', 'Wingdings 3', 'Yu Gothic', 'Yu Gothic Light', 'Yu Gothic Medium', 'Yu Gothic UI', 'Yu Gothic UI Light', 'Yu Gothic UI Semibold', 'Yu Gothic UI Semilight', 'Yu Mincho', 'Yu Mincho Demibold', 'Yu Mincho Light', 'ZWAdobeF', 'cmb10', 'cmex10', 'cmmi10', 'cmr10', 'cmss10', 'cmsy10', 'cmti10', 'cmtt10', 'xkcd', 'xkcd Script', 'ＪＳ平成明朝体W3']

```
fonts = [
    ['Comic Sans MS', 'IPAexGothic'],
    ['Humor Sans', '851tegakizatsu', 'IPAexGothic'],
    ['xkcd Script', 'Natsume', 'IPAexGothic'],
    ['Comic Neue', '851MkPOP', 'IPAexGothic'],
]

with plt.xkcd():
    plt.figure(figsize=(10, 7))
    for i in range(len(fonts)):
        plt.rcParams['font.family'] = fonts[i]
        plt.subplot(2, 2, i + 1)
        plt.plot([1, 2, 3, 4, 5], [1, 2, 4, 8, 16], marker='o')
        plt.title('やる気の伸び（予想）')
        plt.xlabel('授業の回 Lecture')
        plt.ylabel('やる気')
        plt.text(1, 13, fonts[i][0] + ' + ' + fonts[i][1], fontsize=10)
    plt.tight_layout()
    plt.show()
```



- `for i in range(len(fonts)):` で、4組のフォントを順に使って4枚のグラフを描いている（for 文は AH-1 と同じくり返し。第12講）
- 最後の `'IPAexGothic'` は、フォントが入っていないときの予備。入っていない人が実行しても日本語は □ にならない
- 入っていないフォントがあると、`findfont: Font family 'Humor Sans' not found.` のような赤い警告が何行も出る。グラフが出ていれば問題ない
- フォントを入れたのに変わらないとき、`findfont: Font family '851MkPOP' not found.` と出るときは、キャッシュを消していないか、消した後に Kernel → Restart していない（上の「パソコンにインストールして使う」の 3・4）

AJ-3 動くグラフ（アニメーション）

- 線が左から少しずつ伸びていく、**動くグラフ**を作る
- 「1枚目の絵、2枚目の絵、…」と**少しずつ違う絵を何枚も描いて、パラパラ漫画のように続けて見せる**仕組み
- 関数を作る `def`（第13講）を使う。下のコードを写して、数や色を変えて遊べばよい

```
import matplotlib.animation as animation
from IPython.display import HTML

x = np.linspace(0, 2 * np.pi, 40)  # 0 から 2π までを 40 個に区切った数
y1 = np.sin(x)
y2 = np.cos(x)

fig = plt.figure(figsize=(6, 4), dpi=72)  # [1] グラフを用意する（dpi は絵の細かさ）
plt.plot(x[0], y1[0], marker='o', label='sin')  # [2] 最初の1点だけ描いておく
plt.plot(x[0], y2[0], marker='o', label='cos')
lines = plt.gca().get_lines()  # [3] 描いた2本の線を取り出す（lines[0] が sin、lines[1] が cos）
plt.xlim(x[0], x[-1])
plt.ylim(-1.1, 1.1)
plt.title('動くグラフ')
plt.legend(loc='upper right')
```

```
def update(i):  # [4] i 枚目の絵：最初から i 番目までの点で線を描き直す
    lines[0].set_data(x[:i + 1], y1[:i + 1])
    lines[1].set_data(x[:i + 1], y2[:i + 1])

ani = animation.FuncAnimation(fig, update, frames=len(x), interval=50)  # [5] 40 枚を 50 ミリ秒ごとに
plt.close()  # 止まった絵が別に出ないようにする
HTML(ani.to_jshtml())  # [6] notebook の中で再生する
```

（アニメーションは、この講義ノート集では動かない。notebook で実行すると、再生ボタン付きで出る）

- 下の ▶ ボタンで再生する。速さの調整や、1枚ずつ送ることもできる

番号	何をしているか
[1]	`fig = plt.figure()` で、絵を描き直す先のグラフに名前（`fig`）を付けておく。`dpi=72` は絵の細かさで、小さくすると notebook のファイルが軽くなる（何十枚も絵を作るため）
[2]	最初は1点だけ描く。ここで線の色や `marker=` を決める
[3]	`plt.gca().get_lines()` で、描いた線を**リストとして**取り出す。描いた順に `[0]`、`[1]`
[4]	`update(i)` は「i 枚目の絵」を作る関数。`x[:i + 1]` は「先頭から i 番目まで」（第 7 講の `[:-1]` と同じ、リストの一部を取り出す書き方）。`set_data` で線の点を入れ替える
[5]	`FuncAnimation` が、`i` を 0, 1, 2, … と変えながら `update` を `frames=` 回呼ぶ。`interval=` は1枚の表示時間（ミリ秒）
[6]	Jupyter では `plt.show()` だけだと**止まった絵**になる。`HTML(ani.to_jshtml())` で再生ボタン付きにする

変えてみよう

- `interval=50` → `200`（ゆっくり）
- `marker='o'` → `marker='$\\star$'`（AH-2 の記号のマーカー）
- `np.linspace(0, 2 * np.pi, 40)` の `40` → `100`（なめらかになるが、作るのに時間がかかり、ファイルも重くなる）

線が増えたときは for 文で

- 線が3本、4本と増えると、`update` の中に同じ形の行が並ぶ。第12講の **for 文**を使うと、1つにまとめられる

```
Y = [y1, y2]

def update(i):
    for j in range(len(lines)):
        lines[j].set_data(x[:i + 1], Y[j][:i + 1])
```

GIF ファイルに保存する

- `HTML(...)` の行の代わりに次の1行を書くと、いつもの場所に `sin_cos.gif` ができる。スライドや SNS にそのまま貼れる

```
ani.save('sin_cos.gif', writer='pillow')
```

もっと簡単な書き方：絵を先に全部描いておく（ArtistAnimation）

- 上と同じアニメーションを、`def` も `set_data` も使わずに作れる。代わりに for 文（第12講）を使う
- 「パラパラ漫画の1枚ずつを `plt.plot` で描いて、リストにためておき、最後にまとめて動かす」仕組み

```
fig = plt.figure(figsize=(6, 4), dpi=72)  # [1] グラフを用意する
plt.xlim(x[0], x[-1])
plt.ylim(-1.1, 1.1)
plt.title('動くグラフ（ArtistAnimation）')
```

```
frames = []  # [2] 絵をためておく空のリスト
for i in range(len(x)):  # [3] i = 0, 1, 2, … と変えながら、下の2行をくり返す
    line = plt.plot(x[:i + 1], y1[:i + 1], 'C0', x[:i + 1], y2[:i + 1], 'C1')  # i 番目の点までの sin・cos を描く
    frames.append(line)  # [4] 描いた線を1枚の絵としてリストに足す

ani = animation.ArtistAnimation(fig, frames, interval=50)  # [5] ためた絵を 50 ミリ秒ごとに
plt.close()
HTML(ani.to_jshtml())
```

（アニメーションは、この講義ノート集では動かない。notebook で実行すると、再生ボタン付きで出る）

番号	何をしているか
[1]	上と同じ。題名・軸の範囲は動かないので、for の前に1回だけ書く
[2]	`[]` は空のリスト。ここに絵を1枚ずつ足していく
[3]	for 文： `range(len(x))` は 0, 1, 2, ..., 39。字下げした行を、`i` を変えながら 40 回くり返す
[4]	`plt.plot` は描いた線をリストで返す。それを `frames.append(line)` で `frames` の末尾に足す（`append` は第12講）
[5]	`ArtistAnimation` は、ためた絵を順に見せる。`FuncAnimation` と違い、関数は要らない

- **2つの書き方の違い**： `FuncAnimation` は「1枚ずつ、その場で描き直す」、`ArtistAnimation` は「先に全部描いておく」。コードは `ArtistAnimation` のほうが短いが、絵の枚数が多いと時間がかかる
- 凡例は動く線に付けにくいので、この書き方では省いている（色は `'C0'` ＝青、`'C1'` ＝オレンジ）

AJ-4 別の道具で動かす：plotly（紹介）

- **plotly**（プロットリー）は、matplotlib とは別のグラフの道具。マウスを乗せると値が出る、拡大できるなど、**さわれるグラフ**が作れる
- 表（データフレーム）に「年」のような列があれば、`animation_frame='列名'` を足すだけで動くグラフになる。**for も def も要らない**
- 例は、世界の国の豊かさと寿命の移り変わり（1952〜2007年）。plotly に付いてくるデータ `px.data.gapminder()` を使う
- ※ plotly は、自分のパソコンの Jupyter で試す

```
import plotly.express as px

df = px.data.gapminder()  # [1] 142 か国 × 12 年分のデータ
px.scatter(df, x='gdpPercap', y='lifeExp', size='pop', color='continent',  # [2] 横：1人あたり GDP、縦：平均寿命、点の大きさ：人口、色：大陸
           animation_frame='year',  # [3] この列の値（年）ごとに1コマ
           animation_group='country',  # [4] 同じ国の点を、コマをまたいでつなぐ
           hover_name='country',  # [5] マウスを乗せると国名が出る
           log_x=True, size_max=55,  # [6] 横軸を対数目盛に（第８講）、点の最大の大きさ
           range_x=[100, 100000], range_y=[25, 90])  # [7] 軸の範囲を固定する
```

- グラフは**自分で実行すると**出る（この講義ノート集では図が出ないことがある）
- 下の ▶ ボタンで再生する。スライダーで年を選ぶこともできる。点にマウスを乗せると国名と値が出る

番号	何をしているか
[1]	`df.head()` で中身を見ると、`country`（国）・`continent`（大陸）・`year`（年）・`lifeExp`（平均寿命）・`pop`（人口）・`gdpPercap`（1人あたり GDP）の列がある
[2]	`px.scatter(表, x='列名', y='列名', …)` で散布図。列の名前で指定する。`size=`・`color=` にも列名を書ける
[3]	`animation_frame=` に書いた列の値ごとに、1コマずつ絵を作る
[4]	`animation_group=` があると、同じ国の点がなめらかに動く
[5]	`hover_name=` は、マウスを乗せたときに太字で出す列
[6]	`log_x=True` は `plt.xscale('log')` と同じ。`size_max=` は一番大きい点の大きさ
[7]	書かないと、コマごとに軸の範囲が変わってしまう

変えてみよう

- `px.scatter` を `px.bar`、`x='continent'`、`y='pop'` に（大陸ごとの人口の棒グラフが動く。`range_x`・`log_x` は消し、`range_y=[0, 4e9]` に）
- `df = df[df['continent'] == 'Asia']` を `px.scatter` の前に足す（アジアだけ。`df[条件]` は条件に合う行だけを取り出す書き方。第15講）
- **AJ-3 との違い**：plotly は「年」などの列がすでにある表を動かすのが得意。AJ-3 のような「線が伸びていく」動きは matplotlib のほうが書きやすい
- plotly のグラフは、開く環境によっては描くためのプログラムごと notebook に保存され、ファイルが重くなる（数 MB）

動かさなくても：マウスを乗せると値が出る

- plotly のグラフは、アニメーションにしなくても**さわれる**。点にマウスを乗せると名前と値が出る、ドラッグで拡大できる、凡例を押すとその系列を消せる
- 例は、`dsa2026data.xlsx` の `ramen` シート。都市ごとのラーメンの外食費（1世帯あたり、円、第 5 講）を、2024年と2025年で比べる散布図

```
df_ramen = pd.read_excel('dsa2026data.xlsx', sheet_name='ramen')
px.scatter(df_ramen, x='Y2024', y='Y2025', hover_name='city')  # hover_name：マウスを乗せると出る名前の列
```

- 新潟市の点を探してみよう。matplotlib の散布図では、どの点がどの都市かは分からない
- 右上のボタンで、拡大・元に戻す・画像で保存ができる

▶ 解答例（クリックで開く）

AJ-5 スライダーで動かす（ipywidgets）

- **スライダー**を動かすと、その場でグラフが描き直される。数を1つずつ書き換えて実行し直さなくてよい
- 例は、第 9 講のヒストグラム。**棒の数（`bins`）を変えると、分布の見え方がどう変わるか**を自分の手で確かめる

- AJ-3 と同じく、関数を作る `def`（第13講）を使う。写して、数を変えて遊べればよい
- ※ スライダーは**自分で実行したときだけ**出る（この講義ノート集では動かない）

```
from ipywidgets import interact

np.random.seed(0)  # [1] 毎回同じ乱数になるようにする
weight = np.random.normal(55, 6, 200)  # [2] 平均 55、標準偏差 6 の体重のデータを 200 人分つくる

def draw(bins=10):  # [3] bins を受け取って、ヒストグラムを描く関数
    plt.figure(figsize=(6, 4))
    plt.hist(weight, bins=bins, edgecolor='white')
    plt.xlabel('体重 [kg]')
    plt.ylabel('人数')
    plt.title(f'棒の数：{bins}')
    plt.show()

interact(draw, bins=(2, 50));  # [4] bins を 2〜50 のスライダーで動かす
```

番号	何をしているか
[1]	乱数は実行するたびに変わる。`seed(0)` で、毎回同じ数が出るようにする
[2]	`np.random.normal(平均, 標準偏差, 個数)` で、山が1つで左右対称なデータ（第 9 講）を作る
[3]	`def 関数名(引数=最初の値):` の下に字下げして、グラフを描く手順を書く。`f'…{bins}'` は文字の中に変数の値を入れる書き方（f文字列、第15講）
[4]	`interact(関数名, 引数=(最小, 最大))` で、その引数を動かすスライダーができる。動かすたびに `draw` が呼ばれる。最後の `;` は、よけいな表示（ `<function draw …>` ）を出さないため

試してみよう

- `bins` を 2・10・50 にしてみる。少なすぎると山の形が分からず、多すぎるとガタガタになる。**ちょうどよい棒の数は、データの数や目的で変わる**
- スライダーを2本にする： `def draw(bins=10, n=200):` とし、`weight` を作る2行を関数の中に移して `200` を `n` に、最後を `interact(draw, bins=(2, 50), n=(10, 1000));` に（データの数を変えると、ヒストグラムがどれだけなめらかになるか）

まとめ

やりたいこと	書き方
グラフを並べる	`plt.subplot(縦の数, 横の数, 何番目)` 、最後に `plt.tight_layout()`
画像を読む	`img = plt.imread('画像ファイル名')`
画像をマーカーに	AH-1 のひな形を写し、`x`・`y`・`zoom` を変える
記号をマーカーに	`marker='$\\star$'` 、`marker='$A$'`

やりたいこと	書き方
背景に画像	`plt.imshow(img, extent=[x最小, x最大, y最小, y最大], alpha=0.25, aspect='auto')`
ドーナツグラフ	`plt.pie(値のリスト, wedgeprops={'width': 0.3})` 、真ん中に `plt.text(0, 0, '72%')`
ゲージ	合計と同じ大きさの白い部分を足し、`plt.ylim(-0.2, 1.1)` で下半分を切る

第11講：伝わるグラフ・総復習＋α

誤解を招くグラフの例から、**伝わるグラフ**のチェックリストを作る（AT）。次に、ここまで習ったグラフを、使う場面・書き方・共通の指定・書く順番の表で**総復習**する（AU）。最後に、ここまでの道具の組み合わせで描ける**新しい形のグラフ**を見る（AV）。

準備：この講で使うライブラリを読み込み、データを用意する。

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import japanize_matplotlib
```

AT. 伝わるグラフ・誤解を招くグラフ

ここまでのグラフを、「見る人に正しく伝わるか」という目で見直す。よくある失敗を、正しい描き方と並べて見る（並べ方は第10講 AI-1 の `plt.subplot`）。

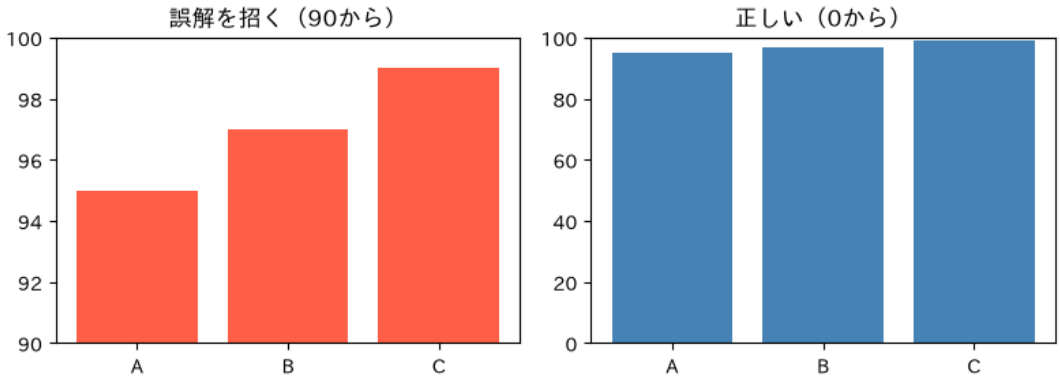
AT-1 縦軸を途中から始めた棒グラフ

- 左は 90 から始めているので、A と B の差が何倍にも見える。**棒グラフの縦軸は 0 から**

```
names = ['A', 'B', 'C']
values = [95, 97, 99]

plt.figure(figsize=(8, 3))
plt.subplot(1, 2, 1)
plt.bar(names, values, color='tomato')
plt.ylim(90, 100)
plt.title('誤解を招く（90から）')

plt.subplot(1, 2, 2)
plt.bar(names, values, color='steelblue')
plt.ylim(0, 100)
plt.title('正しい（0から）')
plt.tight_layout()
plt.show()
```



- 棒グラフは「棒の長さ」で大きさを比べる。途中から始めると、長さの比が実際の値の比と合わなくなる
- 折れ線グラフは「変化」を見るので、途中から始めてもよい（ただし軸の数字ははっきり書く）

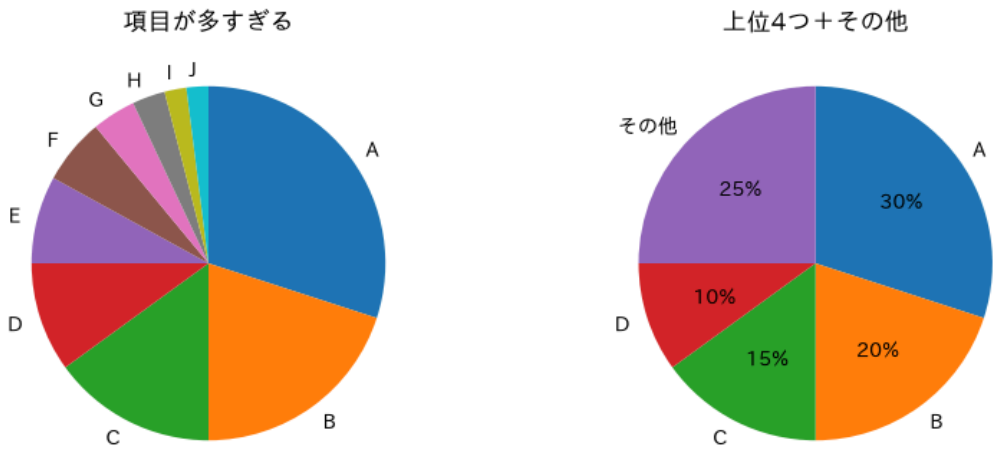
AT-2 項目が多すぎる円グラフ

- 項目が多いと、小さい扇形が読めなくなる。**5つくらいまで**にして、残りは「その他」にまとめる

```
names = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I', 'J']
values = [30, 20, 15, 10, 8, 6, 4, 3, 2, 2]

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.pie(values, labels=names, startangle=90, counterclock=False)
plt.title('項目が多すぎる')

plt.subplot(1, 2, 2)
plt.pie(values[:4] + [sum(values[4:])], labels=names[:4] + ['その他'],
        autopct='%1.0f%%', startangle=90, counterclock=False)
plt.title('上位4つ＋その他')
plt.show()
```



- `values[:4]` は前から4つ、`values[4:]` は5つ目から最後まで（リストの切り出し）。`sum()` で合計する

AT-3 グラフのチェックリスト

よくある失敗	どうする
棒グラフの縦軸が 0 から始まっていない	`plt.ylim(0, …)`
立体（3D）の円グラフ	手前が大きく見える。平らな円グラフか棒グラフにする
項目が多すぎる円グラフ	5つくらいまで。残りは「その他」にまとめる
軸の名前・単位がない	`plt.xlabel`・`plt.ylabel` に単位まで書く（例：`'降水量 (mm)'`）
色が多すぎる・赤と緑で区別している	色は意味のあるところだけ。赤と緑は見分けにくい人がいるので、青とオレンジなどにする
凡例がなく、どの線が何か分からない	`label=` ＋ `plt.legend()`

よくある失敗	どうする
文字が小さくて読めない	`fontsize=14` など。スライドに貼るなら大きめに
データの出どころが書いていない	図の下や説明文に「出典：e-Stat ○○統計」などと書く
画像やイラストでデータが読みにくい	遊び心（第10講 AH）は**データが読める範囲で**

> 自分のグラフで、この表を1つずつ確かめる

AU. 総復習

AU-1 総復習のデータ

- 第11講は、総復習用の Excel `dsa2026review.xlsx` を使う
- ある大学のカフェの 2025 年のデータ（架空）。シートは4つある

シート	中身
`cafe`	月ごとの平均気温・来客数・前年来客数・店内・テイクアウト・ホット・アイス・本店売上・駅前店売上
`cafe_menu`	メニューごとの杯数
`cafe_week`	20週分の、曜日（月〜金）ごとの1日の来客数
`cafe_stock`	カフェを営む会社の、2025年12月の20日分の株価（始値・高値・安値・終値）

- ファイルは GitHub に置いてある。URL は次のとおり

```
https://raw.githubusercontent.com/CreativeStudies-NGT/lectures/main/dsa/dsa2026review.xlsx
```

自分の PC の Anaconda（Jupyter Notebook）で使うとき

ファイルを notebook と同じ場所（いつもの場所）に置く。方法は2つある。

[1] ブラウザでダウンロードする

- 上の URL をブラウザで開くと、`dsa2026review.xlsx` がダウンロードされる
- ダウンロードフォルダ（Windows はエクスプローラー、Mac は Finder で開く）から、いつもの場所に移す

[2] notebook のセルでダウンロードする

- 新しいセルに次の1行を書いて実行すると、いつもの場所にダウンロードされる（Windows・Mac とも同じ）

```
!curl -O https://raw.githubusercontent.com/CreativeStudies-NGT/lectures/main/dsa/dsa2026review.xlsx
```

- 先頭の `!` は「Python ではなく、PC への命令として実行する」という印。`-O` は大文字のオー
- `% Total  % Received …` と `100` の行が出れば、ダウンロードできている
- 1回ダウンロードすれば、次からは実行しなくてよい

置いたら、ファイル名で読む。

```
df_cafe = pd.read_excel('dsa2026review.xlsx', sheet_name='cafe')
```

- `FileNotFoundError` が出たら、置き場所を確かめる
- ファイルを置けないときは、ファイル名の代わりに URL を書いても読める（ネットにつながっている必要がある）

```
df_cafe = pd.read_excel('https://raw.githubusercontent.com/CreativeStudies-NGT/lectures/main/dsa/dsa2026review.xlsx', sheet_name='cafe')
```

確認

- 下のセルで読めれば準備できている

```
df_cafe = pd.read_excel('dsa2026review.xlsx', sheet_name='cafe')
df_cafe.head()
```

	月	平均気温	来客数	前年来客数	店内	テイクアウト	ホット	アイス	本店売上	駅前店売上
0	1	3.5	2900	2700	2160	740	2170	150	1508000	410000
1	2	4.2	1600	1500	1180	420	1170	110	824000	380000
2	3	7.5	1500	1450	1040	460	980	220	795000	450000
3	4	12.8	3200	2950	2020	1180	1680	880	1632000	520000
4	5	17.6	3400	3100	1950	1450	1570	1150	1785000	560000

AU-2 総復習のグラフの一覧

- ここまで習ったグラフを、まとまりごとに並べた。わからないときは、右の「習った回」に戻って、そこのコードを写して書き換える

まとまり	グラフ	習った回
基本の1枚	折れ線2本	第 3 講
	散布図	第 4 講
	棒（値を書く）	第 6 講・第 7 講 R-3
	円（切り出す）	第 6 講
内訳	積み上げ棒	第 7 講
分布・ばらつき（`cafe_week`）	箱ひげ図	第 7 講
	ヒストグラムを重ねる	第 9 講
	エラーバー	第 9 講
2つの量を重ねる・単位	複合グラフ（棒＋折れ線）	第 8 講
	両 y 軸	第 8 講
	単位を変える	第 8 講

まとまり	グラフ	習った回
	指数化（1月 ＝ 100）	第 8 講

- 描けたら、AT-3 のチェックリストで自分のグラフを見直す

AU-3 グラフの型の一覧

困ったら、この表の形を写してデータを入れ替える。

グラフ	使う場面	書き方	習った回
折れ線	時間の流れ・順番に意味がある	`plt.plot(x, y)`	第 3 講
散布図	2つの量の関係	`plt.scatter(x, y)`	第 4 講
棒	項目ごとの大きさ比べ	`plt.bar(項目, 値)`	第 6 講
ヒストグラム	1つの量の分布	`plt.hist(データ, bins=10)`	第 9 講
箱ひげ図	グループごとのばらつき比べ	`plt.boxplot([データ1, データ2])`	第 7 講
円	全体に対する割合	`plt.pie(値, labels=名前)`	第 6 講
エラーバー	平均とばらつき	`plt.errorbar(x, 平均, yerr=標準偏差)`	第 9 講

- 2つの量を1枚に重ねる**複合グラフ**は、`plt.twinx()` の前に書いたものが左の y 軸、後に書いたものが右の y 軸（第 8 講 Z）
- ＋α（AV）で初めて出てくる形のグラフは、AV-9 の表にまとめてある

AU-4 共通の指定

やりたいこと	書き方	習った回
Excel を読む	`df = pd.read_excel('ファイル名', sheet_name='シート名')`	第 5 講
列を取り出す	`df['列名']`	第 5 講
大きさ	`plt.figure(figsize=(8, 5))` を最初に	第 3 講
文字	`plt.title` 、`plt.xlabel` 、`plt.ylabel`	第 3 講
凡例	`label=` ＋ `plt.legend()`	第 3 講
範囲	`plt.xlim` 、`plt.ylim`	第 3 講
補助線	`plt.axhline(y=値)` 、`plt.axvline(x=値)`	第 4 講
注釈	`plt.text(x, y, '文字')`	第 8 講
点・棒ごとの色	散布図は `c=色のリスト` 、棒グラフは `color=色のリスト`	第 4 講・第 6 講
右にもう1本 y 軸	`plt.twinx()` （前に書いたものは左、後に書いたものは右の軸）	第 8 講
保存	`plt.savefig('名前.png', dpi=300)` を `plt.show()` の前に	第 5 講

AU-5 書く順番

1つのグラフに命令をたくさん書くときは、次の順に書く。迷ったら、この順に並べ替える。

順	すること	書き方の例
[0]	大きさ（**最初に**）	`plt.figure(figsize=(8, 5))`
[1]	描く（凡例に出す名前もここで）	`plt.bar(x, y, label='来客数')` 、`plt.plot(…)`
[2]	飾る	`plt.title` 、`plt.xlabel` 、`plt.ylabel` 、`plt.ylim` 、`plt.xticks` 、`plt.legend()`
[3]	右にもう1本 y 軸（必要なときだけ）	`plt.twinx()` → 右の軸のグラフを描く → 右の軸の `ylabel` ・`ylim` ・`legend`
[4]	保存（**飾りの後**）	`plt.savefig('問1.png', dpi=300)`
[5]	表示（**最後に**）	`plt.show()`

```
plt.figure(figsize=(8, 5))          # [0] 大きさ
plt.plot(x, y, label='来客数')       # [1] 描く
plt.title('月ごとの来客数')           # [2] 飾る
plt.ylim(0, 3000)
plt.legend()
plt.savefig('問1.png', dpi=300)      # [4] 保存
plt.show()                          # [5] 表示
```

- `plt.savefig` より**後**に書いた `plt.legend()` や `plt.title` は、保存した画像に入らない（画面の表示には出るので気づきにくい）
- `plt.show()` の後に `plt.savefig` を書くと、真っ白な画像が保存される
- 保存するファイル名は**問題ごとに変える**。同じ名前で保存すると、前の画像が上書きされて消える
- 両 y 軸のときは、タイトル・x 軸・左の軸の飾りを `plt.twinx()` の**前**に、右の軸のものを**後**に書く（第 8 講 Z-2）

AV. ＋α：新しい形のグラフ

AU では、ここまで習った7種類のグラフを整理した。＋α では、**初めて出てくる形のグラフ**を描く。新しい命令は少しだけで、多くは**ここまでの道具の組み合わせ**でできている。

節	グラフ	新しく使うもの
AV-1	値の順に並べた横棒	なし（横棒は第 6 講 N-7、並べ替えは第 6 講 P-1）
AV-2	100% 積み上げ棒	なし（割合に直してから積み上げる）
AV-3	増加率の棒	なし（列どうしの計算）
AV-4	2本の線の間を塗る	`plt.fill_between`
AV-5	ヒートマップ	`plt.imshow` に数の表を渡す

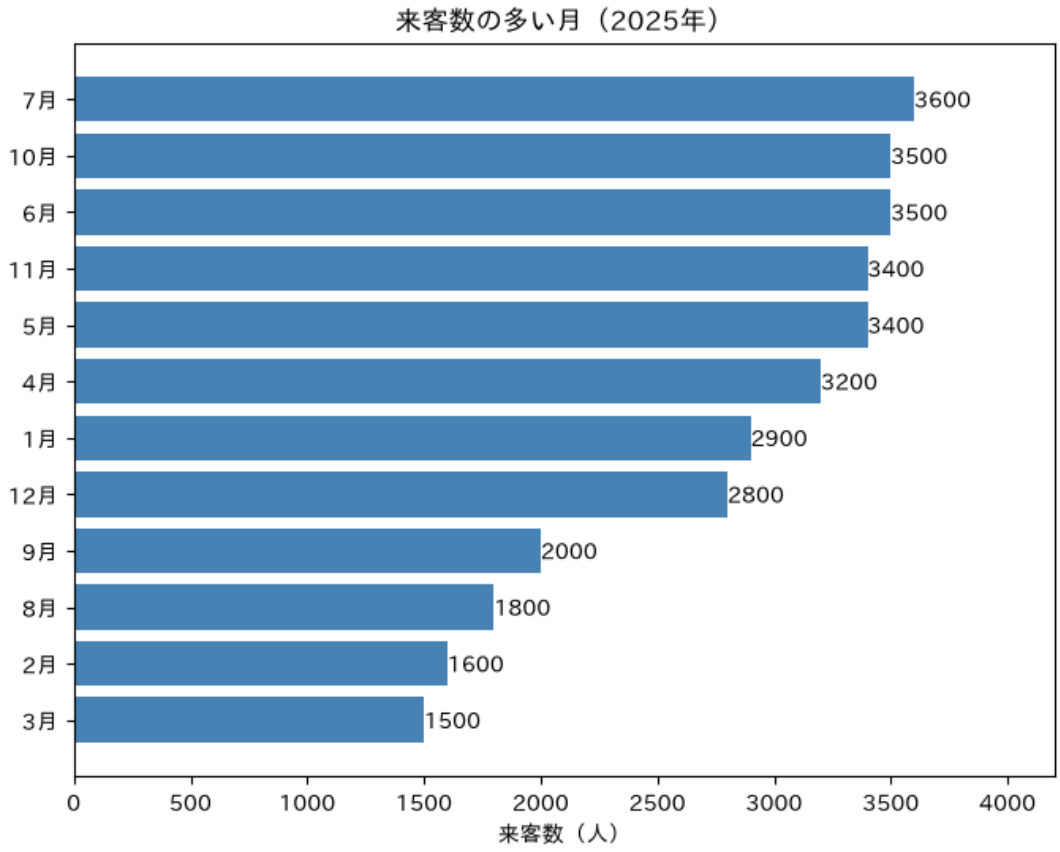
節	グラフ	新しく使うもの
AV-6	並べて1枚にまとめる	なし（第10講 AI-1 の `plt.subplot`）
第 7 講 R-4	ローソク足	`plt.vlines`、`np.where`
AV-8	ベクトル図（矢印）	`plt.quiver`、`np.meshgrid`

AV-1 値の順に並べる（横棒グラフ）

- 「どれが一番多いか」を見せたいときは、**値の順に並べ替えて**から描く
- `df.sort_values('列名')` で、その列の小さい順に行が並ぶ（大きい順は `ascending=False`。第 6 講 P-1）
- `plt.barh(項目, 値)` は横棒グラフ（第 6 講 N-7）。**最初の行が一番下**に描かれるので、小さい順に並べると一番多いものが一番上に来る
- 項目の名前が長いときや、項目が多いときも、横棒の方が読みやすい

```
s = df_cafe.sort_values('来客数')

plt.figure(figsize=(8, 6))
b = plt.barh(s['月'].astype(str) + '月', s['来客数'], color='steelblue')
plt.bar_label(b)
plt.xlim(0, 4200)
plt.xlabel('来客数（人）')
plt.title('来客数の多い月（2025年）')
plt.show()
```



- `s['月'].astype(str) + '月'` は、数字の月を文字に直して「月」を付けたもの。**数字のままだと、**`barh` が数の位置（1〜12 の順）に棒を置くので、並べ替えが効かない

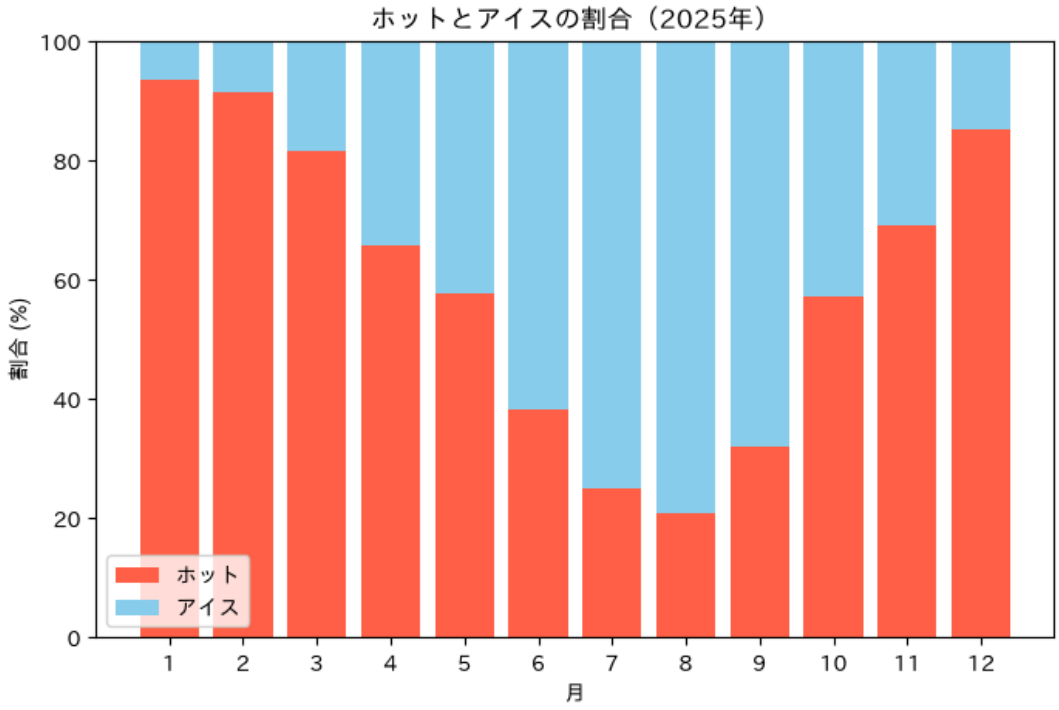
- 7月が一番多く、3月・2月・8月・9月（休みの月）が少ない

AV-2 割合に直して比べる（100% 積み上げ棒）

- 月によって合計の杯数が違うので、杯数のまま積み上げると「割合」が比べにくい
- 合計で割って 100 を掛けると、どの月も合計が 100（%）になる（割り算で比を見るのは第 8 講 Z-3）
- 下の段は `bottom=` なし、上の段は `bottom=下の段`（第 7 講 R-4）。描き方は積み上げ棒と同じで、**積む前に割合に直す**ところだけが新しい

```
total = df_cafe['ホット'] + df_cafe['アイス']
hot_pct = df_cafe['ホット'] / total * 100
ice_pct = df_cafe['アイス'] / total * 100

plt.figure(figsize=(8, 5))
plt.bar(df_cafe['月'], hot_pct, color='tomato', label='ホット')
plt.bar(df_cafe['月'], ice_pct, bottom=hot_pct, color='skyblue', label='アイス')
plt.ylim(0, 100)
plt.ylabel('割合 (%)')
plt.xlabel('月')
plt.xticks(range(1, 13))
plt.title('ホットとアイスの割合（2025年）')
plt.legend(loc='lower left')
plt.show()
```



- 割合にすると、**来客数が少ない月（2月・8月）も、多い月と同じ高さで比べられる。**そのかわり、杯数そのものは分からなくなる

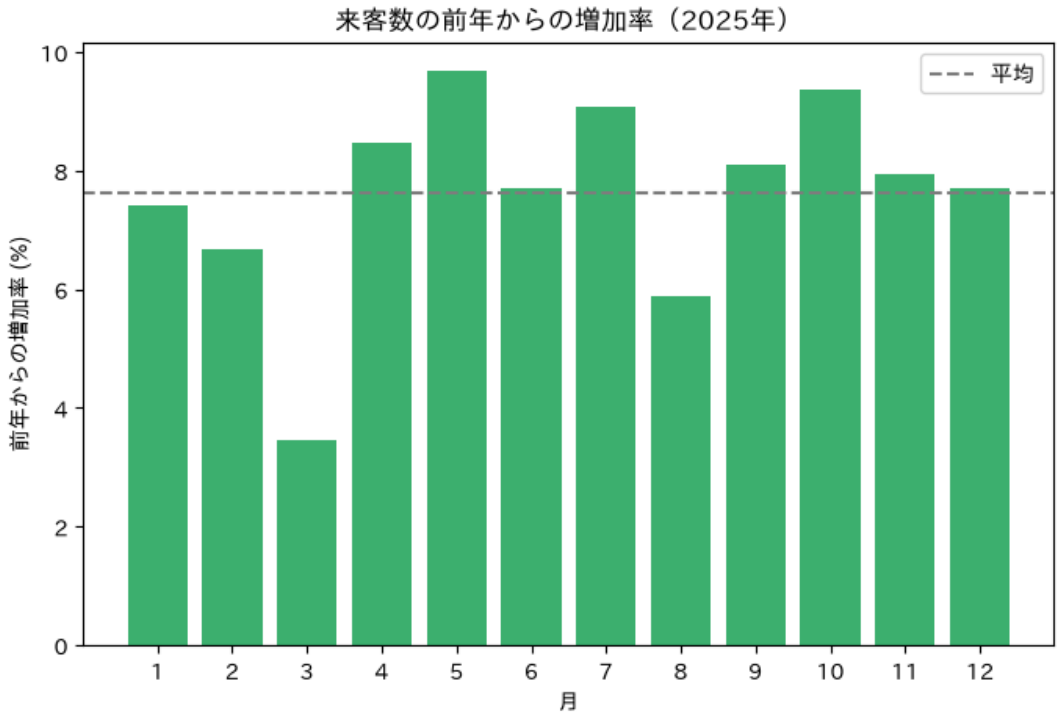
AV-3 2つの列から新しい量を作る

- `df['列A'] - df['列B']`、`df['列A'] / df['列B']` のように列どうしで計算すると、月ごとに計算される（第 8 講 Z-3 の指数化と同じ考え）
- 例：前年からの増加率 (%) ＝ （今年 − 前年） ÷ 前年 × 100

- 全体の平均は `.mean()`（第 4 講 G-1 の `np.mean()` と同じ。この書き方は第 9 講 AC-5）。平均の位置に補助線を引くと、平均より上か下かがすぐ分かる

```
rate = (df_cafe['来客数'] - df_cafe['前年来客数']) / df_cafe['前年来客数'] * 100

plt.figure(figsize=(8, 5))
plt.bar(df_cafe['月'], rate, color='mediumseagreen')
plt.axhline(y=rate.mean(), color='gray', linestyle='--', label='平均')
plt.ylabel('前年からの増加率（%）')
plt.xlabel('月')
plt.xticks(range(1, 13))
plt.title('来客数の前年からの増加率（2025年）')
plt.legend()
plt.show()
```

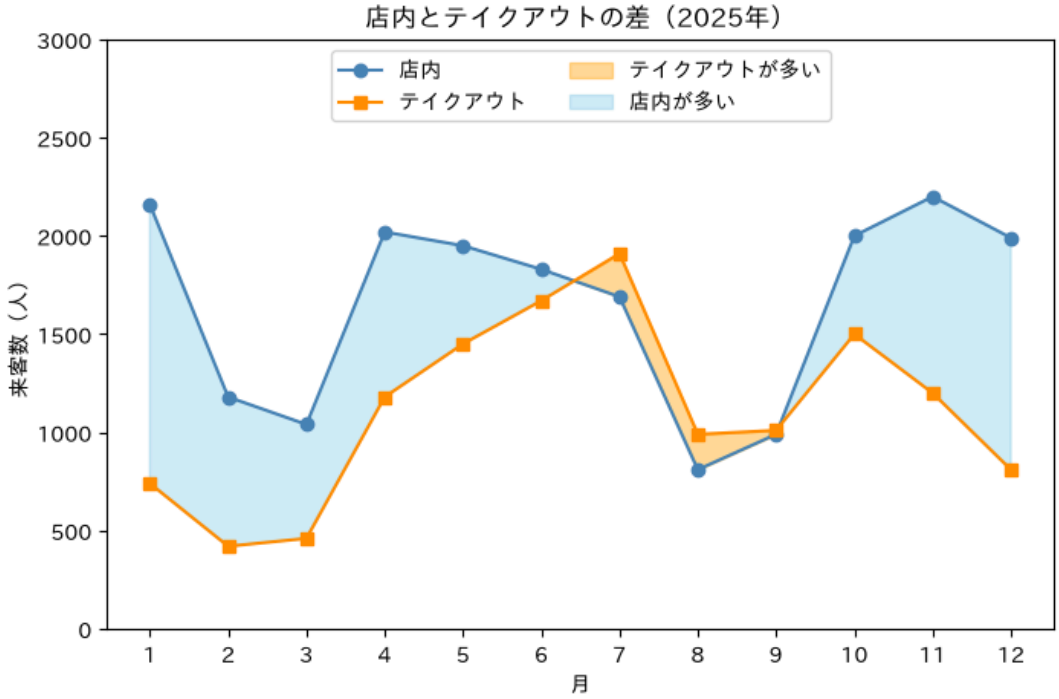


- すべての月で前年より増えた。平均は約 7.6%。一番伸びたのは5月（約 9.7%）、一番伸びなかったのは3月（約 3.4%）
- 人数の差（`来客数 - 前年来客数`）だと、来客数の多い月ほど大きく出やすい。**率にすると、規模の違う月どうしを比べられる**

AV-4 2本の線の間を塗る

- `plt.fill_between(x, 線1, 線2)` で、2本の線の間を塗る
- `where=条件` を付けると、条件に合うところだけを塗る。条件を2つに分けて色を変えると、**どちらが上か**が色で分かる
- `interpolate=True` は、2本の線が交わるところまできちんと塗るための指定（付けないと、交わる前後にすき間ができる）
- `alpha=0.4` で半透明にして、線が見えるようにする
- 第 4 講 I-2 の `plt.axhspan` は横に一定の帯を塗った。`fill_between` は、x ごとに上下が変わる2本の線の間を塗れる

```
plt.figure(figsize=(8, 5))
plt.plot(df_cafe['月'], df_cafe['店内'], color='steelblue', marker='o', label='店内')
plt.plot(df_cafe['月'], df_cafe['テイクアウト'], color='darkorange', marker='s', label='テイクアウト')
plt.fill_between(df_cafe['月'], df_cafe['店内'], df_cafe['テイクアウト'],
                 where=df_cafe['テイクアウト'] > df_cafe['店内'],
                 color='orange', alpha=0.4, interpolate=True, label='テイクアウトが多い')
plt.fill_between(df_cafe['月'], df_cafe['店内'], df_cafe['テイクアウト'],
                 where=df_cafe['テイクアウト'] <= df_cafe['店内'],
                 color='skyblue', alpha=0.4, interpolate=True, label='店内が多い')
plt.xlabel('月')
plt.xticks(range(1, 13))
plt.ylabel('来客数（人）')
plt.ylim(0, 3000)
plt.title('店内とテイクアウトの差（2025年）')
plt.legend(loc='upper center', ncol=2)
plt.show()
```



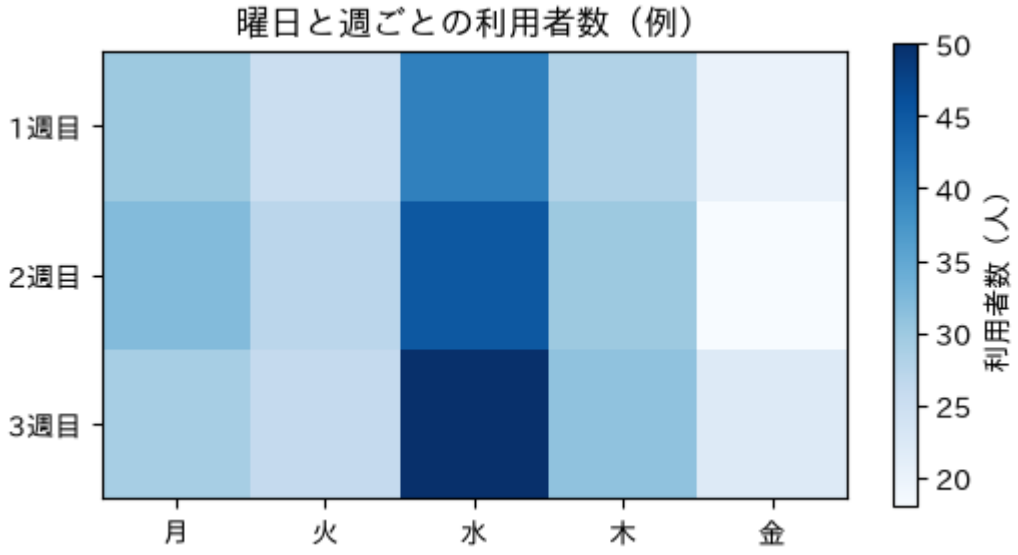
- 7〜9月だけオレンジ（テイクアウトの方が多い）。塗った幅が、2つの差の大きさ

AV-5 色の濃さで表す（ヒートマップ）

- 縦と横に並んだ数の表を、**色の濃さ**で表したものをヒートマップという。「どこが多いか」「どこだけ違うか」をまとめて見られる
- `plt.imshow(表, cmap='色の名前')` で描く。`'Blues'`・`'Oranges'`・`'Greens'` などは、値が大きいほど濃くなる
- `plt.colorbar(label='名前')` で、色と値の対応（カラーバー）を横に付ける（第 5 講の散布図のカラーマップと同じ）
- `plt.imshow` は第10講 AH-3 で背景の画像を敷くのに使った。画像の代わりに**数の表**を渡すと、ヒートマップになる
- マスの位置は 0 から数える。目盛りの名前は `plt.xticks(range(5), 名前のリスト)` のように付け替える

```
data = [[30, 25, 40, 28, 20],
        [32, 27, 45, 30, 18],
        [29, 26, 50, 31, 22]]

plt.figure(figsize=(6, 3))
plt.imshow(data, cmap='Blues')
plt.colorbar(label='利用者数（人）')
plt.xticks(range(5), ['月', '火', '水', '木', '金'])
plt.yticks(range(3), ['1週目', '2週目', '3週目'])
plt.title('曜日と週ごとの利用者数（例）')
plt.show()
```



- 行（横の並び）が週、列（縦の並び）が曜日。水曜の列が濃く、金曜の列が薄い
- データフレームもそのまま `plt.imshow(df)` で描ける。行が多くてマスが細長くなるときは `aspect='auto'` を付ける
- マスに数字を書き込むには `plt.text` を1マスずつ書く。まとめて書くには for 文（第12講）を使う

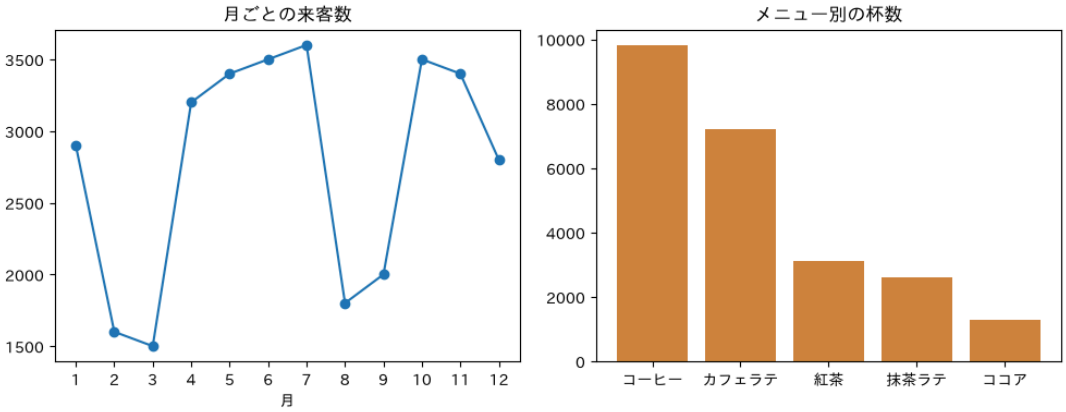
AV-6 グラフを並べて1枚にまとめる

- `plt.subplot(縦の数, 横の数, 何番目)` のあとに書いたものが、その位置に描かれる。最後に `plt.tight_layout()`（第10講 AI-1）
- **違うシートのグラフも並べられる。**報告のまとめの1枚に使える

```
df_menu = pd.read_excel('dsa2026review.xlsx', sheet_name='cafe_menu')

plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
plt.plot(df_cafe['月'], df_cafe['来客数'], marker='o')
plt.xticks(range(1, 13))
plt.xlabel('月')
plt.title('月ごとの来客数')

plt.subplot(1, 2, 2)
plt.bar(df_menu['メニュー'], df_menu['杯数'], color='peru')
plt.title('メニュー別の杯数')
plt.tight_layout()
plt.show()
```



- 1つ1つのグラフのタイトルは短くする（並べると横幅が狭くなる）
- 並べるグラフは、見る人に伝えたい順に置く（左上 → 右上 → 左下 → 右下）

AV-7 株価の動きを描く（ローソク足）

- ローソク足は積み上げ棒の応用なので、**第 7 講 R-4** の「応用：株価の動きを描く（ローソク足）」にまとめた

AV-8 矢印で向きを表す（風・流れ）

- 風向・風速や海流のように、**向きと強さ**を持つ量は、矢印で描く（このような量を**ベクトル**という）
- `plt.quiver(x, y, u, v)` で、点 `(x, y)` から、横に `u`・縦に `v` だけ進む向きの矢印を描く。矢印の長さは強さを表す
- 三角関数（sin・cos）を使う。例題を動かして形を見ればよい

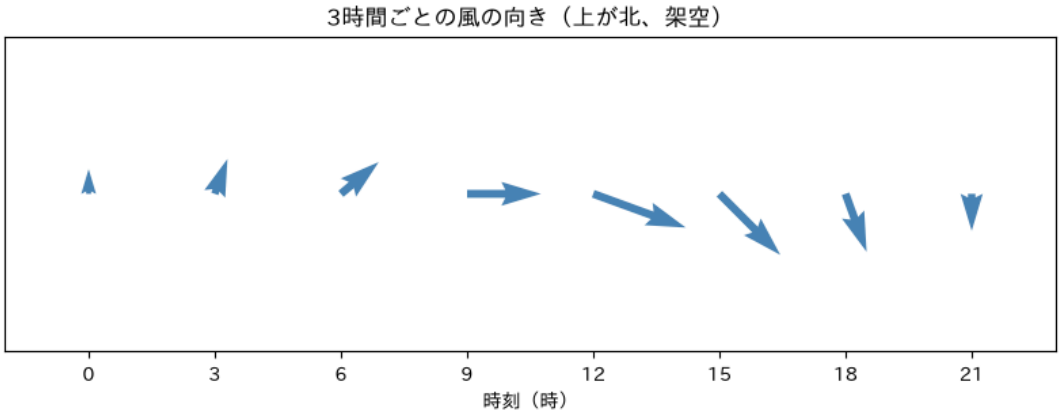
1日の風の向き

- 新潟のある1日の、3時間ごとの風（架空）。風向は気象庁と同じく「**風が吹いてくる**向き」を、北を 0°、東を 90° とした角度で表す
- 角度と風速を、横の成分 `u` と縦の成分 `v` に分ける。吹いてくる向きの**反対**へ矢印を向けるので、マイナスを付ける

```
hour = [0, 3, 6, 9, 12, 15, 18, 21]
direction = [180, 200, 230, 270, 290, 315, 340, 0]
speed = [2, 3, 4, 6, 8, 7, 5, 3]

rad = np.radians(direction)
u = -np.array(speed) * np.sin(rad)
v = -np.array(speed) * np.cos(rad)

plt.figure(figsize=(10, 3))
plt.quiver(hour, np.zeros(8), u, v, color='steelblue')
plt.xticks(hour)
plt.yticks([])
plt.xlim(-2, 23)
plt.xlabel('時刻（時）')
plt.title('3時間ごとの風の向き（上が北、架空）')
plt.show()
```



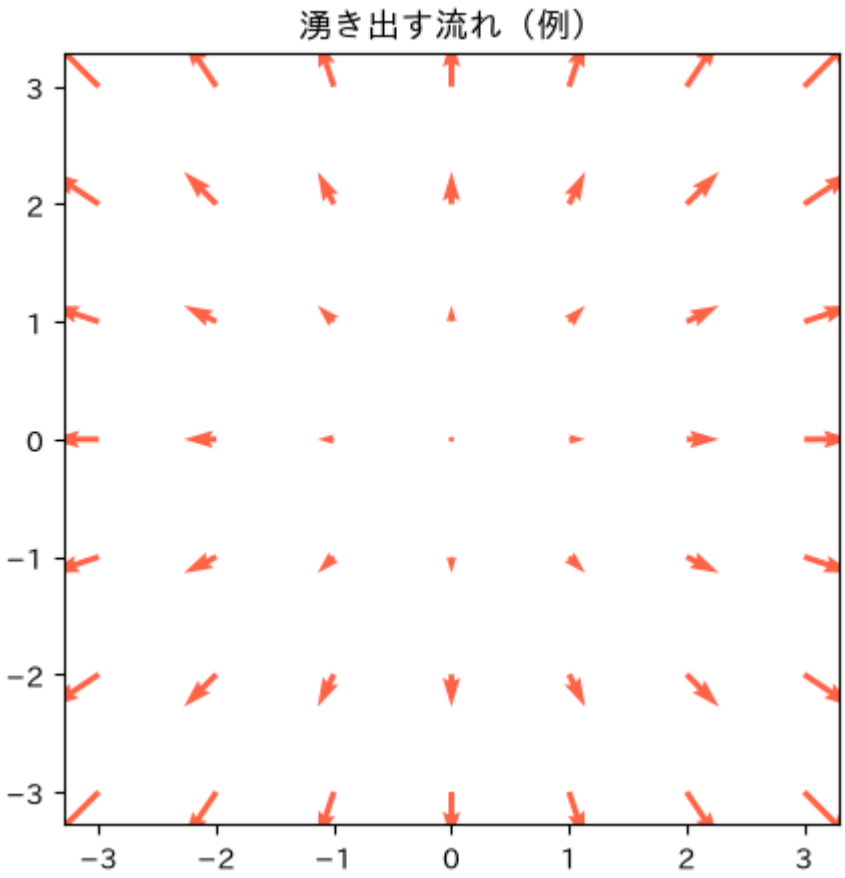
- 夜中から朝は南寄りの弱い風（矢印が上＝北向き）。昼ごろは西寄りの強い風（矢印が右向き）になり、夕方からは北寄りの風（矢印が下向き）に変わって弱まる
- `np.radians(角度)` は、度をラジアンに直す命令。`np.sin`・`np.cos` はラジアンで計算する
- 矢印の長さが風速を表す。昼（12時）の矢印が一番長い
- 本物のデータは、気象庁の「過去の気象データ」で、地点ごとの風向・風速を見られる

面に広がる流れ（湧き出しと渦）

- 地図のような面の上に、格子状に矢印を並べると、流れ全体の様子が見える
- `np.meshgrid(横の値, 縦の値)` で、格子の点の x 座標と y 座標をまとめて作る
- 面の流れの基本の形は2つある。**湧き出し**（1点から外へ広がる）と、**渦**（1点のまわりを回る）
- まず湧き出し。点 `(x, y)` での向きを `(x, y)` にする（中心から、その点へ向かう向きのまま外へ進む）

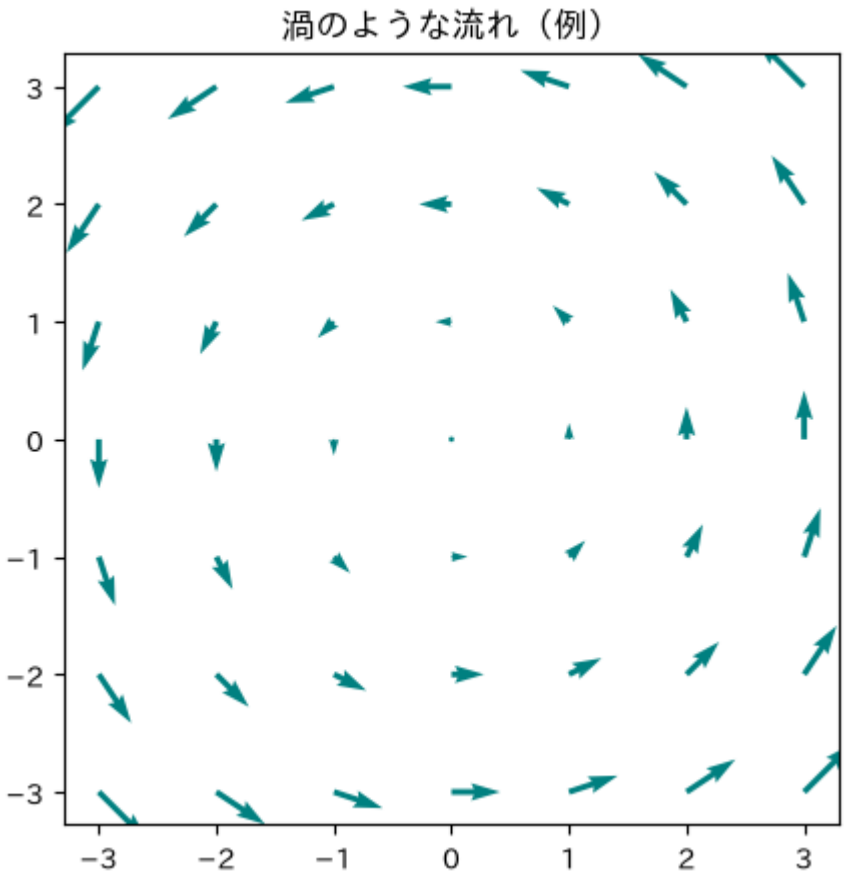
```
x, y = np.meshgrid(range(-3, 4), range(-3, 4))

plt.figure(figsize=(5, 5))
plt.quiver(x, y, x, y, color='tomato')
plt.axis('equal')
plt.title('湧き出す流れ（例）')
plt.show()
```



- 中心から外へ向かって広がり、中心から離れるほど矢印が長い
- 次に渦。点 `(x, y)` での向きを `(-y, x)` にする（湧き出しの向きを左に 90° 回したもの）。左回りの**渦**になる

```
plt.figure(figsize=(5, 5))
plt.quiver(x, y, -y, x, color='teal')
plt.axis('equal')
plt.title('渦のような流れ（例）')
plt.show()
```



- こちらも中心から離れるほど矢印が長い（流れが速い）
- 天気予報の風の図や、海流の図は、この形に地図を重ねたもの

湧き出しと渦を数で表す（発散と回転）

- 大学の理系の数学（**ベクトル解析**）では、流れのその場所での「湧き出しの強さ」を**発散**、「渦の強さ」を**回転**という数で表す。ここは紹介だけで、式が分からなくても先へ進んでよい
- 流れの向きを `(u, v)`（横の成分・縦の成分）とし、`u`・`v` は場所 `(x, y)` によって変わるとする。$\dfrac{\partial u}{\partial x}$ は、`y` を変えずに `x` だけを少し動かしたときの `u` の変わり方（**偏微分**）を表す

$$\text{発散}\ \ \mathrm{div} = \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y}, \qquad \text{回転}\ \ \mathrm{rot} = \frac{\partial v}{\partial x} - \frac{\partial u}{\partial y}$$

- 発散：横に進むと横向きの流れが強まり、縦に進むと縦向きの流れが強まるなら、その場所からは流れ出す量の方が多い。正なら**湧き出し**、負なら**吸い込み**
- 回転：右へ進むと上向きの流れが強まり、上へ進むと右向きの流れが弱まるなら、流れは左へ曲がる。正なら**左回り**、負なら**右回り**
- 上の2つの例に当てはめる

流れ	u	v	発散	回転
湧き出し	x	y	1 + 1 = **2**	0 − 0 = **0**
渦	-y	x	0 + 0 = **0**	1 − (−1) = **2**

- 湧き出しは「発散だけがあり回転がない」流れ、渦は「回転だけがあり発散がない」流れ。どんな流れも、場所ごとに見ると、この2つの成分を持っている

- 天気図では、地表の**低気圧**は風が中心へ吹き込み（発散が負）、北半球では左回りの渦（回転が正）になる。**高気圧**は風が吹き出し（発散が正）、右回りになる。台風は、強い左回りの渦が吸い込まれていく流れ

AV-9 ＋α のグラフの型の一覧

AU-3 の7種類に足して、困ったらこの表の形を写してデータを入れ替える。

グラフ	使う場面	書き方	節
値の順に並べた横棒	一番多いもの・順位を見せる	`s = df.sort_values('列名')` → `plt.barh(項目, 値)`	AV-1
100% 積み上げ棒	合計の違う項目どうしで、内訳の割合を比べる	割合 = 値 ÷ 合計 × 100 → `plt.bar(x, 上の段, bottom=下の段)`	AV-2
増加率の棒	規模の違うものどうしで、伸びを比べる	増加率 = （今年 − 前年） ÷ 前年 × 100 → `plt.bar`	AV-3
線の間を塗る	2本の線のどちらが上か、差の大きさ	`plt.fill_between(x, 線1, 線2, where=条件)`	AV-4
ヒートマップ	縦横に並んだ表の、どこが多いか	`plt.imshow(表, cmap='Blues')` + `plt.colorbar()`	AV-5
並べた1枚	報告のまとめの1枚	`plt.subplot(縦, 横, 何番目)` + `plt.tight_layout()`	AV-6
ローソク足	1日の4つの値（始値・高値・安値・終値）の動き	`plt.vlines(x, 安値, 高値)` + `plt.bar(x, 終値 - 始値, bottom=始値)`	第 7 講 R-4
ベクトル図	向きと強さを持つ量（風・海流）。面の流れは湧き出しと渦	`plt.quiver(x, y, 横の成分, 縦の成分)`	AV-8

AF. よくあるエラーと対処

エラーが出たら、**いちばん最後の行**を見る。

エラー（最後の行）	よくある原因	対処
`SyntaxError`	`'` や `)` の閉じ忘れ、`:` の付け忘れ	指された行の前後を見る
`IndentationError`	for・if の下の字下げがずれている	スペース4つにそろえる
`NameError`	変数名の打ち間違い／上のセルを実行していない	上のセルから実行し直す
`ModuleNotFoundError`	ライブラリが入っていない	`pip install japanize-matplotlib` など
`FileNotFoundError`	Excel が notebook と別のフォルダにある／名前の打ち間違い	notebook と同じ場所（いつもの場所）に置く
`KeyError`	`df['列名']` の列名が違う（空白・全角半角）	`df.head()` で列名を確かめる

エラー（最後の行）	よくある原因	対処
`ValueError: x and y must have same first dimension`	x と y の個数が違う	`len()` で数をそろえる
日本語が □ になる	`import japanize_matplotlib` を忘れた	import のセルを実行する
保存した画像が白紙	`savefig` を `show` の後に書いた	`savefig` を `show` の前に

- エラーの文をそのまま AI や検索に貼ると、原因が分かることが多い。ただし、直したコードは**自分で読んで**から使う

まとめ

- グラフは「何を見せたいか」で選ぶ：**移り変わり**→折れ線、**関係**→散布図、**大きさ比べ**→棒、**分布**→ヒストグラム、**ばらつき比べ**→箱ひげ図、**割合**→円、**平均とばらつき**→エラーバー
- どのグラフも **Excel を読む** → **列を取り出す** → **描く** の3段。困ったら AU-3〜AU-5 の表と、習った回の notebook に戻る
- 単位や大きさの違う2つの量は、**複合グラフ**で重ねる。`plt.twinx()` の前が左の y 軸、後が右の y 軸
- 棒グラフの縦軸は **0 から**。円グラフの項目は **5つくらいまで**。描いたら、AT-3 のチェックリストで自分のグラフを確かめる
- ＋α：新しい形のグラフも、**棒・線・色分けの組み合わせ**で描けることが多い（100% 積み上げ棒、ローソク足）。形の一覧は AV-9
  - 「一番多いもの」を見せたいときは、**値の順に並べ替えて**横棒で描く（AV-1）
  - 列どうしを計算すると、**割合・増加率・1人あたり**などの新しい量が作れる。合計や規模の違うものどうしも比べられる（AV-2・AV-3）
  - 2本の線の差は**間を塗る**と、表の形の数は**色の濃さ**（**ヒートマップ**）で表すと、ひと目で分かる（AV-4・AV-5）
  - 違うシートのグラフも、`plt.subplot` で**1枚にまとめられる**（AV-6）
  - 向きと強さを持つ量（風・海流）は、**矢印**（**ベクトル図**）で描く。面の流れの基本の形は**湧き出し**と**渦**で、その強さはベクトル解析の**発散**と**回転**で表す（AV-8）
- エラーは**最後の行**を見て、AF の表で原因を探す

付録 よくある質問

授業中に出た質問のうち、自分でグラフを描くときに役立つものをまとめた。

Q. 全部覚えないといけない？

覚えなくてよい。各講の「完成形」と部品表のセルをコピーして、変えたいところだけ書き換えれば十分。オプションの名前を忘れたら、関数名の後ろの「(」にカーソルを置いて Shift＋Tab を押すと、説明（英語）が出る（第２講 ショートカットと関数の調べ方）。よく使うものは、使っているうちに覚える。

Q. グラフのコードは、どこに何を書けばよい？

どのグラフのコードも、同じ順番で並んでいる。
[1] import（道具の準備） [2] データ（リストや pd.read_excel） [3] グラフを描く1行（plt.bar・plt.pie・plt.boxplot など） [4] 飾り（title・xlabel・xticks・legend など） [5] 保存（plt.savefig） [6] 表示（plt.show）
新しい命令が出てきたら、[3] と [4] のどちらかを考える。[3] のかっこの中に書くもの（color=、bottom=、showmeans=True など）は「その棒・箱の描き方」、[4] の行は「グラフ全体の飾り」。1行ずつ消したり（行の先頭に # を付けたり）値を変えたりして実行し、グラフのどこが変わるかを見ると、意味がつかめる。

Q. title・xlabel・xlim などは、どの順番で書く？

plt.figure（大きさ）を最初に、グラフ本体（plot・bar など）を次に書けば、title・xlabel・ylabel・xlim・ylim の順番は自由で、どの順でも同じグラフになる。順番が効くのは、plt.legend（label= を書いた後）、plt.twinx（前と後で効く軸が変わる）、plt.savefig と plt.show（最後。savefig は show の前）（第11講 AU-5）。

Q. 両 y 軸のグラフの決まりは？

plt.twinx() より前に書いたものは左の y 軸、後に書いたものは右の y 軸に効く。横軸（xlabel・xticks）と題名は1回だけ（ふつうは twinx の前）書く。凡例は左右で別々に作られるので、loc='upper left' と loc='upper right' のように場所を分ける。「左のグラフ1つ分のコード」＋plt.twinx()＋「右のグラフ1つ分のコード」と読むとよい（第８講 Z-2）。

Q. labels と label、colors と color の違いは？

s が付くのは plt.pie だけ、と覚えるのが確実。plt.plot・plt.scatter・plt.bar の label=・color= は「線や点のまとまり1つ」に名前・色を1つ付けるので単数（棒の色をリストで渡すときも color= のまま）。plt.pie は1つの円を切り分けて、ひと切れずつ名前と色を付けるので labels=・colors=。「円は切り分けるから s」。plt.tick_params(colors=…) も、目盛りと目盛りの文字の2つに色を付けるので colors。

Q. 1文字の違いでエラーになる

綴りの間違いは一番よくあるエラー。True・False は先頭を大文字で書く（true は NameError）。エラーの最後の行に「Did you mean 'wedgeprops'?」のように正しい候補が出ることもある。cmap='RdYlGn_r' のように引用符の中に半角スペースが入っても、別の名前になってエラーになる。エラー文に出ている名前と、自分のコードを1文字ずつ見比べる。エラー文に「U+3000」とあれば、全角スペースが入っている。

Q. 改行の ¥n をコピーすると \n に変わった

コンピューターの中では、¥ と \（バックスラッシュ）は同じ文字。日本語の Windows では、フォントによって ¥ で表示されたり \ で表示されたりするだけなので、どちらでも正しく動く。スラッシュ（/）は別の文字なので、/n と書くと改行にならない。

Q. 棒の上に値を書くには？

b = plt.bar(項目, 値) のように棒グラフを変数 b に入れて、plt.bar_label(b) と書く。b を作らずに plt.bar_label(b) と書くと NameError になる（第７講 R-3）。

Q. 横棒グラフがうまく描けない

plt.bar を plt.barh に変えるだけで描ける。横向きになると、項目は縦軸、量は横軸に移るので、xlabel と ylabel の中身を入れ替える。平均の補助線も、横の線 plt.axhline(y=…) ではなく、縦の線 plt.axvline(x=np.mean(y)) にする（第６講 N-7）。

Q. 棒が多いとき、1本だけ色を変えたい

colorlist = ['c'] * len(x) で同じ色を棒の数だけ並べ、colorlist[5] = 'm' で1つだけ置き換えて、plt.bar(x, y, color=colorlist) に渡す（第６講 P-4）。

Q. 色の組み合わせを変えたい

何も指定しないときの色の組み合わせは tab10（青・オレンジ・緑…）。そのグラフだけ替えるときは plt.pie(…, colors=plt.get_cmap('Set2').colors) のように書き足す。以降のグラフをまとめて替えるときは plt.rcParams['axes.prop_cycle'] の1行（第６講 色の組み合わせを替える）。

Q. カラーマップの色の順を逆にしたい

もとの名前の最後に _r を付ける（例：'RdYlGn_r'）。名前を並べ替えた 'GnYlRd' というカラーマップはないので、エラーになる。

Q. subplot の数字の意味は？

plt.subplot(縦の数, 横の数, 何番目)。plt.subplot(1, 2, 1) は「縦1 × 横2 の枠のうち1番目（左）」、plt.subplot(1, 2, 2) は「2番目（右）」。番号は左上から右へ数え、行が終わったら次の行へ進む（2 × 2 なら上の段が 1・2、下の段が 3・4）。(2, 1, 1)・(2, 1, 2) にすると縦に並ぶ（第10講 AI-1）。

Q. 箱ひげ図の patch_artist=True は何？

箱を「線」として描くか「面」として描くかの選択。面にすると色を塗れるので、patch_artist=True で面にしてから boxprops={'facecolor': '色'} で色を付ける（第７講 S-4）。

Q. ヒストグラムがバーコードのように細くなった

区切り（bins）が多すぎるとこうなる。例えば bins=range(0, 101, 1) と1点刻みにすると棒が100本になる。2つのグループを1回の plt.hist で描くと、1つの区切りに棒が2本並ぶので、もっと細くなる（第９講 AA-2）。

Q. n, bins, patches = plt.hist(...) は何をしている？

plt.hist は、グラフを描くと同時に3つの値を返す。n は各区間の人数、bins は区切りの位置、patches は棒の図形（1本ずつの部品が左から並んだもの。番号は 0 から）。patches[7].set_facecolor('r') で、左から8本目の棒だけ色を変えられる。b = plt.bar(…) を plt.bar_label(b) に渡したのと同じ仕組み（第９講 AC-3）。グラフを描くだけなら plt.hist(…) だけでよい。

Q. 縦軸の範囲を変えてもよい？

箱ひげ図・エラーバー・散布図は位置で比べるグラフなので、plt.ylim() でデータのある範囲に絞ってよい。ただし、比べる図どうしは同じ範囲にそろえる。棒グラフは棒の長さで量を表すので、縦軸は 0 から始めるのが原則（第11講 AT-1）。

Q. 折れ線と散布図はどう使い分ける？

折れ線は、時間などの順番に沿った変化（月ごとの気温など）に使う。散布図は、2つの量の関係（身長と体重、気温と売上など）に使う。オプションの名前の違いは 第４講 参考：折れ線と散布図のオプション比較。

Q. 拡大してもぼやけない形式で保存したい

ベクタ形式で保存する。plt.savefig('graph.svg') のように、拡張子を svg（または pdf）にするだけ。png・jpg はラスタ形式で、拡大すると粗くなる（第５講 K-3）。

Q. ネットで調べても、書き方が違うものが出てくる

matplotlib には、この講義ノート集のように plt.… で書くスタイルと、ax.… で書くスタイルがある。調べるときは「matplotlib plt 2軸 色」のように「plt」とやりたいことを短い言葉で入れると、近い書き方が出やすい（ax. を使った書き方が出てきたら、別のスタイル）。見つからないときは、まず「グラフの種類から探す」表とこのノート集の中を探す。AI に聞いたときも、直したコードは自分で読んでから使う。

Q. Excel ではなくプログラムでグラフを描く良さは？

手順がコードとして残るので、人のファイルをもらっても何をしているか分かる。手順が同じなら、データが変わってもコードをそのまま使い回せる。Markdown セルの文章と組み合わせると、そのままレポートになる。